

ویکی پاور

سایت تخصصی رشته های مهندسی برق ، کامپیوتر و ...



www.WikiPower.ir

هفده	پیشگفتار
۱	فصل ۱
۱	اعداد مبنا در سیستم های دیجیتال
۱	هدف کلی
۱	هدف ساختاری
۲	۱-۱ معماری سیستم های کامپیوتری
۳	۲-۱ سیستم های دیجیتال
۴	۱-۲-۱ عناصر گسسته در سیستم های دیجیتال
۶	۲-۲-۱ زبان توصیف سخت افزاری
۶	۳-۱ نمایش اطلاعات در کامپیوتر

پنج

۷	۱-۳-۱ اعداد دودویی
۱۰	۲-۳-۱ اعداد مبنای هشت و شانزده
۱۱	۳-۳-۱ تبدیل مبنای اعداد
۱۶	۴-۱ متمم اعداد
۱۶	۱-۴-۱ متمم مبنا
۱۸	۲-۴-۱ متمم در مبنای کاهش یافته
۱۹	۳-۴-۱ تفریق به کمک متمم‌ها
۲۲	۴-۴-۱ اعداد دودویی علامت‌دار
۲۵	۵-۱ جمع حسابی
۲۷	۶-۱ تفریق حسابی
۲۸	۷-۱ کدهای دودویی
۲۹	۱-۷-۱ کد BCD
۳۱	۲-۷-۱ جمع BCD
۳۳	۳-۷-۱ حساب دهدهی
۳۴	۴-۷-۱ دیگر کدهای دهدهی
۳۶	۵-۷-۱ کد کاراکتراسکی
۳۷	۶-۷-۱ کدهای کنترل کننده در ASCII
۳۸	۷-۷-۱ کد تشخیص خطا

۴۳	فصل دوم
۴۳	گیت‌های منطقی، جبر بول و توابع بولی
۴۳	هدف کلی
۴۳	هدف ساختاری
۴۴	۱-۲ منطق دودویی
۴۴	۱-۱-۲ تعریف منطق دودویی
۴۶	۲-۱-۲ گیت‌های منطقی
۴۹	۲-۲-۲ جبر بول
۵۲	۱-۲-۲ تعریف اصول اساسی جبر بول
۵۶	۲-۲-۲ قضایای اصلی و خواص جبر بول
۵۸	۲-۲-۲-۲ تقدم عملگرها
۵۹	۳-۲ توابع بول
۶۱	۱-۳-۲ متمم یک تابع
۶۳	۲-۳-۲ سایر اعمال منطقی
۶۶	۴-۲ گیت‌های منطقی دیجیتال
۶۹	۱-۴-۲ گسترش ورودی گیت‌ها
۷۲	۲-۴-۲ مدارهای مجتمع
۷۹	فصل ۳
۷۹	فرم‌های متعارف و استاندارد در جبر بولی

۷۹	هدف کلی
۷۹	هدف ساختاری
۸۰	۳-۱ فرم‌های استاندارد
۸۰	۳-۱-۱ جمع حاصل ضرب‌ها
۸۲	۳-۱-۲ ضرب حاصل جمع‌ها
۸۴	۳-۱-۳ مفهوم فرم‌های متعارف
۸۵	۳-۱-۴ حداقل سازی سطوح گیت
۸۶	۳-۱-۵ مجموع مینترم‌ها
۸۸	۳-۱-۶ ضرب ماکسترم‌ها
۸۹	۳-۲ تبدیل فرم‌های متعارف به یکدیگر
۹۳	فصل ۴
۹۳	ساده کردن عبارات بولی پیچیده
۹۳	هدف کلی
۹۳	هدف ساختاری
۹۳	۴-۱ دستکاری جبری
۹۷	۴-۲ ساده‌سازی با استفاده از نقشه کارنو
۹۸	۴-۲-۱ نقشه دو متغیره کارنو
۹۹	۴-۲-۲ نقشه سه متغیره کارنو
۱۰۵	۴-۲-۳ نقشه چهار متغیره

۱۰۹	۴-۲-۴ نقشه پنج متغیره کارنو
۱۱۲	۴-۲-۵ عناصر اصلی در جدول کارنو
۱۱۴	۴-۳ ساده سازی با ضرب حاصل جمع ها
۱۱۸	۴-۴ حالات بی اهمیت
۱۲۳	فصل ۵
۱۲۳	پیاده سازی مدارهای دیجیتال با گیت های NAND و NOR
۱۲۳	هدف کلی
۱۲۳	هدف ساختاری
۱۲۴	۵-۱ مدارهای NAND
۱۲۵	۵-۱-۱ پیاده سازی دو سطحی گیت NAND
۱۲۸	۵-۱-۲ روال تهیه مدار NAND از تابع بول
۱۲۸	۵-۱-۳ مدارهای NAND چند سطحی
۱۳۱	۵-۲ مدارهای NOR و روش پیاده سازی آنها
۱۳۴	۵-۳ منطق سیمی
۱۳۶	۵-۴ فرم های مفید گیت ها
۱۳۷	۵-۴-۱ پیاده سازی AND-OR-INVERT
۱۳۸	۵-۴-۲ پیاده سازی OR-AND-INVERT
۱۴۰	۵-۵ تابع OR انحصاری
۱۴۳	۵-۵-۱ تابع فرد

۱۴۶	۲-۵-۵ تولید و چک توازن
۱۴۹	۶-۵ زبان توصیف سخت‌افزاری (HDL)
۱۵۱	۱-۶-۵ نمایش مدول
۱۵۳	۲-۶-۵ تاخیر در گیت‌ها
۱۵۶	۳-۶-۵ عبارت بولی
۱۶۱	فصل ۶
۱۶۱	مدارهای ترکیبی
۱۶۱	هدف کلی
۱۶۱	هدف ساختاری
۱۶۱	۱-۶ مدارهای ترکیبی
۱۶۴	۲-۶ روش تحلیل
۱۶۴	۱-۲-۶ تهیه توابع بول خروجی از یک مدار منطقی
۱۶۶	۲-۲-۶ تهیه جدول درستی از نمودار منطقی
۱۶۸	۳-۶ روش طراحی
۱۶۹	۱-۳-۶ مکانیزم‌های تبدیل اعداد در مبناهای متفاوت
۱۷۳	۴-۶ جمع‌کننده‌ها و تفریق‌گرهای دودویی
۱۷۳	۱-۴-۶ نیم جمع‌کننده
۱۷۵	۲-۴-۶ جمع‌کننده کامل
۱۷۷	۳-۴-۶ جمع‌کننده دودویی

۱۷۹	۶-۴-۴ انتشار رقم نقلی
۱۸۵	۶-۴-۵ تفریق دودویی
۱۸۷	۶-۴-۶ مفهوم سرریز
۱۸۹	۶-۴-۷ جمع کننده دهمی
۱۸۹	۶-۴-۸ جمع کننده BCD
۱۹۲	۶-۵ ضرب دودویی
۱۹۴	۶-۶ مقایسه گر مقدار
۱۹۵	۶-۶ مقایسه گر مقدار
۱۹۹	فصل ۷
۱۹۹	مدارهای رمزگذار و رمزگشا
۱۹۹	هدف کلی
۱۹۹	هدف ساختاری
۲۰۰	۷-۱ مدارات رمزگشا (دیکدر)
۲۰۲	۷-۱-۱ پیاده سازی دیکدر با گیت NAND
۲۰۴	۷-۱-۲ پیاده سازی مدار منطقی ترکیبی با دیکدر
۲۰۶	۷-۲ مدارات رمز گذار (انکدر)
۲۰۸	۷-۲-۱ انکدر اولویت
۲۱۰	۷-۳ مولتی پلکسر
۲۱۴	۷-۳-۱ پیاده سازی تابع بول

یازده

۲۱۶	۲-۳-۷ گیت‌های سه حالته
۲۱۹	۴-۷ زبان HDL برای مدارهای ترکیبی
۲۲۰	۱-۴-۷ مدل‌سازی سطح گیت
۲۲۲	۲-۴-۷ گیت‌های سه حالته
۲۲۴	۳-۴-۷ مدل‌سازی روند داده
۲۲۸	۴-۴-۷ مدل‌سازی رفتاری
۲۳۰	۵-۴-۷ نوشتن یک برنامه تست ساده
۲۳۹	فصل ۸
۲۳۹	مدارهای ترتیبی همزمان
۲۳۹	هدف کلی
۲۳۹	هدف ساختاری
۲۴۰	۱-۸ مدارهای ترتیبی
۲۴۱	۱-۱-۸ انواع مدارهای ترتیبی
۲۴۲	۲-۸ فلیپ‌فلاپ‌ها و لچ‌ها
۲۴۳	۱-۲-۸ لچ‌ها
۲۴۹	۳-۸ مکانیزم تغییر حالت لچ‌ها
۲۵۱	۱-۳-۸ فلیپ‌فلاپ D حساس به لبه
۲۵۴	۴-۸ فلیپ‌فلاپ‌های T و JK
۲۵۵	۱-۴-۸ فلیپ‌فلاپ JK

دوازده

۲۵۶	۲-۴-۸ فلیپ فلاپ T
۲۵۶	۳-۴-۸ جدول مشخصه فلیپ فلاپ ها
۲۵۸	۴-۴-۸ معادلات مشخصه
۲۵۸	۵-۴-۸ ورودی های سیستم
۲۶۰	۵-۸ تحلیل مدارهای ترتیبی ساعت دار
۲۶۱	۱-۵-۸ معادلات حالت
۲۶۲	۲-۵-۸ جدول حالت
۲۶۶	۶-۸ تحلیل معادلات ورودی با فلیپ فلاپ
۲۶۷	۱-۶-۸ تحلیل معادلات با کمک فلیپ فلاپ های D
۲۶۹	۲-۶-۸ تحلیل معادلات با کمک فلیپ فلاپ های JK
۲۷۲	۳-۶-۸ تحلیل معادلات با کمک فلیپ فلاپ های T
۲۷۷	فصل ۹
۲۷۷	ثبات ها و شمارنده ها
۲۷۷	هدف کلی
۲۷۷	هدف ساختاری
۲۷۸	۱-۹ ذخیره سازی دودویی و ثبات ها
۲۷۸	۱-۱-۹ ثبات ها
۲۷۹	۲-۱-۹ انتقال بین ثباتی
۲۸۲	۳-۱-۹ شمارنده ها

سیزده

۲۸۲	۲-۹ کاربرد فلیپ فلاپ در ثبات‌ها
۲۸۴	۱-۲-۹ ثبات با بارشدن موازی
۲۸۶	۲-۲-۹ شیفت رجیسترها
۲۸۷	۳-۲-۹ انتقال سریال
۲۹۰	۴-۲-۹ جمع‌کننده سریال
۲۹۴	۵-۲-۹ شیفت رجیستر
۲۹۵	۶-۲-۹ انواع شیفت رجیسترها
۲۹۷	۳-۹ شمارنده‌های موج‌گونه
۲۹۸	۱-۳-۹ شمارنده موج‌گونه دودویی
۳۰۲	۲-۳-۹ شمارنده BCD موج‌گونه
۳۰۵	۴-۹ شمارنده‌های همزمان
۳۰۵	۱-۴-۹ شمارنده دودویی
۳۰۷	۲-۴-۹ شمارنده BCD
۳۰۹	۳-۴-۹ بالا-پایین شمار دودویی
۳۱۱	۴-۴-۹ شمارنده دودویی با بار شدن موازی
۳۱۵	۵-۹ انواع دیگر شمارنده‌ها
۳۱۵	۱-۵-۹ شمارنده حلقوی
۳۱۷	۲-۵-۹ شمارنده جانسون
۳۱۹	۳-۵-۹ شمارنده با حالات بی استفاده

چهارده

۳۲۳	مجموعه سؤالات خودآزمایی
۳۴۲	پاسخ نامه
۳۴۳	سؤالات تشریحی
۳۴۷	واژه نامه
۳۴۷	انگلیسی به فارسی
۳۵۳	واژه نامه
۳۵۳	فارسی به انگلیسی
۳۵۹	لیست منابع و مراجع

شانزده

پیشگفتار

این کتاب با توجه به سر فصل تعیین شده برای دانشجویان دانشگاه پیام نور در رشته کامپیوتر با گرایش نرم افزار تهیه و تنظیم شده است. در تهیه این کتاب سعی بر آن شده است تا مباحثی که برای تدریس درس سه واحدی مدار منطقی لازم به تدریس است، مطرح گردند. این کتاب مشتمل بر نه فصل می باشد.

در ابتدای کتاب لیست سر فصل مطالب قید شده است. در انتهای کتاب مجموعه ای از سؤالات شامل ۹۰ سوال تستی و ۲۳ سوال تشریحی به همراه پاسخ نامه سؤالات تستی ارائه شده است.

نظر به لزوم جاگذاری معادل فارسی کلمات تخصصی برای راحتی فهم دانشجویان دو واژه نامه به صورت انگلیسی به فارسی و فارسی به انگلیسی در انتهای کتاب آمده

هفده

است. در صفحه پایانی کتاب لیست منابع و ماخذ نیز برای آگاهی دانشجویان ارائه شده است.

این اثر با دقت نظر فراوان کارشناسان مدیریت تولید مواد و تجهیزات آموزشی مورد ارزیابی قرار گرفت که بدینوسیله از جناب آقای اکبری به نمایندگی از آن عزیزان قدردانی می‌نمایم.

کتاب حاضر بعنوان منبع درسی در دانشگاه پیام‌نور اعلام شده که بعلت کوتاه بودن زمان امکان رفع کلیه ایرادات تایپی و نگارشی میسر نشد. لذا با وجود سعی و دقت فراوان در پدید آوردن اثری خودخوان ضمن پذیرفتن ایرادات احتمالی، در نوبت اول تیراژ این اثر محدود خواهد بود تا در نیمسال آینده بعد از دریافت پیشنهادات اصلاحی صاحب‌نظران، اساتید و دانشجویان نسبت به چاپ در تیراژ بالاتر اقدام گردد.

در پایان از آقای مهندس کامیار آهنگوب که در تنظیم و تدوین کتاب همکاری شایانی داشته‌اند سپاسگزاری می‌کنم.

داود کریم‌زادگان مقدم

تابستان ۱۳۸۵

فصل ۱

اعداد مبنا در سیستم‌های دیجیتال

هدف کلی

در این فصل مباحث کلی سیستم‌ها و معماری مدارهای دیجیتال به صورت کلی مطرح شده و در ادامه مباحث مربوط به مبناهای اعداد و روش‌های تبدیل اعداد مبنا شرح داده خواهند شد. همچنین متمم‌های اعداد نیز مورد بحث و بررسی قرار گرفته و انواع اعداد دودویی علامت‌دار توضیح داده خواهند شد. در ادامه انواع کدهای دودویی و دهدهی و ... نیز ارائه خواهند شد.

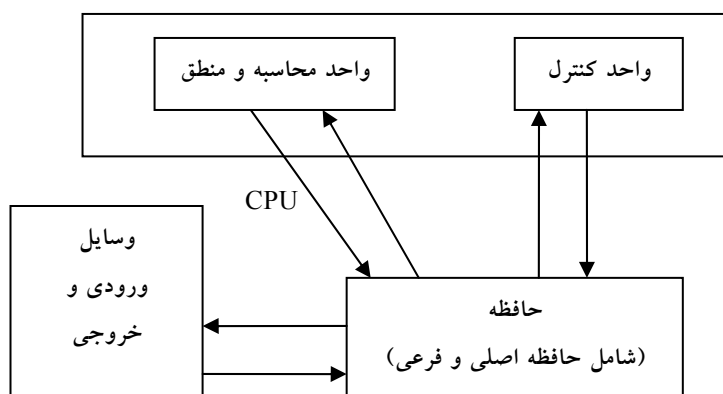
هدف ساختاری

در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند:

- سیستم‌های دیجیتال
- مبناهای اعداد
- روش‌های تبدیل اعداد در مبناهای مختلف
- متمم‌های اعداد
- اعداد دودویی علامت‌دار
- جمع و تفریق اعداد
- انواع جداول و کدهای دودویی و ...

۱-۱ معماری^۱ سیستم‌های کامپیوتری

سیستم‌های کامپیوتری که شامل مجموعه‌ای از دستورالعمل‌ها و سخت‌افزارها می‌باشند، بر اساس ترکیبی از دو نوع معماری نرم‌افزاری و سخت‌افزاری شکل می‌گیرند. معماری نرم‌افزار^۲ به صورت کلی شامل مجموعه‌ای از دستورات و فرمت استفاده از آنها می‌باشد. معماری سخت‌افزار^۳ سیستم نیز شامل مولفه‌های سخت‌افزاری به شرح ذیل می‌باشد:



شکل ۱-۱: شمای کلی از معماری سخت‌افزاری سیستم کامپیوتری

- پردازنده^۴ که ابزاری است برای تفسیر و اجرای دستورالعمل‌ها
- حافظه^۵ که ابزاری است برای ذخیره‌سازی داده‌ها و برنامه‌ها
- ابزار انتقال اطلاعات بین اجزاء داخلی کامپیوتر و یا مابین کامپیوتر و محیط بیرونی

^۱ Architecture

^۲ Software Architecture

^۳ Hardware Architecture

^۴ Processor

^۵ Memory

شکل ۱-۱ معماری یک سیستم کامپیوتری را به صورت شماتیک و کلی نشان می‌دهد که در آن نحوه ارتباط بین عناصر نیز آمده است:

۱-۲ سیستم‌های دیجیتال

امروزه سیستم‌های الکترونیکی یا به بیانی دیگر سیستم‌های دیجیتال که شامل مجموعه‌ای از مدارات برقی و... می‌باشند در تقریباً تمامی علوم مانند مخابرات، تجارت، محاسبات ریاضی و علمی، ناوبری هواپیماها و سفینه‌های فضایی، اعمال جراحی، اینترنت و بسیاری از دیگر زمینه‌های تجاری، صنعتی و علمی به کار می‌روند. بهترین مثال از یک سیستم دیجیتال، کامپیوتر دیجیتال همه منظوره است. مهمترین خاصیت یک کامپیوتر دیجیتال، همگانی بودن آن است. کامپیوتر می‌تواند رشته‌ای از دستورات به نام برنامه را که روی داده‌های مفروض عمل می‌کنند، دنبال نماید. کاربر می‌تواند برنامه یا داده خود را طبق نیاز انتخاب و اجرا کند. به علت این انعطاف، کامپیوترهای همه منظوره دیجیتال می‌توانند عملیات پردازش اطلاعات را در محدوده وسیعی از کاربردها انجام دهند. بخشهای اصلی یک کامپیوتر عبارتند از واحد حافظه، واحد پردازش مرکزی و واحدهای ورودی-خروجی، واحد حافظه برنامه‌ها و داده‌های وارده، خارج شونده و میانی را ذخیره می‌کند. واحد پردازش مرکزی اعمال محاسباتی و دیگر عملیات روی داده‌ها را بر حسب آنچه در برنامه مشخص شده، انجام می‌دهد. داده‌ها و برنامه‌هایی که به وسیله کاربر آماده شده‌اند توسط وسایل ورودی مانند صفحه کلید به حافظه انتقال می‌یابند. یک وسیله خروجی مثل چاپگر نتایج حاصل از محاسبات را دریافت کرده و به کاربر ارائه می‌دهد. یک کامپیوتر دیجیتال می‌تواند به چندین وسیله ورودی-خروجی وصل شود. یکی از وسایل مفید واحد مخابره است که تبادل داده را از طریق اینترنت با دیگر کاربران برقرار می‌سازد. یک کامپیوتر دیجیتال دستگاهی توانمند است که نه تنها می‌تواند محاسبات ریاضی را انجام دهد، بلکه قادر است اعمال

منطقی را هم اجرا نماید. به علاوه می تواند جهت تصمیم گیری بر اساس شرایط داخلی یا خارجی برنامه ریزی شود.

۱-۲-۱ عناصر گسسته^۱ در سیستم های دیجیتال

یکی از ویژگی های سیستم دیجیتال، توانمندی آنها در دستکاری عناصر گسسته اطلاعاتی است. هر مجموعه ای که به تعداد متناهی از عناصر محدود باشد، اطلاعاتی گسسته را داراست. مثال هایی از عناصر گسسته عبارتند از ۱۰ رقم دهدهی، ۲۶ حرف الفباء، ۶۴ مربع بازی شطرنج. کامپیوترهای دیجیتال اولیه برای محاسبات عددی به کار می رفتند. در این حال، عناصر گسسته به کار رفته، ارقام بودند. نام دیجیتال یا رقمی از این مفهوم حاصل شده است. عناصر گسسته اطلاعاتی در یک سیستم دیجیتال با کمیت های فیزیکی به نام سیگنال^۲ نشان داده می شوند. رایج ترین سیگنال های الکتریکی عبارتند از ولتاژ و جریان. وسایل الکترونیکی به نام ترانزیستور در مداراتی که این سیگنال ها را پیاده سازی می کنند به طور چشمگیری به کار می روند. سیگنال ها در بسیاری از سیستم های دیجیتال الکترونیک امروزی، تنها دو مقدار را دارا هستند و بنابراین آنها را دودویی می نامند. یک رقم دودویی که بیت خوانده می شود دو مقدار دارد: 0 و 1. عناصر گسسته اطلاعاتی با گروهی از بیت ها به نام کدهای دودویی نمایش داده می شوند. مثلاً ارقام دهدهی 0 تا 9 در سیستم اعداد دیجیتال با کد چهار بیتی نشان داده می شوند. با به کارگیری تکنیک های مختلف، گروه هایی از بیت ها برای نمایش سمبل های گسسته تعریف می شوند و سپس در توسعه یک سیستم در قالب دیجیتال مورد استفاده قرار می گیرد. در نتیجه، یک سیستم دیجیتال سیستمی است که با عناصر گسسته اطلاعاتی به شکل دودویی کار می کند.

^۱ Discrete Elements

^۲ Signal

اعداد مبنا در سیستم‌های دیجیتال ۵

کمیته‌های اطلاعاتی یا ذاتاً گسسته‌اند و یا از نمونه‌برداری فرآیندهای پیوسته حاصل می‌شوند. به عنوان مثال یک لیست حقوق ذاتاً یک فرآیند یا رویداد گسسته بوده و حاوی: نام کارمند، شماره تامین اجتماعی، حقوق هفتگی، مالیات بر درآمد و غیره است. پرداختی به یک کارمند با استفاده از مقادیر داده گسسته مانند حروف الفبایی (نام‌ها)، ارقام (حقوق)، و نمادها یا سمبل‌های خاص (مانند \$) پردازش می‌گردد. از طرف دیگر یک محقق ممکن است یک پدیده را به صورت پیوسته مشاهده کند، ولی فقط مقادیر خاصی را به صورت جدول ثبت نماید. بنابراین فرد محقق داده پیوسته را نمونه‌برداری می‌نماید ولی هر کمیت در جدول را از عناصر گسسته می‌سازد. در بسیاری از حالات نمونه‌برداری از یک فرآیند به‌طور خودکار به‌وسیله دستگاهی به نام مبدل آنالوگ به دیجیتال انجام می‌شود.

برای استفاده از مدارهای دیجیتال در تولیدات تجاری دلایل اساسی وجود دارد. همچون کامپیوترهای دیجیتال، دستگاههای دیجیتال نیز قابل برنامه‌ریزی‌اند. با تعویض برنامه در یک وسیله برنامه‌پذیر، همان سخت‌افزار یگانه، قابلیت استفاده در کاربردهای متفاوت را خواهد داشت. کاهش قیمت شدید در وسایل دیجیتال به دلیل پیشرفت در تکنولوژی مدارهای مجتمع دیجیتال مرتباً روی می‌دهد. با افزایش تعداد ترانزیستورها در یک قطعه سیلیکان، توابع پیچیده‌تری پیاده‌سازی شده، قیمت هر واحد کاهش یافته و قیمت دستگاههای دیجیتال روز بروز کاهش می‌یابد. دستگاههای ساخته شده با مدارهای مجتمع می‌توانند با سرعتی تا صد میلیون عمل در ثانیه را انجام دهند. می‌توان با استفاده از کدهای اصلاح خطا عملکرد سیستم‌های دیجیتال را به شدت اطمینان بخش نمود. مثالی از این نوع، دیسک چند کاره دیجیتال (DVD) است که در آن

اطلاعات ویدیویی، صوتی و دیگر گونه‌ها بدون از دست رفتن حتی یک قلم داده، ضبط می‌گردد.

۱-۲-۲ زبان توصیف سخت‌افزاری^۱

یک سیستم دیجیتال از به هم پیوستن ماژول‌های دیجیتال^۲ به دست می‌آید. برای درک عمل هر ماژول، دانش و آگاهی مدارهای دیجیتال و عمل منطقی آنها لازم است. یک گرایش مهم در طراحی دیجیتال، استفاده از زبان توصیف سخت‌افزاری است. HDL نوعی زبان برنامه‌ریزی است که برای توصیف مدارهای دیجیتال به صورت متن به کار می‌رود. این زبان برای شبیه‌سازی یک سیستم دیجیتال و اطمینان از صحت عمل آن قبل از ساخت مورد استفاده قرار می‌گیرد.

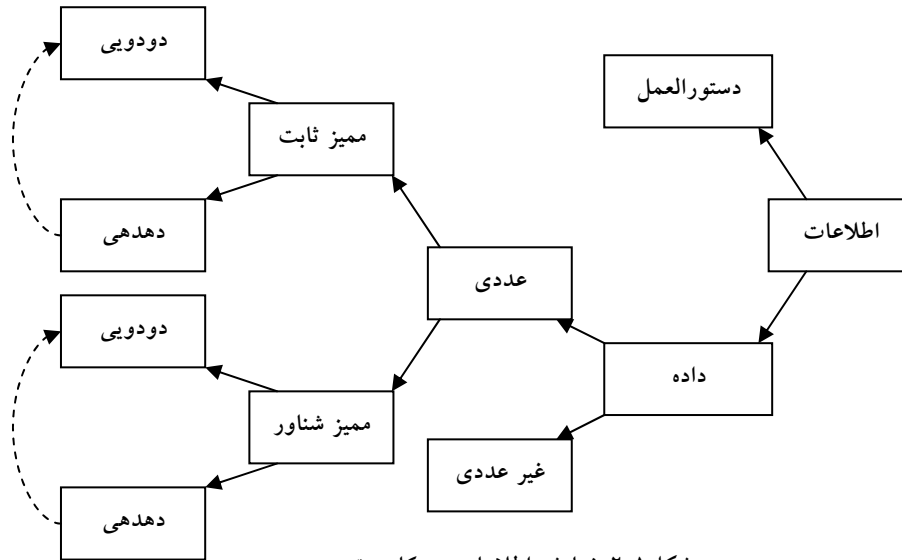
۱-۳ نمایش اطلاعات در کامپیوتر

همانطور که قبلاً گفته شد سیستم‌های دیجیتال کمیت‌های گسسته اطلاعات، که به فرم دودویی نمایش داده شده‌اند را دستکاری می‌نماید. عملوندهای به کار رفته در محاسبات را می‌توان در سیستم دودویی بیان کرد. دیگر عناصر گسسته از جمله ارقام دهدهی به صورت کدهای دودویی نشان داده می‌شوند. پردازش داده به وسیله عناصر منطقی دودویی و با استفاده از سیگنال‌های دودویی انجام می‌گیرد. کمیت‌ها نیز در عناصر دودویی ذخیره می‌شوند. شکل ۱-۲ ساختار درختواره‌ای نمایش انواع اطلاعات را در یک سیستم کامپیوتری نشان می‌دهد. هدف این فصل معرفی مفاهیم دودویی متعدد به صورت یک مرجع برای مطالعات بعدی در فصل‌های آینده است.

^۱ Hardware Description Language (HDL)

^۲ Digital Modules

اعداد مبنا در سیستم‌های دیجیتال ۷



۱-۳-۱ اعداد دودویی

به طور کلی یک عدد با نقطه اعشاری با یکسری ضرایب به صورت زیر نمایش داده می‌شود:

$$a_5a_4a_3a_2a_1a_0.a_{-1}a_{-2}a_{-3}$$

که ضرایب a_x هر یک از ده رقم (0 و 1 و 2 و ... و 9) بوده و x مکان عدد را نشان می‌دهد، و از این رو توان 10 که ضریب در آن ضرب می‌گردد مشخص خواهد شد. این مطلب به صورت زیر بیان می‌شود:

$$10^5a_5+10^4a_4+10^3a_3+10^2a_2+10^1a_1+10^0a_0+10^{-1}a_{-1}+10^{-2}a_{-2}+10^{-3}a_{-3}$$

یک عدد دهدهی مانند 6548 کمیتی معادل با 6 هزartایی، به علاوه 5 صدتایی، به علاوه 4 دهتایی به علاوه 8 واحد را نشان می‌دهد. هزارها، صدها و ... توانی از 10 هستند که با توجه به مکان ضرایب معین می‌گردند. به بیان دقیق‌تر، 6548 را می‌توان به صورت زیر نوشت:

$$6 * 10^3 + 5 * 10^2 + 4 * 10^1 + 8 * 10^0$$

با این وجود معمول این است که فقط ضرایب را بنویسیم و توان‌های لازم 10 را از مکان آنها استنتاج کنیم.

سیستم اعداد دهدهی را در مبنای 10 گویند. زیرا از 10 رقم استفاده می‌کند و ضرایب در توانی از 10 ضرب می‌گردند. سیستم دودویی، یک سیستم اعداد متفاوت است. ضرایب سیستم اعداد دودویی فقط دو مقدار ممکن را دارند: 0 و 1 هر ضریب a_x در 2^x ضرب می‌گردد. مثلاً معادل دهدهی عدد دودویی 11010.11 برابر 26.75 می‌باشد، که از ضرب ضرایب در توان‌هایی از 2 به دست می‌آید:

$$1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 + 1 * 2^{-1} + 1 * 2^{-2} = 26.75$$

به طور کلی، یک عدد در مبنای r به صورت حاصلضرب توانهای r در ضرایب مربوطه‌اش بیان می‌گردد:

$$a_n * r_n + a_{n-1} * r_{n-1} + \dots + a_2 * r_2 + a_1 * r_1 + a_0 + a_{-1} * r_{-1} + a_{-2} * r_{-2} + \dots + r_{-m} a_{-m}.$$

که ضرایب a_x بین 0 تا $r-1$ می‌باشند. برای تفکیک اعداد در مبناهای مختلف، ضرایب را در داخل پرانتزها نوشته و اندیس مبنا را در زیر آن می‌گذاریم (به جز در اعداد دهدهی که محتوا بیانگر دهدهی بودن است).

همانطور که قبلاً اشاره شد، ارقام در یک عدد دودویی بیت خوانده می‌شوند. وقتی که یک بیت برابر 0 است در عمل جمع تبدیل مبنا نقشی ندارد. بنابر این تبدیل دودویی به دهدهی با جمع توانهایی از 2 که ضرایب آن 1 است صورت می‌گیرد. مثلاً عدد زیر را در نظر بگیرید:

$$(101101)_2 = 32 + 8 + 4 + 1 = (45)_{10}$$

در عدد فوق چهار عدد 1 دیده می‌شود. دهدهی مربوطه جمع چهار توان از 2 می‌باشد. 24 اعداد اول (مجموعاً معادل 3 بایت) حاصل از 2 به توان n در جدول

اعداد مبنای در سیستم‌های دیجیتال ۹

شکل ۱-۳ نشان داده شده‌اند. ستون‌ها یک در میان بیانگر یک بایت که معادل ۸ بیت می‌باشند در نظر گرفته شده‌اند تا راحت‌تر بتوان نمایش دودویی را یاد گرفت.

n	2^n	n	2^n	n	2^n
0	1	8	256	16	65536
1	2	9	512	17	131072
2	4	10	1024	18	262144
3	8	11	2048	19	524288
4	16	12	4096	20	1048576
5	32	13	8192	21	2097152
6	64	14	16384	22	4194304
7	128	15	32768	23	8388608

شکل ۱-۳: توانهایی از ۲

اعمال حسابی با اعدادی در مبنای r از همان قواعد دهدهی استفاده می‌کنند. هنگامی که از مبنایی به جز ۱۰ استفاده می‌شود باید دقت کرد که تنها r رقم مجاز به کار گرفته شود. جمع دو عدد دودویی مشابه قوانین دهدهی محاسبه می‌شود، به جز این که ارقام جمع در هر مکان با ارزش فقط می‌تواند ۰ یا ۱ باشد. هر رقم نقلی حاصل در یک مکان مفروض، به وسیله جفت رقم‌های مرتبه بالاتر (بالارزش‌تر) مورد استفاده قرار می‌گیرد. به مثال زیر توجه نمایید:

101101

+100111

1010100

حاصل جمع:

تفریق کمی پیچیده‌تر است. قوانین باز هم همان قوانین دهدهی هستند، به جز این که قرض در یک مکان با ارزش، ۲ را به رقم مفروق‌منه می‌افزاید. (قرض در سیستم دهدهی، ۱۰ واحد به رقم مفروق‌منه اضافه می‌کند).

10110

-10011

000110

باقیمانده:

عمل ضرب خیلی ساده است. ارقام مضروب فیه همیشه 1 یا 0 هستند. بنابراین حاصلضرب های جزئی برابر با 0 یا برابر با مضروب می باشند.

1011	
×101	
1011	
0000	
1011	
10111	حاصل ضرب:

۱-۳-۲ اعداد مبنای هشت و شانزده

تبدیل از مبنای دو به مبنای هشت و شانزده، و بالعکس نقش عمده ای در کامپیوترهای دیجیتال بازی می کند. چون $2^3=8$ و $2^4=16$ است، هر رقم در مبنای هشت متعلق به سه رقم دودویی و هر رقم در مبنای شانزده متعلق به چهار رقم دودویی است.

هنگامی که تعداد ارقام کمتر از 10 باشد مرسوم است که r رقم مورد نیاز برای ضرایب از سیستم دهدهی گرفته شود. هنگامی که مبنای عدد از 10 بزرگتر است از حروف الفبا برای تکمیل 10 رقم دهدهی استفاده می گردد.

در زیر جدولی از شانزده عدد اول در مبنای دهدهی، دودویی، هشت تایی و شانزده تایی نشان داده شده است:

همانطور که در جدول شکل ۱-۴ مشاهده می کنید دو ستون سمت راست مربوط به مبنای هشت و شانزده می باشند که در ادامه به شرح هر یک خواهیم پرداخت. سیستم اعداد هشت هشتی یک سیستم مبنای 8 با هشت رقم 0، 1، 2، 3، 4، 5، 6، 7 می باشد. لازم به ذکر است که ارقام 8 و 9 نمی توانند در یک عدد هشت هشتی ظاهر شوند.

۱۱ اعداد مبنای در سیستم‌های دیجیتال

شانزده تایی (مبنای 16)	هشتایی (مبنای 8)	دودویی (مبنای 2)	دهدهی (مبنای 10)
0	00	0000	00
1	01	0001	10
2	02	0010	02
3	03	0011	03
4	04	0100	04
5	05	0101	05
6	06	0110	06
7	07	0111	07
8	10	1000	08
9	11	1001	09
A	12	1010	10
B	13	1011	11
C	14	1100	12
D	15	1101	13
E	16	1110	14
F	17	1111	15

شکل ۱-۴: اعداد با مبناهای متفاوت

مثالی از یک عدد مبنای هشت عدد 537.4 است. برای تعیین مقدار معادل دهدهی لازم است عدد را به صورت یک سری از توانها با مبنای 8 بسط دهیم.

$$(537.4)_8 = 5 \cdot 8^2 + 3 \cdot 8^1 + 7 \cdot 8^0 + 4 \cdot 8^{-1} = (351.5)_{10}$$

در سیستم اعداد شانزده شانزدهی (مبنای 16)، ده رقم اول از سیستم دهدهی گرفته می‌شوند. حروف A، B، C، D، E و F به ترتیب به جای ارقام 10، 11، 12، 13، 14 و 15 به کار می‌روند. مثالی از یک عدد در مبنای 16 به صورت زیر است.

$$(B65F)_{16} = 11 \cdot 16^3 + 6 \cdot 16^2 + 5 \cdot 16^1 + 15 \cdot 16^0 = (46687)_{10}$$

۳-۳-۱ تبدیل مبنای اعداد

همانطور که قبلاً اشاره شد، تبدیل یک عدد در مبنای r به مبنای ده با بسط عدد به صورت یک سری از توانها و جمع همه جملات انجام می‌شود. اکنون برای تبدیل معکوس یک عدد دهدهی به یک عدد در مبنای r روالی کلی را ارائه می‌کنیم. اگر عدد حاوی نقطه ممیز باشد، لازم است تا عدد به دو بخش صحیح و کسری تفکیک گردد

زیرا هر بخش باید به طور جداگانه تبدیل شود. تبدیل یک عدد صحیح دهدهی به یک عدد در مبنای ۲ با تقسیم عدد و همه خارج قسمت‌های متوالی بر ۲ و جمع‌آوری باقیمانده‌ها انجام می‌گردد.

دودویی به هشت هشتی به سادگی با تفکیک عدد دودویی به گروه‌های سه رقمی در دو طرف نقطه دودویی به دست می‌آید. سپس به هر گروه یک رقم مبنای هشت تعلق می‌گیرد. مثال زیر روال مربوطه را نشان می‌دهد:

$$(10\ 110\ 001\ 101\ 011\ 111\ 1\ 0000\ 0110)_2 = (26153.7406)_8$$

2 6 1 5 3 7 4 0 6

تبدیل از مبنای دو به مبنای شانزده نیز مشابه با روند فوق است، با این تفاوت که عدد دودویی به گروه‌های چهار رقمی تفکیک می‌شوند:

$$(10\ 1101\ 0111\ 1010\ 1111\ 0110)_2 = (2D7A.F6)_{16}$$

2 D 7 A F 6

در ادامه مثال‌های مختلفی ارائه شده است:

مثال ۱: عدد 41 را به دودویی تبدیل کنید.

ابتدا 41 را بر 2 تقسیم می‌کنیم تا خارج قسمت 20 و باقیمانده 1/2 به دست آید خارج قسمت مجدداً به 2 تقسیم می‌گردد تا خارج قسمت و باقیمانده جدیدی به دست آید. این روال تا رسیدن به خارج قسمت 0 ادامه می‌یابد. ضرایب عدد دودویی مورد نظر به طریق زیر از باقیمانده‌ها به دست می‌آید:

ضریب عدد دودویی	باقیمانده	خارج قسمت صحیح
$a_0=1$	$\frac{1}{2}$	20
$a_1=0$	0	10
$a_2=0$	0	5
$a_3=1$	$\frac{1}{2}$	2
$a_4=0$	0	1
$a_5=1$	$\frac{1}{2}$	0

اعداد مبنای در سیستم‌های دیجیتال ۱۳

جواب: $(41)_{10} = (a_5 a_4 a_3 a_2 a_1 a_0)_2 = (101001)_2$

بنابراین، پاسخ $(41)_{10} = (a_5 a_4 a_3 a_2 a_1 a_0)_2 = (101001)_2$ می‌باشد.

روال فوق را می‌توان به طریق ساده‌تر زیر دستکاری کرد:

باقیمانده	خارج قسمت
41	
20	1
10	0
5	0
2	1
1	0
جواب = 101001	

تبدیل اعداد صحیح دهدهی به هر سیستم مبنای r مشابه مثال فوق است به جز این که تقسیم در عوض ۲ بر r انجام می‌گردد.

مثال ۲: عدد $(0.6875)_{10}$ را به دودویی تبدیل کنید.

ابتدا 0.6875 در 2 ضرب می‌شود تا یک عدد صحیح و یک کسر حاصل گردد. کسر دوباره در 2 ضرب می‌شود تا یک عدد صحیح جدید و یک کسر جدید به دست آید. این فرآیند ادامه می‌یابد تا بخش کسری صفر گردد و یا تعداد ارقام دقت مناسبی را ارائه دهند. ضرایب عدد دودویی از اعداد صحیح به صورت زیر به دست می‌آید.

ضرب	کسری	صحیح
$a-1 = 1$	0.3750	$0.6875 * 2 = 1$
$a-2 = 0$	0.7500	$0.3750 * 2 = 0$
$a-3 = 1$	0.5000	$0.7500 * 2 = 1$
$a-4 = 1$	0.0000	$0.5000 * 2 = 1$

بنابراین پاسخ $(0.6875)_{10} = (0. a_1 a_2 a_3 a_4)_2 = (0.1011)_2$ خواهد بود.

برای تبدیل یک عدد کسری از مبنای 10 به یک عدد در مبنای r ، روش مشابهی به کار می‌رود. با این تفاوت که به جای ضرب در 2 ، ضرب در r انجام می‌گردد و ضرایب به جای 0 ، 1 ، از محدوده 0 تا $r-1$ خواهد بود.

مثال ۳: عدد 153 را به مبنای هشت ببرید.

مبنای مورد نظر r برابر 8 است. ابتدا 153 بر 8 تقسیم می‌شود تا خارج قسمت صحیح 19 و باقیمانده 1 حاصل گردد. سپس 19 بر 8 تقسیم می‌شود تا خارج قسمت 2 و باقیمانده 3 را به دست دهد. بالاخره 2 بر 8 تقسیم گردیده تا خارج قسمت 0 و باقیمانده 2 به دست آید. این روند به صورت مناسب زیر انجام می‌گردد:

153	
	باقیمانده
خارج قسمت	
19	1
2	3
	جواب = 231

در تبدیل قسمت کسری مبنای ده به دودویی از روش مشابه با بخش صحیح استفاده می‌شود. با این وجود به جای تقسیم از ضرب و به جای باقیمانده‌ها، بخش‌های صحیح انتخاب می‌گردند. مجدداً بهتر است این روش با مثالی تشریح شود.

مثال ۴: عدد $(0.513)_{10}$ را به مبنای هشت ببرید.

$$0.513 * 8 = 4.104$$

$$0.104 * 8 = 0.832$$

$$0.832 * 8 = 6.656$$

$$0.656 * 8 = 5.248$$

$$0.248 * 8 = 1.984$$

$$0.984 * 8 = 7.872$$

جواب تا هفت رقم با معنی که از بخش صحیح حاصل ضرب‌ها به دست می‌آید برابر

است با

$$(0.513)_{10} = (0.406517...)_{8}$$

اعداد مبنای دیجیتال ۱۵

تبدیل اعداد دهدهی که دارای هر دو بخش صحیح و کسری هستند با تبدیل جداگانه دو بخش و ترکیب جوابها صورت میگیرد. با استفاده از مثالهای ۱ و ۲ داریم:

$$(41.6875)_{10} = (101001.1011)_2$$

با استفاده از مثالهای ۳ و ۴ داریم:

$$(153.513)_{10} = (231.406517)_8$$

تبدیل اعداد از مبنای هشت یا شانزده به مبنای دودویی با روشی عکس روش بالا انجام می‌گردد که این رو به شرح زیر می‌باشد:

هر رقم مبنای هشت با سه رقم مبنای دو معادل خود جایگزین می‌شود. به طور مشابه، هر رقم مبنای شانزده با چهار رقم دودویی معادلش جایگزین خواهد شد. این مطلب در مثالهای زیر تشریح شده است:

$$(673.124)_8 = (110\ 111\ 011.001\ 010\ 100)_2$$

6 7 3 1 2 4

و

$$(306.D)_{16} = (0011\ 0000\ 0110.1101)_2$$

3 0 6 D

اساساً کار با اعداد دودویی، به دلیل اینکه تعداد ارقامشان سه یا چهار برابر معادلشان در مبنای ده می‌باشد، مشکل است. مثلاً عدد دودویی 1111 1111 1111 معادل 4095 است. با این وجود کامپیوترهای دیجیتال اعداد دودویی را به کار می‌برند و گاهی نیز لازم است تا کاربر مستقیماً به وسیله اعداد دودویی با ماشین ارتباط برقرار کند. یک راه برای حفظ سیستم دودویی در کامپیوتر، که در ضمن تعداد ارقام را برای انسان کاهش می‌دهد، استفاده از رابطه بین سیستم اعداد دودویی و هشت هشتی یا شانزده شانزدهی است. با این روش، انسان بر حسب اعداد مبنای هشت یا شانزده فکر کرده و

در مواقعی که ارتباط مستقیم با ماشین لازم است، تبدیل لازمه را با بررسی این اعداد انجام خواهد داد. به این ترتیب عدد دودویی 1111 1111 1111 که دارای 12 رقم است در مبنای هشت به صورت چهار رقم 7777 و یا در مبنای شانزده به شکل FFF در می‌آید. به هنگام تبادل اطلاعات با انسان، نمایش مبنای هشت یا شانزده اعداد دودویی مطلوب تر است زیرا که در این مبناها اعداد با $1/3$ یا $1/4$ تعداد ارقامشان در دودویی قابل نمایش‌اند. بنابراین اغلب کتابچه‌های راهنمای کامپیوتر از اعداد مبنای هشت یا شانزده برای نمایش کمیت‌های دودویی استفاده می‌کنند. گرچه نمایش مبنای شانزده مناسب تر به نظر می‌رسد ولی انتخاب یکی از این دو کاملاً اختیاری است.

۱-۴-۱ متمم اعداد

متمم‌ها در کامپیوترهای دیجیتال برای ساده کردن عمل تفریق و یا عملیات منطقی به کار می‌روند. در هر مبنایی چون r ، دو نوع متمم وجود دارد:

متمم مبنا

متمم مبنای کاهش یافته

فرم اول به متمم r و دومی به متمم $r-1$ موسوم است. وقتی که مقدار مبنا یا پایه را جایگزین کنیم، برای اعداد دودویی، متمم‌های 2 و 1 و برای اعداد دهدهی، متمم‌های 10, 9 را خواهیم داشت.

۱-۴-۱-۱ متمم مبنا

متمم r یک عدد n رقمی مانند N در مبنای r به صورت $r^n - N$ به ازاء $N \neq 0$ و برابر با 0 در ازاء $N = 0$ تعریف می‌شود. از مقایسه این متمم با متمم $(r-1)$ نتیجه می‌شود که متمم r از جمع 1 با متمم $(r-1)$ حاصل می‌شود. زیرا

$$r^n - N = [(r^n - 1) - N] + 1$$

اعداد مبنای دیجیتال ۱۷

می‌باشد. به این ترتیب متمم ۱۰ یک عدد دهدهی مانند 2389 برابر است با $7611 = 7610 + 1$ که از جمع 1 به مقدار متمم 9 حاصل می‌گردد. متمم 2 عدد دودویی 101100 برابر است با $010100 = 010011 + 1$ و از جمع 1 با مقدار متمم 1 به دست می‌آید. چون 10^n عدد است که با یک 1 و n عدد 0 به دنبال آن نمایش داده می‌شود، $10^n - N$ که متمم 10 عدد N است نیز با تغییر ندادن 0های کم ارزش‌تر و تفریق اولین رقم غیر صفر کم ارزش‌تر از 10 و تفریق همه رقم‌های با ارزش‌تر از 9 حاصل می‌گردد. برای نمونه

متمم 10 عدد 012398 برابر 987602 می‌باشد.

متمم 10 عدد 246700 برابر 753300 است.

متمم 10 اولین عدد با تفریق 8 از 10 در کم‌ارزش‌ترین مکان و تفریق دیگر ارقام از 9 حاصل شده است.

متمم 10 دومین عدد بدین ترتیب حاصل گشته است که دو 0 کم‌ارزش‌تر رها می‌شوند، 7 از 10 و دیگر ارقام از 9 تفریق می‌گردند.

به طور مشابه متمم عدد دو می‌تواند با رها کردن همه 0های کم‌ارزش‌تر و نیز تغییر نکردن اولین 1 و جایگزینی همه 0ها با 1ها و 1ها با 0ها در دیگر ارقام با ارزش‌تر حاصل می‌شود. به طور نمونه

متمم 2 عدد 1101100 برابر 0010100 است

متمم 2 عدد 0110111 برابر 1001001 است

متمم 2 اولین عدد با رها کردن دو 0 کم‌ارزش‌تر و اولین 1 و سپس جایگزینی همه 1ها با 0 و 0ها با 1 در چهار رقم با ارزش‌تر باقی مانده به دست می‌آید. متمم 2 دومین عدد با رها کردن اولین 1 و متمم کردن دیگر ارقام حاصل می‌گردد.

در تعاریف قبلی، فرض شد که اعداد دارای نقطه ممیز نیستند. اگر عدد اولیه N حاوی ممیز باشد آن را موقتاً حذف نمود تا متمم r و $(r-1)$ به دست آید. آنگاه آن را به مکان مربوطه اش باز می گردانیم.

۱-۴-۲ متمم در مبنای کاهش یافته

با فرض داشتن عددی n رقمی مانند N در مبنای r ، متمم $(r-1)$ عدد به صورت $N - (r^n - 1)$ تعریف می شود. برای اعداد دهدهی، $r = 10$ و $r-1 = 9$ است، و به این ترتیب متمم ۹ عدد N برابر $N - (10^n - 1)$ خواهد بود. در اینجا 10^n نمایشگر عددی است که متشکل از ۱ و به دنبال آن n عدد ۰ می باشد.

$10^n - 1$ عددی است که با n عدد ۹ نشان داده می شود.

مثلاً اگر $n = 4$ باشد، داریم $10^4 = 10000$ و $10^4 - 1 = 9999$. به این ترتیب نتیجه می شود که متمم ۹ یک عدد دهدهی با تفریق هر رقم از ۹ حاصل خواهد شد. به چند مثال عددی زیر توجه کنید.

متمم ۹ عدد 546700 برابر است با $546700 - 999999 = 453299$

متمم ۹ عدد 012398 برابر است با $012398 - 999999 = 987601$

برای اعداد دودویی، $r=2$ و $r-1=1$ است، بدین ترتیب متمم ۱ عدد N ،

$N - (2^n - 1)$ خواهد بود. مجدداً، 2^n برابر با یک عدد دودویی است که از یک ۱،

و n عدد ۰ تشکیل شده است. $2^n - 1$ یک عدد دودویی متشکل از n عدد ۱ می باشد.

مثلاً اگر $n = 4$ باشد، داریم $2^4 = (10000)_2$ و $2^4 - 1 = (1111)_2$. بنابراین متمم ۱ یک عدد دودویی از تفریق هر رقم از ۱ به دست می آید. با این وجود، هنگام تفریق ارقام دودویی از عدد ۱، یکی از دو حالت $1-1=0$ و یا $1-0=1$ را خواهیم داشت، که سبب

اعداد مبنای دیجیتال ۱۹

می‌شود هر بیت از 0 به 1 و از 1 به 0 تبدیل شود. بنابراین متمم یک عدد دودویی با تغییر 1ها به 0 و 0ها به 1 حاصل می‌گردد. در زیر مثالهایی آورده شده است:

متمم 1 عدد 1011000 برابر است با 0100111

متمم 1 عدد 0101101 برابر است با 1010010

متمم $(r-1)$ اعداد مبنای هشت و شانزده به ترتیب از تفریق ارقام آنها از 7 یا F (15 دهمی) حاصل می‌شود.

توجه: توجه داشته باشید که متمم یک متمم، عدد را به حالت اولیه‌اش باز می‌گرداند. متمم r عدد N برابر $r^n - N$ است. متمم یک متمم برابر است با

$$r^n - (r^n - N) = N$$

که همان عدد اولیه است.

۱-۴-۳ تفریق به کمک متمم‌ها

روش مستقیم تفریق که در مدارس ابتدایی بیان شد از مفهوم قرض کردن استفاده می‌نماید. در این روش وقتی که یک رقم در مفروق‌منه کوچکتر از مفروق باشد یک 1 از رقم با ارزش‌تر قرض گرفته می‌شود. این روش هنگامی که تفریق با قلم و کاغذ انجام شود به خوبی کار می‌کند. با این وجود، هنگام پیاده‌سازی تفریق با سخت‌افزار دیجیتال، روش کمتر از روش‌های متمم کارایی دارد. تفریق دو عدد n رقمی بی‌علامت $M - N$ در مبنای r به صورت زیر انجام می‌شود:

مفروق‌منه M را به متمم r مفروض N ، اضافه کنید. یعنی

$$M + (r^n - N) = M - N + r^n$$

اگر $M \geq N$ باشد، عمل جمع یک رقم نقلی انتهای تولید می‌کند که باید چشم پوشی شود؛ آنچه باقی می‌ماند نتیجه $M - N$ است.

اگر $M < N$ باشد، عمل جمع هیچگونه رقم نقلی انتهایی تولید نمی‌کند و جواب برابر با $(M-N) \cdot r^n$ است که همان متمم r عمل $(M-N)$ می‌باشد. برای یافتن جواب معمول، متمم r حاصل جمع را به دست می‌آوریم و سپس یک علامت منفی در جلو آن می‌گذاریم.

مثال‌های زیر روال را تشریح می‌کنند.

مثال ۵: با استفاده از متمم 10 تفریق $72532 - 3250$ را انجام دهید.

$$\begin{array}{rcl} M & = & 72532 \\ N \text{ عدد } 10 \text{ متمم} & = & +96750 \\ \text{حاصل جمع} & = & 169383 \\ \text{چشم پوشی از رقم نقلی انتهایی } 10^5 & = & -100000 \\ \text{جواب} & = & 69282 \end{array}$$

توجه کنید که M دارای 5 رقم ولی N فقط 4 رقمی است. چون هر دو عدد باید دارای تعداد ارقام برابری باشند، پس N به صورت 03250 نوشته می‌شود. متمم 10 این عدد N ، یک 9 در با ارزش‌ترین مکان تولید می‌نماید. تولید رقم نقلی در با ارزش‌ترین مکان دلالت بر $M \geq N$ دارد و نتیجه نیز مثبت است.

مثال ۶: با استفاده از متمم 10 تفریق $72532 - 3250$ را به دست آورید.

$$\begin{array}{rcl} M & = & 03250 \\ N \text{ عدد } 10 \text{ متمم} & = & +27468 \\ \text{حاصل جمع} & = & 30718 \end{array}$$

که رقم نقلی در آن وجود ندارد. بنابراین

$$\text{جواب (متمم 10 عدد 30718)} = -69282$$

توجه کنید که چون $3250 < 72532$ است نتیجه منفی است. نظر به این که ما با اعداد بی علامت سر و کار داریم، نمی‌توان برای این حالت نتیجه بدون علامتی به دست آورد. وقتی تفریق را با متمم‌ها انجام می‌دهیم، جواب منفی از نبود رقم نقلی انتهایی و نتیجه متمم تشخیص داده می‌شود. هنگام کار با کاغذ و قلم، می‌توانیم جواب را به یک

۲۱ اعداد مبنای در سیستم‌های دیجیتال

عدد منفی علامت‌دار تغییر دهیم تا به فرم معمول‌تر درآید. تفریق با متمم‌ها برای اعداد دودویی به روش مشابهی در مثال‌های زیر آمده است.

مثال ۷: با فرض دو عدد دودویی $X = 1010100$ و $Y = 1000011$ ، تفریق‌های زیر را انجام دهید.

(الف) $X - Y$ و (ب) $Y - X$ با استفاده از متمم 2

$$\begin{array}{rcl}
 X = & 1010100 & \text{(الف)} \\
 Y \text{ عدد 2 متمم} = & +0111101 & \\
 \text{حاصل جمع} = & 10010001 & \\
 \text{چشم پوشی از نقلی انتهای } 2^7 = & -10000000 & \\
 X - Y = \text{جواب} & 0010001 &
 \end{array}$$

(ب)

$$\begin{array}{rcl}
 Y = & 1000011 & \\
 X \text{ عدد 2 متمم} = & +0101100 & \\
 \text{حاصل جمع} = & 1101111 &
 \end{array}$$

رقم نقلی انتهای وجود ندارد. بنابراین، جواب

$$Y - X = (\text{متمم 2 عدد } 1101111) - 0010001 =$$

تفریق اعداد بی علامت را می‌توان با متمم $(r-1)$ نیز انجام داد. به خاطر دارید که متمم $(r-1)$ ، یک واحد کمتر از متمم r است. به این علت، نتیجه جمع مفروق منه با متمم مفروق حاصل جمعی تولید می‌کند که یکی کمتر از تفاضل صحیح به هنگام رخداد نقلی انتهای است. حذف نقلی انتهای و افزودن 1 به حاصل جمع را رقم نقلی چرخشی می‌خوانند.

مثال ۸: مثال ۷ را با استفاده از متمم ۱ انجام دهید.

$$X - Y = 1010100 - 1000011 \text{ (الف)}$$

$$\begin{array}{rcl} X & = & 1010100 \\ Y & = & +0111100 \\ \text{حاصل جمع} & = & 1001000 \\ \text{رقم نقلی انتهایی} & = & +1 \\ \text{جواب } X - Y & = & 0010001 \end{array}$$

$$Y - X = 1000011 - 1010100 \text{ (ب)}$$

$$\begin{array}{rcl} Y & = & 1000011 \\ X & = & +0101011 \\ \text{حاصل جمع} & = & 1101110 \end{array}$$

که رقم نقلی انتهایی وجود ندارد.

$$\text{بنابراین جواب } Y - X = (\text{متمم ۱ عدد } 1101110) = -0010001 \text{ است.}$$

توجه کنید که نتیجه منفی با اخذ متمم ۱ از حاصل جمع به دست آمده است زیرا این نوع متمم به کار رفت. روال رقم نقلی انتهایی چرخشی در تفریق اعداد ددهی بی علامت با متمم ۹ نیز قابل استفاده است.

۱-۴-۴ اعداد دودویی علامت دار

اعداد صحیح مثبت و از آن جمله صفر را می توان با اعداد بی علامت نشان داد. با این وجود برای نمایش اعداد صحیح منفی به علامتی نیاز نداریم. در حساب معمولی، یک عدد منفی را با یک علامت منها و عدد مثبت را با علامت بعلاوه نشان می دهند. به دلیل محدودیت در سخت افزار، کامپیوترها باید هر چیزی را با ارقام دودویی نشان دهند. مرسوم است که علامت را با یک بیت واقع در سمت چپ ترین مکان عدد نمایش دهند. معمولاً اعداد مثبت را با گذاشتن ۰ و اعداد منفی را با گذاشتن ۱ در محل بیت مزبور معرفی می نمایند.

لازم است بدانیم که هر دو گروه اعداد دودویی علامت‌دار و بی علامت هنگام ارائه به کامپیوتر از رشته بیت‌ها تشکیل شده‌اند. معمولاً کاربر علامت‌دار بودن یا نبودن عدد را معین می‌نماید. اگر عدد دودویی علامت‌دار باشد سمت چپ‌ترین بیت، علامت، و بقیه بیت‌ها علامت هستند. اگر عدد دودویی بدون علامت فرض شود، سمت چپ‌ترین بیت، با ارزش‌ترین بیت عدد خواهد بود. مثلاً رشته بیت‌های 01001 می‌تواند به عنوان 9 (دودویی بی علامت) و یا +9 (دودویی علامت‌دار) در نظر گرفته شود زیرا سمت چپ‌ترین بیت 0 است. رشته بیت‌های 11001 به عنوان 25 بی علامت و یا 9 علامت‌دار منفی خواهد بود زیرا در سمت چپ‌ترین مکان عدد، رقم 1 وجود دارد که بیانگر منفی بودن عدد، و بقیه چهار بیت عدد 9 را نشان می‌دهد. معمولاً اگر نوع عدد از قبل مشخص باشد هیچگونه اشتباهی در تشخیص وجود نخواهد داشت.

نمایش اعداد علامت‌دار در آخرین مثال فوق، نمایش مقدار علامت‌دار منفی نامیده می‌شود. در این نامگذاری، عدد شامل مقدار و یک سمبل (+ یا -) یا یک بیت (0 یا 1) برای مشخص نمودن علامت است. این روش در محاسبات معمولی مورد استفاده می‌باشد. وقتی که عملیات حسابی در یک کامپیوتر پیاده‌سازی می‌شوند، بهتر است روش دیگری به نام سیستم متمم علامت‌دار منفی برای ارائه اعداد منفی به کار گرفته می‌شود. در این سیستم، یک عدد منفی با متمم‌اش مشخص می‌شود. در حالی که سیستم مقدار علامت‌دار منفی، عدد را با تغییر علامتش منفی می‌کند، سیستم متمم علامت‌دار منفی، منفی عدد را با متمم سازی‌اش تهیه می‌نماید. چون اعداد مثبت همواره با 0 در سمت چپ‌شان شروع می‌شوند متمم همواره با 1 آغاز می‌گردد، که بیانگر عدد منفی خواهد بود. سیستم متمم علامت‌دار منفی می‌تواند از متمم 1 یا متمم 2 استفاده کند ولی متمم 2 رایج‌تر است. به عنوان مثال فرض کنید که عدد 9 با هشت بیت در دودویی نشان داده شده باشد. +9 با یک بیت 0 در سمت چپ‌ترین مکان و به دنبال آن معادل دودویی 9 می‌آید که نتیجه 00001001 خواهد بود. توجه داشته باشید که تمام هشت بیت باید مقدار داشته باشند، بنابراین پس از بیت علامت بقیه مکان‌ها تا

اولین 1 از سمت چپ با 0 پر می‌شوند. هر چند که برای نمایش +9 فقط یک راه وجود دارد، برای نمایش 9- سه روش موجود است.

- نمایش مقدار علامت‌دار منفی 10001001
- نمایش متمم 1 علامت منفی دار 11110110
- نمایش متمم 2 علامت منفی دار 11110111

در سیستم مقدار علامت‌دار منفی، با تغییر بیت علامت در سمت چپ‌ترین مکان، از 0 به 1، عدد 9- به 9+ تبدیل می‌شود. در متمم 1 علامت منفی دار، 9- را با متمم کردن همه بیت‌های 9+، از جمله بیت علامت به دست می‌آوریم. در متمم 2 علامت منفی دار، 9- با متمم کردن تمام بیت‌های عدد مثبت از جمله بیت علامت حاصل می‌شود.

شکل ۱-۵، همه اعداد دودویی 4 بیت را به هر سه فرم نمایش، نشان می‌دهد. عدد دهدهی معادل نیز به منظور وجود مرجع آورده شده است. توجه کنید که اعداد مثبت در هر سه نمایش یکسانند و دارای یک 0 در سمت چپ‌ترین مکان می‌باشند.

مقدار علامت منفی دار	متمم 1 علامت منفی دار	متمم 2 علامت منفی دار	دهدهی
0111	0111	0111	+7
0110	0110	0110	+6
0101	0101	0101	+5
0100	0100	0100	+4
0011	0011	0011	+3
0010	0010	0010	+2
0001	0001	0001	+1
0000	0000	0000	+0
1000	1111	-	-0
1001	1110	1111	-1
1010	1101	1110	-2
1011	1100	1101	-3
1100	1011	1100	-4
1101	1010	1011	-5
1110	1001	1010	-6
1111	1000	1001	-7
-	-	1000	-8

شکل ۱-۵: اعداد دودویی علامت‌دار

اعداد مبنای در سیستم‌های دیجیتال ۲۵

سیستم متمم 2 علامت منفی دار تنها یک نمایش برای 0 دارد که همیشه مثبت است. دو سیستم دیگر دارای 0 مثبت و 0 منفی‌اند، چیزی که در محاسبات معمولی با آن مواجه نمی‌شویم. مجدداً توجه کنید که همه اعداد منفی دارای 1 در سمت چپ‌ترین بیت‌اند. به این ترتیب ما آنها را از اعداد مثبت تفکیک می‌نماییم. با چهار بیت قادریم 16 عدد دودویی را نشان دهیم.

در سیستم مقدار علامت‌دار منفی و متمم 1، هشت عدد مثبت و هشت عدد منفی و از جمله دو عدد صفر وجود دارد. در نمایش متمم 2، هشت عدد مثبت از جمله صفر و هشت عدد منفی موجود است. سیستم مقدار علامت‌دار منفی در حساب معمولی مورد استفاده است و هنگامی که در کامپیوتر به کار رود، مشکلاتی به همراه دارد زیرا باید علامت و مقدار به طور جداگانه دستکاری شوند. بنابراین، معمولاً متمم علامت منفی به کار گرفته می‌شود. متمم 1 نیز مشکلاتی را به بار می‌آورد و به ندرت در محاسبات به کار می‌رود. متمم 1 برای اعمال منطقی مفید است چون تبدیل 0 به 1 و 1 به 0 معادل با عمل متمم منطقی است.

۱-۵ جمع حسابی

جمع دو عدد در سیستم مقدار علامت منفی از قوانین معمول در حساب تبعیت می‌نماید. اگر علامت‌ها یکسان باشند دو مقدار را با هم جمع کرده و به حاصل جمع علامت مشترک را تخصیص می‌دهیم. اگر علامت‌ها یکی نباشند، مقدار کوچکتر را از بزرگتر کم می‌کنیم و به نتیجه حاصل علامت عدد بزرگتر را اختصاص می‌دهیم. مثلاً:

$$-25 = -(39-14) = (-39) + (+14)$$
 است، که با تفریق مقدار کوچکتر 14 از مقدار بزرگتر 39 و استفاده از علامت 39 برای علامت نتیجه انجام شده است. این فرایند به مقایسه علامت‌ها و اندازه‌ها و سپس اجرای جمع و تفریق نیاز دارد. روال مشابهی در نمایش مقدار علامت منفی برای اعداد دودویی قابل اعمال است. بر عکس قوانین جمع

اعداد در سیستم مقدار علامت منفی نیازی به مقایسه و تفریق ندارد، بلکه فقط باید آنها را جمع کرد. روال بسیار ساده بوده و برای اعداد دودویی به صورت زیر بیان می‌گردد:

+6	00000110	-6	11111010
+13	00001101	+13	00001101
+19	00010011	+7	00000111
+6	00000110	- 6	11111010
- 13	11110011	- 13	11110011
- 7	11111001	- 19	11101101

جمع دو عدد دودویی علامت‌دار با اعداد منفی که به فرم متمم 2 نمایش داده شده‌اند از جمع دو عدد از جمله بیت‌های علامت حاصل می‌شود. رقم نقلی حاصل از بیت علامت چشم‌پوشی می‌گردد. مثال‌های عددی برای جمع در زیر آمده است.

توجه کنید که اعداد منفی باید از ابتدا به صورت متمم 2 باشد و حاصل جمع اگر منفی باشد به صورت متمم 2 خواهد بود.

در هر یک از چهار حالت فوق، عمل انجام شده جمعی است که در آن بیت علامت هم لحاظ شده است. در این روش هر رقم نقلی خروجی نادیده گرفته می‌شود و نتایج منفی به فرم متمم عدد دو هستند.

برای یافتن یک جواب صحیح، باید مطمئن بود که برای جای دادن نتیجه، تعداد کافی بیت وجود دارد. اگر با دو عدد n بیت شروع کنیم و حاصل جمع $n+1$ بیت را اشغال کند گوییم سرریز رخ داده است. هنگامی که جمع با کاغذ و قلم انجام می‌شود سرریز مسئله‌ای نیست زیرا ما از نظر عرض صفحه محدودیت نداریم. در این‌گونه موارد فقط یک 0 به بالاترین مکان عدد مثبت و یا یک 1 به بالاترین مکان عدد منفی می‌افزاییم تا آنها را به $n+1$ بیت گسترش دهیم و سپس جمع را اجرا نماییم. ولی سرریز در کامپیوتر مشکل‌ساز است زیرا تعداد بیت‌هایی که عدد را نگه می‌دارند

محدود می‌باشد، و نتیجه‌ای که از مقدار نهایی به میزان 1 واحد تجاوز کند را نمی‌توان در آن جای داد.

فرم متمم اعداد منفی برای کسانی که به سیستم مقدار علامت منفی عادت کرده‌اند، ناآشنا است. برای تعیین یک عدد منفی وقتی که به فرم متمم 2 علامت‌دار باشد، لازم است که آن را به یک عدد مثبت تبدیل کنیم تا به شکل آشناتری درآید. مثلاً عدد دودویی علامت‌دار 11111001 یک عدد منفی است زیرا سمت چپ‌ترین بیت برابر 1 است. متمم 2 آن 00000111 می‌باشد که معادل دودویی عدد +7 است. به این ترتیب تشخیص می‌دهیم که عدد منفی اولیه برابر 7- بوده است.

۱-۶ تفریق حسابی

تفریق دو عدد دودویی علامت‌دار، وقتی که اعداد منفی به صورت متمم 2 باشند بسیار ساده است و می‌تواند به صورت زیر بیان شود:

۱. متمم 2 مفروق (از جمله بیت علامت) را به دست آورید.

۲. آنرا به مفروق منہ (از جمله بیت علامت) اضافه کنید.

۳. رقم نقلی خروجی از مکان بیت علامت چشم پوشی شود.

این پدیده به این علت رخ می‌دهد که اگر علامت مفروق عوض شود، تفریق به جمع تبدیل خواهد شد. این نکته با روابط زیر نشان داده شده است:

$$(\pm A) - (+B) = (\pm A) + (-B);$$

$$(\pm A) - (-B) = (\pm A) + (+B).$$

اما تغییر یک عدد مثبت به یک عدد منفی به سادگی با به دست آوردن متمم 2 آن امکان‌پذیر است. عکس مطلب فوق نیز صحیح می‌باشد زیرا متمم یک عدد منفی متمم، یک عدد مثبت معادل تولید می‌نماید. تفریق $7 + (-13) = (-6)$ را در نظر بگیرید. در دودویی با هشت بیت، این تفریق به صورت $(11110011 - 11111010)$ است. با یافتن

متمم 2 مفروق (13-)، یعنی (13+)، تفریق به فرم جمع در می آید. در دودویی این عمل به صورت زیر می باشد:

$$11111010 + 00001101 = 100000111$$

با حذف رقم نقلی انتهایی پاسخ صحیح 00000111 (7 +) خواهد بود. لازم به تذکر است که جمع و تفریق اعداد دودویی در سیستم متمم علامت منفی مشابه با قوانین جمع و تفریق معمولی است. بنابراین کامپیوترها دارای سخت افزار مشترک برای هر دو نوع عمل حسابی می باشند. کاربر یا برنامه نویس باید نتایج چنین جمع یا تفریقی را به طور متفاوت تفسیر کند و این تفسیر به فرض اولیه وی یعنی علامت دار بودن یا بی علامت بودن اعداد بستگی دارد.

۱-۷ کدهای دودویی

سیستم های دیجیتال از سیگنال هایی که دو مقدار مجزا و عناصری از مدار که دو حالت باثبات دارند استفاده می کنند. بین سیگنال های دودویی، عناصر مدار دودویی و ارقام دودویی رابطه مستقیمی وجود دارد. مثلاً یک عدد دودویی n رقمی را می توان با n عنصر مدار دودویی که هر یک دارای یک سیگنال خروجی معادل 0 یا 1 اند، نشان داد.

سیستم های دیجیتال نه تنها اعداد دودویی بلکه بسیاری از اجزا گسسته اطلاعات را هم دستکاری و نمایش می دهند و روی آنها عمل می کنند. هر عنصر گسسته اطلاعاتی را در میان یک گروه از مقادیر می توان با استفاده از کد دودویی نشان داد. کدها باید به صورت دودویی باشند زیرا کامپیوترها فقط قادرند 0 ها و 1 ها را نگه دارند. باید توجه داشت که کدها فقط نماد یا سمبل نمایش اطلاعات را عوض می کنند و نه مفهوم آنها را. اگر بیت های یک کامپیوتر را به طور تصادفی مورد بررسی قرار دهیم، ملاحظه خواهیم کرد که اغلب به جای اعداد دودویی، اطلاعات کد شده در آنها وجود دارد.

اعداد مبنای دیجیتال ۲۹

یک کد دودویی n بیتی، گروهی متشکل از n بیت است که 2^n ترکیب ممکن از ۱ها و ۰ها را داراست، و هر ترکیب یک عنصر از مجموعه کد شده را نمایش می‌دهد. یک مجموعه چهار عنصری با دو بیت کد می‌شود که به هر عنصر یکی از ترکیبات بیتی زیر تخصیص می‌یابد:

11, 10, 01, 00

یک مجموعه هشت عضوی به کد 3 بیت نیاز دارد و برای یک مجموعه 16 عنصری یک کد 4 بیت لازم است. ترکیب بیتی یک کد n بیتی با شمارش دودویی از 0 تا $2^n - 1$ حاصل می‌گردد. به هر عنصر باید یک ترکیب بیتی دودویی منحصر بفرد اختصاص یابد و هیچ دو عنصر دارای مقدار یکسانی نمی‌باشند؛ در غیر این صورت تخصیص کد گنگ و بی معنی خواهد بود.

گرچه حداقل تعداد بیت‌های لازم برای 2^n مجزا، برابر n است، حداکثر تعداد بیت‌ها برای تعریف یک کد دودویی وجود ندارد. مثلاً 10 رقم دهدهی با 10 بیت قابل کد شدن است، و هر رقم دهدهی به یکی از ترکیبات نه 0 و یک 1 تخصیص می‌یابد. در این کد دهدهی، رقم 6 به ترکیب بیتی 000100000 اختصاص می‌یابد.

۱-۷-۱ کد BCD

گرچه سیستم اعداد دودویی طبیعی ترین سیستم برای یک کامپیوتر است، ولی بسیاری از مردم به سیستم دهدهی عادت دارند. یکی از راههای حل این مشکل تبدیل اعداد دهدهی به دودویی، اجرای همه محاسبات به دودویی و سپس تبدیل نتایج دودویی به دهدهی است. این روش لازم می‌دارد تا اعداد دهدهی را در کامپیوتر ذخیره کنیم تا بتوانند به دودویی تبدیل شوند. چون کامپیوتر فقط می‌تواند مقادیر دودویی را قبول کند، باید ارقام دهدهی را با کدی مرکب از ۱ها و ۰ها نشان دهیم. هنگامی که این ارقام به فرم کد شده در کامپیوتر ذخیره شوند، می‌توان مستقیماً عملیات حسابی را روی این اعداد دهدهی اجرا نمود.

اگر تعداد عناصر در مجموعه به صورت توانی از 2 نباشد، کد دودویی دارای ترکیبات بیتی تخصیص نیافته خواهد بود. 10 رقم دهدهی چنین مجموعه‌ای را می‌سازند. یک کد دودویی که بتواند 10 عنصر را از هم تفکیک کند باید حداقل 4 بیت داشته باشد، ولی 6 ترکیب از 16 ترکیب ممکن تخصیص نیافته باقی می‌ماند. با مقدار دهی 10 گانه به 4 بیت می‌توان کدهای دودویی متفاوتی به دست آورد. کدی که برای ارقام دهدهی معمولاً به کار می‌رود در شکل ۱-۶ نشان داده می‌شود. این کد را دهدهی کد شده به دودویی می‌خوانند و به طور خلاصه آنرا با BCD نمایش می‌دهند. چند کد دهدهی دیگر بعداً در این بخش نمایش داده خواهند شد.

همان گونه که در شکل بالا مشاهده می‌شود، هر کد 4 بیتی به یک رقم دهدهی نسبت داده می‌شود. در این بین توجه به چند نکته الزامی است:

رقم BCD	سمبل دهدهی
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9

شکل ۱ - ۶: دهدهی کد شده به دودویی (BCD)

یک عدد k رقمی در BCD به $4k$ بیت نیاز دارد.

هرگاه عدد دهدهی در BCD بین 0 تا 9 باشد با عدد دودویی‌اش معادل است.

یک عدد BCD بزرگتر از 10 با عدد دودویی معادلش، هر چند که از 0ها و 1ها تشکیل شده، متفاوت است. ترکیبات دودویی 1010 تا 1111 در BCD مفهومی ندارند.

اعداد مبنای دیجیتال ۳۱

عدد دهدهی 396 در BCD با 12 بیت به صورت 0011 1001 0110 نمایش داده می‌شود، که در آن هر گروه 4 بیتی یک رقم دهدهی را نشان می‌دهد. بعلاوه عدد دهدهی 185 و مقدار مربوطه آن را در BCD و دودویی ملاحظه کنید:

$$(185)_{10} = (0001\ 1000\ 0101)_{BCD} = (10111001)_2$$

مقدار BCD دارای 12 بیت است، ولی عدد دودویی معادل آنها تنها 8 بیت لازم دارد. واضح است یک عدد BCD نسبت به مقدار دودویی به بیت‌های بیشتری احتیاج دارد. با این وجود، استفاده از اعداد دهدهی دارای مزیت است زیرا داده‌های ورودی و خروجی به وسیله انسانهایی که سیستم‌های دهدهی را به کار می‌برند تولید می‌شود.

توجه داشته باشید که اعداد BCD اعداد دهدهی هستند و نه اعداد دودویی، هرچند که آنها در ساختارشان از بیت‌ها استفاده می‌کنند. تنها تفاوت بین یک عدد دهدهی و BCD این است که اعداد دهدهی سِمبل‌های 0، 1، 2، ...، 9 و اعداد BCD سِمبل‌های 0000، 0001، 0010، ...، 1001 را به کار می‌برند. مقدار دهدهی دقیقاً یکی است. عدد 10 دهدهی در BCD با هشت بیت 0001 0000 و عدد 15 با 0001 0101 نمایش داده می‌شوند. مقادیر دودویی معادل آنها به ترتیب 1010 و 1111 است که تنها چهار بیت دارند.

۱-۷-۲ جمع BCD

جمع دو رقم دهدهی در BCD، همراه با رقم نقلی احتمالی از جفت رقم کم ارزش‌تر قبلی را در نظر بگیرید. چون هر رقم از 9 تجاوز نمی‌کند، جمع نمی‌تواند بزرگتر از $9+9+1=19$ باشد که در آن 1، رقم نقلی قبلی است. فرض کنید می‌خواهیم ارقام BCD را به شکل اعداد دودویی با هم جمع کنیم. جمع دودویی، نتیجه‌ای بین 0 تا 19 را تولید خواهد کرد. این مقادیر به دودویی برابرند با 0000 تا 10011، ولی به فرم BCD برابر با 0000 تا 11001 می‌باشند. اولین رقم، رقم نقلی و چهار بیت بعدی رقم جمع BCD است. وقتی حاصل جمع دودویی برابر یا کمتر از 1001 است (بدون نقلی)، رقم

BCD مربوطه صحیح است. با این وجود، وقتی جمع دودویی بزرگتر یا مساوی 1010 باشد، نتیجه یک رقم BCD نامعتبر است. جمع $6 = (0110)_2$ با حاصل جمع دودویی، آن را به رقم صحیح بدل کرده و در صورت لزوم رقم نقلی نیز تولید خواهد کرد. دلیل این است که اختلاف بین یک رقم نقلی در با ارزش‌ترین مکان بیتی حاصل از جمع دودویی و نقلی دهنده‌ی برابر است با $6 = 10 - 16$. جمع BCD زیر را در نظر بگیرید:

4	0100	4	0100	8	1000
+5	+0101	+8	+1000	+9	1001
9	1001	12	1100	17	10001
			+0110		+0110
			10010		10111

در هر حالت دو رقم BCD و نیز فرم دودویی آنها با هم جمع می‌شوند. اگر جمع دودویی بزرگتر یا برابر 1010 باشد، به آن 0110 را می‌افزاییم تا رقم BCD حاصل جمع و نقلی صحیح حاصل شود.

در مثال اول حاصل جمع برابر 9 می‌باشد که یک رقم حاصل جمع BCD صحیح است. در مثال دوم، جمع دودویی یک رقم BCD نامعتبر تولید می‌کند. افزایش 0110 به آن رقم حاصل جمع BCD صحیح 10010 را همراه با یک رقم نقلی به وجود می‌آورد. در مثال سوم، جمع دودویی یک رقم نقلی خواهد داشت. این وضعیت هنگامی رخ می‌دهد که حاصل جمع مساوی یا بزرگتر از 16 باشد. گر چه چهار بیت دیگر کمتر از 1001 است، حاصل جمع نیاز به اصلاح دارد زیرا دارای رقم نقلی است. با جمع 0110 رقم حاصل جمع BCD $7 = (0111)$ و یک نقلی BCD حاصل می‌گردد.

BCD carry	1	1		
	0001	1000	0100	184
	+0101	0111	0110	+576
Binary sum	0111	10000	1010	
Add 6	-----	0110	0110	-----
BCD sum	0111	10110	0000	760

اعداد مبنای دیجیتال ۳۳

جمع دو عدد BCD بی علامت n رقمی روال مشابهی دارد. جمع $184 + 576 = 760$ را به BCD در نظر بگیرید:

اولین جفت رقم BCD کم ارزش‌تر، یک رقم BCD برابر با 0000 و یک رقم نقلی برای جفت رقم بعدی را تولید می‌کند. جفت رقم دوم بعلاوه نقلی قبلی حاصل جمع 0110 و یک رقم نقلی برای جفت رقم بعدی را به وجود می‌آورد. جفت رقم سوم بعلاوه یک رقم نقلی حاصل جمع دودویی 0111 را تولید کرده و نیاز به اصلاح ندارد.

۱-۷-۳ حساب دهمی

نمایش اعداد دهمی علامت‌دار در BCD مشابه اعداد علامت‌دار در دودویی است. ما می‌توانیم از هر یک از هر دو سیستم مقدار علامت‌دار منفی یا متمم علامت منفی‌دار استفاده کنیم. علامت یک عدد دهمی معمولاً با چهار بیت نمایش داده می‌شود تا با کد 4 بیت ارقام دهمی همسان باشد. معمولاً علامت مثبت با چهار 0 و علامت منها با 9 BCD یعنی 1001 نشان داده می‌شود.

سیستم مقدار علامت‌دار منفی به ندرت در کامپیوترها به کار می‌رود. این سیستم می‌تواند متمم 9 یا متمم 10 باشد، ولی اغلب متمم 10 به کار گرفته می‌شود. برای به‌دست آوردن متمم 10 یک عدد BCD، ابتدا متمم 9 را به‌دست آورده و به کم ارزش‌ترین رقم 1 واحد می‌افزاییم. متمم 9 نیز با کسر هر رقم از 9 حاصل می‌گردد.

روالی که در بخش قبل برای سیستم متمم 2 علامت‌دار بنا نهاده شد به سیستم متمم 10 علامت‌دار در اعداد دهمی نیز قابل اعمال است. جمع با افزودن همه ارقام، از جمله رقم علامت و چشم پوشی از رقم نقلی انتهایی انجام می‌شود. در اینجا فرض می‌شود که همه اعداد منفی به فرم متمم 10 باشند. جمع $135 + (-240) = +375$ را در نظر بگیرید که در سیستم متمم علامت‌دار انجام شده است.

$$\begin{array}{r} 0\ 375 \\ +9\ 760 \\ \hline 0\ 135 \end{array}$$

عدد 9 واقع در سمت چپ‌ترین مکان عدد دوم نمایشگر یک علامت منفی و 9760 متمم 10 عدد 0240 است. برای به‌دست آوردن +135، دو عدد با هم جمع و رقم نقلی نادیده گرفته می‌شود. البته اعداد دهدهی داخل کامپیوتر، از جمله ارقام علامت باید BCD باشند. همانطور که قبلاً اشاره شد جمع با ارقام BCD انجام می‌شود.

تفریق اعداد دهدهی اعم از علامت‌دار یا بی‌علامت در سیستم متمم 10 مشابه با حالت دودویی است. متمم 10 مفروق را به‌دست آورید و آن را به مفروق منه اضافه کنید. بسیاری از کامپیوترها برای انجام محاسبات حسابی اعداد دهدهی در BCD، سخت‌افزار خاصی دارند. کاربر کامپیوتر می‌تواند برای انجام عمل حسابی با اعداد دهدهی، بدون نیاز به تبدیل آنها، به دودویی برنامه‌نویسی کند.

۷-۱-۴ دیگر کدهای دهدهی

کدهای دودویی برای ارقام دهدهی به حداقل چهار بیت در قبال هر رقم نیاز دارند. با

۸-۴-۲-۱	افزونی ۳	۲۴۲۱	BCD ۸۴۲۱	رقم دهدهی
0000	0011	0000	0000	0
0111	0100	0001	0001	1
0110	0101	0010	0010	2
0101	0110	0011	0011	3
0100	0111	0100	0100	4
1011	1000	0101	0101	5
1010	1001	0110	0110	6
1001	1010	0111	0111	7
1000	1011	1000	1000	8
1111	1100	1001	1001	9
0001	0000	0101	1010	ترکیبات
0010	0001	0110	1011	
0011	0010	0111	1100	بیتی بکار
1100	1101	1000	1101	نرفته
1101	1110	1001	1110	
1110	1111	1010	1111	

شکل ۷-۱: چهار کد دودویی متفاوت برای ارقام دهدهی

اعداد مبنای در سیستم‌های دیجیتال ۳۵

ایجاد 10 ترکیب مختلف در چهار بیت کدهای مختلف را می‌توان ایجاد کرد. کدهای BCD و سه نوع کد دیگر در جدول ۵-۱ نشان داده شده‌اند. هر کد تنها 10 ترکیب بیتی از 16 ترکیب ممکن را در چهار بیت به کار می‌برند. شش ترکیبی که در هر حال به کار نروند دارای مفهوم نیستند و باید از آنها اجتناب کرد.

کدهای BCD و 2421 از جمله کدهای وزین هستند. در یک کد وزین به هر مکان از بیت وزنی تخصیص داده شده است به نحوی که هر رقم با جمع اوزان تمام ۱ها در ترکیب کد به دست می‌آید. کد BCD دارای وزن های 8، 4، 2، 1 است که مربوط به توازنی از دو برای هر بیت است. مثلاً تخصیص بیتی 0110 با توجه به وزن ۱ها برای 6 تفسیر می‌شود زیرا

$$8 * 0 + 4 * 1 + 2 * 1 + 1 * 0 = 6$$

ترکیب بیتی 1101 وقتی با کد 2421 وزین شود معادل دهدهی $1 * 1 + 2 * 1 + 4 * 1 + 8 * 0 = 7$ را خواهد داد توجه کنید که در کد 2421 بعضی از ارقام به دو طریق کدگذاری می‌شوند. عدد 4 دهدهی به ترکیب های بیتی 0100 یا 1010 متعلق است زیرا هر دو ترکیب عدد 4 را نشان می‌دهند.

کدهای 2421 و افزونی-3 مثال‌هایی از کدهای خود متمم هستند. این کدها خواصی دارند که با توجه به آن متمم 9 عدد دهدهی مستقیماً از تغییر 0ها به 1 و 1ها به 0 در کد حاصل می‌شود. مثلاً عدد دهدهی 395 در افزونی-3 به صورت 0110 1100 1000 می‌باشد. متمم 9 آن یعنی 604 به صورت 0111 0011 1001 می‌باشد که در واقع با متمم هر بیت از کد به دست می‌آید (مثل متمم 1 عدد دودویی). کد افزونی-3 به دلیل خود متممی‌اش در کامپیوترهای قدیمی به کار رفت. این کد بی وزن است. و هر ترکیب کدی در آن از جمع مقدار دودویی متناظرش با 3 حاصل می‌شود. توجه داشته باشید که کد BCD خود متمم نیست. کد 8، 4، 2، 1 - مثالی از تخصیص هر دو نوع وزن مثبت و منفی به یک کد دهدهی است. در این حال، ترکیب بیتی 0110 برای 2 دهدهی تخصیص یافته و از رابطه

$$0 = 2 * (-1) + 1 * (-2) + 1 * 4 + 0 * 8 \text{ محاسبه می شود.}$$

۱-۷-۵ کد کاراکتراسکی

در بسیاری از کاربردهای کامپیوترهای دیجیتال نه تنها نیاز به دستکاری روی داده‌های عددی بلکه روی حروف نیز وجود دارد. برای مثال یک کمپانی بیمه با میلیون‌ها سند، از یک کامپیوتر دیجیتال برای پردازش فایل‌هایش استفاده می‌کند. برای نمایش نام و سایر مشخصات طرفهای قرار داد، داشتن یک کد دودویی برای حروف الفبا ضروری است. به علاوه همان کد دودویی می‌باید اعداد دهدهی و بعضی کاراکترهای خاص دیگر مانند \$ را نیز نمایش دهد. یک مجموعه کاراکتر الفبا عددی مجموعه‌ای از عناصر، متشکل از 10 رقم عدد، 26 حروف الفبا و تعداد معینی از علائم خاص است. چنین مجموعه‌ای بین 36 تا 64 عنصر برای حروف بزرگ و یا بین 64 تا 128 عنصر با حروف بزرگ و کوچک برای هر کلید دارد. در حالت اول شش بیت و در حالت دوم به هفت بیت نیاز است.

	b7b6b5							
b4b3b2b1	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	P
0001	SOH	DC1	!	1	A	Q	A	Q
0010	STX	DC2	"	2	B	R	B	R
0011	ETX	DC3	#	3	C	S	C	S
0100	EOT	DC4	\$	4	D	T	D	T
0101	ENQ	NAK	%	5	E	U	E	U
0110	ACK	SYN	&	6	F	V	F	V
0111	BEL	ETB	'	7	G	W	G	W
1000	BS	CAN	(	8	H	X	H	X
1001	HT	EM	)	9	I	Y	I	Y
1010	LF	SUB	*	:	J	Z	J	Z
1011	VT	ESC	+	;	K	[	K	{
1100	FF	FS	,	<	L	\	L	
1101	CR	GS	-	=	M	]	M	}
1110	SO	RS	.	>	N	^	N	~
1111	SI	US	/	?	O	_	o	DEL

شکل ۱-۸: کدهای استاندارد آمریکایی اسکی (ASCII)

اعداد مبنای در سیستم‌های دیجیتال ۳۷

کد دودویی استاندارد برای کاراکترهای الفبا عددی، اسکی (ASCII) است. این کد از هفت بیت برای کد کردن 128 کاراکتر، طبق جدول (۷-۱) استفاده می‌کند. هفت بیت کد با b1 تا b7 مشخص شده‌اند که b7 با ارزش‌ترین بیت را تشکیل می‌دهد. مثلاً حروف A در اسکی به صورت 1000001 (ستون 100 و سطر 0001) می‌باشد. جدول کدهای استاندارد اسکی حاوی:

94 کاراکتر گرافیکی

34 کاراکتر غیر چاپی برای عملیات کنترلی مختلف

می‌باشد که در آن کاراکترهای گرافیکی نیز از 26 حرف بزرگ (A تا Z)، 26 حرف کوچک (a تا z)، 10 عدد (0 تا 9) و 32 کاراکتر قابل چاپ مانند %، * و \$ تشکیل شده است.

۱-۷-۶ کدهای کنترل کننده در ASCII

34 کاراکتر کنترل در جدول اسکی با اسامی خلاصه شده‌ای مشخص شده‌اند. این کاراکترها در پایین شکل همراه با نوع عملشان ذکر شده‌اند. سه نوع کاراکترهای کنترلی وجود دارند:

- افکتورهای فرمت
- جداسازی اطلاعات
- کاراکترهای کنترل تبادل اطلاعات

افکتور فرمت قالب آنچه را که باید چاپ شود کنترل می‌نماید. این گروه شامل پس‌بر (BS)، جدول‌بندی افقی (HT) و بازگشت نورد (CR) است. جداسازی‌های اطلاعات، داده‌ها را به صورت پاراگراف‌ها و صفحات دسته‌بندی می‌کند. از جمله آنها می‌توان از جداساز رکورد (RS) و جداساز فایل (FS) نام برد. کاراکترهای کنترل تبادل اطلاعات در حین انتقال متن بین پایانه‌های دور از هم مفیدند. مثال‌هایی از این نوع

عبارتند از کاراکتر شروع متن (STX) و ختم متن (ETX) که برای قاب بندی یک پیام متنی به هنگام انتقال از خط تلفن به کار می‌روند.

اسکی یک کد 7 بیتی است ولی اغلب کامپیوترها واحدهای هشت بیتی اطلاعات که بایت نام دارند را دستکاری می‌کنند. بنابراین کاراکترهای اسکی اغلب هر کدام در یک بایت ذخیره می‌شوند. بیت اضافی گاهی برای اهداف دیگری به کار می‌رود و اغلب

NUL	Null	DLE	Data-link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End-of-transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	From feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

شکل ۱-۹: لیست کاراکترهای کنترلی

به کاربرد بستگی دارد. مثلاً بعضی از چاپ‌گرها کاراکترهای اسکی را به صورت 8 بیتی می‌شناسند که در آن با ارزش‌ترین بیت برابر 0 است. 128 کاراکتر 8 بیتی دیگر را با قرار دادن 1 در با ارزش‌ترین مکان برای فونت های نوع ایتالیک یا الفبای یونانی می‌توان به کار برد.

۷-۷-۱ کد تشخیص خطا

برای تشخیص خطاها در مخابره یا پردازش داده، گاهی بیت هشتمی به نام بیت توازن به کاراکتر اسکی اضافه می‌شود. بیت توازن، بیتی اضافی است که حاوی پیامی بوده و

طی آن تعداد 1های کل، زوج یا فرد خواهد شد. دو کاراکتر زیر به همراه توازن زوج یا فرد دیده می‌شوند:

		توازن زوج	با توازن فرد
ASCII A	1000001	01000001	11000001
ASCII T	1010100	11010100	01010100

در هر حالت یک بیت به سمت چپ‌ترین مکان کد می‌افزاییم تا تعداد 1ها در کاراکتر برای توازن زوج، زوج و یا اینکه تعداد 1ها در کاراکتر برای توازن فرد، فرد گردد. به طور کلی یکی از دو توازن اختیار می‌شود ولی توازن زوج معمول تر می‌باشد.

بیت توازن در تشخیص خطا در حین انتقال اطلاعات از یک مکان به مکان دیگر مفید است. این کار با تولید یک بیت توازن زوج برای هر کاراکتر در سمت فرستنده انجام می‌گردد. کاراکترهای 8 بیتی که به همراه بیت‌های توازن می‌باشند به مقصد ارسال می‌گردند. سپس توازن هر کاراکتر در سمت گیرنده چک می‌شود. اگر توازن کاراکتر دریافتی زوج نباشد، حداقل یک بیت در حین انتقال تغییر کرده است. این روش یک، سه یا هر تعداد فردی از خطا را در هر کاراکتر انتقال یافته تشخیص می‌دهد. در این حالت تعداد زوجی از خطاها قابل تشخیص نخواهد بود. کدهای خطای دیگر برای محافظت از خطاهای زوج لازم است.

اینکه پس از شناسایی خطا چه کاری باید انجام داد به کاربرد مربوطه بستگی دارد. یک امکان این است که با فرض اتفاقی بودن خطا و عدم تکرار، تقاضای ارسال مجدد گردد. در این حالت اگر گیرنده یک خطای توازن را شناسایی کند، یک کاراکتر کنترل اسکی، ASCII NAK، (تصدیق نفی) را متشکل از هشت بیت با توازن زوج 1001 0101 باز پس می‌فرستند. اگر خطایی شناسایی نشد، گیرنده کاراکتر کنترل ACK (تصدیق) را

با کد 00000110 باز می‌فرستد. سمت فرستنده دوباره با ارسال پیام NAK پاسخ می‌دهد تا این که توازن صحیح دریافت شود. اگر پس از چند نوبت تکرار، انتقال همچنان دارای خطا بود، پیام خطایی به اپراتور برای چک کردن عامل خطا در خط انتقال فرستاده می‌شود.

سؤالات

- ۱- عدد $(0.2498)_{10}$ را به دودویی تبدیل کنید.
- ۲- عدد 2406 را به مبنای هشت ببرید.
- ۳- عدد 673.124 در مبنای هشت معادل چه اعدادی در مبنای دودویی و دهدهی است.
- ۴- متمم 10 عدد 256703 چه عددی خواهد بود.
- ۵- متمم 2 عدد 0100101 چه عددی خواهد بود.
- ۶- متمم 9 عدد 062374 چه عددی خواهد بود.
- ۷- با استفاده از متمم 10 تفریق 46532-3248 را به دست آورید.

فصل دوم

گیت‌های منطقی، جبر بول و توابع بولی

هدف کلی

در این فصل مباحث کلی مربوط به منطق دودویی به همراه مفاهیم اساسی جبر بول مورد بحث و بررسی قرار خواهند گرفت. در ادامه مفاهیم و تئوری‌های اساسی جبر بول بررسی شده و در ادامه توابع بول نیز به صورت کامل مورد نقد و بررسی قرار خواهند گرفت. در ادامه نیز انواع گیت‌های منطقی به همراه جداول درستی هر یک بررسی خواهند شد.

هدف ساختاری

در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند:

- منطق دودویی
- انواع گیت‌های منطقی
- اصول اساسی جبر بول
- تئوری‌های اساسی جبر بول
- تقدم عمل‌گرها در جبر بول
- توابع بول
- متمم توابع بول
- مدارهای مجتمع

۲-۱ منطق دودویی

منطق دودویی با متغیرهایی که دو ارزش گسسته و عملیاتی که مفهوم منطقی دارند، سرو کار دارد و ارزشی که متغیرها اختیار می‌کنند ممکن است با اسامی مختلفی نام‌گذاری شوند (مثل صحیح و غلط، بله و خیر و غیره)، اما برای ما بهتر است آن را بر حسب بیت تصور کنیم و مقادیر 1 و 0 را به آن تخصیص دهیم. منطق دودویی معرفی شده در این بخش معادل با جبری به نام جبر بول است. در این بخش جبر بول به روشی غیر مستدل بوده و ارتباط آن با مدارهای منطقی دیجیتال و سیگنال‌های دودویی بیان شده است.

۲-۱-۱ تعریف منطق دودویی

منطق دودویی شامل متغیرهای دودویی و عملیات منطقی است. متغیرها با حروف الفبایی مانند A, B, C, X, Y, Z و غیره نام‌گذاری می‌شوند، که هر متغیر فقط و فقط دو مقدار مجزای 0 و 1 دارد. سه نوع عملیات منطقی اصلی وجود دارند: AND, OR و NOT. در ادامه به شرح هر یک از عملیات می‌پردازیم:

۱- AND: این عمل به وسیله یک "." یا بدون ذکر هر عملگری نمایش داده می‌شود. مثلاً $x.y = z$ یا $x y = z$ را چنین می‌خوانیم "x AND y برابر است با z".

عمل منطقی AND چنین تفسیر می‌شود که، $z=1$ است اگر و فقط اگر $x=1$ و $y=1$ باشد؛ در غیر این صورت $z=0$ است. (به یاد داشته باشید که x و y و z متغیرهای دودویی هستند و نمی‌توانند به جز 1 و 0 چیز دیگری باشند.)

۲- OR: عملی است که با علامت بعلاوه نشان داده می‌شود. مثلاً $x+y=z$ را چنین می‌خوانیم "x OR y برابر است با z" و به این معنی است که $z=1$ است به شرطی که $x=1$ و $y=1$ و یا هر دو $x=1$ و $y=1$ باشند. اگر هر دو $x=0$ و $y=0$ باشد آنگاه $z=0$ خواهد بود.

گیت‌های منطقی، جبر بول و توابع بولی ۴۵

AND			OR		
x	y	x . y	x	y	x + y
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	1	0	1
1	1	1	1	1	1

NOT	
x	x'
0	1
1	0

شکل ۱-۲: جدول درستی عملیات منطقی

۳- NOT: این علامت با یک علامت پریم نشان داده می‌شود (و گاهی با یک خط بار). مثلاً $x' = z$ (یا $x = z$) و چنین خوانده می‌شود، "NOT x برابر است با z" و به این معنی است که z چیزی است که x نیست. به بیان دیگر اگر $x = 1$ باشد آنگاه $z = 0$ ؛ اما اگر $x = 0$ باشد، آنگاه $z = 1$ است. عمل NOT را متمم هم می‌گویند چون 1 را به 0 و 0 را به 1 تبدیل می‌کند.

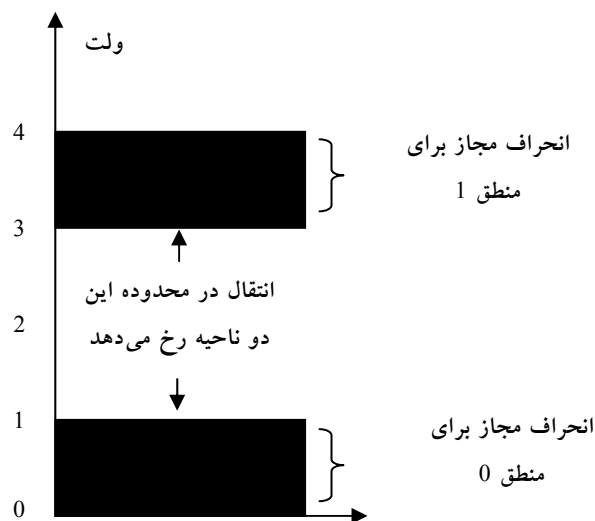
منطق دودویی شبیه حساب دودویی است، و اعمال AND و OR به ترتیب به اعمال ضرب و جمع شباهت دارند. در حقیقت سمبل‌های به کار رفته برای AND و OR همان هایی هستند که برای ضرب و جمع مورد استفاده قرار می‌گیرند. معهذا منطق دودویی را نباید با حساب دودویی اشتباه کرد. مسئله‌ای که باید مورد توجه قرار گیرد این است که یک متغیر حسابی، عددی را مشخص می‌کند که ممکن است دارای چندین رقم باشد. یک متغیر منطقی همیشه 0 و یا 1 است.

مثلاً در حساب دودویی داریم $1 + 10 = 1$ (می‌خوانیم: "یک بعلاوه یک برابر است با 2")، در صورتیکه در منطق دودویی، داریم $1 + 1 = 1$ (می‌خوانیم: "یک OR یک، برابر است با 1").

برای هر ترکیبی از مقادیر x و y ، مقدار معینی برای z وجود دارد که این مقدار پس از اعمال یا تعریف عمل منطقی مشخص می‌گردد. این تعاریف را می‌توان به صورت خلاصه یا استفاده از جدول درستی فهرست کرد. یک جدول درستی، جدولی است متشکل از تمام ترکیبات ممکن متغیرها و بیانگر ارتباط بین مقادیر آنها و نتایج حاصل از عمل مربوطه روی آنها می‌باشد. به عنوان مثال جداول درستی برای عملگرهای AND و OR با متغیرهای x و y ، با لیست کردن همه مقادیر ممکن آنها وقتی به صورت زوج ترکیب شده‌اند، حاصل می‌شود. نتیجه عمل برای هر ترکیب به طور جداگانه آمده است. جداول درستی AND و OR و NOT در جدول زیر نشان داده شده‌اند. این جداول تعریف عملیات مذکور را به طور شفاف بیان می‌دارند.

۲-۱-۲ گیت‌های منطقی

گیت‌های منطقی، مدارهایی الکترونیک هستند که روی یک یا چند سیگنال ورودی عمل می‌کنند تا یک سیگنال خروجی تولید نمایند. سیگنال‌های الکترونیکی مانند ولتاژها یا جریان‌هایی که در سرتاسر یک سیستم دیجیتال وجود دارند دو مقدار جدا از هم را اختیار می‌کنند. مدارهایی که با ولتاژ کار می‌کنند به دو سطح ولتاژ که نمایشگر



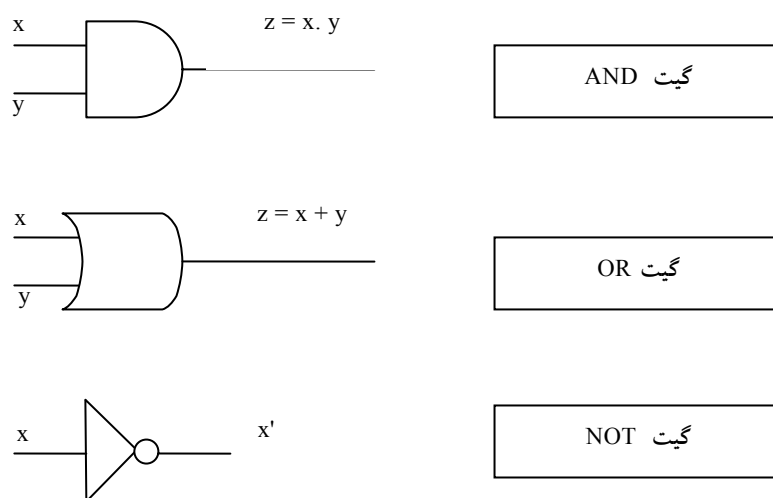
شکل ۲-۲: محدوده انتقال سیگنال 0 و 1

گیت‌های منطقی، جبر بول و توابع بولی ۴۷

یک متغیر دودویی و برابر با منطق 1 و منطق 0 اند واکنش نشان می‌دهند. مثلاً یک سیستم دیجیتال خاص ممکن است منطق 0 را به عنوان سیگنالی برابر با 0 ولت و منطق 1 را به صورت سیگنالی برابر با 4 ولت تعریف کند. در عمل، هر سطح ولتاژ، محدوده مورد قبولی مانند شکل ۲-۲ را داراست.

پایانه‌های ورودی مدارهای دیجیتال سیگنال‌های دودویی را در محدوده مجازی می‌پذیرند و در پایانه‌های خروجی در محدوده مجازی پاسخ می‌دهند. ناحیه میانی بین دو ناحیه مجاز تنها هنگام گذر از یک حالت به حالت دیگر قطع می‌شود. هر اطلاعات محاسباتی یا کنترلی مورد نظر را می‌توان با عبور سیگنال‌هایی دودویی از میان ترکیباتی از گیت‌ها مورد استفاده قرار داد، که هر سیگنال بیانگر یک متغیر دودویی بوده و یک بیت از اطلاعات را حمل می‌کند.

سمبل‌های گرافیکی مورد استفاده برای سه نوع گیت در شکل ۲-۳ دیده می‌شوند:



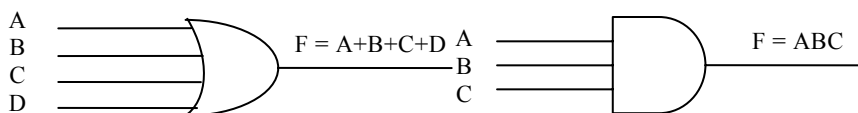
شکل ۲-۳: سمبل‌های مداری منطقی دیجیتال

گیت‌ها، بلوک‌هایی سخت‌افزاری‌اند که با ورودی منطقی مناسبی، در خروجی خود 0 یا 1 تولید می‌نمایند. سیگنال ورودی x و y در گیت‌های AND و OR در یکی از چهار

X	0	1	1	0	0
Y	0	0	1	1	0
AND : $x \cdot y$	0	0	1	0	0
OR : $x + y$	0	1	1	1	0
NOT : x'	1	0	0	1	1

شکل ۲-۴: سیگنال‌های ورودی-خروجی برای گیت‌های AND , OR , NOT

حالت ممکن قرار دارند: 00، 01، 10 و 11. این سیگنال‌ها همراه با خروجی خود در شکل ۲-۴ دیده می‌شوند. نمودارهای زمانی پاسخ هر گیت را برای چهار گیت فوق نشان می‌دهند. محور افقی نمودار زمان، و محور عمودی سیگنال‌ها را ضمن تغییر بین دو سطح ولتاژ ممکن نمایش می‌دهد. سطح پایین منطق 0 و سطح بالا منطق 1 را نشان می‌دهد. هنگامی در خروجی گیت AND منطق 1 وجود دارد که هر دو سیگنال ورودی در منطق 1 باشند. گیت OR هنگامی خروجی 1 دارد که یکی از سیگنال‌های ورودی در منطق 1 باشد. گیت NOT را معمولاً وارون‌گر یا معکوس‌گر هم می‌گویند. دلیل انتخاب این نام با توجه به پاسخ سیگنال در نمودار زمانی مشخص است، و در آن نشان داده شده است که سیگنال خروجی مفهوم منطق ورودی را معکوس کرده است.



ب- گیت OR با چهار ورودی

الف- گیت AND با سه ورودی

شکل ۲-۵: مدار سوئیچینگ نمایش دهنده منطق دودویی

گیت‌های منطقی، جبر بول و توابع بولی ۴۹

گیت‌های AND و OR ممکن است بیش از دو ورودی داشته باشند. یک گیت AND با سه ورودی و یک OR با چهار ورودی در شکل ۲-۵ ملاحظه می‌شود. گیت AND سه ورودی به شرطی خروجی 1 دارد که هر سه ورودی آن 1 باشد. اگر هر یک از ورودی‌ها 0 باشند، خروجی AND برابر 0 خواهد بود. گیت OR چهار ورودی هنگامی خروجی 1 تولید می‌کند که یکی از ورودی‌ها در 1 منطقی باشد. خروجی هنگامی 0 می‌شود که همه ورودی‌ها در منطق 0 باشند.

۲-۲- جبر بول

جبر بول را می‌توان مانند هر سیستم منتهی ریاضی، به وسیله مجموعه‌ای از عناصر، یک مجموعه از الگوها و تعدادی اصول اثبات نشده یا بدیهیات تعریف نمود. یک مجموعه از عناصر کلکسیونی از اشیاء است که دارای خواص مشترکی باشند. اگر s یک مجموعه و x و y عناصر مشخصی از آن باشند، آنگاه $x \in s$ به این معنی است که x عضوی از مجموعه s است و $y \notin s$ یعنی y عضوی از مجموعه s نیست. یک مجموعه با تعداد قابل شمارشی از عناصر با یک جفت آکولاد مشخص می‌شود: $A = \{1, 2, 3, 4\}$

یعنی عناصر مجموعه A عبارتند از 1, 2, 3, 4. یک عملگر دودویی روی یک مجموعه از عناصر، S ، قانونی است که به هر جفت از عناصر S ، یک عنصر منحصر به فرد از S را تخصیص دهد. به عنوان مثال رابطه $a * b = c$ را در نظر بگیرید. (*) را یک عملگر دودویی می‌خوانیم به شرطی که بتواند عنصر c را به جفت عنصر a و b منتسب نماید ضمن اینکه رابطه $a, b, c \in S$ معتبر باشد. با این وجود اگر $a, b \in S$ و $c \notin S$ باشد، (*) یک عملگر دودویی نیست.

اصول یک سیستم ریاضی، فرضیات اولیه را تشکیل می‌دهند که با استفاده از آنها می‌توان قوانین و تئوری‌ها و خواص سیستم را نتیجه گرفت. مهمترین اصول به کار رفته در فرموله کردن ساختارهای جبری عبارتند از:

۱. بسته بودن

۲. عنصر شناسه
۳. عنصر معکوس
۴. اصل شرکت پذیری
۵. اصل جابجایی
۶. اصل توزیع پذیری

۱- **بسته بودن:** یک مجموعه S نسبت به عملگر دودویی بسته است به شرطی که برای هر جفت عنصر از S ، این عملگر عنصر منحصر به فردی از آن را به جفت عنصر منتسب نماید. به عنوان مثال، مجموعه اعداد طبیعی $N = \{1, 2, 3, 4, \dots\}$ را نسبت به عملگر جمع $(+)$ بسته گوئیم زیرا برای هر دو عنصر $a, b \in N$ عنصر دیگری مانند $c \in N$ می توان یافت بطوری که $a + b = c$ باشد. مجموعه اعداد طبیعی نسبت به عملگر تفریق بسته نیست چون داریم $2 - 3 = -1$ در حالی که $2, 3 \in N$ و $-1 \notin N$ است.

۲- **عنصر شناسه:** مجموعه S نسبت به عملگر $(*)$ روی مجموعه S دارای عنصر شناسه است، اگر عنصر $e \in S$ با خاصیت زیر موجود باشد.

$$e * x = x * e = x \text{ به ازای هر } x \in S \text{ داشته باشیم}$$

مثال: عنصر 0 یک عنصر شناسه نسبت به عملگر $(+)$ روی مجموعه اعداد صحیح $I = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$ است، چون

$$x + 0 = 0 + x = x \text{ به ازای هر } x \in I \text{ داشته باشیم}$$

مجموعه اعداد طبیعی N دارای عنصر شناسه نیست زیرا 0 جزو مجموعه نمی باشد.

گیت‌های منطقی، جبر بول و توابع بولی ۵۱

۳- عنصر معکوس: مجموعه‌ای چون S با عنصر شناسه e نسبت به عملگر $(*)$ دارای معکوس است به شرطی که برای هر $x \in S$ ، یک $y \in S$ وجود داشته باشد به نحوی که:

$$x * y = e$$

مثال: در مجموعه اعداد صحیح I ، با $e = 0$ ، معکوس عنصر a برابر $(-a)$ است چون

$$a + (-a) = 0$$

۴- اصل شرکت‌پذیری: یک عملکرد دودویی $(*)$ روی مجموعه S شرکت‌پذیر است اگر داشته باشیم:

$$(x * y) * z = x * (y * z) \quad \text{به ازای همه مقادیر } x, y, z \in S \text{ داشته باشیم}$$

۵- اصل جابجایی: یک عملگر $(*)$ روی مجموعه دارای خاصیت جابجایی است هرگاه:

$$x * y = y * x \quad \text{به ازای هر } x, y \in S \text{ داشته باشیم}$$

۶- اصل توزیع‌پذیری: اگر $(*)$ و $(.)$ دو عملگر روی مجموعه S باشند، $(*)$ را روی $(.)$ توزیع‌پذیر گوئیم هرگاه:

$$x * (y * z) = (x * y) * (x * z)$$

مثالی جبری در این مورد میدان یا حوزه است. میدان مجموعه‌ای از عناصر است، همراه با دو عملگر دودویی، که هر یک دارای خواص ۱ تا ۵ بوده و هر دو عملگر برای تشکیل خاصیت ۶ با یکدیگر ترکیب می‌شوند. مجموعه اعداد حقیقی، همراه با عملگرهای دودویی $(+)$ و $(.)$ ، میدان اعداد حقیقی را تشکیل می‌دهند. میدان اعداد حقیقی مبنای جبر معمولی و حساب است. عملگرها و اصول دارای مفاهیم زیر هستند:

عملگر دودویی $(+)$ جمع را تعریف می‌کند.

معکوس جمع، تفریق می‌باشد..

شناسه جمع، 0 است

شناسه ضرب 1 می باشد.

عملگر دودویی (.) ضرب را تعریف می نماید.

معکوس ضرب $a = 1/a$ تقسیم را تعریف می کند، یعنی

$$a.(1/a) = 1$$

تنها اصل توزیع پذیری قابل اعمال مربوط به عملگر (.) روی (+) است:

$$a.(b + c) = (a.b) + (a.c)$$

۲-۲-۱ تعریف اصول اساسی جبر بول

در سال ۱۸۵۴ جورج بول یک سیستم جبری را که امروزه آن را جبر بول می نامیم پایه ریزی کرد. در سال ۱۹۳۸ نیز شانون یک جبر بول دو مقداری به نام جبر سوئیچینگ را معرفی کرد که در آن خواص مدارهای سوئیچینگ با این جبر قابل ارائه است. برای تعریف مستدل جبر بول، ما اصول فرموله شده به وسیله هانتینگتون در سال ۱۹۰۴ را به کار می بریم. **اصول هانتینگتون** به شرح زیر بودند:

۱- (a) مجموعه نسبت به عملگر (+) بسته باشد.

(b) مجموعه نسبت به عملگر (.) بسته باشد.

۲- (a) یک عنصر شناسه 0 برای (+) وجود داشته باشد.

(b) یک عنصر شناسه 1 برای (.) وجود داشته باشد.

۳- برای هر عنصر $x \in B$ ، عنصری مثل $x' \in B$ وجود دارد (به آن متمم x می گوئیم)

به نحوی که: (a) $x' + x = I$ و (b) $x'.x = 0$

۴- حداقل دو عنصر $x, y \in B$ وجود دارد به نحوی که $x \neq y$ باشد.

۵- (a) مجموعه نسبت به (+) دارای خاصیت جابجایی باشد: $x + y = y + x$

گیت‌های منطقی، جبر بول و توابع بولی ۵۳

(b) مجموعه نسبت به $(.)$ دارای خاصیت جابجایی باشد: $x \cdot y = y \cdot x$

۶- (a) $(.)$ نسبت به $(+)$ توزیع پذیر است: $x \cdot (y+z) = (x \cdot y) + (x \cdot z)$

(b) $(+)$ نسبت به $(.)$ توزیع پذیر است: $x + (y \cdot z) = (x+y) \cdot (x+z)$

جبر بول یک ساختار جبری است که با عناصر مجموعه، یعنی B ، همراه با دو عملگر دودویی $(+)$ و $(.)$ تعریف می‌شود به شرطی که اصول زیر (هانتینگتون) در آن معتبر باشد. به بیانی دیگر به منظور داشتن یک جبر بول باید:

۱- عناصر مجموعه B مشخص باشند.

۲- قوانین عمل دو عملگر دودویی معین باشند.

۳- مجموعه عناصر، B ، همراه با دو عملگر، برای شش اصل هانتینگتون معتبر باشد.

۲-۱-۱ تفاوت‌های جبر بول با جبر معمولی

با مقایسه جبر بول با حساب و جبر معمولی (حوزه یا میدان اعداد حقیقی) تفاوت‌های زیر قابل ملاحظه‌اند:

۱- اصول هانتینگتون فاقد اصل شرکت‌پذیری است. با این وجود، این اصول برای جبر بول معتبر و برای هر دو عملگر از دیگر اصول قابل استنتاج است.

۲- اصل توزیع‌پذیری $(+)$ روی $(.)$ ، یعنی $x + (y \cdot z) = (x+y) \cdot (x+z)$ ، برای جبر بول معتبر است، ولی در جبر معمولی قابل قبول نیست.

۳- جبر بول دارای معکوس‌های جمع و ضرب نیست؛ بنابراین عملگرهای تفریق و تقسیم وجود ندارند.

۴- اصل ۳ عملگری به نام متمم را معرفی می‌نماید که در جبر معمولی وجود ندارد.

۵- جبر معمولی در مورد اعداد حقیقی بحث می‌کند، که یک مجموعه با بی نهایت عنصر را شامل می‌شود. جبر بول در مورد مجموعه‌ای از عناصر، B ، بحث می‌نماید که هنوز آن را معرفی نکرده‌ایم، ولی بعداً در جبر بول دو مقداری یا دو ارزشی معرفی خواهد شد (کاربرد بعدی ما از این جبر مورد توجه است)، و در آن B به صورت مجموعه‌ای از دو عنصر 0 و 1 تعریف می‌شود.

لازم است تا اختلاف بین یک مجموعه از عناصر متعلق به یک ساختار جبری و متغیرهای یک سیستم جبری را بدانیم. مثلاً اجزاء حوزه اعداد حقیقی، اعداد هستند، در صورتی که متغیرهایی مانند a ، b و c که در جبر معمولی به کار می‌روند، سمبل‌هایی هستند که به جای اعداد حقیقی به کار می‌روند. به طور مشابه در جبر بول مجموعه B دارای متغیرهایی مانند x ، y و z می‌باشد که صرفاً سمبل هستند و عناصر را نشان می‌دهند.

بسته به انتخاب عناصر B و قوانین عملیات، می‌توان چندین جبر بول را فرموله کرد. در ادامه کار ما فقط با جبر دو ارزشی که تنها دو عنصر دارد، سرو کار خواهیم داشت. جبر بول دو ارزشی در تئوری مجموعه‌ها و منطق کاربرد دارد. هدف ما در این کتاب کاربرد جبر بول در مدار منطقی گیتی است.

۲-۱-۲-۲ جبر بول دو ارزشی

جبر بول دو ارزشی روی مجموعه دو عنصری، $\{0, 1\}$ ، به همراه قوانین برای دو

x	y	$x \cdot y$	x	y	$x + y$	X	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

عملگر دودویی (+) و (.) که در جدول زیر نشان داده شده تعریف می‌شود (قانون عملگر متمم برای تصدیق اصل ۳ است): این قوانین دقیقاً مثل اعمال AND، OR و

گیت‌های منطقی، جبر بول و توابع بولی ۵۵

NOT در شکل ۱-۲ می‌باشند. اکنون نشان می‌دهیم که اصول هانتینگتون برای مجموعه، $\{0=1\}$ و دو عملگرهای دودویی که قبلاً تعریف شده‌اند، معتبر است.

۱- با توجه به جداول، بسته بودن کاملاً روشن است زیرا نتیجه هر عملگر 1 یا 0 بوده و $0, 1 \in B$ می‌باشند.

۲- با توجه به جداول می‌بینیم که:

$$(a) 1+1=0+1=1+0=1$$

$$(b) 1 \cdot 0=0 \cdot 1=0 \cdot 0=0$$

این روابط بیانگر وجود عناصر شناسه 0 برای (+) و 1 برای (.)، طبق تعریف می‌باشند.

۳- اصول جابجایی با توجه به تقارن جداول عملگرها آشکار است.

۴- (a) صحت اصل توزیع‌پذیری $x \cdot (y+z) = (x \cdot y) + (x \cdot z)$ را با ایجاد جدول برای تمام مقادیر ممکن x, y, z ، می‌توان تحقیق کرد. برای هر ترکیب، $x \cdot (y+z)$ را به دست آورده و نشان می‌دهیم که برابر $(x \cdot y) + (x \cdot z)$ است.

x	y	z	y+z	$x \cdot (y+z)$	x.y	x.z	$(x \cdot y) + (x \cdot z)$
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

(b) صحت اصل توزیع‌پذیری (+) روی (.) را نیز می‌توان مانند بند قبل تحقیق نمود.

۵- با استفاده از جدول متمم، به سادگی می‌توان دید که:

(a) $x+x'=1$ ، زیرا $0+0'=0+1=1$ ، $1+1'=1+0=1$ است.

(b) $x.x'=0$ ، زیرا $0.0'=0.1=0$ ، $1.1'=1.0=0$ است، که اصل ۵ را تصدیق می‌کند.

۶- اصل ۶ نیز صادق است زیرا جبر بول دو ارزشی دارای دو مقدار مجزای ۰ و ۱ با $1 \neq 0$ است.

تا اینجا ما یک جبر دو ارزشی با عملگرهای AND, OR و یک عملگر متمم معادل با NOT ایجاد کردیم. بنابراین جبر بول به روش مستدل ریاضی بنا گردید و نشان داده شد که معادل با منطق دودویی غیر مستدل است. بیان غیر مستدل برای درک کاربرد جبر بول در مدارهای گیتی مفید است. روش مستدل برای بیان و ایجاد تئوری‌ها و خواص سیستم جبری مورد توجه است. جبر بول دو ارزشی تعریف شده در این بخش را "جبر سوئیچینگ" نیز می‌نامند. برای تاکید بر تشابه بین جبر بول دو ارزشی و دیگر سیستم‌های دودویی، این جبر در بخش قبل "منطق دودویی" نامیده شد. از این پس، کلمه "دو ارزشی" را در بحث‌های بعدی از جبر بول حذف می‌کنیم.

۲-۲-۲ قضایای اصلی و خواص جبر بول

اصول هانتینگتون به صورت جفت جفت لیست شده‌اند و به صورت بخشهای (a) و (b) مشخص شدند. هر یک از این دو را با تعویض عملگرها و عنصر شناسه می‌توان از دیگری به دست آورد. این خاصیت در جبر بول به اصل دوگانگی معروف است. خصوصیات فوق بیان می‌دارد که هر عبارت جبری منتج از اصول جبر بول با تعویض عملگرها و شناسه‌ها باز هم معتبر باقی می‌ماند. در جبر بول دو ارزشی، عناصر شناسه و عناصر مجموعه B یکسانند: یعنی ۱ و ۰ اند. اصل دوگانگی کاربردهای متعددی دارد. اگر دوگان یک عبارت جبری مورد نظر باشد تنها کافی است عملگرهای AND و OR تعویض و ۰ها به ۱ و ۱ها به ۰ تبدیل گردند.

۲-۲-۱ تئوری‌های اساسی جبر بول

تئوری‌ها و اصول لیست شده اساسی‌ترین روابط در جبر بول اند. تئوری‌ها نیز مانند اصول به صورت جفت جفت ارائه شده‌اند و هر رابطه دوگان زوج خود است. اصول، بدیهیات ساختار جبری بوده و اثباتی لازم ندارند. تئوری‌ها باید با توجه به اصول ثابت شوند، اثبات تئوری‌ها با یک متغیر در زیر نشان داده شده‌اند. در سمت مقابل روابط، شماره اصل به کار رفته نوشته شده است.

تئوری ۱ (a): $x+x=x$

$$\begin{aligned} x+x &= (x+x).1 \\ &= (x+x)(x+x') \\ &= x+xx' \\ &= x+0 \\ &= x \end{aligned}$$

تئوری ۱ (b): $x.x=x$

$$\begin{aligned} x.x &= xx+0 \\ &= xx+xx' \\ &= x(x+x') \\ &= x.1 \\ &= x \end{aligned}$$

توجه کنید که تئوری ۱ (b) دوگان ۱ (a) است و هر مرحله از اثبات در بخش (b) دوگان بخش (a) می‌باشد. به این ترتیب هر تئوری دوگان از اثبات زوجش حاصل می‌گردد.

تئوری ۲ (a): $x+1=1$

$$\begin{aligned} x+1 &= 1.(x+1) \\ &= (x+x')(x+1) \\ &= x+x'.1 \\ &= x+x' \\ &= 1 \end{aligned}$$

تئوری ۲ (b): بر اساس دوگانگی $x.0=0$

تئوری ۳: $x = (x')'$. با توجه به اصل ۳ ، داریم $x+x'=1$ ، $x.x'=0$ ، که متمم x را تعریف می کند . متمم x' برابر x است و در نتیجه همان (x') می باشد. بنابراین، چون متمم منحصر به فرد است داریم $(x')' = x$.

با استفاده از اصول و تئوری های اثبات شده قبلی می توان تئوری های دو یا سه متغیره را به صورت جبری ثابت کرد. مثلاً: تئوری جذب را در نظر بگیرید.

تئوری ۴: شرکت پذیری

$$(a): x + (y + z) = (x + y) + z$$

$$(b): x (y z) = (x y) z$$

تئوری ۵:

$$(a): (x + y)' = x' y'$$

$$(b): (x y)' = x' + y'$$

$$\begin{aligned} x+xy &= x.1+xy \\ &= x(1+y) \\ &= x(y+1) \\ &= x.1 \\ &= x \end{aligned}$$

تئوری ۶ (a): $x+xy=x$

تئوری ۶ (b): بر اساس دوگانگی $x(x+y)=x$

۲-۲-۲-۲-۲ مقدم عملگرها

در ارزیابی عبارت جبر بول تقدم اول با پرانتز، دوم با NOT، سوم با AND و چهارم OR است. به بیان دیگر، عبارت داخل پرانتز باید قبل از سایر عملگرها ارزیابی شود. عملگر مقدم بعدی متمم است. پس از آن AND و بالاخره OR قرار دارد. به عنوان مثال، جدول درستی را برای تئوری دموورگان تشکیل می دهیم سمت چپ عبارت

گیت‌های منطقی، جبر بول و توابع بولی ۵۹

$(x+y)'$ است. بنابراین داخل پرانتز ابتدا ارزیابی می‌شود و سپس نتیجه متمم می‌گردد. سمت راست عبارت $x'y'$ است. بنابراین متمم x و y هر دو ابتدا ارزیابی شده و حاصل AND می‌گردد. توجه کنید که در محاسبات معمولی هم روال مشابهی (به جز برای متمم) برای ضرب و جمع به ترتیب به جای AND و OR برقرار است.

۳-۲ توابع بول

جبر بول جبری است که با متغیرهای دودویی و عملیات منطقی سروکار دارد. یک تابع به وسیله یک عبارت جبری متشکل از متغیرهای دودویی، ثابت‌های 0 یا 1 و سمبل‌های عملیاتی منطقی تشکیل شده است. برای مقدار مفروضی از متغیرهای دودویی، تابع می‌تواند 1 یا 0 باشد. یک تابع بول رابطه‌ای منطقی را بین متغیرها بیان می‌کند. این تابع با تعیین مقدار دودویی عبارت بر حسب همه مقادیر ممکن متغیرها ارزیابی می‌شود.

یک جدول بولی به صورت یک جدول درستی هم می‌تواند نشان داده شود. جدول درستی لیستی از 1ها و 0ها است که به متغیرهای دودویی تخصیص می‌یابد، و ستونی که مقدار نتایج را برای هر ترکیب نشان می‌دهد. تعداد سطرها در جدول درستی 2^n است، که n تعداد متغیرها در تابع است. ترکیبات دودویی برای جدول درستی از شمارش اعداد دودویی و از 0 تا 2^n-1 به دست می‌آید. یک تابع بول را می‌توان از یک عبارت جبری به یک نمودار مداری متشکل از گیت‌های منطقی تبدیل کرد. برای درک بهتر موضوع دو تابع زیر را در نظر بگیرید:

$$F_1 = x + y'z$$

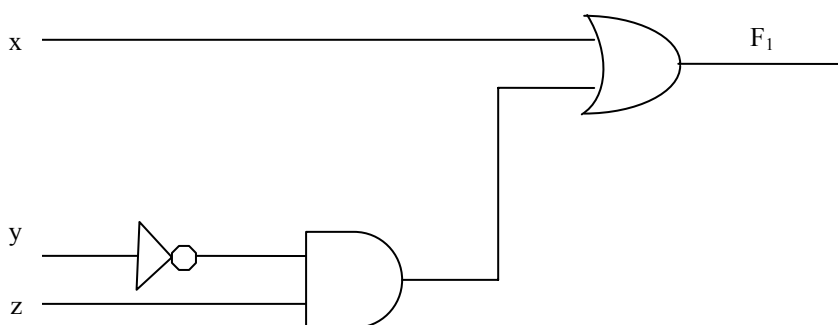
$$F_2 = x'y'z + x'y z + x y'$$

جدول شکل ۶-۲، درستی دو تابع F_1 و F_2 را نشان می‌دهد:

x	y	z	F ₁	F ₂
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	0

شکل ۷-۲: جدول درستی توابع سه متغیره F₁ و F₂

در این جدول هشت ترکیب دودویی ممکن برای تخصیص بیتی به سه متغیر z, y, x وجود دارد. ستونی که بر چسب F₁ دارد در ازاء هر ترکیب 0 یا 1 است. جدول نشان می‌دهد که وقتی $x=1$ یا $yz=1$ باشد تابع F₁ برابر 1 است. در غیر این صورت 0 خواهد بود. برای نمایش F₁ در یک جدول درستی تنها یک راه وجود دارد. با این وجود، وقتی تابع به فرم یک عبارت جبری است، می‌تواند به فرم‌های متفاوتی نشان داده شود. عبارت خاصی که برای مشخص کردن تابع مورد استفاده قرار می‌گیرد اتصالات میان گیت‌ها در نمودار مدار منطقی را دیکته می‌نماید. نمودار مدار منطقی تابع بولی F₁ در شکل ۷-۲ نشان داده شده است:



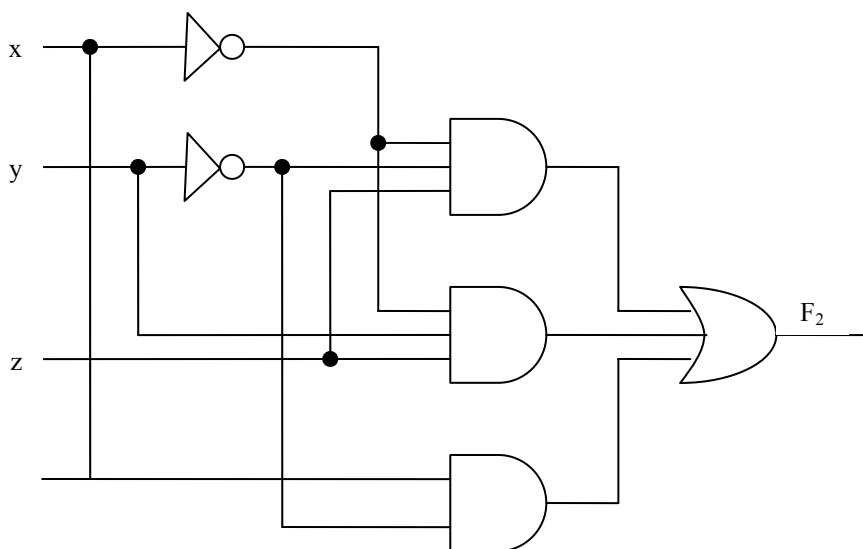
شکل ۷-۲: پیاده‌سازی با گیت $F_1 = x + y'z$

برای تولید متمم ورودی y از گیت NOT استفاده شده است. برای جمله $y'z$ یک گیت AND و برای ترکیب آن دو یک گیت OR به کار رفته است. در نمودارهای مدار

گیت‌های منطقی، جبر بول و توابع بولی ۶۱

منطقی، متغیرهای تابع به عنوان ورودی مدار و متغیر دودویی F_1 به عنوان خروجی مدار در نظر گرفته می‌شوند.

در تابع بولی F_2 که جدول درستی آن در بالا آمده است، متغیرهای x, y به کمک وارون‌گر متمم شده‌اند تا x', y' به دست آیند. سه جمله در عبارت با سه گیت AND پیاده‌سازی شده‌اند. گیت OR نیز، گیت منطقی سه جمله را فراهم می‌سازد. نمودار مدار منطقی تابع بولی F_2 در شکل ۸-۲ نشان داده شده است:



شکل ۸-۲: پیاده‌سازی تابع بول F_2 با گیت

۱-۳-۲ متمم یک تابع

متمم یک تابع F برابر F' است و از تعویض ۰ ها با ۱ و ۱ ها با ۰ در مقدار F به دست می‌آید. متمم یک تابع را می‌توان به صورت جبری از تئوری دموگن نیز به دست آورد. تئوری‌های دموگن به سه یا چند متغیر هم قابل گسترش اند. با استفاده از اصول و تئوری‌های ارائه شده، فرم سه متغیره اولین تئوری دموگن به طریق زیر ثابت می‌شود.

$$\text{با فرض } (B + C = x) \text{ داریم: } (A + B + C)' = (A + x)'$$

با تئوری ۵ (a) دموگران $(A + B + C)' = A'x'$

$B + C = x$ را جایگزین کنید $(A + B + C)' = A' (B + C)'$

با تئوری ۵ (a) دموگران $(A + B + C)' = A'(B'C')$

با تئوری ۴ (b) شرکت‌پذیری $= A'B'C'$

تئوری‌های دموگران برای هر تعدادی از متغیرها، مشابه حالت دو متغیره بوده و با روش جایگزینی متوالی، مشابه روشی که در فوق مشاهده شد، می‌توان آن را به‌دست آورد. **فرم عمومی تئوری دموگران** به صورت زیر است:

$$(A + B + C + D + \dots + F)' = A'B'C'D'\dots F'$$

$$(ABCD\dots F)' = A' + B' + C' + D' + \dots + F'$$

این تئوری بیان می‌دارد که متمم یک تابع با تعویض عملگرهای AND و OR و متمم کردن هر لیترال حاصل می‌شود.

مثال ۱: متمم توابع $F_1 = x'y'z' + x'y'z$ و $F_2 = x(y'z' + yz)$ را به‌دست آورید. متمم‌ها را با اعمال هر تعداد تئوری دموگران به‌صورت زیر به‌دست آورید:

$$\begin{aligned} F'_1 &= (x'y'z' + x'y'z)' \\ &= (x'y'z')'(x'y'z)' \\ &= (x + y' + z)(x + y + z') \end{aligned}$$

$$\begin{aligned} F'_2 &= [x(y'z' + yz)]' \\ &= x' + (y'z' + yz)' \\ &= x' + (y'z')'(yz)' \\ &= x' + (y + z)(y' + z') \end{aligned}$$

گیت‌های منطقی، جبر بول و توابع بولی ۶۳

روال ساده‌تری برای به‌دست آوردن متمم یک تابع این است که دوگان تابع و متمم هر لیترال به‌دست آید. این روش با توجه به فرم عمومی تئوری دموگان نتیجه می‌شود. به خاطر داشته باشید که دوگان یک تابع با تبدیل عملگر AND به OR و تبدیل ۱ها و 0ها به یکدیگر به‌دست می‌آید.

مثال ۲: متمم توابع F_1 و F_2 مثال ۲-۲ را با استفاده از دوگان‌ها و متمم‌های هر لیترال به‌دست آورید.

$$1) F_1 = x'yz' + x'y'z'$$

دوگان تابع F_1 برابر است با: $(x' + y + z')(x' + y' + z')$

متمم هر لیترال برابر است با: $(x + y' + z)(x + y + z') = F'_1$

$$2) F_2 = x(yz' + yz)$$

دوگان تابع F_1 برابر است با: $x + (y' + z')(y + z)$

متمم هر لیترال برابر است با: $x' + (y + z)(y' + z') = F'_2$

۲-۳-۲ سایر اعمال منطقی

وقتی که عملگرهای AND و OR بین دو متغیر x, y قرار می‌گیرند، به ترتیب دو تابع بولی $x + y$, $x \cdot y$ را تشکیل می‌دهند. قبلاً بیان شد که برای n متغیر 2^{2^n} تابع دودویی وجود دارد. برای دو متغیر، $n=2$ و تعداد توابع بولی ممکن 16 است. بنابراین توابع AND و OR تنها دو تابع از 16 تابع ممکن با دو متغیر دودویی هستند. جدول درستی 16 تابع که با دو متغیر دودویی x, y تشکیل گردیده در جدول زیر لیست شده است. هر یک از 16 ستون F_0, F_{15} ، جدول درستی یک تابع ممکن برای دو متغیر x, y را نشان می‌دهد. توجه داشته باشید که توابع از 16 ترکیب ممکن تخصیص یافته به F معین می‌گردند. 16 تابع را می‌توان با توابع بول نشان داد (شکل ۲-۹).

x	y	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13	F14	F15
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

شکل ۲-۹: جدول درستی برای ۱۶ تابع از دو متغیر دودویی

اگر چه هر تابع را می‌توان بر حسب عملگرهای دودویی NOT, OR, AND بیان کرد، اما دلیلی وجود ندارد که کسی عملگر خاصی را برای بیان سایر توابع تعیین ننماید. چنین عملگرهایی در ستون دوم جدول شکل ۲-۱۰ لیست شده‌اند. با این وجود همه سمبل‌های جدید که در جدول نشان داده شده‌اند، بجز OR انحصاری ($\oplus$) کاربرد چندانی به وسیله طراحان ندارند.

به دنبال هر یک از توابع در جدول زیر، نام و توضیحی که تابع را به نحوی تشریح می‌کند، آورده شده است. ۱۶ تابع لیست شده فوق به سه گروه زیر تقسیم می‌شوند.

توضیحات	نام	سمبل عملگر	توابع بول
Binary constant 0	Null	.	$F_0=0$
x and y	AND	$x.y$	$F_1=xy$
x, but not y	Inhibition	x/y	$F_2=xy'$
x'	Transfer	.	$F_3=x$
y, but not x	Inhibition	y/x	$F_4=x'y$
y	Transfer	.	$F_5=y$
x or y, but not both	Exclusive-OR	$x \oplus y$	$F_6=xy' + x'y$
x or y	OR	$x+y$	$F_7=x+y$
Not-OR	NOR	$x \downarrow y$	$F_8=(x+y)'$
x equals y	Equivalence	$(x \oplus y)'$	$F_9=xy+x'y'$
Not y	Complement	y'	$F_{10}=y'$
If y, then x	Implication	xy	$F_{11}=x+y'$
Not x	Complement	x'	$F_{12}=x'$
If x, than y	Implication	xy	$F_{13}=x'+y$
Not-AND	NAND	$x \uparrow y$	$F_{14}=(xy)'$
Binary constant 1	Identity	.	$F_{15}=1$

شکل ۲-۱۰: عبارات بولی ۱۶ تابع تعریف شده در شکل ۲-۹

۱- دو تابع که ثابت‌های ۱, ۰ را تولید می‌کنند (Identity, Null)

گیت‌های منطقی، جبر بول و توابع بولی ۶۵

۲- چهار تابع یکانی از نوع متمم^۱ و انتقال^۲

۳- ده تابع باقیمانده شامل هشت عمل مختلف به شرح زیر می‌باشد:

- AND
- OR
- NAND
- NOR
- XOR (یا OR انحصاری)
- XNOR (یا هم ارزی)
- نهی^۳
- استلزام (استنباط)^۴

ثابت‌ها برای توابع دودویی فقط می‌توانند 0 یا 1 باشند. تابع متمم، متمم هر متغیر دودویی را تولید می‌نمایند. تابعی که برای یک متغیر ورودی است را انتقال^۵ می‌نامند، زیرا متغیر x یا y از طریق یک گیت بدون تغییر مقدار عبور کرده است. از هشت عملگر دودویی، دوتای آنها (نهی و استلزام) به وسیله طراحان مدارات منطقی به کار می‌روند، ولی به ندرت در منطق کامپیوتر از آنها استفاده می‌شود. عملگرهای AND و OR قبلاً در جبر بول ذکر شدند. چهار تابع دیگر به طور گسترده در طراحی دستگاه‌های دیجیتال مورد استفاده‌اند.

تابع NOR متمم OR بوده و نام آن از NOT-OR اخذ شده است. به طور مشابه NAND، متمم AND است و از NOT-AND مشتق می‌شود. OR انحصاری یا XOR

¹ _ Complement

² _ Transfer

³ Inhibition

⁴ Implication

⁵ Transfer - Buffer

مشابه با OR است ولی حالتی که در آن هر دو متغیر y, x متفقا برابر 1 باشند، را شامل نمی‌شود. تابع XNOR یا هم ارزی تابعی است که هنگام مساوی بودن دو متغیر برابر 1 می‌شود، یعنی وقتی هر دو 0 یا هر دو 1 باشند. توابع XOR و XNOR متمم یکدیگرند و این خاصیت بسادگی با ملاحظه جدول شکل ۲-۹ قابل تشخیص است. جدول درستی برای OR عبارت است از F6 و برای XNOR نیز F9 است. این دو تابع متمم یکدیگرند. به این دلیل تابع هم ارزی را NOR انحصاری هم می‌گویند و با XNOR نشان می‌دهند.

جبر بول دو عملگر دودویی با نام های AND , OR و یک عملگر یکانی با نام NOT (متمم) دارد. ما با توجه به تعاریف، برخی از خواص آنها را استنتاج نمودیم و در این بخش تعدادی از عملگرهای دودویی دیگر را برحسب آنها معرفی کردیم. این روال منحصر به فرد نیست. به عنوان مثال می‌توانستیم ابتدا NOR ($\downarrow$) را تعریف کرده و سپس AND , OR , NOT را بر حسب آنها تعریف کنیم. به هر حال دلایل مکفی برای روش منتخب وجود دارد و در واقع مفاهیم AND , OR , NOT بیشتر بین مردم مصطلح بوده و بر تفکرات حاکم هستند. علاوه بر آن اصول هانتینگتون منعکس کننده طبیعت دوگانی این جبر است و این خود بر خاصیت تقارن (+) و (.) نسبت به یکدیگر دلالت دارد.

۲-۴ گیت‌های منطقی دیجیتال

- چون توابع بول بر حسب عملگرهای AND , OR , NOT بیان شده‌اند، پیاده کردن آنها با استفاده از اینگونه گیت‌ها ساده‌تر خواهد بود. امکان ساخت گیت‌ها برای دیگر اعمال منطقی در عمل مورد توجه است. فاکتورهایی که باید به هنگام ساخت آنها در نظر گرفته شوند عبارتند از:
- امکان سنجی و اقتصادی بودن روش ساخت به هنگام استفاده از قطعات فیزیکی
- امکان گسترش ورودی گیت‌ها به بیش از دو

گیت‌های منطقی، جبر بول و توابع بولی ۶۷

- در نظر گرفتن خواص اصلی عملگرهای دودویی مثل جابجایی و شرکت‌پذیری
- توانایی گیت در پیاده‌سازی توابع به تنهایی یا همراه با سایر گیت‌ها

از شانزده تابع معرفی شده در قسمت قبل، دو تابع برابر با مقدار ثابت و چهار تای دیگر دوبار تکرار شده‌اند. بنابراین تنها ده تابع برای تهیه گیت‌های منطقی کاندید هستند. دو تابع نهی و استلزام دارای خاصیت جابجایی یا شرکت‌پذیری نیستند و لذا به عنوان گیت‌های منطقی استاندارد مورد استفاده نمی‌باشند.

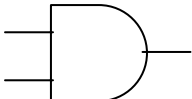
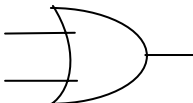
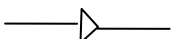
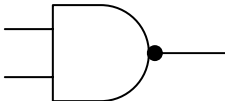
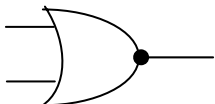
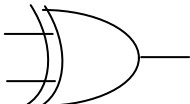
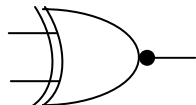
هشت تابع دیگر یعنی: Buffer, NOT, AND, OR, NAND, NOR, XOR و XNOR به عنوان گیت‌های استاندارد در طراحی سیستم‌های دیجیتال به کار می‌روند.

سمبل‌های گرافیکی و جدول درستی هشت گیت فوق در شکل ۲-۱۱ نشان داده شده‌اند. هر گیت موجود در شکل، دارای دو متغیر ورودی دودویی x, y و یک متغیر خروجی دودویی F می‌باشد. مدارهای AND, OR, NOR از قبل تعریف شده بودند.

مدار NOT یا وارون‌گر وضعیت منطقی یک متغیر دودویی را معکوس می‌نماید و در واقع متمم متغیر را تولید می‌کند. دایره کوچک در خروجی سمبل گرافیکی یک وارون‌گر (که به آن حباب می‌گویند) بیانگر متمم شدن است.

سمبل مثبت به تنهایی علامت بافر می‌باشد. یک بافر عمل انتقال را انجام می‌دهد، ولی یک عمل منطقی تولید نمی‌کند زیرا مقدار دودویی خروجی برابر مقدار ورودی دودویی است. این مدار صرفاً در تقویت توان سیگنال‌ها استفاده شده و معادل با دو مدار متوالی وارون‌گر (معکوس‌گر) است.

تابع NAND متمم AND است و همانطور که از سمبل گرافیکی آن مشخص است از یک سمبل AND و یک حباب تشکیل شده است.

نام	سمبل گرافیکی	تابع جبری	جدول درستی															
AND		$F=XY$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F=x+y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F=x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
NAND		$F=(xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F= (x+y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Exclusive-OR (XOR)		$F= xy' + x'y = x\oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Exclusive-NOR or equivalence		$F=xy+ x'y'=(x\oplus y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

شکل ۲-۱۱: گیت‌های منطقی به همراه مشخصات و جدول درستی

تابع NOR هم متمم OR است و با یک سمبل OR و به دنبال آن یک حباب نمایش داده می‌شود.

گیت‌های منطقی، جبر بول و توابع بولی ۶۹

گیت‌های NAND و NOR به‌طور گسترده‌ای به عنوان گیت‌های استاندارد مورد استفاده قرار گرفته و بیشتر OR و AND مورد توجه اند. این بدان علت است که گیت‌های NAND و NOR به سادگی به وسیله مدارات ترانزیستوری قابل تولید بوده و می‌توان به راحتی توابع بول را با آنها پیاده‌سازی کرد.

گیت NOR دارای سمبل مشابهی با OR است، بجز اینکه یک خط منحنی در سمت ورودی‌اش کشیده شده است. گیت XNOR متمم XOR است و لذا حباب کوچکی در خروجی آن وجود دارد.

۲-۴-۱ گسترش ورودی گیت‌ها

گیت‌هایی که در شکل ۲-۱۱ نشان داده شدند، بجز برای وارون‌گر و انتقال، قابل گسترش به بیش از دو ورودی می‌باشند. اگر عمل دودویی یک گیت جابجا و شرکت‌پذیر باشد، می‌توان ورودی‌های آن را گسترش داد. اعمال AND و OR که در جبر بول تعریف شده‌اند این خاصیت را از خود به نمایش گذاشته اند. برای تابع OR داریم:

$$x + y = y + x \quad (\text{جابجایی})$$

و

$$(x+y) + z = x + (y + z) = x + y + z \quad (\text{شرکت‌پذیر})$$

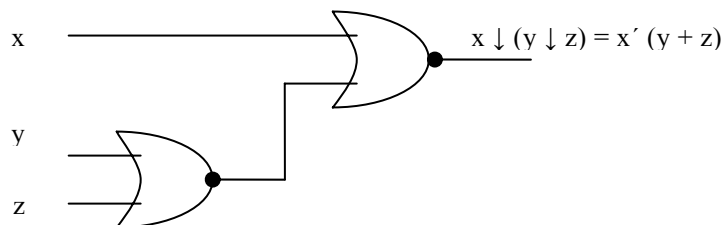
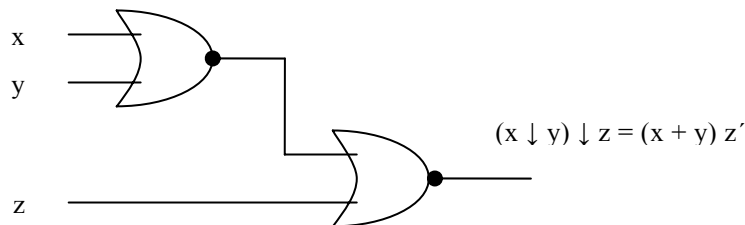
این روابط بیانگر تعویض پذیری ورودی‌های گیت و قابل گسترش بودن متغیرهای ورودی به بیش از دو در تابع OR است.

توابع NAND و NOR

توابع NAND و NOR جابجا پذیرند و ورودی آنها می‌تواند به بیش از دو افزایش یابد، مشروط بر این که در تعریف تابع مختصر تغییری صورت گیرد. مشکل این است که NAND و NOR شرکت‌پذیر نیستند. یعنی:

$$[(x \downarrow y) \downarrow z] \neq x(y \downarrow z)$$

این نکته در شکل ۱۲-۲ و معادلات زیر مشاهده می‌گردد:



شکل ۱۲-۲ شرکت ناپذیری عملگر NOR - $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$

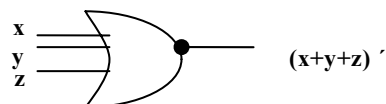
$$(x \downarrow y) \downarrow z = [(x + y)' + z]' = (x + y)z' = xz' + yz'$$

$$x \downarrow (y \downarrow z) = [x + (y + z)']' = x'(y + z) = x'y + x'z$$

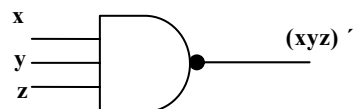
برای غلبه بر این مشکل، گیت NOR (یا NAND) چند ورودی را به عنوان متمم

OR (یا AND) آن تعریف می‌کنیم. بنابراین:

$$x \downarrow y \downarrow z = (x + y + z)'$$



$$x \uparrow y \uparrow z = (xyz)'$$



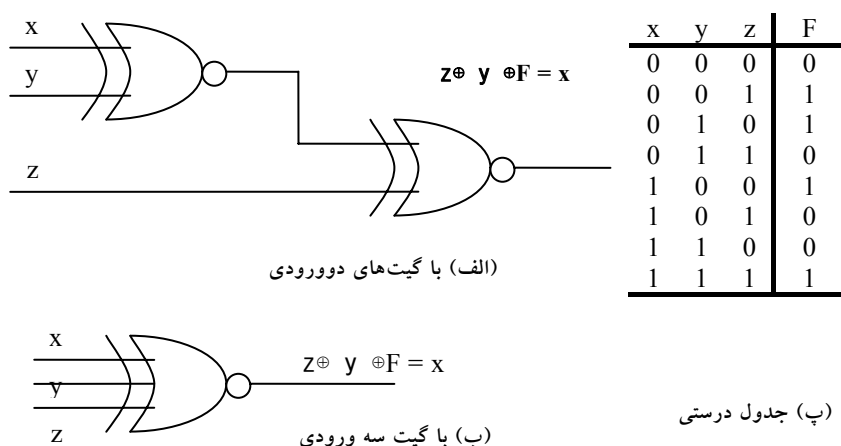
سمبل‌های گرافیکی گیت‌های سه ورودی در شکل ۱۳-۲ نشان داده شده‌اند. در

نوشتن متوالی اعمال NAND, NOR باید پرانتزها به فرم صحیحی انتخاب شوند تا بیانگر

ترتیب صحیح گیت‌ها باشند.

گیت‌های XOR و XNOR

XOR و XNOR هر دو خواص جابجایی و شرکت‌پذیری را دارند و ورودی‌هایشان قابل توسعه به بیش از دو می‌باشد. با این وجود گیت‌های XOR چند ورودی از نقطه نظر سخت‌افزاری متداول نیستند. در واقع حتی فرم دو ورودی آن نیز معمولاً از سایر



شکل ۲-۱۳: گیت XOR

گیت‌ها ساخته می‌شود. علاوه بر این، تعریف این توابع باید به هنگام گسترش ورودی‌ها تصحیح گردد. تابع XOR یک تابع فرد است یعنی هرگاه ورودی‌ها تعداد فردی 1 داشته باشند، این تابع (خروجی) برابر 1 خواهد بود. ساختمان یک گیت XOR با سه ورودی در شکل ۲-۱۳ دیده می‌شود.

این مدار معمولاً با گیت‌های دو ورودی تهیه می‌شود، شکل ۲-۱۳ (الف). به صورت گرافیکی، آن را می‌توان با یک گیت سه ورودی نشان داد، شکل ۲-۱۳ (ب). جدول درستی (پ) آشکارا مشخص می‌نماید که خروجی F برابر 1 است به شرطی که فقط یکی از ورودی‌ها و یا هر سه ورودی برابر 1 باشند؛ یعنی وقتی تعداد کل‌ها در متغیرهای ورودی فرد است، تابع 1 است.

۲-۴-۲ مدارهای مجتمع

با گسترش علم الکترونیک و طراحی مدارات ترکیبی و پیچیده نیاز به طراحی بسته‌های یکپارچه و کوچکتر از مدارها بیش از پیش احساس می‌شد. یک مدار مجتمع (IC) یک کریستال نیمه‌هادی از جنس سیلیکان است که به آن تراشه می‌گویند و حاوی اجزاء الکترونیکی در ساخت گیت‌های دیجیتال می‌باشد و انواع گیت‌ها در داخل تراشه به هم وصل می‌شوند تا مدار مورد نیاز ایجاد گردد. تراشه روی یک محفظه سرامیک یا پلاستیک نصب شده و اتصالات به پایه‌های بیرون برای ایجاد مدار مجتمع، متصل می‌گردند. تعداد پایه‌ها ممکن است گاه چند هزار در یک بسته بزرگ برسد. در روی هر IC یک شماره برای شناسایی چاپ می‌شود.

۲-۴-۲-۱ سطوح مجتمع سازی

IC های دیجیتال اغلب بر اساس پیچیدگی مدار درونی‌شان که به تعداد گیت‌های منطقی مرتبط است دسته بندی می‌شوند. تفکیک تراشه‌هایی که تنها چند یا چند صد و یا چندین هزار گیت دارند با ارجاع به بسته و دسته بندی آنها به وسایل مجتمع با فشردگی کم، متوسط، زیاد و خیلی زیاد صورت می‌گیرد.

مدارهای مجتمع با فشردگی کم (SSI) دارای چند گیت مستقل در بسته اند. ورودی‌ها خروجی‌های گیت‌ها مستقیماً به پایه‌های بسته متصل می‌شوند. تعداد گیت‌ها معمولاً کمتر از 10 بوده و به وسیله پایه‌های موجود در IC محدود می‌گردند.

مدارهای مجتمع با فشردگی متوسط (MSI) دارای فشردگی بین 10 تا 1000 گیت در یک بسته دارند. این دسته از مدارات معمولاً اعمال دیجیتال خاصی را اجرا می‌کنند. توابع دیجیتال MSI با عناوین دیگرها، جمع کننده‌ها و مولتی‌پلکسرها در فصل ۴ و ثبات‌ها و شمارنده‌ها در فصل ۶ مطرح شده‌اند.

گیت‌های منطقی، جبر بول و توابع بولی ۷۳

مدارهای مجتمع با فشردگی زیاد (LSI) حاوی هزاران گیت در یک بسته می‌باشند. این دسته از مدارات، پردازنده‌ها، حافظه‌ها و مدارات منطقی برنامه پذیر را شامل می‌شوند. بعضی از اجزا LSI در فصل ۸ معرفی شده‌اند.

مدارهای مجتمع با فشردگی خیلی زیاد (VLSI) صدها هزار گیت در یک بسته دارند. از جمله مثالها می‌توان از آرایه‌های حافظه، تراشه میکرو کامپیوتر های پیچیده نام برد. به دلیل کوچکی و ارزانی، وسایل VLSI تکنولوژی طراحی سیستم کامپیوتر را متحول ساخته و به طراح قابلیت ساخت وسایلی را می‌دهد که قبلاً اقتصادی نبودند.

۲-۲-۴ منطق مثبت و منفی

سیگنال دودویی در ورودی‌ها یا خروجی هر گیت، بجز در حالت گذرا، یکی از دو مقدار را دارد. یک مقدار سیگنال، منطق 1 و دیگری منطق 0 را نمایش می‌دهد. چون دو مقدار سیگنال متعلق به دو ارزش منطقی است، لذا دو انتساب متفاوت برای دو ارزش منطقی می‌توان اختیار کرد، شکل ۲-۱۴ سطح سیگنال بالاتر با H و سطح سیگنال

مقدار سیگنال	مقدار منطقی	مقدار سیگنال	مقدار منطقی
H	1	H	1
L	0	L	0

(الف) منطق مثبت (ب) منطق منفی

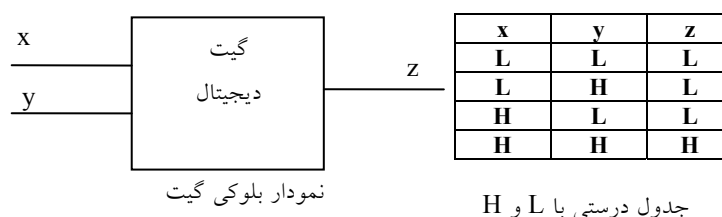
شکل ۲-۱۴: تخصیص سیگنال و قطبیت منطق

پایین‌تر با L مشخص شده است. اگر سطح بالا، H، برای منطق 1 به کار رود یک سیستم منطق مثبت تعریف شده است. انتخاب L برای منطق 1 سیستم منطقی منفی را معرفی می‌نماید.

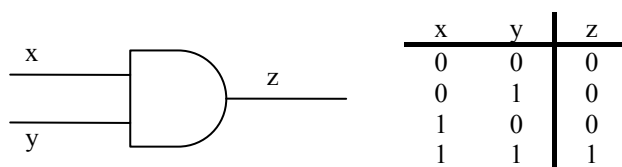
کلمات مثبت یا منفی گاهی گمراه کننده هستند زیرا هر دو سیگنال ممکن است مثبت یا منفی باشند. در واقع، این قطب های سیگنال نیستند که بیانگر نوع منطق

می‌باشند، بلکه انتخاب مقادیر منطقی بر حسب سطوح نسبی سیگنال‌ها نسبت به هم، نوع منطق را مشخص می‌کنند.

گیت‌های دیجیتال سخت‌افزاری بر حسب مقادیر سیگنال L , H تعریف می‌شوند. از این پس انتخاب منطق مثبت و منفی به عهده کاربر است. به عنوان مثال گیت الکترونیک شکل زیر را به همراه جدول درستی آن در نظر بگیرید:



این جدول رفتار فیزیکی گیت را وقتی H برابر $3V$ و L برابر 0 ولت است نشان می‌دهد. شکل ۱۵-۲ جدول درستی منطق مثبت را فرض می‌کند که در آن $H=1$ و $L=0$ است. این جدول درستی همانند جدول عمل AND با منطق مثبت در شکل زیر دیده می‌شود.

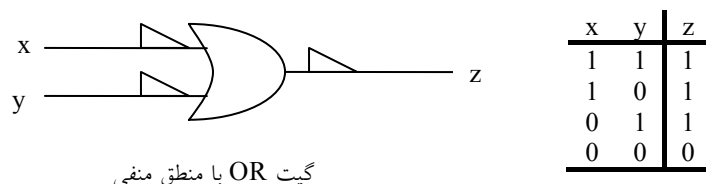


شکل ۱۵-۲: نمایش جدول درستی گیت AND با منطق مثبت

اکنون تخصیص منطق منفی را برای همان گیت فیزیکی با $L=1$ و $H=0$ در نظر بگیرید. نتیجه جدول درستی شکل زیر خواهد بود. گرچه داده‌ها معکوس شده‌اند ولی

گیت‌های منطقی، جبر بول و توابع بولی ۷۵

جدول عمل OR را نشان می‌دهد. سمبل گرافیکی گیت OR با منطق منفی در شکل ۱۶-۲ دیده می‌شود.



گیت OR با منطق منفی

شکل ۱۶-۲: نمایش جدول درستی گیت OR با منطق منفی

مثلت‌های کوچک در ورودی‌ها و خروجی نشانگر قطبیت هستند. وجود علائم قطبیت همراه با مشخصات پایانه بیانگر فرض منطق منفی برای سیگنال است. بنابراین گیت فیزیکی فوق می‌تواند یک گیت AND با منطق مثبت و نیز یک گیت OR با منطق منفی باشد. تبدیل منطق مثبت به منفی و بالعکس، عملی است که طی آن در ورودی و خروجی یک گیت 0 ها به 1 و 1 ها به 0 تبدیل می‌شوند. چون این عمل دوگان تابع را تولید می‌کند، تعویض پایانه‌ها از یک قطبیت به قطبیت دیگر نتیجه‌اش همان یافتن دوگان تابع است.

نتیجه این تبدیل این است که همه عملگرهای AND به OR و بالعکس تبدیل شوند. به علاوه نباید از ذکر مثلث در سمبل‌های گرافیکی که بیانگر قطبیت است در منطق منفی، فراموش کرد. در این کتاب از گیت‌ها با منطق منفی استفاده نمی‌کنیم و فرض خواهیم که همه گیت‌ها با منطق مثبت کار کنند.

۲-۴-۳ خانواده‌های منطقی دیجیتال

جدا از بحث پیچیدگی و عمل منطقی مدارهای مجتمع دیجیتال که باعث دسته‌بندی آنها نیز می‌گردد، این مدارات بر اساس تکنولوژی مدار خاصی که به آن تعلق دارند نیز دسته بندی می‌گردند. تکنولوژی مدار به نام خانواده مدار منطقی خوانده می‌شود. هر

خانواده منطقی دارای مدار الکترونیک مبنای خاص خود بوده و سایر توابع و مدارات پیچیده دیجیتال با استفاده از آنها ساخته می‌شوند. مدار مبنا در هر خانواده، گیت NAND، NOR یا NOT است. قطعات الکترونیک به کار رفته در ساخت مدار مبنا معمولاً برای نام‌گذاری تکنولوژی مورد استفاده قرار می‌گیرد. به لحاظ تجاری انواع متفاوتی از خانواده‌های منطقی مدارات مجتمع معرفی شده‌اند. انواع رایج آنها در زیر لیست شده‌اند.

emitter-coupled logic	ECL
transistor-transistor logic	TTL
metal-oxide semiconductor	MOS
complementary metal- oxide semiconductor	CMOS

ECL: در سیستم‌هایی که به سرعت بالا نیاز دارد ارجحیت دارد.

TTL: مدت مدیدی است که مورد استفاده بوده و به عنوان یک گیت استاندارد شناخته شده است.

MOS: در مدارهایی که نیاز به چگالی قطعه بالایی دارند مورد استفاده است.

CMOS: در مواقعی که توان مصرفی باید کم باشد مورد توجه می‌باشد.

نظر به اینکه توان مصرفی کم در طراحی VLSI از اصول است، CMOS تبدیل به یک خانواده منطقی غالب شده است در حالی که از کاربرد خانواده‌های ECL, TTL به تدریج کاسته می‌شود.

گیت‌های منطقی، جبر بول و توابع بولی ۷۷

سؤالات

۱- با استفاده از جدول درستی نشان دهید که گیت‌های NAND و NOR (هر یک با سه ورودی) متمم یکدیگر هستند یا خیر؟

۲- جداول درستی توابع ذیل را تهیه کنید.

$$F_1 = (x+y). (x'+z).(x+y'+z')$$

$$F_2 = x' + yz'$$

۳- متمم توابع زیر را به دست آورید.

$$F_1 = x' y z' + x' y'$$

$$F_2 = x (y' z' + y z)$$

$$F_3 = (xy' + z) x' z'$$

۴- با استفاده از جدول درستی نشان دهید که گیت‌های X-NOR و X-OR (هر یک با دو ورودی x و y) متمم یکدیگرند.

۵- نمودار منطقی عبارات ذیل را رسم نمایید.

$$F_1 = (x + y). (x' + y' + z)$$

$$F_2 = x + (y. z') + (x'. y'. z) + x' z'$$

فصل ۳

فرم‌های متعارف و استاندارد در جبر بولی

هدف کلی

در این فصل مباحث اصلی مربوط به استانداردسازی عبارات بولی و روش‌های حداقل‌سازی عبارات بولی به منظور کاهش هزینه ساخت گیت‌ها مورد بحث و بررسی قرار خواهد گرفت. تهیه عبارات متعارف و استاندارد به عنوان هدف اصلی این فصل می‌باشد.

هدف ساختاری

در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند:

- فرم‌های استاندارد
- جمع حاصل ضرب‌ها
- ضرب حاصل جمع‌ها
- مفاهیم فرم‌های متعارف
- حداقل‌سازی سطوح گیت‌ها
- جمع مینترم‌ها
- حاصل ضرب ماکسترم‌ها
- تبدیل فرم‌های متعارف به یکدیگر

۳-۱ فرم‌های استاندارد

همانطور که می‌دانید طراحان قادر هستند بر اساس نیاز خود اقدام به تعریف توابع بولی متنوع نمایند که این توابع بولی می‌توانند بر اساس دو عملگر (+) و (.) دسته‌بندی شوند. با گسترش دامنه استفاده از توابع بولی در ساختن مدارات منطقی دیجیتال، طراحان را با شرایطی مواجه کرد که در آن توابع دارای حداقل متغیرها نبودند. لذا لزوم داشتن استاندارد نگارش عبارات بولی بیش از پیش احساس شد. بر این اساس محققین با ارائه الگوهایی، سعی کردند روش‌های استاندارد را برای نوشتن عبارات بولی طراحی نمایند. در این فرم جمله‌هایی که تابع را تشکیل می‌دهند ممکن است یک، دو یا هر تعدادی از متغیرها را دارا باشند. دو نوع فرم استاندارد وجود دارد:

- جمع حاصل ضرب‌ها
- ضرب حاصل جمع‌ها

۳-۱-۱ جمع حاصل ضرب‌ها

یک متغیر دودویی ممکن است به فرم معمولی (x) یا متمم (x') ظاهر شود. اکنون تصور کنید که دو متغیر دودویی x و y با عملگر AND با هم ترکیب شوند. چون هر متغیر ممکن است به هر یک از دو شکل فوق ظاهر گردد، چهار ترکیب برای آنها

مینترم‌ها				
علامت	جمله	z	y	x
m_0	$x'y'z'$	0	0	0
m_1	$x'y'z$	1	0	0
m_2	$x'yz'$	0	1	0
m_3	$x'yz$	1	1	0
m_4	$xy'z'$	0	0	1
m_5	$xy'z$	1	0	1
m_6	$x yz'$	0	1	1
m_7	$x yz$	1	1	1

شکل ۳-۱: جدول نمایش مینترم‌ها برای سه متغیر

فرم‌های متعارف و استاندارد در جبر بولی ۸۱

متصور است: $xy, xy', x'y, x'y'$. هر یک از این چهار جمله AND را یک مینترم یا یک جمله ضرب استاندارد گویند. به‌طور مشابه n متغیر را می‌توان ترکیب کرده و 2^n مینترم به وجود آورد. 2^n مینترم مختلف را می‌توان با روشی مشابه با آنچه در شکل ۳-۱ آمده، نشان داد.

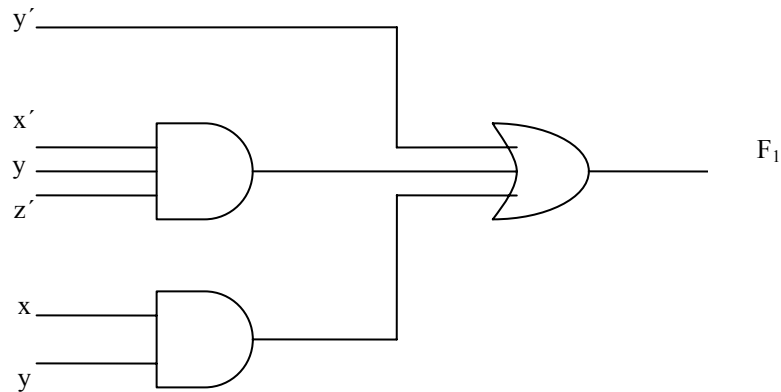
اعداد دودویی از صفر تا $2^n - 1$ زیر ستون n متغیر لیست شده‌اند. هر مینترم از AND تمام n متغیر به‌دست می‌آید که در آن هر متغیر پریم‌دار متعلق به بیت 0 و بدون پریم با 1 نشان داده می‌شود. سمبل هر مینترم نیز در جدول با m_j نشان داده شده است. که در آن J معادل دهدهی عدد دودویی مربوط به مینترم است.

جمع حاصل‌ضرب‌ها، یک عبارت بولی است شامل جملات AND که آنها را جملات ضرب می‌گوییم و هر یک دارای یک یا چند لیترا است. علامت جمع به معنی OR این جملات است.

مثالی از یک تابع به صورت جمع حاصل‌ضرب‌ها را در زیر ملاحظه نمایید.

$$F_1 = y' + xy + x'yz'$$

این عبارت سه جمله، با یک، دو و سه لیترا دارد. جمع آنها اثر OR را داراست.



شکل ۳-۲: مدار منطقی عبارت $F_1 = y' + xy + x'yz'$

نمودار منطقی جمع حاصلضربها متشکل از گروهی گیت AND است که بدنبال یک گیت OR می‌آید. الگوی این آرایش در شکل ۳-۲ آمده است. هر جمله ضرب نیاز به یک گیت AND دارد. این نکته در یک ورودی تک لیتراست مستثنی است. جمع منطقی با یک گیت OR صورت می‌گیرد که ورودی‌هایش خروجی گیت‌های AND و نیز تک ورودی مذکور است. ضمناً فرض بر این است که متمم متغیرهای ورودی مستقیماً موجودند بنابراین وارون‌گر در نمودار لحاظ نشده است. این آرایش را پیاده‌سازی دو سطحی یا دو طبقه می‌گویند.

۳-۱-۲ ضرب حاصل جمعها

همانند آنچه که در بالا گفته شد، n متغیر یک جمله OR تشکیل می‌دهند که هر متغیر ممکن است پریم‌دار یا بدون پریم باشد. 2^n ترکیب ممکن را ماکسترم یا جمع استاندارد گویند. هشت ماکسترم برای سه متغیر، همراه با سمبل آنها در جدول ۳-۳ لیست شده‌اند. هر 2^n ماکسترم برای n متغیر به طریق مشابهی حاصل می‌شود. هر ماکسترم از یک جمله OR با n متغیر به دست می‌آید که در آن متغیر پریم‌دار با 1 و بدون پریم با 0 نشان داده می‌شود. توجه کنید که هر ماکسترم، متمم می‌تترم مربوطه‌اش می‌باشد و بالعکس.

ماکسترم‌ها				
علامت	جمله	z	y	x
m_0	$x+y+z$	0	0	0
m_1	$x+y+z'$	1	0	0
m_2	$x+y'+z$	0	1	0
m_3	$x+y'+z'$	1	1	0
m_4	$x'+y+z$	0	0	1
m_5	$x'+y+z'$	1	0	1
m_6	$x'+y'+z$	0	1	1
m_7	$x'+y'+z'$	1	1	1

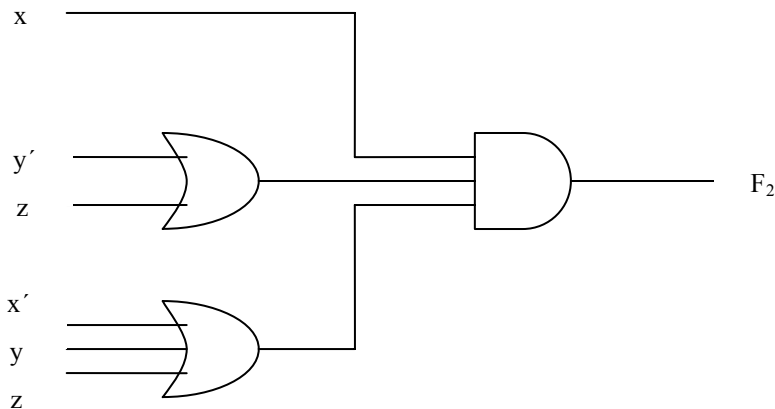
شکل ۳-۳: جدول نمایش ماکسترم‌ها برای سه متغیر

فرم‌های متعارف و استاندارد در جبر بولی ۸۳

ضرب حاصل جمع‌ها، یک عبارت بولی حاوی جملات OR است که به آن جملات جمع می‌گویند. هر جمله می‌تواند به هر تعداد لیترال داشته باشد. ضرب به معنی AND این جملات است.

مثالی از یک تابع به صورت ضرب حاصل جمع‌ها چنین است:

$$F_2 = x (y' + z) (x' + y + z')$$



شکل ۳-۴: مدار منطقی عبارت $F_2 = x (y' + z) (x' + y + z')$

این تابع دارای سه جمله جمع، با یک، دو و سه لیترال است. ضرب نیز یک عملگر AND می‌باشد. استفاده از لغات ضرب و جمع از شباهت عمل AND با ضرب حسابی و شباهت عمل OR با جمع حسابی مشتق شده است. ساختار گیتی ضرب حاصل جمع متشکل از گروهی گیت OR برای جملات جمع (به جز برای تک لیترال) و به دنبال آن یک گیت AND می‌باشد. این نکته در شکل ۳-۴ دیده می‌شود، این نوع استاندارد عبارت به یک ساختار دو سطحی (یا دو طبقه) از گیت‌ها منجر می‌گردد.

۳-۱-۳ مفهوم فرم‌های متعارف

یک تابع بول می‌تواند به صورت جبری با استفاده از جدول درستی و با تشکیل مینترم‌های هر ترکیب از متغیرهایی که برای تابع، 1 را تولید می‌کنند، و اجرای هر عملگر روی OR همه این جملات ایجاد شود.

مثلاً F_1 در جدول زیر با ترکیبات 001, 100, 111 به صورت $xyz, xy'z', x'y'z$ بیان می‌شود. چون هر یک از مینترم‌ها $F_1=1$ را ایجاد می‌نمایند پس:

$$F_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

به سادگی می‌توان نشان داد که:

$$F_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

x	y	z	F_1	F_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

این مثال‌ها خصوصیت مهمی از جبر بول را به نمایش می‌گذارند: یعنی هر تابع بولی را می‌توان به صورت جمع مینترم‌ها نشان داد "جمع به معنی OR جملات است".

اکنون متمم تابع بول را ملاحظه نمایید، می‌توان آن را با تشکیل مینترم‌هایی در جدول درستی که 0 تابع را تولید می‌کنند، ایجاد کرد و سپس آنها را OR نمود. متمم F_1 چنین است:

$$F'_1 = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

اگر متمم F'_1 را به دست آوریم تابع F_1 به دست خواهد آمد.

$$F_1 = (x+y+z) (x+y'+z) (x'+y+z') (x'+y'+z) = m_0.m_2.m_3.m_5.m_6$$

فرم‌های متعارف و استاندارد در جبر بولی ۸۵

به‌طور مشابه، می‌توان عبارت F_2 را از جدول به‌دست آورد:

$$F_2 = (x+y+z) (x+y+z') (x+y'+z) (x'+y+z) = m_0 m_1 m_2 m_4$$

این مثال‌ها نیز دومین خاصیت جبر بول را به نمایش می‌گذارند: هر تابع بول را می‌توان به‌صورت ضرب ماکسترم‌ها (ضرب به معنی AND جملات است) درآورد. روال تهیه ضرب ماکسترم‌ها مستقیماً از جدول درستی به فرم زیر میسر است. برای هر ترکیبی از متغیرها ماکسترم‌هایی که در تابع 0 تولید می‌کنند را تشکیل دهید، و سپس AND همه ماکسترم‌ها را به‌دست آورید. توابع بول که به‌صورت جمع مینترم‌ها یا ضرب ماکسترم بیان شوند را **فرم متعارف** نامند.

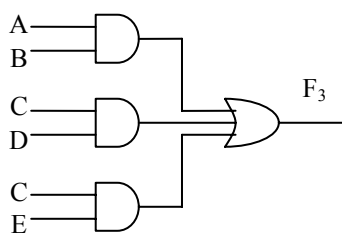
۳-۱-۴ حداقل سازی سطوح گیت

یک تابع بول ممکن است به‌صورت غیر استاندارد نیز بیان شود. مثلاً تابع F_3 که در زیر آمده است نه جمع حاصلضرب و نه ضرب حاصل جمع است.

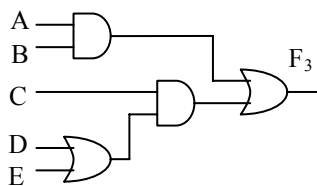
$$F_3 = AB + C (D + E)$$

پیاده‌سازی این عبارت در شکل ۳-۵ (الف) دیده می‌شود. این مدار به دو گیت AND و دو گیت OR نیاز دارد. در این مدار سه سطح گیت وجود دارد. می‌توان در آن با استفاده از اصل توزیع‌پذیری پرانتز را حذف و آن را به فرم استاندارد درآورد.

$$F_3 = AB + C (D + E) = AB + CD + CE$$



(ب) $AB + CD + CE$



(الف) $AB + C (D + E)$

شکل ۳-۵: پیاده‌سازی سه و دو سطحی

عبارت جمع حاصلضرب در شکل ۵-۳ (ب) پیاده شده است. به طور کلی، یک پیاده سازی دو سطحی ترجیح داده می شود. زیرا به هنگام انتشار ورودی ها به سمت خروجی ها حداقل مقدار تاخیر را در گیت تولید می کند.

۵-۱-۳ مجموع مینترم ها

قبلاً بیان شد که برای هر n متغیر دودویی 2^n مینترم مجزا وجود دارد و هر تابع بولی می تواند به صورت مجموعی از مینترم ها در آید. مینترمی که جمع آنها توابع بول را تعریف می کنند، آنهایی هستند که اهای تابع را در جدول درستی تشکیل می دهند. چون تابع در قبال هر مینترم می تواند 0 و یا 1 باشد، و چون 2^n مینترم وجود دارد، می توان تعداد توابع ممکن که با n متغیر ایجاد می شود را 2^{2^n} دانست. گاهی بهتر است تابع بول را بر حسب جمع مینترم ها بیان کرد. اگر در این فرم نبود، می توان ابتدا آن را به صورت جمع جملات AND در آورد. آنگاه هر ترم برای یافتن همه متغیرها در آن واریسی می شود. اگر یک یا چند متغیر وجود نداشته باشند، می توان جمله را در عبارتی مثل $x+x'$ AND نمود، که x یکی از متغیرهای مفقود شده است. مثال زیر مطلب را روشن می کند.

مثال ۱: تابع بولی $F=A+B'C$ را به صورت جمع مینترم ها در آورید. تابع سه متغیر A, B, C دارد. در اولین جمله A ، دو متغیر مفقود است؛ بنابراین:

$$A = A(B+B') = AB + AB'$$

این تابع هنوز هم یک متغیر کسر دارد

$$\begin{aligned} A &= AB(C+C') + AB'(C+C') \\ &= ABC + ABC' + AB'C + AB'C' \end{aligned}$$

جمله دوم $B'C$ یک متغیر کم دارد

$$B'C = B'C(A+A') = AB'C + A'B'C$$

با ترکیب همه جملات داریم:

فرم‌های متعارف و استاندارد در جبر بولی ۸۷

$$F = A + B'C$$

$$= ABC + ABC' + AB'C + AB'C' + A'B'C$$

دیده می‌شود که $AB'C$ دوباره تکرار شده است و بر حسب تئوری ($x+x = x$) می‌توان یکی از آنها را حذف کرد. با مرتب نمودن میترم‌ها به ترتیب صعودی داریم:

$$F = A'B'C + AB'C + AB'C + ABC' + ABC$$

$$= m_1 + m_4 + m_5 + m_6 + m_7$$

گاهی بهتر است تابع بول را وقتی به صورت جمع میترم‌هاست به فرم خلاصه زیر نشان دهیم:

$$F(A,B,C) = \Sigma(1,4,5,6,7)$$

سمبل جمع Σ به معنی OR جملات است. اعدادی که به دنبال آن می‌آیند نیز میترم‌های تابع هستند. حروف داخل پرانتز در جلو F ، لیستی از متغیرهای تشکیل دهنده جملات میترم را نشان می‌دهند. روش دیگری برای تشکیل میترم‌های تابع بول تهیه مستقیم جدول درستی تابع از عبارت جبری و سپس خواندن میترم‌ها از جدول درستی است. تابع بول زیر را در نظر بگیرید:

$$F = A + B'C$$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

شکل ۳-۶: جدول درستی برای $F = A + B'C$

جدول درستی در جدول ۳-۶ مستقیماً از عبارت جبری با لیست هشت ترکیب زیر متغیرهای A, B, C و اعمال 1 زیر ستون F برای ترکیباتی که در آن $A=1$ و $BC=01$

است، فراهم شده است. سپس از جدول درستی می‌توان مشاهده کرد که میترم‌های تابع، جملات 1، 4، 5، 6 و 7 می‌باشند.

۳-۱-۶ ضرب ماکسترم‌ها

هر یک از 2^{2^n} تابع متشکل از n متغیر را می‌توان به صورت ضرب ماکسترم‌ها نیز بیان داشت. برای بیان توابع بول به عنوان ضرب ماکسترم‌ها، ابتدا باید جملات OR را تشکیل دهیم. این کار را می‌توان با استفاده از قانون توزیع پذیری انجام داد. سپس هر متغیر مفقود در هر جمله OR با xx' می‌شود. روش با مثال زیر روشن تر خواهد شد.

$$x + (yz) = (x+y)(x+z)$$

مثال ۲: تابع بول $F = xy + x'z$ را به صورت ضرب جملات ماکسترم نشان دهید.

ابتدا تابع را با استفاده از اصل توزیع پذیری به فرم جملات OR در آورید:

$$\begin{aligned} F = xy + x'z &= (xy + x')(xy + z) \\ &= (x+x')(y+x')(x+z)(y+z) \\ &= (x'+y)(x+z)(y+z) \end{aligned}$$

تابع سه متغیر دارد: x, y, z . هر جمله فاقد یک متغیر است؛ بنابراین

$$\begin{aligned} x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\ x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\ y + z &= y + z + xx' = (x + y + z)(x' + y + z) \end{aligned}$$

با ترکیب همه جملات و حذف تکراری‌ها، خواهیم داشت:

$$\begin{aligned} F &= (x+y+z)(x+y'+z)(x'+y+z)(x'+y+z') \\ &= M_0 M_2 M_4 M_5 \end{aligned}$$

نمایش ساده تر به شکل زیر است:

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

فرم‌های متعارف و استاندارد در جبر بولی ۸۹

سمبل ضرب، Π ، بیانگر AND ماکسترم‌هاست. اعداد داخل پرانتز شماره ماکسترم‌های تابع‌اند.

۲-۳ تبدیل فرم‌های متعارف به یکدیگر

متمم یک تابع که به صورت مجموع مینترم‌ها نشان داده شده برابر است با مجموع مینترم‌هایی که در تابع اصلی وجود ندارند. دلیل این است که تابع اصلی با آن دسته از مینترم‌ها بیان شده است که تابع را 1 می‌کنند، در صورتی که متمم آن در ازاء مینترم‌هایی 1 می‌شود که تابع را 0 نموده‌اند. به عنوان مثال تابع زیر را در نظر بگیرید:

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

متمم این تابع به شکل زیر است:

$$F'(A, B, C) = \Sigma(0, 2, 3) = m_0 + m_2 + m_3$$

اکنون اگر متمم F' را با روش تئوری دمورگان به دست آوریم، F را به فرم متفاوتی خواهیم داشت:

$$F = (m_0 + m_2 + m_3)' = m'_0 \cdot m'_2 \cdot m'_3 = M_0 M_2 M_3 = \Pi(0, 2, 3)$$

آخرین تبدیل در رابطه فوق از تعریف مینترم‌ها و ماکسترم‌ها در جدول (۲-۳) حاصل می‌شود. با توجه به جدول درستی رابطه زیر معتبر است:

$$m'_j = M_j$$

یعنی ماکسترم M_j ، متمم مینترم m_j است و بالعکس.

آخرین مثال تبدیل یک تابع مینترمی به معادل ماکسترمی را نشان می‌دهد. بحث مشابهی نشان می‌دهد که تبدیل ضرب ماکسترم‌ها به جمع مینترم‌ها نیز به طریق فوق است. اکنون یک روال کلی را بیان می‌کنیم. برای تبدیل یک فرم متعارف به فرم متعارف دیگر، سمبل‌های Σ و Π را باهم عوض کنید و شماره‌های مفقود شده را از فرم اصلی تابع، لیست نمایید. برای یافتن جملات مفقود، باید بدانیم که تعداد کل جملات 2^n است، که در آن n تعداد متغیرهای دودویی در تابع می‌باشد.

یک تابع بولی می‌تواند از یک عبارت جبری به کمک جدول درستی و روال تبدیل متعارف به ضربی از ماکسترم‌ها تبدیل شود. به عنوان مثال عبارت بولی زیر را ملاحظه نمایید.

$$F = xy + x'z$$

ابتدا جدول درستی تابع را طبق جدول ۳-۷ به دست می‌آوریم. ۱های زیر ستون F از ترکیب $xy = 11$ یا $xz = 01$ به دست می‌آیند. مینترم‌های تابع در جدول درستی شماره‌های 1، 3، 6 و 7 می‌باشند. تابع بر حسب مجموع مینترم‌ها چنین است:

$$F(x, y, z) = \Sigma (1, 3, 6, 7)$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

شکل ۳-۷: جدول درستی برای تابع $F = xy + xz$

چون جمعاً در یک تابع از سه متغیر هشت مینترم یا ماکسترم وجود دارد، جملات مفقود عبارتند از 0، 2، 4 و 5. تابعی که بر حسب ضرب ماکسترم‌ها بیان شود برابر زیر است:

$$F(x, y, z) = \Pi (0, 2, 4, 5)$$

سؤالات

۱- تابع بولی زیر را به صورت جملات جمع ماکسترم نشان دهید.

$$F = xy + x'z + y'z'$$

۲- متمم توابع زیر را به صورت جمع میترم‌ها بنویسید.

$$F(x,y,z) = \Pi(0,1,5,7)$$

$$F(x,y,z,w) = \Pi(0,2,4,11,14)$$

$$F(x,y,z) = \Sigma(1,4,5,6,7)$$

$$F(x,y,z,w) = \Sigma(0,3,5,9,12,13)$$

۳- اگر تابع $F_1(x,y,z)$ به صورت زیر باشد، متمم تابع F_1 را به دست آورید.

$$F_1(x,y,z) = M_0.M_2.M_5$$

۴- تابع ذیل را با حداقل تعداد سطح پیاده سازی و نمودار منطقی آنرا رسم نمایید.

$$F_1 = AB' + C(D + E) + AD'$$

فصل ۴

ساده کردن عبارات بولی پیچیده

هدف کلی

در این فصل مباحث اصلی مربوط به ساده کردن عبارات بولی پیچیده با استفاده از منطق جدول کارنو مورد بحث و بررسی قرار خواهد گرفت. عناصر اصلی جدول کارنو بررسی شده و جدول‌های کارنو با تعداد دو الی پنج متغیر مورد بحث قرار خواهند گرفت.

هدف ساختاری

در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند

- ساده‌سازی با استفاده از نقشه کارنو
- عناصر اصلی جدول کارنو
- نقشه‌های دو الی پنج متغیره کارنو
- بررسی حالات بی‌اهمیت در جدول کارنو

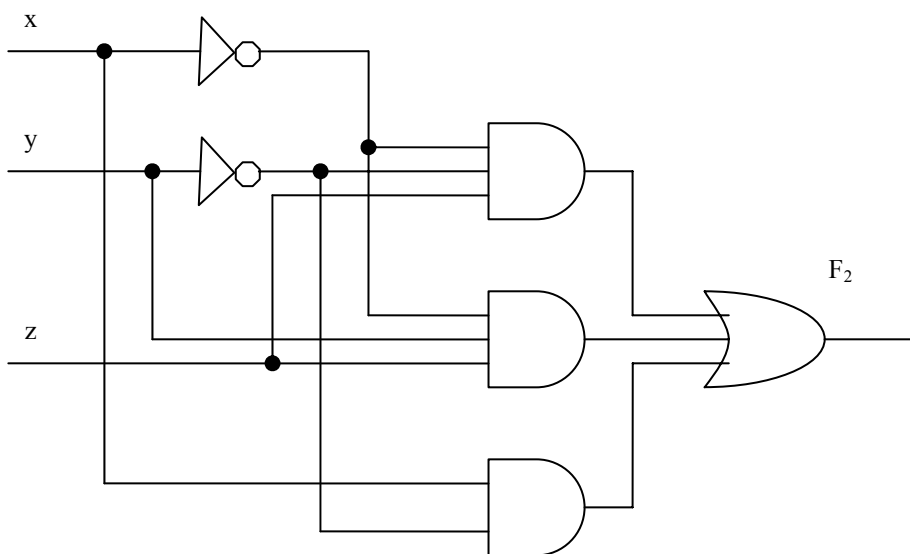
۴-۱ دستکاری جبری

گاهی اوقات ممکن است با دستکاری یک عبارت بولی توسط قوانین جبر بول، عبارت ساده‌تری برای یک تابع به دست آوریم. وقتی که یک عبارت بولی با گیت‌های منطقی پیاده‌سازی شود، هر جمله به یک گیت نیاز دارد و هر متغیر در جمله یک ورودی به

یک گیت است. بنابراین با ساده کردن عبارات، تعداد گیت‌ها در مدار و تعداد ورودی‌ها به هر گیت را کاهش دهیم. مثلاً تابع بولی زیر را در نظر بگیرید:

$$F_2 = x' y' z + x' y z + x y'$$

مدار منطقی این تابع که دارای سه جمله و هشت لیترال است که در شکل زیر نشان داده شده‌است. منظور از لیترال، یک متغیر تک در یک جمله‌است که ممکن است متمم شود یا نشود.



شکل ۴-۱: مدار منطقی سه سطحی تابع $F_2 = xy'z + x'yz + xy'$

آنگونه که مشاهده می‌کنید متغیرهای y, x به کمک وارون‌گر متمم شده‌اند تا x', y' به‌دست آیند. سه جمله در عبارت با سه گیت AND پیاده‌سازی شده‌اند. گیت OR نیز، OR منطقی سه جمله را فراهم می‌سازد. جدول درستی تابع F_2 در زیر آمده‌است:

اغلب در تابع بول با کاهش تعداد جملات، تعداد لیترال‌ها، یا هر دو مدار ساده‌تری حاصل می‌شود. هدف از دستکاری جبر بول غالباً کاهش یک عبارت به منظور دستیابی

۹۵ ساده کردن عبارات بولی پیچیده

به یک مدار ساده تر است. اکنون ساده سازی ممکن برای تابع را با اعمال بعضی از

x	y	z	F ₂
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

شکل ۴-۲: جدول درستی تابع F₂

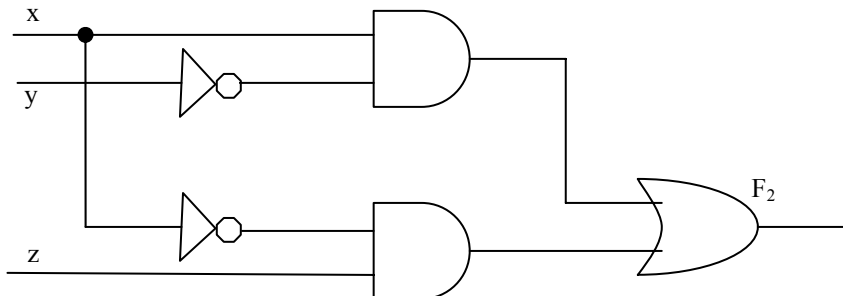
ویژگی های جبر بول ملاحظه کنید:

$$\begin{aligned} F_2 &= x'y'z + x'yz + xy' \\ &= x'z(y' + y) + xy' \\ &= x'z + xy' \end{aligned}$$

همانگونه که مشاهده می کنید تابع جدید دو جمله و چهار لیترال دارد.

$$F_2 = x'z + xy'$$

مدار منطقی عبارت ساده شده به صورت زیر می باشد:



شکل ۴-۳: مدار منطقی تابع F₂ به صورت ساده شده

تابع تنها به دو جمله کاهش یافته و قابل پیاده سازی با گیت مطابق شکل ۴-۳ است. بدیهی است که این مدار از مدار اولیه ساده تر می باشد، ولی هر دو یک تابع را پیاده سازی می کنند. تساوی دو عبارت را می توان به کمک جدول درستی هم تحقیق

کرد. عبارت ساده شده، وقتی $xz=01$ یا $xy=10$ باشد، برابر 1 است. این تابع هم همان چهار 1 را در جدول تولید می‌کند. چون هر دو عبارت جدول درستی یکسانی را تولید می‌کنند به آنها معادل گوییم. بنابراین، دو مدار به ازاء همه ترکیبات ممکن متغیرهای ورودی، خروجی‌های یکسانی دارند. هر دو عبارت تابع یکسانی را تولید می‌کنند ولی یکی از آنها گیت‌ها و ورودی‌های کمتری نسبت به دیگری دارد و بنابراین چون سیم‌بندی و قطعات کمتری نیاز است بر دیگری ترجیح داده می‌شود.

در ادامه برای درک بهتر موضوع مثال‌های دیگری ارائه می‌گردد:

مثال ۱: توابع بولی زیر را با حداقل لیترال‌ها ساده کنید.

$$x(x' + y) \quad -۱$$

$$\begin{aligned} x(x' + y) &= xx' + xy \\ &= 0 + xy \\ &= xy \end{aligned}$$

$$x + x'y \quad -۲$$

$$\begin{aligned} x + x'y &= (x + x')(x + y) \\ &= 1(x + y) \\ &= x + y \end{aligned}$$

$$(x + y)(x + y') \quad -۳$$

$$\begin{aligned} (x + y)(x + y') &= x + xy + xy' + yy' \\ &= x(1 + y + y') \\ &= x \end{aligned}$$

$$xy + x'z + yz \quad -۴$$

$$\begin{aligned} xy + x'z + yz &= xy + x'z + yz(x + x') \\ &= xy + x'z + xyz + x'yz \\ &= xy(1 + z) + x'z(1 + y) \\ &= xy + x'z \end{aligned}$$

۴-۲ ساده سازی با استفاده از نقشه کارنو

توابعی که تا پنج متغیر دارند قابل ساده سازی با روش جدول کارنو هستند. برای توابع بول پیچیده تر، طراحان دیجیتال از برنامه های کامپیوتر کوچک سازی استفاده می کنند. تنها روش موجود، روال سعی و کاشش می باشد که از روابط ساده و تکنیک های دستکاری آشنا استفاده می کند.

پیچیدگی گیت های منطقی دیجیتال که یک تابع بول را پیاده سازی می کنند، مستقیماً به پیچیدگی عبارات جبری که توسط آن تابع پیاده سازی می شوند بستگی دارد. گرچه جدول درستی یک تابع نمایش منحصر به فردی دارد، اما وقتی به صورت جبری بیان شود، می تواند فرم های متفاوتی داشته باشد. عبارت بول را می توان به صورت جبری ساده کرد. با این وجود، این روش حداقل سازی به دلیل کمبود قوانین خاص در پیشگویی مرحله بعدی فرآیند دستکاری، مشکل است.

روش نقشه، روالی ساده را برای ساده سازی توابع بول پیش پا می گذارد. این روش را می توان فرم مصور جدول درستی تصور کرد. روش نقشه را **نقشه کارنو** یا **نقشه k** هم می نامند.

نقشه نموداری است متشکل از مربعات که هر مربع یک مینترم از تابع را نشان می دهد. چون هر تابع بول را می توان به مجموعی از مینترم ها نشان داد، بنابراین نتیجه می شود که یک تابع بولی در نقشه را می توان با مربعاتی که مینترم های متعلق به آنها در تابع وجود دارد به صورت گرافیکی شناسایی کرد. در واقع نقشه، نمایشی عینی از همه راه هایی است که یک تابع ممکن است در فرم استاندارد داشته باشد. با تشخیص همه الگوهای مختلف، کاربر می تواند عبارت جبری مختلفی برای یک تابع به دست آورده و از میان آنها ساده ترین را انتخاب کند.

عبارت ساده شده حاصل از نقشه همیشه به یکی از دو فرم استاندارد زیر می باشد:

- جمع حاصل ضرب ها

• ضرب حاصل جمع‌ها

فرض بر این است که ساده‌ترین عبارت جبری، دارای حداقل جملات با کمترین لیترال در هر جمله باشد. این فرض نموداری با حداقل گیت را فراهم نموده و تعداد ورودی‌ها به گیت نیز حداقل خواهد بود. بعد خواهیم دید که ساده‌ترین عبارت منحصر به فرد نیست. گاهی ممکن است دو یا چند عبارت بیابیم که معیار حداقل سازی را برآورد. در این حالت هر یک از دو حل رضایت بخش خواهد بود. در ادامه روش حل جدول کارنو برای حداکثر پنج متغیر توضیح داده خواهد شد. لازم به ذکر است که روش نقشه کارنو صرفاً برای توابع دارای دو الی پنج متغیر کاربرد دارد و برای توابع با تعداد متغیر بیشتر قابل استفاده نیست.

۴-۲-۱ نقشه دو متغیره کارنو

نقشه دو متغیره در شکل ۴-۴ نشان داده شده‌است. در این نقشه چهار مینترم برای دو متغیر وجود دارد. از این رو نقشه متشکل از چهار مربع است، که هر یک متعلق به یک مینترم می‌باشد. 0, 1 موجود در هر سطر و ستون مقدار متغیر را نشان می‌دهند. متغیر x در سطر 0 پریم‌دار و در سطر 1 بدون پریم است. به طور مشابه y در ستون 0 پریم‌دار و در ستون 1 بدون پریم می‌باشد.

x	y	
	0	1
0	$x'y'$	$x'y$
1	xy'	xy

(ب)

m_0	m_1
m_2	m_3

(الف)

شکل ۴-۴: نقشه دو متغیره کارنو

ساده کردن عبارات بولی پیچیده ۹۹

اگر مربع‌هایی را که مینترم آنها متعلق به تابع مفروضی است با علامتی مشخص کنیم، روش مفید دیگری برای نمایش هر یک از 16 تابع ممکن از دو متغیر به دست می‌آید. به عنوان مثال تابع xy در شکل ۴-۴ (الف) دیده می‌شود. چون m_3 برابر xy است، یک 1 در داخل مربع متعلق به m_3 قرار می‌دهیم. به طور مشابه تابع $x+y$ در نقشه شکل ۴-۴ (ب) نشان داده شده است که در آن سه مربع با 1 علامت زده شده‌اند. این مربعات تابع به دست آمده‌اند:

$$m_1 + m_2 + m_3 = x'y + xy' + xy = x + y$$

سه مربع از تلافی x در سطر دوم و متغیر y در ستون دوم، که ناحیه متعلق به x یا y را پوشش می‌دهند، نیز به دست می‌آید.

۴-۲-۲ نقشه سه متغیره کارنو

یک نقشه سه متغیره در شکل ۴-۵ مشاهده می‌شود. برای سه متغیر هشت مینترم وجود دارد. بنابراین نقشه از هشت مربع تشکیل یافته است. توجه کنید که مینترم‌ها بر اساس ترتیب دودویی مرتب نشده‌اند. بلکه ترتیب این است که هنگام عبور از یک ستون به ستون مجاور تنها یک بیت از نظر مقدار تغییر می‌کند. برای نشان دادن رابطه بین مربع‌ها و سه متغیر نقشه، بخش (ب) با اعدادی در هر سطر و هر ستون علامت‌گذاری شده است. مثلاً مربع متعلق به m_5 مربوط به سطر 1 و ستون 01 است. وقتی دو عدد در کنار هم قرار گیرند عدد دودویی 101 حاصل می‌شود که معادل دهدهی آن عدد 5 می‌باشد.

		y							
		yz x		00	01	11	10		
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$	m_0	m_1	m_3	m_2
	1	$xy'z'$	$xy'z$	xyz	xyz'	m_4	m_5	m_7	m_6

شکل ۴-۵: نمایش توابع در نقشه

به طریقی دیگر هم می توان به مربع $m_5 = xy'z$ نگاه کرد به این ترتیب که بگوییم m_5 در سطر مربوط به x و ستون متعلق به $y'z$ است (ستون 01). توجه کنید که هر متغیر در چهار مربع مقدار 0 و در چهار مربع دیگر مقدار 1 را دارد. به منظور تفکیک، هر متغیر را در خانه های 1 بدون پریم و در خانه های 0 با پریم نشان می دهیم. برای سادگی، متغیر را با سمبل حرفی اش در زیر مربعاتی که بدون پریم هستند می نویسیم.

جهت درک برتری های جدول کارنو در ساده سازی توابع بول، باید خاصیت مربع های همجوار را مشخص کنیم. تنها اختلاف بین هر دو مربع مجاور در نقشه این است که در یکی متغیری با پریم و در دیگری بدون پریم ظاهر می شود. مثلاً، m_7, m_5 در دو مربع مجاور قرار دارند. متغیر y در m_5 پریم دار و در m_7 بدون پریم است، ضمن این که دو متغیر دیگر در هر دو مربع یکسانند. با توجه به اصول جبر بول، نتیجه می گیریم که جمع دو مینترم در مربع های مجاور را می توان به یک جمله AND متشکل از دو لیترال ساده کرد. برای روشن شدن مطلب، مجموع دو مربع همجوار مانند m_7, m_5 را ملاحظه کنید.

$$\begin{aligned} m_5 + m_7 &= xy'z + xyz \\ &= xz(y' + y) \\ &= xz \end{aligned}$$

در اینجا دو مربع در متغیر y با هم اختلاف دارند که هنگام تشکیل جمع دو مینترم حذف می شود. بنابراین هر دو مینترم که در دو مربع مجاور با هم OR شوند موجب حذف متغیری می گردند که در آن دو مینترم متفاوت اند. مثال های زیر روال حداقل سازی یک تابع بول را با یک نقشه توضیح می دهد.

مثال ۲: تابع بولی زیر را ساده کنید.

$$F(x, y, z) = \Sigma(2, 3, 4, 5)$$

ابتدا در هر مربعی که مینترم تابع را نشان دهد، مقدار 1 قرار می دهیم. این کار در شکل ۴-۶ به این ترتیب انجام شده است که مربعات مینترم های 010, 011, 100, 101 با 1

۱۰۱ ساده کردن عبارات بولی پیچیده

علامت زده شده‌اند. قدم بعدی یافتن مربع‌های مجاور است. این کار در نقشه با زیر مربع‌هایی که هر یک دو عدد 1 را در بر می‌گیرند صورت گرفته‌است. زیر مربع یا مستطیل بالای سمت راست ناحیه پوشش یافته با $x'y$ را شامل می‌شود. این دو مربع در سطر 0 قرار دارند که با x' و نیز در دو ستون آخر با y نشان داده می‌شوند. به طور

	yz		y	
	00	01	11	10
0			1	1
x 1	1	1		
	z			

شکل ۴-۶: جدول کارنو تابع $F(x, y, z) = \Sigma(2, 3, 4, 5)$

مشابه مستطیل پایین سمت چپ جمله ضرب xy' را نشان می‌دهد (سطر دوم نشان دهنده x و دو ستون چپ نیز y' است). جمع منطقی این دو جمله ضرب، عبارت ساده شده را نتیجه می‌دهد.

مواردی وجود دارد که در آنها دو مربع همجوارند ولی به هم نچسبیده‌اند. در شکل ۴-۵، m_0 مجاور m_2 ، m_4 مجاور m_6 است زیرا مینترم‌ها تنها در یک متغیر با هم اختلاف دارند. این مطلب به راحتی با کمک جبر قابل اثبات است.

$$\begin{aligned}
 m_0 + m_2 &= x'y'z' + x'yz' \\
 &= x'z'(y' + y) \\
 &= x'z' \\
 m_4 + m_6 &= xy'z' + xyz' \\
 &= xz'(y' + y) \\
 &= xz'
 \end{aligned}$$

در نتیجه ما باید تعریف مربع‌های همجوار را اصلاح کنیم تا این حالت و دیگر حالات مشابه را نیز شامل شود. این تصحیح بدین صورت انجام می‌گیرد که نقشه کشیده شده در یک سطح از دو لبه سمت چپ و راست مجاور تصور شوند.

مثال ۳: تابع بول زیر را ساده کنید.

$$F(x, y, z) = \Sigma(3, 4, 6, 7)$$

نقشه این تابع در شکل ۴-۷ ترسیم شده است. در این شکل ۴ مربع با ۱ علامت خورده‌اند که هر کدام متعلق به یک مینترم است. دو مربع همجوار در ستون سوم با هم ترکیب شده‌اند تا جمله دو لیترال yz را به وجود آورند. دو مربع باقیمانده هم بر اساس تعریف جدید مجاورند و در نمودار با نیم مربع‌ها محصور شده‌اند. این دو مربع، وقتی ترکیب شوند جمله دو لیترالی xz' را به دست می‌دهند. بنابراین تابع ساده شده به فرم زیر است.

$$F = yz + xz'$$

اکنون به ترکیب چهار مربع همجوار در نقشه سه متغیره توجه نمایید. چنین ترکیبی نشان دهنده جمع منطقی چهار مینترم مجاور است و نتیجه این ترکیب، تولید عبارتی با تنها یک متغیر است. به عنوان مثال جمع منطقی چهار مینترم مجاور ۶,۴,۲,۰ عبارت را به جمله یک لیترالی z' کاهش می‌دهد.

$$\begin{aligned} m_0 + m_2 + m_4 + m_6 &= x'y'z' + x'yz' + xy'z' + xyz' \\ &= x'z'(y' + y) + xz'(y' + y) \\ &= x'z' + xz' \\ &= z'(x' + x) \\ &= z' \end{aligned}$$

۱۰۳ ساده کردن عبارات بولی پیچیده

	yz		y	
	00	01	11	10
0			1	
x 1	1		1	1

شکل ۴-۷: جدول کارنو تابع $F(x, y, z) = \Sigma(3, 4, 6, 7)$

در حل جدول کارنو لازم است تا به نکات زیر توجه گردد:

- تعداد مربعات مجاوری که ممکن است ترکیب شوند همواره برابر توانی از ۲، مانند ۱، ۲، ۴، ۸ می باشد.
- هر چقدر تعداد بیشتری از مربعات همجوار ترکیب شوند جمله حاصلضرب نتیجه، تعداد کمتری لیترال خواهد داشت.
- یک مربع یک مینترم را نمایش می دهد و دارای سه لیترال است.
- دو مربع مجاور یک جمله دو لیترال را نشان می دهند.
- چهار مربع همجوار یک جمله با یک لیترال را نشان می دهند.
- هشت مربع همجوار که تمام نقشه را می پوشانند همواره تابع ۱ را تولید می کنند.

مثال ۴: تابع بول زیر را ساده کنید.

$$F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$$

نقشه تابع F در شکل ۴-۸ نشان داده شده است. ابتدا چهار مربع مجاور در اولین و آخرین ستون را با هم ترکیب می کنیم تا جمله تک لیترال z' به دست آید. تنها مینترم باقیمانده که متعلق به مینترم ۵ است با مربع مجاورش که قبلاً به کار رفته، ترکیب می گردد. این کار نه تنها مجاز است بلکه مفید نیز می باشد، زیرا دو مربع مجاور جمله دو لیترالی xy' را تولید می کنند در حالی که یک مربع تنها، جمله سه لیترال $xy'z$ را نمایش می دهد. تابع ساده به صورت زیر است.

$$F(x,y,z) = \Sigma (0, 2, 4, 5, 6) = z' + xy'$$

		yz		y	
		00	01	11	10
x	0	1			1
	1	1	1		1
		z			

شکل ۴-۸: جدول کارنو تابع $F(x, y, z) = \Sigma (0, 2, 4, 5, 6)$

اگر تابعی به صورت مجموع میترم‌ها بیان نشود، می‌توان از نقشه برای به‌دست آوردن میترم‌های تابع استفاده کرد و سپس تابع را به صورت جملاتی با حداقل لیترال‌ها ساده نمود. البته باید عبارت جبری حتماً به صورت جمع حاصلضرب‌ها باشد. هر جمله ضرب را می‌توان با نقشه‌ای متشکل از یک، دو یا چند مربع در نقشه نشان داد. آنگاه میترم‌های تابع مستقیماً از جدول استخراج می‌شوند.

مثال ۵: تابع بول زیر را ساده کنید.

$$F = A'C + A'B + AB'C + BC$$

(الف) آن را به مجموع میترم‌ها نشان دهید.

(ب) و سپس عبارت مجموع حاصلضرب حداقل را پیدا کنید.

سه جمله ضرب در عبارت دو لیترال دارند و در نقشه سه متغیره، هر یک با دو مربع نشان داده شده‌اند.

دو مربع مربوط به جمله اول، $A'C$ ، در شکل ۴-۹ از تلاقی A' (اولین سطر) و C (دو ستون میانی) به‌دست می‌آید تا مربعات 001 و 011 را بدهند. توجه کنید که وقتی 1ها را در مربعات می‌گذارید، ممکن است یک 1 را که از جمله قبلی در آن قرار داده شده بیابید. این نکته برای دومین جمله، $A'B$ ، رخ می‌دهد که دو عدد 1 در مربع‌های 01 و 010 قرار دارند. مربع 011 با جمله اول، $A'C$ ، مشترک است، بنابراین تنها یک 1 در آن

ساده کردن عبارات بولی پیچیده ۱۰۵

قرار داده می شود. به همین ترتیب می بینیم که جمله $AB'C$ متعلق به مربع 101، یعنی مینترم 5 است، و جمله BC متعلق به دو مربع 011 و 111 می باشد.

تابع جمعاً پنج مینترم دارد و در نقشه شکل ۴-۹ هم با پنج عدد 1 نشان داده شده است. مینترم هایی که مستقیماً از نقشه خوانده می شوند عبارتند از 1، 2، 3، 5 و 7. تابع را می توان به صورت جمع مینترم ها نشان داد.

$$F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$$

		BC		B	
		00	01	11	10
A	0		1	1	1
	1		1	1	

شکل ۴-۹: جدول کارنو تابع $F = A'C + A'B + AB'C + BC$

عبارت جمع حاصلضرب مفروض اولیه چندین جمله دارد. همانطور که در نقشه مشاهده می شود می توان آن را ساده کرده و عبارتی دو جمله ای به دست آورد.

$$F = C + A'B$$

۴-۲-۳ نقشه چهار متغیره

نقشه توابع بول چهار متغیره در شکل ۴-۱۰ نشان داده شده است. در (الف) 16 جمله مینترم فهرست شده به هر یک مربعی تخصیص داده شده است. در (ب) نقشه دوباره رسم شده تا بیانگر ارتباط بین چهار متغیر باشد. سطرها و ستون ها بر اساس کد گری شماره گذاری شده اند، و بین هر دو سطر یا ستون مجاور تنها یک رقم تغییر می کند. مینترم متعلق به هر مربع از ترکیب شماره سطر و شماره ستون آن به دست می آید. مثلاً وقتی اعداد سطر سوم (11) و ستون دوم (01) ترکیب شوند عدد دودویی 1101 حاصل

می‌گردد، که معادل 13 دهنده‌ی است. بنابراین، مربع در سطر سوم و ستون دوم مینترم m_{13} را نمایش می‌دهد.

		y							
		yz		00	01	11	10		
w	x	y	z	00	01	11	10	00	01
	00	00	00	$w'x'y'z'$	$w'x'y'z$	$w'x'yz$	$w'x'yz'$	m_0	m_1
	01	01	01	$w'xy'z'$	$w'xy'z$	$w'xyz$	$w'xyz'$	m_4	m_5
	11	11	11	$wxy'z'$	$wxy'z$	$wxyz$	$wxyz'$	m_{12}	m_{13}
	10	10	10	$wx'y'z'$	$wx'y'z$	$wx'yz$	$wx'yz'$	m_8	m_9
				(ب)				(الف)	

شکل ۴-۱۰: جدول چهار متغیره کارنو

ساده کردن توابع بول چهار متغیره مشابه با روش به کار رفته برای توابع سه متغیره است. مربعات مجاور مربعاتی هستند که در کنار یکدیگرند. به علاوه نقشه در سطحی واقع است و لبه‌های بالا و پایین و چپ و راست نیز مجاور است تا به این ترتیب مربعات همجوار را بسازند. مثلاً m_0 و m_2 و نیز m_{11} و m_3 هر کدام مربعات مجاور را می‌سازند. ترکیب مربعات همجوار به راحتی با بررسی نقشه چهار متغیره قابل تشخیص است. در جدول کارنو چهار متغیره نکات زیر باید مورد توجه قرار گیرند:

- یک مربع یک مینترم را نمایش می‌دهد، و جمله آن چهار لیترالی است.
- دو مربع همجوار یک جمله سه لیترالی را می‌سازند.
- چهار مربع همجوار یک جمله دو لیترالی را نشان می‌دهند.
- هشت مربع همجوار یک جمله یک لیترالی را نمایش می‌دهند.
- شانزده مربع همجوار تابعی برابر 1 را تولید می‌کنند.
- هیچ ترکیب دیگری از مربع‌ها نمی‌تواند تابع را ساده کند.

ساده کردن عبارات بولی پیچیده ۱۰۷

مثال ۶: تابع بول زیر را ساده کنید.

$$F(w, x, y, z) = \Sigma (0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

چون تابع چهار متغیر دارد، باید از نقشه چهار متغیره استفاده کرد. مینترم‌های لیست شده در مجموع فوق با 1ها در نقشه شکل ۴-۱۱ علامت زده شده‌اند. هشت 1 مجاور می‌توانند با هم ترکیب شده و جمله تک لیترالی y' را نتیجه دهند. سه 1 باقیمانده در سمت راست نمی‌توانند با هم ترکیب و جمله ساده‌ای بدهند. آنها باید به صورت دو یا چهار مربع مجاور با هم ترکیب شوند. هر چقدر تعداد مربعات ترکیب شده بیشتر باشد، تعداد لیترال‌ها در جمله کمتر خواهد بود.

		y			
		00	01	11	10
wx	yz				
	00	1	1		1
	01	1	1		1
	11	1	1		1
	10	1	1		
		z			

شکل ۴-۱۱: جدول کارنو مثال ۵

در این مثال دو 1 فوقانی سمت راست با دو 1 فوقانی در سمت چپ ترکیب شده و جمله $w'z'$ را می‌دهند. توجه داشته باشید که می‌توان یک مربع را بیش از یک بار به کار برد. حال فقط یک مربع در سطر سوم و ستون چهارم (مربع 1110) باقیمانده‌است. در عوض انتخاب این مربع به تنهایی، آن را با مربع‌هایی که قبلاً به کار رفته‌اند برای ایجاد مربع‌های مجاور ترکیب می‌کنیم. این مربعات شامل دو سطر میانی و دو ستون انتهایی بوده و جمله xz' را تولید می‌کنند. تابع ساده شده به صورت زیر است:

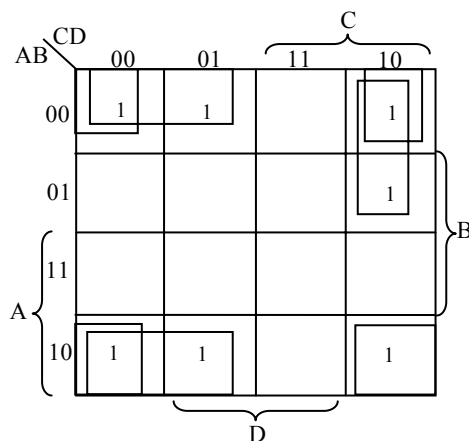
$$F = y' + w'z' + xz'$$

مثال ۷: تابع زیر را ساده کنید.

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

ناحیه مفروش شده با این تابع شامل مربعاتی است که در شکل ۴-۱۲ با ۱ علامت زده شده است. این تابع دارای چهار متغیره بوده و همانطور که دیده می شود سه جمله سه لیترالی و یک جمله چهار لیترالی دارد. هر جمله سه لیترالی در نقشه با دو مربع نمایش داده شده است. مثلاً $A'B'C'$ در مربعات 0001, 0000 نشان داده شده است. تابع را می توان با انتخاب چهار ۱ در گوشه ها و ترکیب آنها برای به دست آوردن جمله $B'D'$ ساده کرد. این عمل مجاز است زیرا وقتی نقشه را سطحی تصور کنیم که لبه های چپ و راست و لبه های پایین و بالای آن با هم مجاورند، این چهار مربع همجوار خواهند بود. دو ۱ سمت چپ در سطر بالا و دو ۱ در سطر پایین ترکیب می شوند تا جمله $B'C'$ حاصل شود. تنها ۱ باقیمانده را به صورت دو مربع ترکیب می کنیم تا $A'CD'$ حاصل گردد. تابع ساده شده به صورت زیر خواهد بود.

$$F = B'D' + B'C' + A'CD'$$



شکل ۴-۱۲: جدول کارنو مثال ۶

۴-۲-۴ نقشه پنج متغیره کارنو

استفاده از نقشه‌هایی که بیش از چهار متغیر دارند چندان ساده نیست. یک نقشه پنج متغیره به ۳۲ مربع و نقشه شش متغیره به ۶۴ مربع نیاز دارد. وقتی تعداد متغیرها زیاد شود، تعداد مربعات هم به طور بی‌رویه‌ای افزایش می‌یابد و یافتن مربعات همجوار بیش از پیش به شکل هندسی وابسته می‌گردد. یک نقشه پنج متغیره در شکل ۴-۱۳ نشان داده شده است. این نقشه، از دو نقشه چهار متغیره با متغیرهای A, B, C, D, E تشکیل یافته و متغیر A آن دو را از هم تفکیک کرده است. نقشه چهار متغیره سمت چپ ۱۶ مربعی را نشان می‌دهد که در آن $A = 0$ است، و دیگر نقشه چهار متغیره، مربعات مربوط به $A = 1$ را نمایش می‌دهد. مینترم‌های ۰ تا ۱۵ متعلق به $A = 0$ و مینترم‌های ۱۶ تا ۳۱ متعلق به $A = 1$ است. هر نقشه چهار متغیره وقتی جداگانه بررسی

A=0					A=1						
BC \ DE		D				BC \ DE		D			
		00	01	11	10			00	01	11	10
B	00	0	1	3	2	00	16	17	19	18	
	01	4	5	7	6	01	20	21	23	22	
	11	12	13	15	14	11	28	29	31	30	
	10	8	9	11	10	10	24	25	27	26	
		(ب) E						E (الف)			

شکل ۴-۱۳: جدول پنج متغیره کارنو (دو جدول مجزا)

شود همجواری تعریف شده قبلی خود را حفظ می‌کند. به علاوه هر مربع از نقشه $A=0$ با مربع متناظرش در مربع $A=1$ همجوار است. مثلاً مینترم ۴ با مینترم ۲۰ و مینترم

15 با 31 مجاور است. بهترین راه تجسم این قانون برای مربع‌های همجوار این است که این دو نیم نقشه را بر روی یکدیگر تصور کنیم. هر دو مربعی که روی هم قرار گیرند مجاور شناخته می‌شوند.

با پیگیری روشی که برای نقشه پنج متغیره به کار رفت، می‌توان نقشه شش متغیره را با 4 نقشه چهار متغیره به دست آورد تا 64 مربع مورد نیاز حاصل گردد. نقشه‌هایی با شش یا تعداد بیشتری متغیر، نیاز به تعداد بی شماری مربع داشته و استفاده از آنها غیر عملی است. روش دیگر، استفاده از برنامه‌های کامپیوتری در ساده‌سازی توابع بول با متغیرهای بی شمار می‌باشد.

با بررسی و در نظر گرفتن تعریف جدید همجواری مربعات، می‌توان نشان داد که 2^k مربع همجوار به ازاء $k = (0, 1, 2, \dots, n)$ در یک نقشه n متغیره ناحیه را مشخص می‌کند که نمایش دهنده جمله‌ای با $n-k$ لیترال است. برای این که عبارت فوق مفهوم داشته باشد باید همیشه n بزرگتر از k باشد. وقتی $n=k$ است، تمام سطح نقشه ترکیب شده و تابع یکانی (1) را تولید می‌کند. جدول ۴-۱۴ رابطه بین تعداد مربعات مجاور و تعداد لیترال در هر جمله را نشان می‌دهد. مثلاً هشت مربع مجاور ناحیه‌ای را در نقشه پنج متغیره ترکیب می‌کنند تا یک جمله دو متغیره حاصل شود.

تعداد مربعات مجاور		تعداد لیترال‌ها در یک جمله در یک نقشه n متغیره			
k	2^k	$n=2$	$n=3$	$n=4$	$n=5$
0	1	2	3	4	5
1	2	1	2	3	4
2	4	0	1	2	3
3	8		0	1	2
4	16			0	1
5	32				0

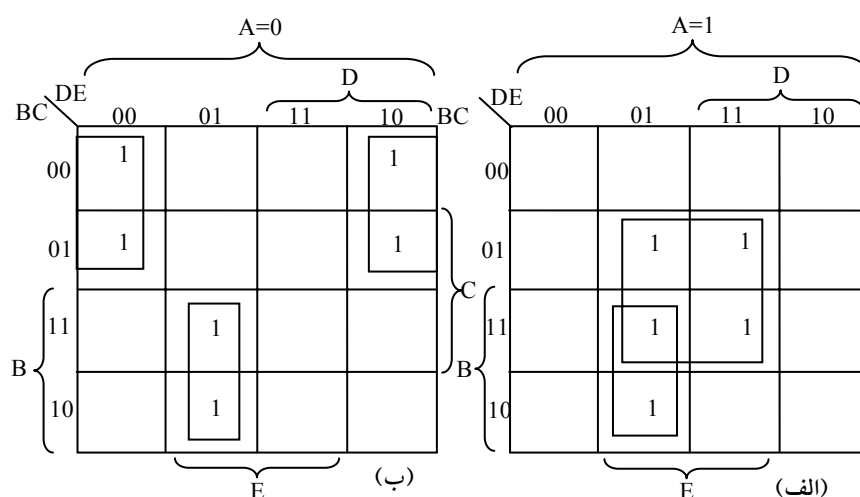
شکل ۴-۱۴: رابطه بین تعداد مربعات مجاور و تعداد لیترال‌ها در یک

مثال ۸: تابع بول زیر را ساده کنید.

$$F(A,B,C,D,E) = \Sigma(0, 2, 4, 6, 9, 13, 21, 23, 25, 29, 31)$$

نقشه پنج متغیره برای این تابع در شکل ۴-۱۵ دیده می شود. در بخشی از نقشه که متعلق به مینترم های 0 تا 15 است، $A=0$ بوده و در آن شش مینترم مقدار 1 را دارند. پنج مینترم دیگر به بخش $A=1$ متعلق است.

چهار مربع مجاور در نقشه $A=0$ با هم ترکیب شده اند تا جمله $A'B'E'$ را بدهند. توجه کنید که باید A' را نیز در جمله منظور کنیم زیرا تمام مربع ها متعلق به نقشه $A=0$ می باشند. دو مربع در ستون 01 و دو سطر آخر در هر دو بخش نقشه مشترکند. بنابراین آنها چهار مربع مجاور را تشکیل داده و جمله سه متغیره $BD'E$ را می سازند. در اینجا



شکل ۴-۱۵: نقشه مثال ۷

متغیر A آورده نشده است زیرا مربع های مجاور به هر دو $A=0$ و $A=1$ متعلق اند. جمله ACE از چهار مربع همجوار در نقشه $A=1$ به دست می آید. تابع ساده شده جمع منطقی سه جمله می باشد.

$$F = A'B'E' + BD'E + ACE$$

۴-۲-۵ عناصر اصلی در جدول کارنو

هنگام انتخاب مربع‌های مجاور در یک نقشه باید مطمئن شویم که همه مینترم‌های تابع هنگام ترکیب مربع‌ها پوشش داده شده‌اند. همچنین باید تعداد جملات در عبارت حداقل شود و هر جمله‌ای که مینترم آن قبلاً به وسیله دیگر جملات به کار رفته نیز کنار گذاشته شود. گاهی نیز ممکن است دو یا سه عبارت بر معیار ساده‌سازی صحه بگذارند. روش ترکیب مربع‌ها در نقشه را می‌توان سیستماتیک‌تر کرد به شرطی که مفهوم جملات عناصر اصلی و عناصر اصلی اساسی خوب فهمیده شوند.

یک **عنصر اصلی** جمله‌ای حاصلضربی است که از ترکیب حداکثر مربعات مجاور به هم حاصل می‌گردد. اگر مینترمی در یک مربع تنها با یک عنصر اصلی پوشش یابد، به آن عنصر اصلی اساسی گوئیم.

عناصر اصلی یک تابع را می‌توان با ترکیب حداکثر تعداد مربعات ممکن به دست آورد. این بدان معنی است که یک 1 تنها اگر در مجاورت هر 1 دیگر در نقشه نباشد، یک عنصر اصلی است. دو 1 مجاور به شرطی یک عنصر اصلی را ایجاد می‌کنند که در داخل یک گروه چهار تایی مربع‌ها واقع نباشند. چهار 1 مجاور یک عنصر اصلی را تشکیل می‌دهند بشرطی که در یک گروه از هشت مربع همجوار نباشند و به همین ترتیب. عنصر اصلی اساسی با نظاره بر مربعات 1 و واریسی تعداد عناصر اصلی که آن را پوشش می‌دهد تعیین می‌گردد. یک عنصر اصلی، اساسی است اگر تنها عنصر اصلی باشد که مینترم را پوشش می‌دهد.

مثال ۹: تابع چهار متغیره زیر را در نظر بگیرید:

$$F(A, B, C, D) = \Sigma (0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

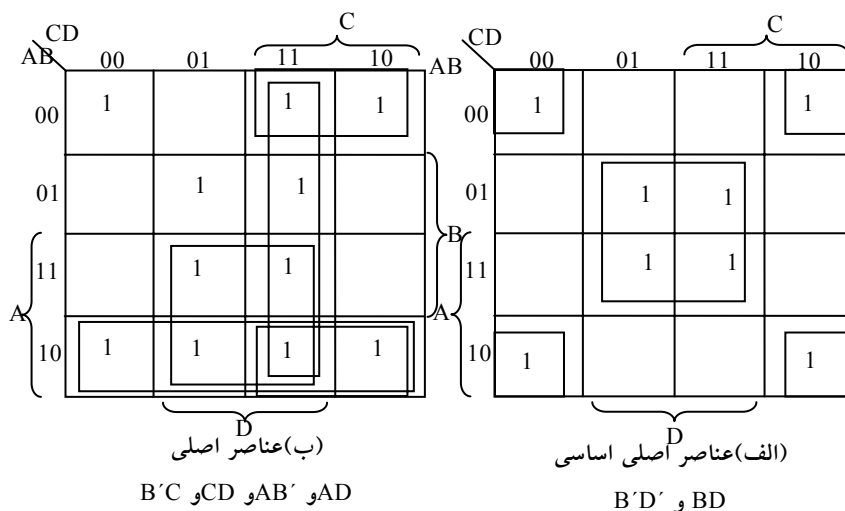
مینترم‌های تابع با 1 در نقشه‌های شکل ۴-۱۶ علامت زده شده‌اند. بخش (الف) از شکل، دو عنصر اصلی اساسی را نشان می‌دهد. یک موجب، اساسی است زیرا تنها یک راه برای پوشش m_0 در چهار مربع مجاور وجود دارد. این چهار مربع جمله $B'D'$ را

۱۱۳ ساده کردن عبارات بولی پیچیده

تعریف می کنند. به طور مشابه، برای ترکیب m_5 با چهار مربع مجاور تنها یک راه وجود دارد و جمله BD از آن حاصل می گردد. این دو عنصر اصلی اساسی هشت مینترم را پوشش می دهند. سه مینترم باقیمانده m_3 و m_9 و m_{11} باید بعد ملاحظه شوند.

شکل ۴-۱۶ (ب) همه راه های ممکن که سه مینترم با عناصر اصلی پوشش می یابند را نشان می دهد. مینترم m_3 می تواند با عنصر اصلی CD یا $B'C$ پوشش یابد. مینترم m_9 با هر یک از AD یا AB' پوشش می یابد. مینترم m_{11} نیز با هر یک از چهار عنصر اصلی می تواند پوشش پیدا کند. عبارت ساده شده از جمع منطقی دو عنصر اصلی اساسی، و هر دو عنصر اصلی دیگر که مینترم های m_3 و m_9 و m_{11} را پوشش دهند به دست می آید. چهار امکان برای بیان تابع با چهار جمله ضرب که هر یک دو لیترال دارند وجود دارد:

$$\begin{aligned} F &= BD + B'D' + CD + AD \\ &= BD + B'D' + CD + AB' \\ &= BD + B'D' + B'C + AD \\ &= BD + B'D' + B'C + AB' \end{aligned}$$



شکل ۴-۱۶: نمایش ساده سازی با استفاده از عناصر اصلی مثال ۸

مثال فوق نشان داد که شناسایی عناصر اصلی در نقشه در تعیین صور متفاوت تابع ساده شده کمک موثری می‌نمایند.

روال یافتن عبارت ساده شده از نقشه لازم می‌دارد که ابتدا تمام عناصر اصلی اساسی را معین کنیم. تابع ساده شده از جمع منطقی همه عناصر اصلی اساسی، به علاوه دیگر عناصر اصلی حاصل می‌گردد. این عناصر اصلی ممکن است برای پوشش میترم‌های باقیمانده‌ای که در عنصر اصلی اساسی وجود ندارد لازم باشد. گاهی بیش از یک راه برای ترکیب مربعات وجود دارد و هر ترکیب هم ممکن است عبارت ساده شده یکسانی را تولید کند.

۳-۴ ساده‌سازی با ضرب حاصل جمع‌ها

در تمام مثال‌های قبلی، توابع بول حاصل از نقشه به فرم جمع حاصلضرب‌ها بیان شدند. با کمی اصلاح می‌توان فرم ضرب حاصل جمع‌ها را به دست آورد.

روال تهیه یک تابع حداقل بر حسب ضرب حاصل جمع‌ها از خواص اصلی توابع بول حاصل می‌گردد. 1های واقع در مربع‌های نقشه نشانگر میترم‌های تابع است. میترم‌هایی که در تابع ذکر نشوند متمم تابع را بیانگرند. با توجه به این مطلب مشاهده می‌کنیم که متمم یک تابع به وسیله مربع‌هایی که با 1 علامت زنی نشده‌اند بیان می‌گردد. اگر در مربع‌های خالی 0 قرار داده و آنها را با روش مربع‌های همجوار ترکیب کنیم عبارت ساده شده متمم تابع یعنی F' را به دست خواهیم آورد. متمم F' به ما تابع F را باز می‌گرداند. به دلیل عمومیت تئوری دمورگان تابع حاصل به طور خودکار به صورت ضرب حاصل جمع‌هاست. برای درک بهتر موضوع، مثالی در این رابطه ارائه می‌گردد:

مثال ۱۰: تابع بولی زیر را (الف) به صورت جمع حاصلضرب‌ها، (ب) ضرب حاصل جمع‌ها ساده کنید.

$$F(A, B, C, D) = \Sigma (0, 1, 2, 5, 8, 9, 10)$$

ساده کردن عبارات بولی پیچیده ۱۱۵

۱های موجود در نقشه شکل ۴-۱۷، همه مینترم‌های تابع را نمایش می‌دهند. مربع‌هایی که با ۰ علامت‌زده شده‌اند مینترم‌های غایب در F را نشان می‌دهند، بنابراین متمم F را بیانگر هستند.

CD \ AB		C			
		00	01	11	10
A	00	1	1	0	1
	01	0	1	0	0
	11	0	0	0	0
	10	1	1	0	1

شکل ۴-۱۷: جدول کارنو مثال ۹

ترکیب مربعات حاوی ۱ها تابع ساده شده را به صورت جمع حاصلضرب‌ها به دست می‌دهد:

$$F = B'D' + B'C' + A'C'D \quad (\text{الف})$$

اگر مربعات حاوی ۰ها را ترکیب کنیم، تابع متمم ساده شده به دست خواهد آمد:

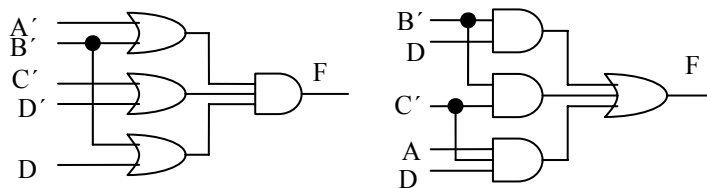
$$F' = AB + CD + BD'$$

با اعمال تئوری دمورگان (استفاده از دوگان و متمم کردن هر متغیر) تابع ساده شده را به صورت ضرب حاصل جمع‌ها به دست می‌آوریم:

$$F = (A' + B')(C' + D')(B' + D) \quad (\text{ب})$$

پیاده‌سازی عبارت ساده شده حاصل از مثال ۹ در شکل ۴-۱۸ دیده می‌شود. عبارت جمع حاصلضرب‌ها در بخش (الف) با گروهی از گیت‌های AND پیاده‌سازی شده‌است. خروجی گیت‌های AND نیز به ورودی‌های یک گیت OR متصل گردیده است. همان

تابع به صورت ضرب حاصل جمع‌ها در شکل (ب) با تعدادی گیت OR، که هر یک متعلق به یک جمله OR است، پیاده‌سازی شده و خروجی آنها به یک گیت AND منتهی گشته‌است. در هر دو حال فرض بر این است که متهم متغیرها نیز مستقیماً در دست‌رسند و بنابراین نیازی به وارون‌گر نمی‌باشد.



$$F = (A' + B')(C' + D')(B' + D) \text{ (ب)} \quad F = B'D' + B'C' + A'C'D \text{ (الف)}$$

شکل ۴-۱۸: پیاده‌سازی با گیت تابع مثال ۹

الگوهای ایجاد شده در شکل ۴-۱۸ یک سری روش‌های کلی می‌باشند که به وسیله آنها هر تابع بول استاندارد قابل پیاده‌سازی است. در جمع حاصلضرب‌ها، گیت‌های AND به یک OR ختم می‌شوند و در ضرب حاصل جمع‌ها گیت‌های OR به یک AND متصل می‌گردند. هر یک از دو پیکر بندی فوق دارای دو سطح از گیت‌ها می‌باشند. بنابراین پیاده‌سازی یک تابع استاندارد دو سطحی می‌گویند.

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

شکل ۴-۱۹: جدول درستی تابع F

مثال ۱۰: روالی را برای محاسبه فرم ساده شده یک تابع بر حسب ضرب حاصل جمع‌ها، وقتی تابع ابتدا به صورت جمع مینترم‌ها است، نشان می‌دهد. این روال

۱۱۷ ساده کردن عبارات بولی پیچیده

هنگامی که تابع در آغاز بر حسب ماکسترم‌ها بیان شود نیز معتبر است. برای مثال به جدول درستی تابع F که در شکل ۴-۱۹ آمده است توجه نمایید:

این تابع به صورت جمع مینترم‌ها چنین بیان می‌شود:

$$F(x, y, z) = \Sigma (1, 3, 4, 6)$$

در ضرب ماکسترم‌ها تابع به صورت زیر است:

$$F(x, y, z) = \Pi (0, 2, 5, 7)$$

به بیان دیگر، ۱های تابع، مینترم‌ها را نشان می‌دهند و ۰های آن بیانگر جملات ماکسترم هستند. نقشه این تابع در شکل ۴-۲۰ دیده می‌شود. برای ساده کردن این تابع، در مربع مربوط به هر جمله مینترم که تابع به ازاء آن، مقدار ۱ گذاشته و بقیه مربع‌ها را با ۰ پر می‌کنیم. از طرف دیگر اگر تابع به فرم ضرب ماکسترم‌ها داده شده باشد در ابتدا در مربعاتی که جملات آن در تابع است ۰ قرار می‌دهیم و بقیه مربع‌ها با ۱ پر می‌شوند. سپس تابع می‌تواند به یکی از فرم‌های استاندارد ساده شود.

		yz		y	
		00	01	11	10
x	0	0	1	1	0
	1	1	0	0	1

شکل ۴-۲۰: جدول کارنو شکل ۴-۱۹

برای جمع حاصلضرب‌ها، ۱ها را با هم ترکیب می‌کنیم و خواهیم داشت:

$$F = x'z + xz'$$

برای ضرب حاصل جمع‌ها، ۰ها را با هم ترکیب می‌کنیم تا متمم تابع ساده شده به صورت زیر حاصل شود:

$$F' = xz + x'z'$$

که نشان می‌دهد تابع XOR متمم تابع هم ارزی (XNOR) است. با متمم‌گیری مجدد از F' تابع ساده شده را به ضرب حاصل جمع‌ها به دست خواهیم آورد.

$$F = (x' + z')(x + z)$$

برای وارد کردن یک تابع در یک نقشه که بر حسب ضرب حاصل جمع‌ها بیان شده است، می‌باید متمم تابع را به دست آورد و در آن مربع‌های مربوطه را با 0 پر کرد. مثلاً تابع

$$F = (A' + B' + C')(B + D)$$

را می‌توان با متمم‌گیری از آن وارد جدول کرد.

$$F' = ABC + B'D'$$

آنگاه مربع‌هایی که مینترم‌های F' را تشکیل می‌دهند با 0 پر می‌کنیم. بقیه مربع‌ها را با 1 پر می‌نماییم.

۴-۴ حالات بی‌اهمیت

جمع منطقی مینترم‌های مربوط به یک تابع شرایطی را که تحت آن تابع برابر 1 است، مشخص می‌نماید. تابع در ازاء بقیه مینترم‌ها 0 است. در این حالت فرض بر این است که همه ترکیبات مقادیر برای متغیرهای تابع معتبرند. در عمل کاربردهایی وجود دارند که در آنها در ازاء ترکیبات معینی از متغیرها، تابع مشخص نیست. مثلاً یک کد دودویی چهار بیتی برای ارقام دهدهی دارای شش ترکیب است که به کار نرفته‌اند و در نتیجه نامشخص تصور می‌گردند. توابعی که در ازاء ترکیبی از ورودی‌ها خروجی‌های نامشخص دارند، **تابع غیر کامل** نامیده می‌شود. در بسیاری از کاربردها، توجهی به مقدار منتسب به تابع در ازاء مینترم‌های نامعین نخواهیم داشت. به این دلیل مرسوم است که همه مینترم‌های نامشخص در تابع را حالات بی‌اهمیت بخوانیم. از حالات بی‌اهمیت می‌توان برای ساده‌سازی بیشتر عبارت بول در یک نقشه استفاده کرد.

باید توجه داشت که یک مینترم بی‌اهمیت، ترکیبی از متغیرهاست که مقدار منطقی آن نامشخص است. به این دلیل نمی‌توان یک حالت بی‌اهمیت را در نقشه با 1 نشان داد زیرا این عمل به این معنی است که تابع برای ترکیب خاص از ورودی‌ها همواره برابر 1 می‌باشد. به طور مشابه گذاشتن 0 در مربع‌های نقشه به معنی 0 بودن همیشگی تابع در آن حالت است. برای تفکیک حالت بی‌اهمیت از 1ها و 0ها از x استفاده می‌کنیم. بنابراین هر x در داخل یک مربع از نقشه به این معنی است که تخصیص 1 یا 0 به تابع به‌ازاء یک مینترم خاص فاقد اهمیت است.

وقتی مربع‌های مجاور انتخاب می‌گردند تا تابع در جدول ساده شود، مینترم‌های بی‌اهمیت با این ایده که ساده‌ترین فرم برای تابع به‌دست آید، برابر 1 یا 0 فرض می‌شوند. در ساده‌سازی تابع می‌توانیم با توجه به ساده‌ترین فرم ممکن برای تابع، به حالات بی‌اهمیت 0 یا 1 دهیم. برای درک بهتر موضوع مثالی در زیر آمده‌است که در آن حالات بی‌اهمیت نشان داده می‌شوند:

مثال ۱۱: تابع بول زیر

$$F(w, x, y, z) = \Sigma (1, 3, 7, 11, 15)$$

که حالات بی‌اهمیت زیر را دارا می‌باشد، ساده کنید.

$$d(w, x, y, z) = \Sigma (0, 2, 5)$$

مینترم‌های F ترکیباتی از متغیرها هستند که تابع را برابر 1 می‌کنند. مینترم‌های d مینترم‌های بی‌اهمیتی هستند که ممکن است به آنها 0 یا 1 تخصیص داده شود. ساده‌سازی نقشه در شکل ۴-۲۱ نشان داده شده‌است. مینترم‌های F با 1 علامت زده شده‌اند، مینترم‌های d با x علامت گذاری شده‌اند و بقیه مربع‌ها با 0 پر شده‌اند. برای به‌دست آوردن عبارت جمع حاصلضرب‌های ساده شده باید هر پنج 1 موجود در نقشه به حساب آیند، ولی بسته به روش ساده‌سازی ممکن است xها را در نظر بگیریم و یا نگیریم. جمله yz چهار مینترم در سومین ستون را پوشش می‌دهد. مینترم باقیمانده m_1 می‌تواند با مینترم m_3 ترکیب شده و جمله سه لیترا $w'x'z$ را بدهند. با این وجود با

احتساب یک یا دو x همجوار، می‌توانیم چهار مربع مجاور را ترکیب نماییم تا جمله دو متغیره حاصل گردد. در بخش (الف) از نمودار، مینترم‌های بی‌اهمیت 0 و 2 با 1 جایگزین شده‌اند و تابع ساده شده به صورت زیر است.

$$F = yz + w'x'$$

در بخش (ب) از نمودار، مینترم بی‌اهمیت 5 با 1 جایگزین شده و آنگاه تابع ساده شده به فرم زیر است:

$$F = yz + w'z$$

هر یک از دو عبارت شرایط بیان شده برای این مثال را دارا هستند.

		D						y							
		yz		00	01	11	10			yz		00	01	11	10
w	x	00	X	1	1	X		x	00	X	1	1	X		
		01	0	X	1	0			01	0	X	1	0		
		11	0	0	1	0			11	0	0	1	0		
		10	0	0	1	0			10	0	0	1	0		
				z						z					
				F=yz+ w'z (ب)						F=yz+w'x' (الف)					

شکل ۴-۲۱: جدول کارنو دارای حالات بی‌اهمیت (مثال ۱۰)

مثال قبل نشان داد که مینترم‌های بی‌اهمیت در نقشه در ابتدا با x ها علامت خورده‌اند و فرض می‌شود که بتوانند 0 و یا 1 بشوند. انتخاب 0 و یا 1 به روش ساده کردن تابع غیر کامل وابسته است. پس از انتخاب، تابع ساده شده حاصل، متشکل از مجموع مینترم‌ها است و در آنها مینترم‌هایی که در آغاز نامعلوم بوده ولی بعد به عنوان 1 انتخاب شده‌اند نیز وجود خواهند داشت. دو عبارت ساده شده حاصل در مثال ۱۰ را در نظر بگیرید:

۱۲۱ ساده کردن عبارات بولی پیچیده

$$F(w, x, y, z) = yz + w'x' = \Sigma(0, 1, 2, 3, 7, 11, 15)$$

$$F(w, x, y, z) = yz + w'z = \Sigma(1, 3, 5, 7, 11, 15)$$

هر دو عبارت شامل مینترم‌های 1، 3، 7، 11 و 15 می‌باشند که تابع F را برابر 1 می‌کنند. مینترم‌های بی‌اهمیت در آن دو به طور متفاوتی به کار گرفته شده‌اند و در اولین عبارت مینترم‌های 0 و 2 برابر 1 گرفته شده و مینترم 5 با انتخاب 0 حذف شده‌است. در دومین عبارت مینترم 5 برابر 1 و مینترم‌های 0 و 2 با مقدار 0 جایگزین شده‌اند. دو عبارت توابعی را نشان می‌دهند که فرم جبری متفاوتی دارند. هر دو مینترم‌های مشخص شده را می‌پوشانند ولی هر یک مینترم‌های بی‌اهمیت متفاوتی را پوشش می‌دهند. مادامی که تابع مشخص شده غیر کامل است، هر دو عبارت قابل قبول‌اند زیرا تنها اختلاف در مقدار F مینترم‌های بی‌اهمیت می‌باشند.

می‌توان عبارت ضرب حاصل جمع‌ها را هم برای تابع شکل ۴ - ۲۱ به دست آورد. در این حالت، تنها راه برای ترکیب 0ها جایگزینی مینترم‌های بی‌اهمیت شماره 0 و 2 با مقدار 0 می‌باشد و به این ترتیب تابع متمم ساده شده به دست می‌آید:

$$F' = z' + wy'$$

با متمم گیری از طرفین، عبارت ساده شده به صورت ضرب حاصل جمع‌ها خواهد بود:

$$F(w, x, y, z) = z(w' + y) = \Sigma(1, 3, 5, 7, 11, 15)$$

در این حال، ما مینترم‌های شماره 0 و 2 را با مقدار 0 و مینترم 5 را با 1 جایگزین کرده‌ایم.

سؤالات

۱- عبارت بولی زیر را ساده کرده و نمودار آن را رسم نمایید

$$F_1 = x' y z + x' y z' + x y'$$

۲- جدول کارنو تابع زیر را رسم کرده و سپس تابع را ساده نمایید.

$$F(x, y, z) = \Sigma (2, 3, 4, 6)$$

۳- تابع بولی زیر را با استفاده از جدول کارنو ساده نمایید و متمام آنرا به دست آورید.

$$F(x, y, z) = \Sigma (0, 1, 4, 6, 7)$$

۴- تابع بول چهار متغیره زیر را ساده کنید.

$$F(w, x, y, z) = \Sigma (0, 2, 4, 5, 7, 8, 9, 14)$$

۵- تابع بول زیر را با استفاده از جدول کارنو ساده کنید.

$$F(A, B, C, D, E) = \Sigma (4, 6, 8, 9, 13, 21, 22, 25, 27, 30)$$

۶- تابع بول $F(w, x, y, z) = \Sigma (0, 3, 7, 8, 14)$ که حالات بی اهمیت زیر را دارا می باشد، ساده کنید.

$$d(w, x, y, z) = \Sigma (1, 2, 5)$$

فصل ۵

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR

هدف کلی

در این فصل مباحث اصلی مربوط به پیاده‌سازی گیت‌ها با استفاده از مدارهای NAND و NOR مورد بحث و بررسی قرار گرفته و علت استفاده از این نوع مدارها به همراه سادگی طراحی مطرح خواهند شد. همچنین مباحث تکمیلی سایر گیت‌های خاص که به نحوی در تکمیل مدارهای فوق‌الذکر تاثیر گذار هستند نیز مورد بحث خواهند بود.

هدف ساختاری

در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند:

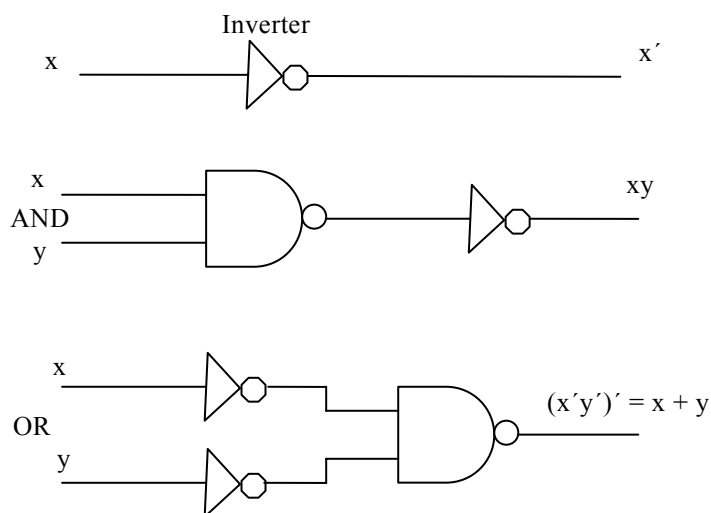
- مدارهای NAND و NOR
- علل استفاده از مدارهای NAND و NOR
- مدارهای AND-OR-INVERT
- مدارهای OR-AND-INVERT
- مدارهای OR انحصاری

مدارهای دیجیتال اغلب به جای AND , OR با گیت‌های NAND و NOR ساخته می‌شوند. ساختن گیت‌های NAND و NOR با اجزاء الکترونیکی ساده‌تر بوده و به عنوان گیت‌های پایه در تمام خانواده‌های ICهای دیجیتال به کار می‌روند. به دلیل مزیت

گیت‌های NAND و NOR در طراحی مدارهای دیجیتال، قواعد و روال‌هایی برای تبدیل توابع بول بیان شده بر حسب AND , OR , NOT به نمودارهای منطقی معادل بر حسب NAND و NOR بوجود آمده است.

۱-۵ مدارهای NAND

گیت NAND را یک گیت یونیورسال می‌گویند زیرا هر سیستم دیجیتالی را می‌توان با آن پیاده‌سازی کرد. برای اینکه نشان دهیم هر تابع بولی قابل پیاده‌سازی با گیت‌های NAND می‌باشد، کافی است فقط نشان دهیم که اعمال منطقی NOT , OR , AND را می‌توان با NAND پیاده‌سازی کرد. این کار در شکل ۱-۵ نشان داده شده است.



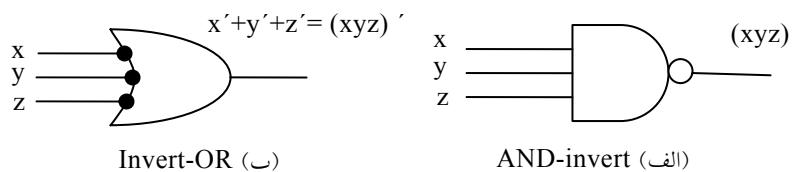
شکل ۱-۵: عملیات منطقی با گیت‌های NAND

عمل متمم از یک گیت NAND یک ورودی که دقیقاً مثل NOT عمل می‌کند حاصل می‌گردد. عمل AND نیاز به دو گیت NAND دارد. اولی عمل NAND و دومی عمل NOT را انجام می‌دهد. عمل OR از طریق یک گیت NAND و دو NOT در هر ورودی حاصل می‌شود.

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۲۵

راهی مناسب برای پیاده‌سازی یک تابع بول با گیت‌های NAND، به دست آوردن تابع بول ساده شده بر حسب عملگرهای بولی و سپس تبدیل تابع به منطق NAND است. تبدیل یک عبارت جبری از AND, OR, NOT به NAND به سادگی با دستکاری نمودار AND-OR به نمودار NAND انجام می‌شود.

برای ساده‌سازی تبدیل به منطق NAND، بهتر است سمبل گرافیکی دیگری برای گیت تعریف کنیم. دو سمبل گرافیکی معادل برای گیت NAND در شکل ۲-۵ دیده می‌شود. سمبل AND-invert قبلاً معرفی شد و متشکل بود از یک سمبل AND و به دنبال آن یک دایره کوچک که به آن حباب گفته شد و نقش متمم‌سازی را داشت. به همین ترتیب می‌توان گیت NAND را با سمبل گرافیکی OR با حبابی در هر ورودی نشان داد. سمبل invert-OR برای گیت NAND از تئوری دمورگان و با توجه به این قرار داد که دواير کوچک به منزله متمم کردن هستند به دست می‌آید.



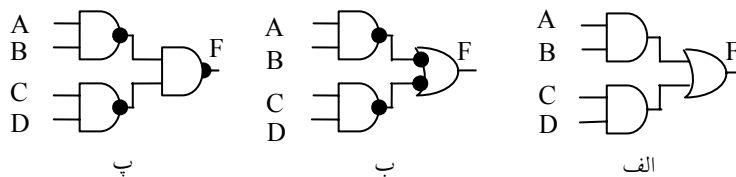
شکل ۲-۵: دو سمبل گرافیکی برای گیت NAND

دو سمبل گرافیکی فوق در طراحی و تحلیل مدارهای NAND مفید هستند. وقتی هر دو سمبل در یک نمودار به کار روند گوییم مدار با علائم مخلوط نشان داده شده است.

۱-۱-۵ پیاده‌سازی دو سطحی گیت NAND

برای پیاده‌سازی توابع بول با گیت‌های NAND، تابع باید به فرم جمع حاصلضرب‌ها باشد. برای درک ارتباط بین عبارت جمع حاصلضرب‌ها و معادل NAND آن، به نمودارهای منطقی شکل ۳-۵ توجه کنید. هر سه نمودار معادل بوده و تابع زیر را پیاده می‌نمایند.

$$F = AB + CD$$



شکل ۳-۵: سه راه پیاده‌سازی تابع $F = AB + CD$

در (الف) تابع با گیت‌های AND , OR پیاده‌سازی شده است. در (ب) گیت‌های AND با گیت‌های NAND و گیت OR نیز با یک گیت NAND که با سمبل OR-invert مشخص شده پیاده‌سازی شده است. توجه داشته باشید که یک حباب به معنی متمم و دو حباب در یک مسیر دوبار متمم‌سازی را نشان می‌دهند، پس می‌توانند حذف شوند. حذف حباب‌ها در گیت‌های (ب) مدار شکل (الف) را نتیجه می‌دهد. بنابراین دو نمودار یک تابع را پیاده‌سازی می‌کنند پس معادل‌اند.

در شکل ۳-۵ (پ)، خروجی گیت NAND با سمبل گرافیکی AND-invert ترسیم شده است. هنگام رسم نمودارهای منطقی NAND، هر یک از دو مدار (ب) یا (پ) پذیرفته است. مدار (ب) از علائم مخلوط استفاده کرده است و رابطه مستقیم‌تری را با عبارت بول پیاده شده نشان می‌دهد. صحت پیاده‌سازی NAND در شکل ۳-۵ (پ) می‌تواند به صورت جبری تحقیق شود. تابعی که این شکل را پیاده کرده است به سادگی با تئوری دموورگان قابل تبدیل به جمع حاصلضرب‌هاست:

$$F = ((AB)' (CD)')' = AB + CD$$

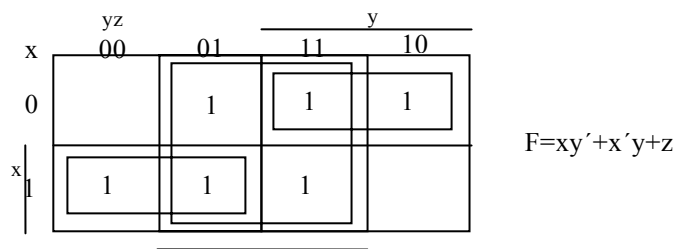
مثال ۱: تابع بول زیر را با گیت‌های NAND پیاده کنید.

$$F(x, y, z) = (1, 2, 3, 4, 5, 7)$$

اولین قدم در تبدیل، ساده‌سازی تابع در جمع حاصلضرب‌هاست. این کار به کمک نقشه شکل ۴-۵ انجام شده است و تابع حاصل به صورت زیر است.

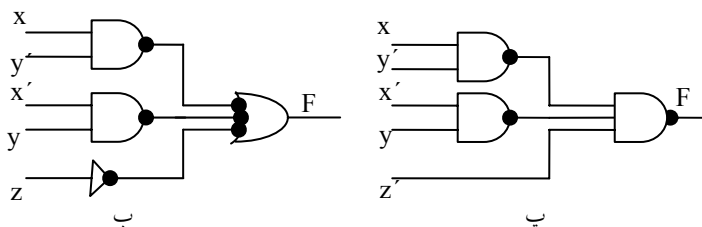
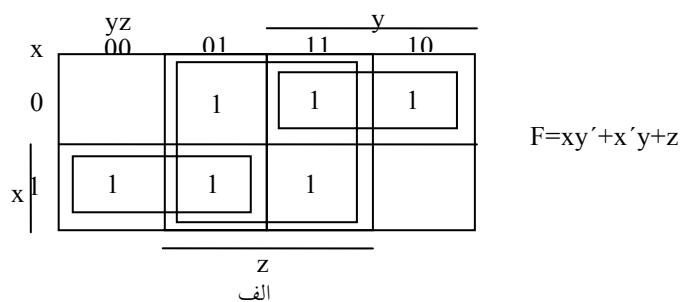
$$F = xy' + x'y + z$$

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۲۷



شکل ۵-۴: جدول کارنو تابع مثال ۱

پیاده‌سازی NAND دو سطحی در شکل ۵-۵ (الف) به صورت علائم مخلوط دیده می‌شود. توجه کنید که ورودی z باید یک گیت NAND یک ورودی باشد تا حباب موجود در گیت سطح دوم را جبران کند.



شکل ۵-۵: مدار منطقی مثال ۱

روش دیگری برای ترسیم نمودار منطقی در شکل ۵-۵ (ب) نشان داده شده است. در اینجا تمام گیت‌های NAND با سمبل یکسان ترسیم شده‌اند. وارون‌گر با ورودی z حذف شده است ولی متغیر ورودی متمم شده و با z' نشان داده شده است.

۵-۱-۲ روال تهیه مدار NAND از تابع بول

روالی که در مثال قبل توصیف شد بیان می‌دارد که یک تابع بول می‌تواند با دو سطح (یا دو طبقه) گیت NAND پیاده‌سازی شود. روال تهیه نمودار منطقی NAND از تابع بول به قرار زیر است:

- تابع را ساده کرده آن را به فرم جمع حاصل ضرب‌ها بنویسید.
- برای هر جمله ضرب موجود در تابع که حداقل دو لیترال دارد یک گیت NAND بکشید. ورودی به هر یک گیت NAND لیترال‌های جمله‌اند. این مجموعه، گیت‌های سطح اول را تشکیل می‌دهد.
- در سطح دوم، یک گیت NAND با ورودی‌هایی که از خروجی‌های سطح اول می‌آیند بکشید. از سمبل گرافیکی AND-invert یا OR-invert استفاده نمایید.
- یک جمله با یک لیترال نیاز به یک وارون‌گر در اولین سطح دارد. با این وجود، اگر تک لیترال متمم شده است می‌توان آن را مستقیماً به گیت NAND سطح دوم وصل کرد.

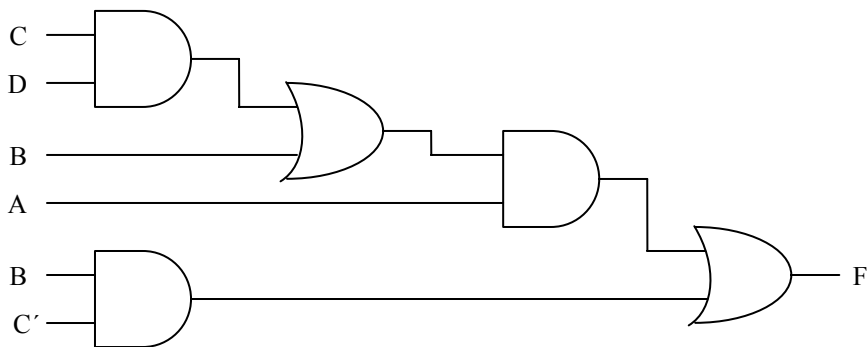
۵-۱-۳ مدارهای NAND چند سطحی

فرم استاندارد بیان توابع بول، پیاده‌سازی دو سطحی (طبقه) را نتیجه می‌دهد. مواردی وجود دارد که طراحی سیستم‌های دیجیتال یک ساختار گیتی با سه یا چهار طبقه را نتیجه می‌دهد. رایج‌ترین روش طراحی مدارهای چند طبقه بیان تابع بول بر حسب عملیات AND، OR، NOT می‌باشد. سپس می‌توان تابع را با گیت‌های AND، OR، پیاده‌سازی کرد. آنگاه در صورت لزوم تمام مدار را می‌توان به NAND تبدیل نمود. به عنوان مثال تابع بول زیر را ملاحظه کنید:

$$F = A(CD + B) + BC'$$

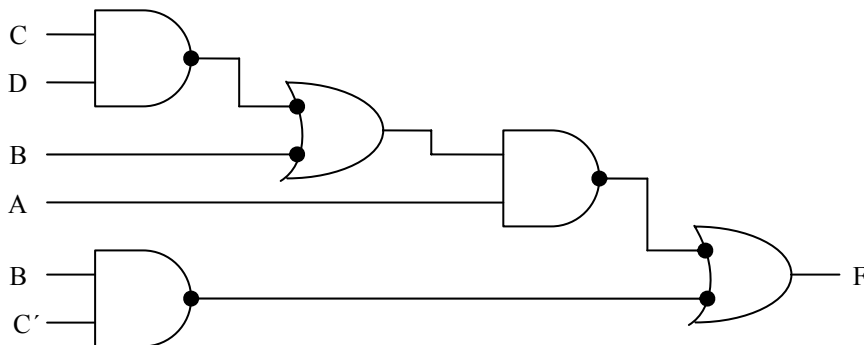
پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۲۹

گرچه می‌توان پرانتزها را حذف کرده و عبارت را به صورت جمع حاصلضرب استاندارد در آورد، ولی آن را به عنوان یک مدار چند طبقه مورد بررسی قرار می‌دهیم. پیاده‌سازی AND-OR برای آن در شکل ۶-۵ نشان داده شده است. در مدار، چهار طبقه گیت دیده می‌شود. اولین طبقه دو گیت AND دارد. دومین طبقه یک گیت OR و به دنبال آن یک AND در طبقه سوم آمده و در طبقه چهارم هم یک OR ملاحظه می‌شود.



شکل ۶-۵: مدار تابع $F = A(CD + B) + BC'$ با استفاده از گیت AND - OR

با استفاده از علائم مخلوط، می‌توان یک نمودار منطقی با الگویی از سطوح متناوب AND , OR را به سادگی به مدار NAND تبدیل کرد. این تبدیل در شکل ۷-۵ دیده می‌شود. روال این است که هر گیت AND را به سمبل AND-invert و هر گیت OR را



شکل ۷-۵: مدار تابع $F = A(CD + B) + BC'$ با استفاده از گیت NAND

به OR-invert تبدیل کنیم. مدار NAND حاصل، عملکرد یکسانی با نمودار AND-OR دارد به شرطی که در هر مسیر دو حباب وجود داشته باشد. حباب مربوط به ورودی B موجب می‌شود تا یک متمم اضافی صورت گیرد که باید آن را با متغیر ورودی مذکور به لیترال B' جبران کرد.

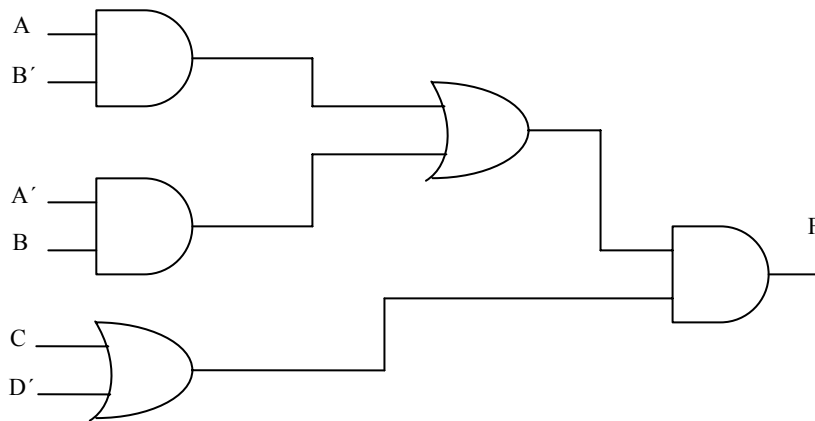
روال کلی نمودار چند طبقه AND-OR به نمودار تمام NAND با استفاده از علائم مخلوط به شرح زیر است:

- همه گیت‌های AND را با استفاده از سمبل‌های گرافیکی AND-invert به گیت NAND تبدیل کنید.
- همه گیت‌های OR را با سمبل‌های گرافیکی OR-invert به NAND تبدیل نمایید.

همه حباب‌ها را در نمودار چک کنید. برای هر حبابی که در یک مسیر جبران نشده است یک وارون‌گر (گیت NAND یک ورودی) وارد کنید و یا لیترال ورودی را متمم نمایید.

به عنوان مثالی دیگر تابع بول چند سطحی زیر را ملاحظه کنید.

$$F = (AB' + A'B)(C + D') \quad \bullet$$

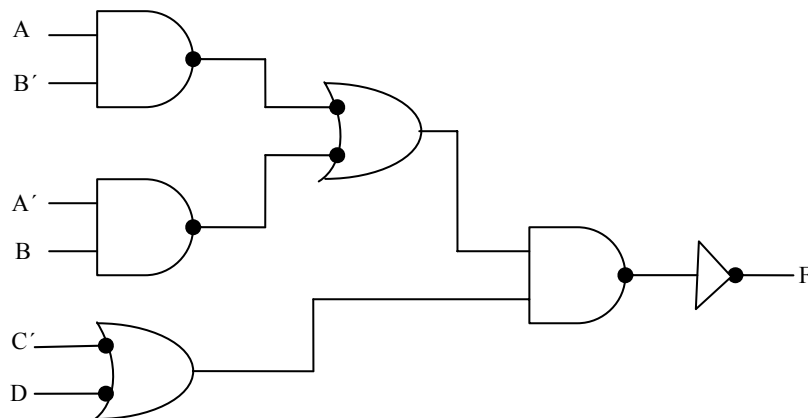


شکل ۵-۸: مدار تابع $F = (AB' + A'B)(C + D')$ با استفاده از گیت AND-OR

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۳۱

پیاده‌سازی AND-OR تابع در شکل ۵-۸ ملاحظه می‌شود که در آن سه طبقه گیت به کار رفته است.

در شکل ۵-۹ نمودار فرم تبدیل شده به NAND آن با علائم مخلوط دیده می‌شود. دو حباب اضافی مربوط به ورودی‌های C و D' موجب متمم شدن آنها به C' و D می‌گردد. حباب موجود در گیت NAND خروجی، مقدار خروجی را متمم می‌کند بنابراین برای به دست آوردن مقدار اصلی تابع مجبوریم یک گیت وارون‌گر در خروجی به کار ببریم.

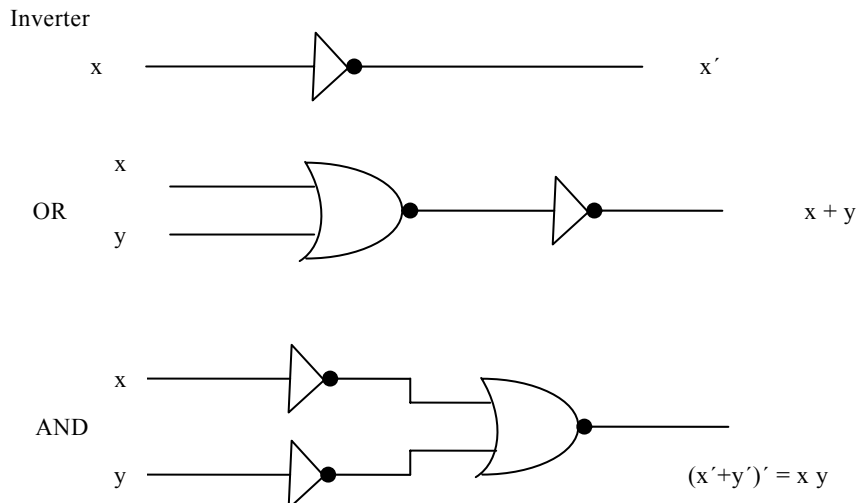


شکل ۵-۹: مدار تابع $F = (AB' + A'B)(C + D')$ با استفاده از گیت AND-OR

۵-۲ مدارهای NOR و روش پیاده‌سازی آنها

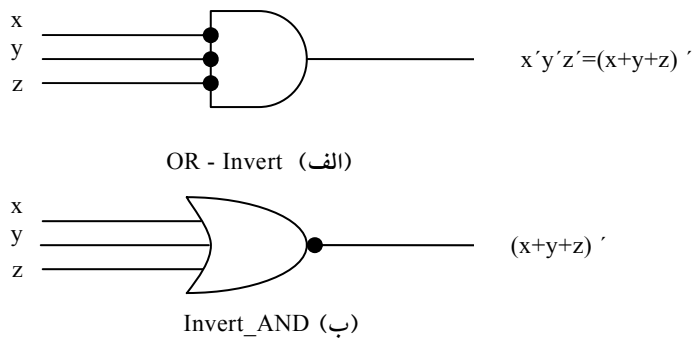
گیت NOR به عنوان گیت یونیورسال دیگری است که برای پیاده‌سازی هر تابع بول به کار می‌رود. عمل NOR دوگان NAND است. بنابراین تمام روال‌ها و قوانین منطق NOR دوگان روال‌های متناظر و قوانین حاصل در منطق NAND هستند. پیاده‌سازی اعمال AND، OR، NOT با گیت‌های NOR در شکل ۵-۱۰ ملاحظه می‌گردد. عمل متمم، با گیت NOR یک ورودی حاصل شده و عیناً مثل وارون‌گر عمل می‌کند. عمل

OR نیاز به دو گیت NOR و عمل AND از یک گیت NOR که در هر ورودی اش یک وارون گر دارد حاصل می شود.



شکل ۵-۱۰: عملیات منطقی با گیت های NOR

دو سمبل گرافیکی برای علائم مخلوط در شکل ۵ - ۱۱ دیده می شود. سمبل OR-invert عمل NOR را با یک OR و به دنبال آن یک متمم تعریف می کند. هر دو سمبل عمل NOR یکسانی را به نمایش می گذارند و از نظر منطقی با توجه به تئوری دمورگان یکی هستند.



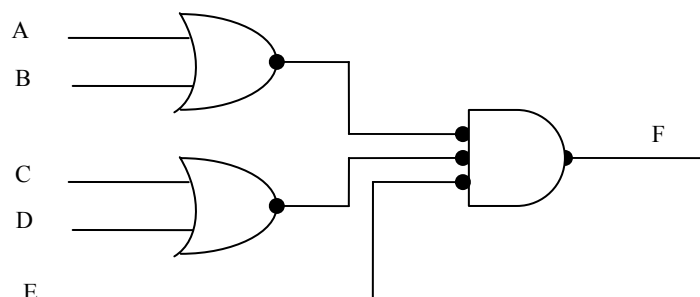
شکل ۵-۱۱: دو سمبل گرافیکی برای گیت NOR

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۳۳

پیاده‌سازی دو طبقه با گیت NOR لازم می‌دارد تا تابع به صورت ضرب حاصل جمع‌ها ساده شود. به خاطر دارید که عبارت ضرب حاصل جمع‌های ساده شده از نقشه با ترکیب 0ها و متمم کردن آنها به دست می‌آید. عبارت ضرب حاصل جمع‌ها با گیت‌های OR در اولین سطح که جملات جمع را تولید می‌کنند پیاده‌سازی می‌شود. به دنبال آنها گیت AND برای تولید ضرب دیده می‌شود. تبدیل نمودار OR-AND به NOR با تبدیل گیت‌های OR با گیت NOR با استفاده از سمبل گرافیکی invert-AND صورت می‌گیرد. یک جمله تک لیترال که به یک گیت سطح دوم برود باید متمم گردد. شکل ۵-۱۲ پیاده‌سازی یک تابع را به فرم ضرب حاصل جمع‌ها نشان می‌دهد:

$$F = (A + B) (C + D) E$$

الگوی OR-AND را به سادگی با حذف حباب‌ها در طول هر مسیر می‌توان شناسایی کرد. متغیر E برای جبران سومین حباب در ورودی گیت سطح دوم متمم شده است.

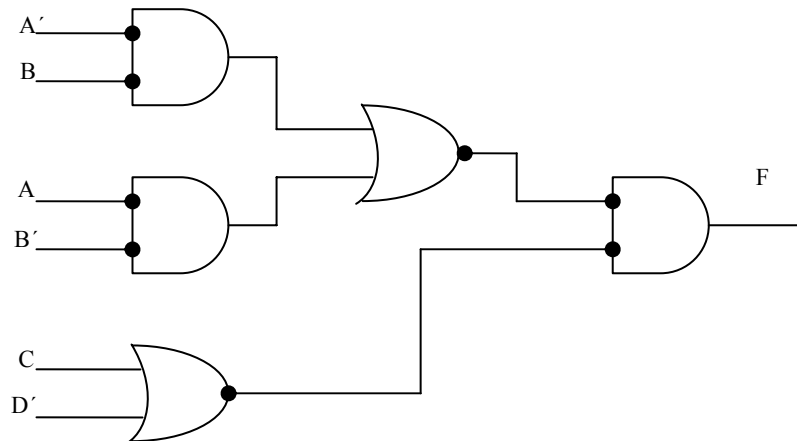


شکل ۵-۱۲: پیاده‌سازی $F = (A + B) (C + D) E$

روال تبدیل یک نمودار AND-OR چند سطح به نمودار تمام NOR مشابه آنچه برای گیت‌های NAND دیدیم، می‌باشد. در حالت NOR، باید هر گیت OR را به یک سمبل OR-invert و هر گیت AND را به یک invert-AND تبدیل نماییم. هر حبابی که به وسیله حباب دیگر در همان مسیر جبران نشود نیاز به یک وارون‌گر یا متمم شدن لیترال ورودی دارد. برای مثال تابع بول برای این مدار به شکل زیر است:

$$F = (AB' + A'B)(C + D')$$

نمودار معادل AND-OR را می‌توان با حذف حباب‌ها تشخیص داد. برای جبران حباب‌ها در چهار ورودی، لازم است لیترال‌های ورودی مربوطه متمم شوند.



شکل ۱۳-۵: پیاده‌سازی $F = (AB' + A'B)(C + D')$ با گیت‌های NOR

۳-۵ منطق سیمی

دیگر پیاده‌سازی‌های دو سطحی گیت‌هایی که بیشتر در مدارهای مجتمع یافت می‌شوند از نوع NAND, NOR هستند. به همین دلیل، پیاده‌سازی‌های منطقی NAND, NOR از دیدگاه عملی مهم‌تراند. در بعضی از گیت‌های NAND یا NOR (و نه همه آنها) این امکان وجود دارد تا با اتصال سیم بین خروجی‌های دو گیت، یک تابع منطقی مشخص تولید کرد. این منطق را منطق سیم‌بندی یا سیمی می‌نامند. مثلاً گیت‌های TTL NAND کلکتور باز وقتی به هم گره زده شوند تولید منطق AND سیمی (Wired-AND) را می‌نمایند. منطق AND سیمی که با گیت‌های NAND انجام می‌شود در شکل ۱۴-۵ (الف) ترسیم شده است. گیت AND با ترسیم خطوط تا مرکز گیت نشان داده شده تا بدین ترتیب از گیت‌های AND معمولی تفکیک شود. گیت AND سیمی (یا اتصال)

پایاده سازی مدارهای دیجیتال با گیت های NAND و NOR ۱۳۵

یک گیت فیزیکی نیست، بلکه فقط سمبلی برای نمایش تابع حاصل از اتصال سیمی است.

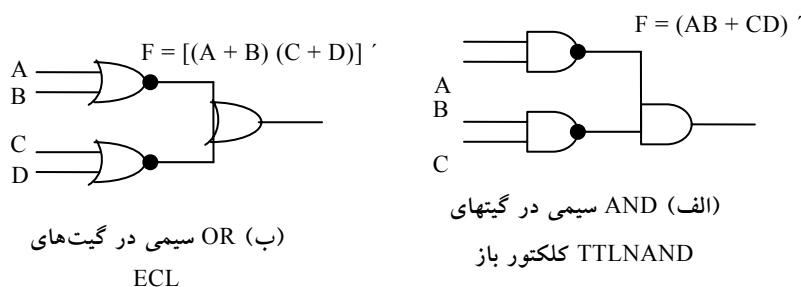
تابع منطقی پیاده شده با مدار شکل ۵-۱۴ (الف) برابر زیر است.

$$F = (AB)' \cdot (CD)' = (AB + CD)'$$

و به آن تابع AND-OR - INVERT می گویند. به طور مشابه خروجی NOR گیت های ECL برای اجرای یک تابع OR سیمی به هم گره زده می شوند. تابع منطقی پیاده سازی شده با مدار شکل ۵-۱۴ (ب) چنین است:

$$F = (A + B)' + (C + D)' = [(A + B)(C + D)]'$$

و به آن تابع OR - AND - INVERT می گویند.



شکل ۵-۱۴: منطق سیمی

یک گیت منطقی سیمی تولید گیت سطح دوم فیزیکی را نمی کند، زیرا تنها یک اتصال سیمی است. با این وجود به هنگام بحث، مدارهای شکل ۵-۱۴ را به عنوان پیاده سازی های دو سطحی یا دو طبقه در نظر می گیریم. اولین طبقه متشکل از گیت های NAND (یا NOR) و دومین طبقه تنها یک گیت AND یا OR دارد. در بحث های بعدی اتصال سیمی در سمبل گرافیکی حذف می گردد.

۵-۴ فرم‌های مفید گیت‌ها

از نقطه نظر تئوری یافتن ترکیب‌های دو سطحی ممکن گیت‌ها آموزنده است. در اینجا چهار نوع گیت AND، OR، NAND و NOR را بررسی می‌کنیم. اگر یکی از انواع گیت‌ها را به سطح اول و نوع دیگر را به سطح دوم نسبت دهیم، در می‌یابیم که 16 ترکیب ممکن از فرم دو سطحی وجود دارد. می‌توان در هر دو سطح یک نوع گیت مانند NAND-NAND را هم به کار برد. هشت ترکیب از آنها، فرم زاید خوانده می‌شوند زیرا در حقیقت یک عمل ساده منطقی را انجام می‌دهند. این نکته در مواردی که هر دو سطح اول و دوم از گیت‌های AND تشکیل شده‌اند به خوبی مشهود است. خروجی مدار صرفاً تابع AND از همه متغیرهای ورودی است. هشت فرم مفید دیگر نوعی پیاده‌سازی جمع حاصل ضرب‌ها و یا ضرب حاصل جمع‌ها را تولید می‌کند این هشت فرم مفید عبارتند از:

- AND-OR
- OR-AND
- NOR-NOR
- NAND-NAND
- NAND-AND
- NOR-OR
- AND-NOR
- OR-NAND

اولین گیت در هر یک از فرم‌های فوق سطح اول پیاده‌سازی را تشکیل می‌دهد. دومین گیت در لیست تنها گیتی است که در سطح دوم قرار گرفته است. توجه کنید هر دو فرمی که در یک سطر آمده‌اند دوگان یکدیگرند. فرم‌های AND-OR و OR-AND، فرم‌های دو سطح اصلی بحث شده در بخش‌های قبل می‌باشند. فرم‌های

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۳۷

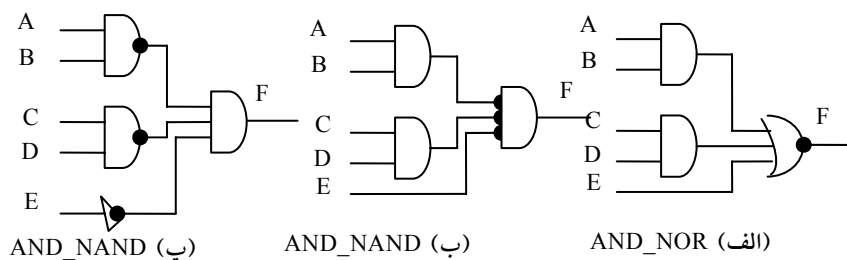
NAND-NAND و NOR-NOR در بخش‌های قبل ارائه شدند. چهار فرم باقیمانده نیز در این بخش بررسی می‌شوند.

۵-۴-۱ پیاده‌سازی AND-OR-INVERT

دو فرم NAND-AND و AND-NOR معادل یکدیگرند و می‌توان آنها را همزمان شرح داد. هر دو تابع طبق شکل ۵-۱۵، عمل AND-OR-INVERT را اجرا می‌کنند. فرم AND-NOR، همان عمل AND-OR با یک وارون‌گر در خروجی است. این فرم تابع زیر را پیاده‌سازی می‌کند.

$$AB=(F)' + CD + E$$

با استفاده از سمبل گرافیکی معادل دیگری برای گیت NOR، نمودار شکل ۵-۱۵ (ب) به دست می‌آید. توجه کنید که تک متغیر E متمم نشده است زیرا تنها تغییر، در سمبل گرافیکی گیت NOR صورت گرفته است. اکنون حباب‌ها را از پایانه‌های ورودی گیت سطح دوم به پایانه‌های خروجی گیت‌های سطح اول انتقال می‌دهیم. برای جبران هر حباب یک وارون‌گر در ازاء هر متغیر لازم است. به همین ترتیب می‌توان وارون‌گر را حذف کرد به شرطی که E متمم شود. مدار شکل ۵-۱۵ (پ)، فرم NAND-AND است و برای پیاده‌سازی تابع AND-OR-INVERT نشان داده شده است.



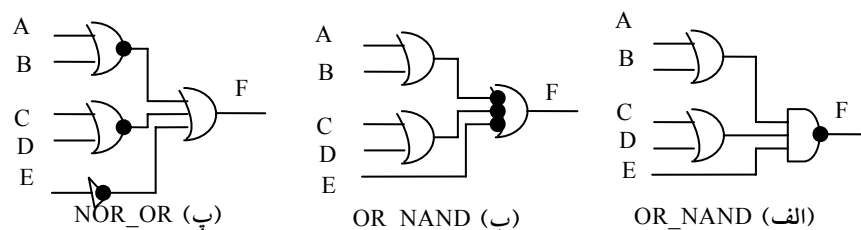
شکل ۵-۱۵: مدارهای AND-OR-INVERT ؛

پیاده‌سازی AND-OR نیاز به یک عبارت جمع حاصلضربها دارد. پیاده‌سازی AND-OR-INVERT مشابه آن است، به جز اینکه یک وارون‌گر اضافی دارد. بنابراین اگر متمم تابع به صورت جمع حاصلضربها ساده شود (با ترکیب 0ها در نقشه)، می‌توان F' را با بخش AND-OR تابع پیاده‌سازی کرد. وقتی که F' از داخل وارون‌گر عبور کند، خروجی F تابع را تولید می‌نماید. مثالی در مورد پیاده‌سازی AND-OR-INVERT بدنبال آمده است.

۲-۴-۵ پیاده‌سازی OR-AND-INVERT

فرم‌های OR-NAND و NOR-OR تابع OR-AND-INVERT را اجرا می‌کنند. این فرم‌ها در شکل ۱۶-۵ نشان داده شده‌اند. فرم OR-NAND، فرم OR-AND را تداعی می‌کند. به جز اینکه در خروجی گیت NAND، عمل متمم با حباب انجام می‌شود. در این شکل تابع زیر پیاده‌سازی شده است.

$$F = [(A+B)(C+D)E]'$$



شکل ۱۶-۵: مدارهای OR-AND-INVERT ؛ $F = [(A+B)(C+D)E]'$

با استفاده از فرم دیگر گیت NAND نمودار شکل ۱۶-۵ (ب) به دست می‌آید. مدار در (پ) با انتقال دوایر کوچک از ورودی‌های گیت سطح دوم به خروجی‌های گیت‌های سطح اول حاصل می‌گردد. مدار شکل ۱۶-۵ (پ) یک فرم NOR-OR است و قبلاً برای پیاده‌سازی تابع OR-AND-INVERT در شکل ۱۴-۵ نشان داده شد. پیاده‌سازی OR-AND-INVERT به عبارتی به فرم ضرب حاصل جمع‌ها احتیاج دارد.

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۳۹

اگر متمم تابع به صورت ضرب حاصل جمع‌ها ساده شود می‌توانیم F' را با بخش OR-AND تابع پیاده‌سازی کنیم. پس از عبور F' از بخش INVERT، متمم F' یعنی F در خروجی حاصل خواهد شد.

مثال ۲: تابع شکل ۱۷-۵ (الف) را به فرم دو سطحی پیاده‌سازی کنیم.

متمم تابع با ترکیب ۰ها در نقشه به فرم جمع حاصل ضرب‌های ساده شده به دست می‌آید.

$$F' = x'y + xy' + z$$

خروجی نرمال این تابع می‌تواند به صورت زیر بیان شود.

$$F = (x'y + xy' + z)'$$

$$F = x'y'z' + xyz'$$

که به فرم AND-OR-INVERT است. پیاده‌سازی‌های AND-NOR و NAND-AND در شکل ۱۷-۵ (ب) دیده می‌شوند. توجه کنید که در پیاده‌سازی NAND-AND به گیت NAND یک ورودی یا گیت وارون‌گر نیاز است، ولی در حالت AND-NOR به آن نیازی نیست. اگر در عوض z از z' استفاده کنیم به وارون‌گر احتیاجی نیست. فرم‌های OR-AND-INVERT نیاز به عملیات ساده شده‌ای از متمم تابع به فرم حاصل جمع‌ها دارد. برای تهیه این عبارت، ابتدا ۱ها را در نقشه ترکیب می‌کنیم.

آنگاه متمم تابع را به دست می‌آوریم.

$$F' = (x+y+z)(x'+y'+z)$$

خروجی نرمال F اکنون به فرم زیر نوشته می‌شود:

$$F = [(x+y+z)(x'+y'+z)]'$$

که به فرم OR-AND-INVERT بیان شده است. با استفاده از این عبارت تابع را می‌توانیم به فرم OR-NAND یا NOR-OR که در شکل ۱۷-۵ (پ) نشان داده شده است، نیز پیاده کنیم.

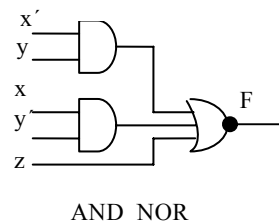
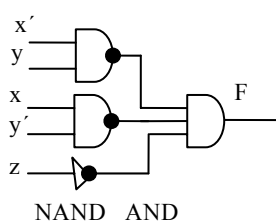
		yz		y	
		00	01	11	10
x	0	1	0	0	0
	1	0	0	0	1

z

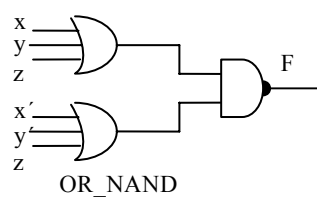
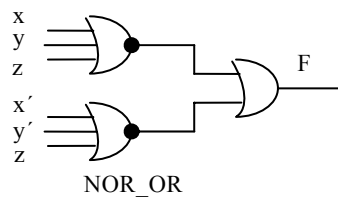
$$F = x'y'z' + xyz'$$

$$F = x'y + xy' + z$$

(الف) $F = (x'y + xy' + z)'$



(ب) $F = (x'y + xy' + z)'$



(پ) $F = [(x + y + z)(x' + y' + z)]'$

شکل ۵-۱۷: پیاده‌سازی دو طبقه تابع F

۵-۵ تابع OR انحصاری

OR انحصاری (XOR) که با علامت $\oplus$ نشان داده می‌شود یک عملگر منطقی است که

تابع بولی زیر را اجرا می‌نماید:

$$x \oplus y = xy' + x'y$$

۱۴۱ پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR

این تابع هنگامی برابر 1 است که فقط x یا y برابر 1 باشند، ولی هردو آنها به طور همزمان 1 نباشند. NOR انحصاری (XNOR) که به آن هم ارزی هم می‌گویند عمل زیر را اجرا می‌نماید.

$$(x \oplus y)' = xy + x'y'$$

هنگامی که این تابع برابر 1 است که هر دو متغیر x و y به طور همزمان برابر 1 یا برابر 0 باشند. به کمک جدول درستی یا دستکاری جبری می‌توان نشان داد که NOR انحصاری متمم OR انحصاری است:

$$\begin{aligned}(x \oplus y)' &= (xy' + x'y)' \\ &= (x' + y)(x + y') \\ &= xy + x'y'\end{aligned}$$

روابط زیر در مورد OR انحصاری معتبرند:

$$x \oplus 0 = x$$

$$x \oplus 1 = x'$$

$$x \oplus x = 0$$

$$x \oplus x' = 1$$

$$x \oplus y' = x' \oplus y = (x \oplus y)'$$

هر یک از این روابط می‌تواند با به‌کارگیری جدول درستی و جایگزینی $\oplus$ با عبارت بولی هم ارزی ثابت شود. و نیز می‌توان نشان داد که عمل OR انحصاری خاصیت جابجایی و شرکت‌پذیری را دارد. یعنی:

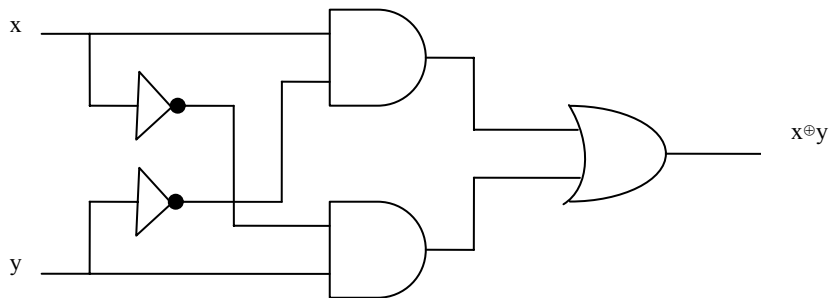
$$A \oplus B = B \oplus A$$

و

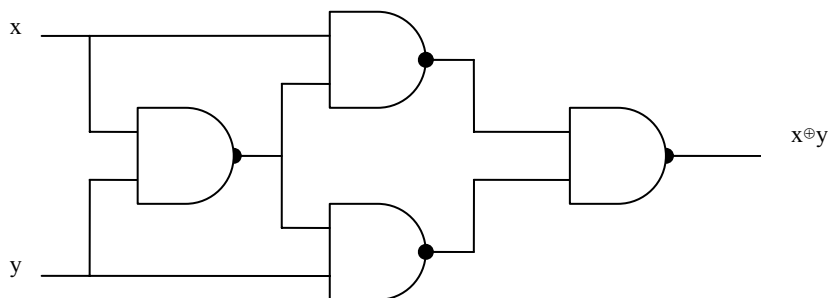
$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

این بدان معنی است که دو ورودی گیت OR انحصاری بدون تاثیر بر عمل قابل تعویض‌اند. و نیز به این معنی است که یک عمل OR انحصاری سه متغیره را نیز می‌توانیم ارزیابی کنیم و به همین دلیل سه متغیر یا بیشتر را بدون پراکنش بیان می‌نماییم.

با این وجود گیت‌های OR انحصاری با ورودی‌های متعدد مشکل ساخت سخت‌افزاری را دارند. در واقع، حتی تابع دو ورودی آن هم با عنوان گیت‌های دیگر ساخته می‌شود. یک تابع OR انحصاری دو ورودی که با گیت‌های معمولی AND، OR و NOT ساخته شده در شکل ۱۸-۵ (الف) دیده می‌شود.



(الف) با گیت‌های AND-OR-NOT



(ب) با گیت‌های NAND

شکل ۱۸-۵: پیاده‌سازی گیت XOR

شکل ۱۸-۵ (ب) پیاده‌سازی OR انحصاری را با چهار گیت NAND نشان می‌دهد. گیت NAND اول عمل $(xy)' = (x' + y')$ را اجرا می‌کند. مدارهای NAND دو طبقه دیگر جمع حاصلضرب ورودی‌ها را تهیه می‌نمایند:

$$\begin{aligned} (x' + y')x + (x' + y')y &= xy' + x'y \\ &= x \oplus y \end{aligned}$$

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۴۳

تنها توابع بولی محدودی بر حسب OR انحصاری بیان می‌شوند. با این وجود این تابع به کرات ضمن طراحی سیستم‌های دیجیتال به کار گرفته می‌شود. خصوصاً در عملیات حسابی و خطایابی و تصحیح خطا بسیار مفید است.

۵-۱-۵ تابع فرد

عملگر OR انحصاری با سه متغیر یا بیشتر را می‌توان با جایگزینی سمبل $\oplus$ با عبارت بولی معادلش به یک تابع بولی معمولی تبدیل کرد. بخصوص، حالت سه متغیره را می‌توان به یک عبارت بولی مطابق زیر تبدیل نمود.

$$\begin{aligned} A \oplus B \oplus C &= (AB' + A'B)C' + (AB + A'B')C \\ &= AB'C' + A'BC' + ABC + A'B'C \\ &= \Sigma(1,2,4,7) \end{aligned}$$

عبارت بول به وضوح نشان می‌دهد که تابع OR انحصاری سه متغیره برابر با 1 است به شرطی که فقط یک متغیر 1 باشد و یا هر سه متغیر برابر 1 باشند. برخلاف حالت دو متغیره، که فقط یک متغیر باید برابر 1 می‌بود، در حالت سه یا چند متغیره، نیاز این است که تعداد فردی از متغیرها برابر 1 باشند. در نتیجه عمل XOR چند متغیره را تابع فرد می‌خوانند.

تابع بول حاصل از عمل XOR سه متغیره به صورت جمع چهار مینترم است که مقادیر عددی آنها 001، 010، 100 و 111 می‌باشد. هر یک از این اعداد دودویی تعداد فردی 1 دارند. چهار مینتروم دیگری که در تابع لحاظ نشده‌اند 000، 011، 101 و 110 بوده و تعداد زوجی 1 در مقدار دودویی آنها وجود دارد. به‌طور کلی تابع XOR با n متغیر یک تابع فرد است که به صورت جمع 2^{n-1} مینترم که مقادیر عددی آنها تعداد فردی 1 دارد بیان می‌شود.

تعریف یک تابع فرد با ترسیم آن در یک نقشه شفاف تر می‌کنیم. شکل ۵-۱۹ (الف) نقشه را برای تابع XOR سه متغیره نشان می‌دهد. چهار مینترم تابع یک مربع در میان با

هم فاصله دارند. تابع فرد از چهار مینترمی که مقادیر دودویی‌اش تعداد فردی 1 دارند شناسایی می‌شود.

A	BC		B	
	00	01	11	10
0	1		1	
1		1		1

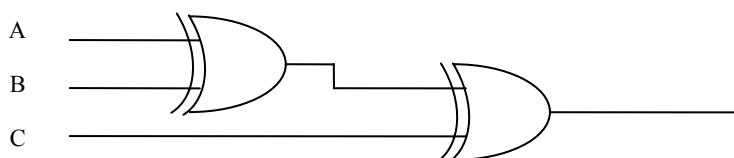
ب- تابع زوج
 $F = (A \oplus B \oplus C)'$

A	BC		B	
	00	01	11	10
0		1		1
1	1		1	

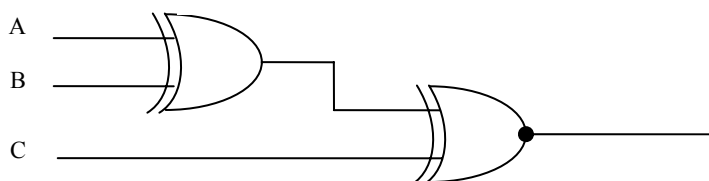
الف- تابع فرد
 $F = A \oplus B \oplus C$

شکل ۵-۱۹ نقشه تابع XOR سه متغیره

متمم یک تابع فرد، یک تابع زوج است. طبق شکل ۵-۱۹ (ب)، تابع زوج سه متغیره هنگامی 1 است که تعداد زوجی متغیر در یک مینترم، 1 باشد (از جمله مینترمی که هیچ یک از متغیرها در آن 1 نیست).



(الف) تابع فرد سه ورودی



(ب) تابع زوج سه ورودی

شکل ۵-۲۰: نمودار منطقی توابع فرد و زوج

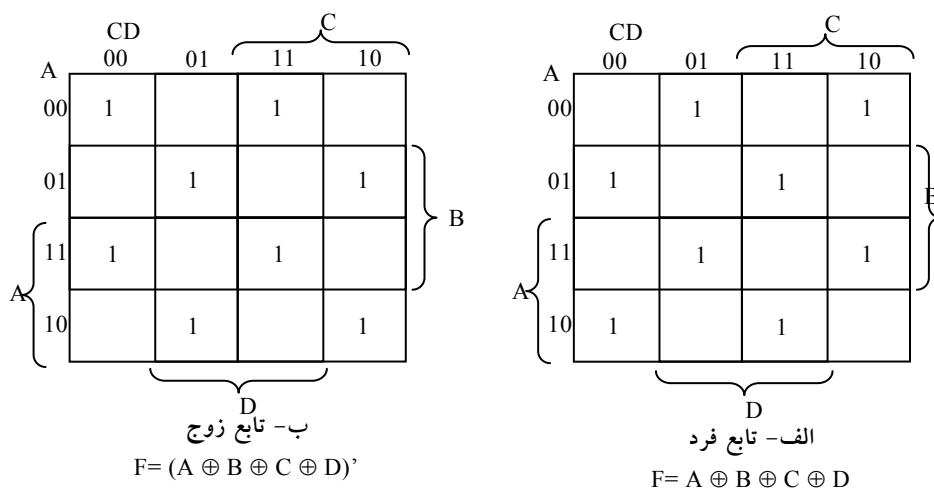
۱۴۵ پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR

تابع فرد ۳ ورودی را می‌توان با گیت دو ورودی هم طبق شکل ۲۰-۵ (الف) پیاده‌سازی کرد. متمم یک تابع فرد با جایگزینی گیت خروجی با یک گیت XNOR طبق شکل ۲۰-۵ (ب) به دست می‌آید.

اکنون عملکرد XOR چهار متغیره را ملاحظه کنید. با دستکاری جبری، می‌توانیم جمع میترم‌های این تابع را به دست آوریم.

$$\begin{aligned} A \oplus B \oplus C \oplus D &= (AB' + A'B)(CD' + C'D) \\ &= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D) \\ &= \Sigma(1, 2, 4, 7, 8, 11, 13, 14) \end{aligned}$$

برای تابع بول چهار متغیره ۱۶ میترم وجود دارد. نیمی از میترم‌ها دارای تعداد فردی ۱ در مقادیر عددی خود هستند؛ نیمه دیگر دارای تعداد زوجی ۱ در میترم می‌باشند. هنگام ترسیم تابع در نقشه، مقدار عدد دودویی هر میترم از اعداد سطر و ستون مربعی که میترم را نمایش می‌دهد به دست می‌آید.



شکل ۲۱-۵: نقشه برای تابع XOR چهار متغیره

نقشه شکل ۲۱-۵ (الف) مربوط به تابع XOR چهار متغیره است. این یک تابع فرد است زیرا مقادیر دودویی همه میترم‌ها تعداد فردی ۱ دارند. متمم یک تابع فرد هم

یک تابع زوج است. طبق شکل ۵-۲۱ (ب) تابع زوج چهار متغیره هنگامی 1 است که تعداد زوجی از متغیرها در مینترم برابر 1 باشد.

۵-۵-۲ تولید و چک توازن

توابع XOR در سیستم‌هایی که به کدهای عیب یاب و تصحیح کننده خطا نیاز دارند بسیار مفیدند. همانطور که در فصل اول ملاحظه شد، یک بیت توازن به منظور تشخیص خطا در حین انتقال اطلاعات دودویی به آن اضافه می‌شود. بیت توازن، بیتی اضافی است که با پیام دودویی همراه می‌شود تا تعداد 1ها را زوج یا فرد کند. پیام، از جمله بیت توازن، ارسال و سپس در مقصد برای تشخیص خطا چک می‌شود. اگر توازن چک شده با آنچه ارسال شده است تطابق نداشته، یک خطا اعلام می‌گردد. مداری که بیت توازن را در فرستنده تولید می‌کند، مولد توازن نامیده می‌شود مداری که توازن را در سمت گیرنده چک می‌کند چک کننده توازن خوانده می‌شود.

فرض کنید بخواهیم یک پیام سه بیتی را همراه با یک بیت توازن زوج ارسال کنیم. جدول شکل ۵-۲۲، جدول درستی را برای مولد توازن نشان می‌دهد. سه بیت x و y و z که پیام را تشکیل می‌دهند ورودی به مدار هستند. بیت توازن P ، خروجی است. برای توازن زوج، بیت P باید طوری باشد که تعداد کل 1ها را زوج کند (از جمله P). از جدول درستی می‌بینیم که P یک تابع فرد را تشکیل می‌دهد زیرا برای مینترم‌هایی که

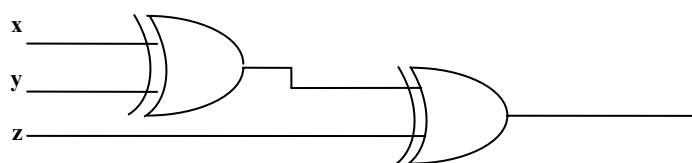
پیام سه بیتی			بیت
x	y	z	P
0	0	0	0
0	0	0	1
0	0	1	1
0	0	1	0
1	0	0	1
1	0	0	0
1	0	1	0
1	0	1	1

شکل ۵-۲۲: جدول درستی مولد توازن زوج

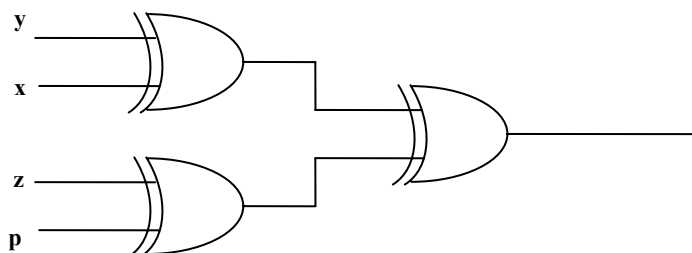
پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۴۷

تعداد فردی 1 دارند باید برابر 1 شود. بنابراین P به صورت یک تابع XOR سه متغیره بیان می‌شود.

نمودار منطقی مولد توازن در شکل ۵-۲۳ (الف) ملاحظه می‌شود. سه بیت پیام، همراه با بیت توازن به مقصد ارسال می‌شوند و در آنجا به مدار چک کننده توازن برای چک کردن خطای محتمل به هنگام ارسال، وارد می‌گردند. چون اطلاعات با توازن زوج ارسال شده است، چهار بیت دریافتی باید تعداد زوجی 1 داشته باشد.



(الف) مولد توازن زوج ۳ بیتی



(ب) چک کننده توازن زوج ۴ بیتی

شکل ۵-۲۳: نمودار منطقی مولد و چک کننده توازن

$$P=x\oplus y\oplus z$$

خطا در حین انتقال هنگامی رخ می‌دهد که چهار بیت دریافتی تعداد فردی 1 دارد، و این به معنی رخ داد خطا در حین انتقال است. خروجی چک کننده توازن که با C مشخص شده است به هنگام رخ داد خطا برابر 1 می‌شود. یعنی اگر چهار بیت دریافتی تعداد فرد 1 داشته باشد خطا رخ داده است. جدول شکل ۵-۲۴ جدول درستی برای چک کننده توازن زوج است.

با توجه به آن ملاحظه می‌شود که تابع C متشکل از هشت مینترم با مقدار دودویی دارای تعداد فردی ۱ است. این مطلب مربوط به شکل ۵-۲۳ (الف) است که تابع فرد را نشان می‌دهد. می‌توان چک کننده توازن را با گیت‌های XOR پیاده‌سازی کرد:

$$C = x \oplus y \oplus z \oplus p$$

چهار بیت دریافتی			چک خطای توازن	
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

شکل ۵-۲۴: جدول درستی چک کننده توازن زوج

نمودار منطقی چک کننده توازن در شکل ۵-۲۳ (ب) ملاحظه می‌گردد. لازم به تذکر است که مولد توازن را با مدار شکل ۵-۲۳ (ب) نیز می‌توان تولید کرد به شرطی که ورودی p به منطق ۰ متصل گردد و خروجی نیز با p نام‌گذاری شود دلیل این است که $z \oplus 0 = z$ بوده و موجب می‌شود تا z از گیت بدون تغییر عبور کند. مزیت این است که برای هر دو مدار تولید و چک کننده توازن از یک مدار مشابه می‌توان استفاده کرد.

با توجه به مثال قبل واضح است که تابع مولد توازن و نیز چک کننده دارای نیمی از کل مینترم‌ها هستند و مقادیر عددی آنها تعداد زوج یا فردی ۱ دارند. در نتیجه می‌توان

پایاده سازی مدارهای دیجیتال با گیت های NAND و NOR ۱۴۹

آنها را با گیت های XOR پیاده سازی کرد. تابعی با تعداد زوجی 1 متمم تابع فرد است. این تابع با XOR پیاده سازی می شود ولی گیت واقع در خروجی باید XNOR باشد تا متمم لازم را تولید نماید.

۵-۶ زبان توصیف سخت افزاری (HDL)

زبان توصیف سخت افزاری، زبانی است که سخت افزار سیستم های دیجیتال را به فرم متنی توصیف می نماید. در واقع این زبان، یک زبان برنامه نویسی است، ولی خصوصا حول توصیف ساختارهای سخت افزاری و رفتار آنها بنا نهاده شده است. می توان از آن برای نمایش نمودارهای منطقی، عبارت بولی و دیگر مدارهای دیجیتال پیچیده استفاده کرد. HDL به عنوان یک زبان مستندسازی برای نمایش و مستند کردن سیستم های دیجیتال به کار می رود به نحوی که قابل خواندن به وسیله انسان ها و کامپیوترها می باشد. این زبان به عنوان زبان تبادل بین دو طراح هم به کار می رود. محتوای زبان به طور موثر و نیز به سادگی قابل ذخیره ، بازیابی و پردازش به وسیله نرم افزار کامپیوتر است.

در پردازش HDL دو کاربرد وجود دارد:

- شبیه سازی
- سنتز

شبیه سازی منطقی نمایشی از ساختار و رفتار یک سیستم منطقی دیجیتال به کمک کامپیوتر است. یک شبیه ساز توصیف HDL را تفسیر کرده و یک خروجی قابل خواندن مانند نمودار زمانی، تولید می نماید و بدین وسیله رفتار سخت افزار را قبل از ساخت پیش بینی می نماید. HDL امکان تشخیص خطای عملیاتی در طراحی را بدون نیاز به خلق فیزیکی آن، فراهم می سازد. خطاهایی که در حین شبیه سازی شناسایی می شوند را می توان با اصلاح عبارت مربوطه در HDL رفع کرد. امکاناتی که عملیات

طرح را تست می‌کند، برنامه تست می‌نامند. بنابراین برای شبیه‌سازی یک سیستم، طرح ابتدا در HDL توصیف شده و سپس صحت عمل آن با شبیه‌سازی طرح و تست آن به وسیله برنامه تست که در HDL نوشته می‌شود، تحقیق می‌گردد.

سنتز منطقی فرایندی است که طی آن از قطعات و اتصال بین آنها به نام netlist در مدل سیستم دیجیتالی که در HDL توصیف شده است لیستی تهیه می‌گردد. netlist سطح گیت را می‌توان در ساخت یک مدار مجتمع یا طرح برد مدار چاپی به کار برد. سنتز منطقی مشابه با کامپایل یک برنامه در زبان سطح بالاست. تفاوت در این است که، در عوض تولید کد منتج، یک بانک اطلاعاتی تولید می‌نماید که در آن دستور العمل‌های ساخت یک قطعه سخت‌افزاری دیجیتال فیزیکی توصیف شده با کد HDL آمده است. سنتز منطقی بر روال‌های مبتنی است که مدارهای دیجیتال را پیاده‌سازی می‌کنند و شامل آن بخش از یک طراحی دیجیتال که قابل اتوماتیک شدن با نرم‌افزار کامپیوتر باشد.

در صنعت HDL‌های انحصاری متعددی وجود دارند که به وسیله کمپانی‌ها برای طراحی یا کمک به طراحی مدارهای مجتمع ساخته شده‌اند. دو استاندارد HDL به وسیله IEEE پشتیبانی می‌شوند: VHDL و Verilog HDL. VHDL یک زبان تحت کنترل وزارت دفاع بود که در حال حاضر به صورت تجاری و در دانشگاه‌ها استفاده می‌شود. Verilog به عنوان یک زبان انحصاری که به وسیله کمپانی Cadence Data System ارتقاء یافت، ولی بعد کنترل آن را به مجموعه‌ای از کمپانی‌ها به نام Open Verilog International (OVI) محول کرد. VHDL نسبت به Verilog زبان سخت‌تری است. چون Verilog برای یادگیری ساده‌تر است، ما آن را در این کتاب انتخاب کرده ایم. با این وجود، توصیف‌های Verilog HDL لیست شده در سرتاسر این کتاب تنها درباره Verilog نیست، بلکه معرفی نمایش سیستم‌های دیجیتال به کمک کامپیوتر به وسیله نوعی زبان توصیف سخت‌افزاری است.

۵-۶-۱ نمایش مدول

زبان Verilog HDL دارای دستور زبانی است که دقیقاً ساختارهای مجازی که در زبان می‌توانند به کار روند را توصیف می‌نماید. خصوصاً، Verilog حدود 100 کلمه کلیدی از پیش تعریف شده، حروف کوچک و شناسه‌هایی دارد که ساختار زبان را تعریف می‌کنند. تعدادی از کلمات کلیدی این زبان که از اهمیت بالایی برخوردارند، به شرح ذیل می‌باشد:

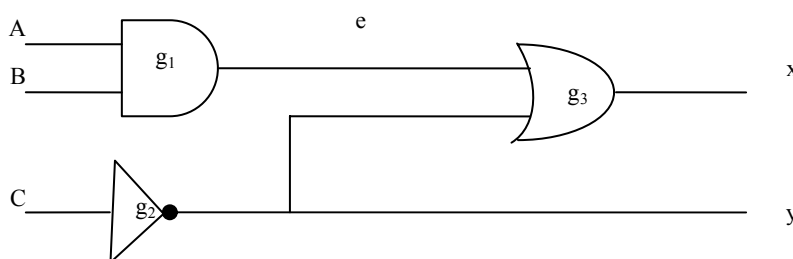
module	input	wire	Or
endmodule	output	and	not

هر متن بین دو اسلش (//) و انتهای خط به عنوان یک توضیح تفسیر می‌گردد. فاصله‌های Blank و نام‌های حساس به اندازه هستند، و این بدان معنی است که حروف بزرگ و کوچک با هم متفاوتند. در Verilog مدول یک بلوک ساختاری است. این دستور با کلمه کلیدی module آغاز و با کلمه کلیدی end module پایان می‌یابد. اکنون برای تشریح بعضی از مفاهیم زبان مثال ساده‌ای را تشریح می‌کنیم.

توصیف HDL مدار شکل ۵-۲۵ در مثال ۳ نشان داده شده است. خطی که دو اسلش دارد توضیحات است و عمل مدار را توضیح می‌دهد. دومین خط، مدول را همراه با نام و لیستی از پورت‌ها مشخص می‌کند. نام (در اینجا smpl-circuit) یک شناسه است که برای ارجاع به مدول به کار رفته است. شناسه‌ها نام‌هایی هستند که به متغیرها داده می‌شوند و به این ترتیب در طراحی قابل ارجاع می‌گردند. آنها از کاراکترهای الفبا عددی و زیر خط (-) ساخته می‌شوند و حساس به اندازه اند. شناسه‌ها باید با کاراکتر الفبایی و یا خط تیره شروع شوند. آنها را نمی‌توان با عدد شروع کرد. Port List رابطی بین مدول برای تبادل اطلاعات (مخابره) با محیط است. در این مثال پورت‌ها، ورودی‌ها و خروجی‌های مدارند. Port List بین پورت‌ها محصور شده و از ویرگول برای جدا کردن عناصر لیست استفاده می‌شود. عبارت با نقطه ویرگول (;) پایان می‌یابد.

مثال ۳:

```
//Description of simple circuit
Module smpl-circuit (A, B, C ,X, Y);
Input A , B , C;
Output X , Y;
Wire e;
And g1 (e ,A, B);
Not g2 (y,c);
Or g3(x ,e, y);
End module
```



شکل ۵-۲۵: مداری برای نمایش عبارت HDL مثال ۳

همه کلمات کلیدی که باید به حروف کوچک باشند به منظور وضوح با خط پر رنگ چاپ می‌شوند، ولی این زبان نیاز نیست. input، output بیان می‌دارند که کدام پورت‌ها ورودی و کدام خروجی هستند. اتصالات درونی در نقش سیم‌ها می‌باشند. مدار دارای یک اتصال داخلی در e بوده و با کلمه کلیدی wire بیان می‌شود. ساختار مدار با گیت‌های اصلی از پیش تعریف شده به عنوان کلمه کلیدی مشخص می‌گردد. معرفی هر گیت با یک نام اختیاری مثل g1 و g2 و غیره و به دنبال آن خروجی و ورودی‌هایی که با ویرگول از هم جدا شده و در داخل پرانتزاند، صورت می‌گیرد. همواره خروجی در ابتدا معرفی می‌شود و سپس از آن ورودی ذکر می‌گردد. مثلاً گیت OR که g3 نامیده شده، دارای خروجی x و ورودی‌های e و y است. توصیف مدول با

پایه‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۵۳

کلمه کلیدی endmodule خاتمه می‌یابد. توجه کنید که هر عبارت با یک نقطه ویرگول (;) پایان می‌پذیرد، ولی پس از endmodule نقطه ویرگول گذاشته نمی‌شود.

۵-۶-۲ تاخیر در گیت‌ها

یکی از مباحث مهم در طراحی مدارات دیجیتال، ایجاد تاخیر در بین گیت‌ها می‌باشد. هنگام استفاده از HDL در شبیه‌سازی، گاهی لازم است مقداری تاخیر بین ورودی تا خروجی گیت در نظر گرفته شود. در Verilog تاخیر بر حسب واحدهای زمانی و سمبل # معین می‌گردد. ارتباط یک واحد زمانی با زمان فیزیکی با استفاده از رهنمون کامپایلر timescale انجام می‌شود. رهنمون‌های کامپایلر با سمبل " " ' (backquote) شروع می‌شوند. چنین رهنمونی قبل از اعلان مدول مشخص می‌گردد. مثالی از رهنمون timescale در زیر آمده است.

```
timescale 1ns/ 100 Ps'
```

عدد اول نشان دهنده واحد اندازه‌گیری برای زمان‌های تاخیر است. عدد دوم دقتی که تحت آن تاخیرها گرد شده‌اند را نشان می‌دهد که در این حالت 0.1ns است. اگر timescale مشخص نشود، شبیه‌ساز واحد زمان معینی را، مثل 1ns، پیش فرض می‌کند. در این کتاب، واحد زمان پیش فرض را انتخاب خواهیم کرد.

مثال ۴: HDL توصیف مثال قبلی را همراه با تاخیر در هر گیت مشخص می‌نماید. گیت‌های AND، OR، NOT به ترتیب زمان تاخیر 30ns، 20ns و 10ns را دارند. اگر مدار شبیه‌سازی شود و ورودی‌ها از 000 به 111 تغییر یابند، خروجی‌ها طبق جدول شکل ۵-۲۶ تغییر می‌کنند.

خروجی وارون‌گر در y پس از تاخیر 10ns از 1 به 0 تغییر می‌یابد. خروجی گیت AND در e پس از 30ns تاخیر از 0 به 1 تغییر می‌کند. خروجی گیت OR در x در t=30ns از 1 به 0 می‌رود و سپس در t=50ns به 1 باز می‌گردد. در هر دو حالت، تغییر در خروجی گیت OR از تغییری که در 20ns قبل در ورودی‌اش اتفاق می‌افتد، ناشی

می‌شود. واضح است که هر چند خروجی x پس از تغییرات ورودی نهایتاً در 1 ثابت پیدا می‌کند، تاخیرهای گیتی قبل از آن برای مدت 20ns یک جرقه منفی ایجاد می‌نمایند.

```
//Description of circuit with delay
Module circuit – with - delay (A, B, C ,X, Y );
Input A , B , C;
Output X , Y;
Wire e;
And # (30) g1 (e ,A, B);
Or # (20) g3(x ,e, y);
Not# (10) g2 (y,c);
End module
```

	واحد زمانی (ns)	ورودی ABC	خروجی yex
تغییر اولیه	–	000	101
	–	111	101
	10	111	001
	20	111	001
	30	111	010
	40	111	010
	50	111	011

شکل ۵-۲۶: جدول خروجی گیت‌ها پس از تاخیر

برای شبیه‌سازی یک مدار با HDL، لازم است ورودی‌ها را برای شبیه ساز به مدار اعمال کنیم تا پاسخ خروجی تولید گردد. یک توصیف HDL که محرک را برای یک طرح فراهم می‌کند برنامه تست خوانده می‌شود. در اینجا بدون آن که توضیحاتی اضافی را ارائه کنیم روال را برای مثال ساده‌ای شرح می‌دهیم.

۱۵۵ پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR

مثال ۵: این مثال HDL یک برنامه تست را برای شبیه‌سازی مدار تاخیر دار نشان

می‌دهد. دو ماژول در این برنامه تست لحاظ شده است:

- ماژول محرک
- ماژول توصیف مدار

مدول محرک stimcrt پورت ندارد. ورودی‌ها به مدار با کلمه کلیدی reg و خروجی نیز با کلمه کلیدی wire معرفی می‌شوند. circuit-with-delay با cwd نام‌گذاری یا ذکر شده است. مثال ۵ عبارت HDL

```
//stimulus for simple circuit
Module stimulus ;
reg A , B , C;
Wire X , Y ;
circuit – with - delay  cwd (A, B, C ,X, Y);
initial
begin
A= 1' b0 ;  B= 1' b0 ;  C= 1' b0 ;
#100
A= 1' b1 ;  B= 1' b1 ;  C= 1' b1 ;
#100 $ finish ;
End
endmodule

//Description of circuit with delay
Module circuit – with - delay (A, B, C ,X, Y);
Input A , B , C;
Output X , Y ;
Wire e;
and # (30) g1(e ,A, B );
or # (20) g3(x ,e, y);
not# (10) g2(y,c );
endmodule
```

عبارت initial ورودی‌های بین کلمات کلیدی begin و end را مشخص می‌نماید. در آغاز $ABC = 000$ است. (هر یک از A، B و C با b0 1 تنظیم شده‌اند، و به معنی یک رقم دودویی با مقدار صفر است.) پس از 100ns ثانیه دیگر شبیه‌سازی خاتمه می‌یابد. (\$ finish یک تکلیف در سیستم است.) زمان کل شبیه‌سازی 200ns طول می‌کشد. ورودی‌های A، B و C پس از 100ns، از 0 به 1 تغییر می‌یابند. در اولین 10ns، خروجی y غیر مشخص است، خروجی x هم در اولین 30ns نامعین می‌باشد. خروجی y پس از 110ns از 1 به 0 می‌رود. خروجی x پس از 130ns از 1 به 0 می‌رود و در 150ns به 1 باز می‌گردد که این مقادیر دقیقاً در جدول ۳-۵ پیش‌بینی شده بود.

۳-۶-۵ عبارت بولی

عبارت بولی در Verilog HDL با عبارت تخصیص مداوم یا پیوسته متشکل از کلمه کلیدی assign که پس از آن یک عبارت بولی آمده مشخص می‌گردد. برای تفکیک علامت جمع حسابی از علامت OR، Verilog HDL از سمبل‌های (&)، (l) و (~) به ترتیب برای AND، OR و NOT استفاده می‌کند. بنابراین برای توصیف مدار ساده شکل ۳-۵ با یک عبارت بولی عبارت زیر را به کار می‌بریم.

$$\text{assign } x = (A \& B) \text{ l } \sim C);$$

مثال ۶: عبارت HDL

این مثال توصیف مداری که با دو عبارت بولی زیر بیان شده را نشان می‌دهد:

$$x = A + + BC B'D$$

$$y = B'C + BC'D'$$

مدار دارای دو خروجی x و y و چهار ورودی A، B، C و D است. دو عبارت assign معادلات بول را توصیف می‌نمایند.

```
// circuit specified with Boolean expressions
Module circuit – bln (x, y, A, B, C, D);
Input A, B, C, D;
```

پایاده سازی مدارهای دیجیتال با گیت های NAND و NOR ۱۵۷

Output X , Y;

Assign x = A | (B&C) | (~B & D);

Assign y = (~B & C) | (B & ~C & ~D);

End module

دیدیم که یک مدار دیجیتال می تواند با عبارت HDL درست مثل ترسیم در یک نمودار مداری، یا با عبارت بولی توصیف گردد. مزیت HDL این است که برای پردازش به وسیله کامپیوتر مناسب است.

گیت های به کار رفته در توصیف های HDL، با لغات کلیدی and، or و غیره به وسیله سیستم تعریف می شوند و Primitives سیستم نام گذاری می گردند. کاربر می تواند Primitive های دیگری را با تعریف آنها به صورت جدول اضافه نماید. این نوع مدارها را تعریف شده به وسیله کاربر یا UDP می نامند. یکی از راه های معرفی مدار به فرم جدول، معرفی آن با جدول درستی است. توصیف های UDP از کلمه کلیدی module استفاده نمی کنند. در عوض با کلمه کلیدی primitive (اصلی) تعریف می شوند. بهترین راه معرفی primitive ارائه یک مثال می باشد.

مثال ۷: عبارت HDL یک UDP را با یک جدول درستی تعریف می کند. حل آن بر اساس دستورالعمل زیر است:

از کلمه کلیدی Primitive استفاده شده و به دنبال آن یک نام و لیست پورت ها آورده می شود. تنها یک خروجی می تواند وجود داشته باشد که با به کارگیری کلمه کلیدی output و قبل از همه در لیست پورت اعلام می شود. به هر تعداد ورودی (input) می تواند تعریف شود. ترتیب معرفی آنها با اعلام input با ترتیب مقادیرشان در جدولی که به دنبال می آید، باید همخوانی داشته باشد. جدول درستی باید در داخل کلمات کلیدی table و endtable محصور شود. مقادیر ورودی با (:) پایان می یابند. خروجی همواره آخرین وارده در هر سطر است و بعد از آن (;) می آید و در پایان endprimitive ذکر می شود.

```
//User defined primitive (UDP)
Primitive crcpt ( x, A , B, C);
Output X;
Input A , B , C;
// Truth table for  $x(A, B, C) = \sum(0, 2, 4, 6, 7)$ 
table
// A B C : X (note that this is only a comment )
0 0 0 : 1;
0 0 1 : 0;
0 1 0 : 1;
0 1 1 : 0;
1 0 0 : 1;
1 0 1 : 0;
1 1 0 : 1;
1 1 1 : 1;
Endtable
Endprimitive

// Instantiate primitive
Module declare- crcpt ;
Reg x , y , z ;
Wire w;
Crcpt ( w, x,y, z );
endmodule
```

توجه کنید که متغیرهای لیست شده در بالای جدول بخشی از توضیحات بوده و به منظور آشنایی ذکر شده‌اند. سیستم متغیرها را به ترتیبی که در بخش ورودی ذکر شده‌اند تشخیص می‌دهد. یک UDP نیز مثل Primitive سیستم به کار گرفته می‌شود. مثلاً

$$\text{Crcpt}(w, x, y, z)$$

مداری با تابع

$$W(x, y, z) = \sum(0, 2, 4, 6, 7)$$

پیاده‌سازی مدارهای دیجیتال با گیت‌های NAND و NOR ۱۵۹

و ورودی‌های x و y و z و خروجی w را پیاده می‌کند. گرچه Verilog HDL این نوع توصیف را فقط برای UDP به کار می‌برد. دیگر HDL ها و سیستم‌های طراحی کامپیوتری (CAD) روال‌های دیگری را برای مشخص کردن مدارهای دیجیتال به صورت جدول استفاده می‌کنند. جداول می‌توانند با نرم‌افزار CAD برای به دست آوردن یک ساختار گیتی بهینه پردازش شوند. در این بخش، HDL را معرفی و مثال‌های ساده‌ای از مدل‌سازی ساخت یافته را ارائه دادیم.

سؤالات

۱- تابع بولی زیر را با استفاده از گیت‌های NAND طراحی نمایید.

$$F = xy' + x'y + z$$

۲- تابع بولی زیر را با استفاده از گیت‌های معمولی و گیت‌های NAND پیاده‌سازی نمایید.

$$F = A' (CD' + B) + BC'$$

۳- تابع بولی زیر را با استفاده از گیت‌های NOR طراحی نمایید.

$$F = (AB' + A'B) (C + D')$$

۴- تابع زیر را با استفاده از مدارهای AND-OR-INVERT طراحی نمایید.

$$F = (AB + CD + E)'$$

فصل ۶

مدارهای ترکیبی

هدف کلی

در این فصل مباحث اصلی مربوط به مدارهای ترکیبی به همراه روشهای تحلیل و طراحی مورد بحث و بررسی قرار گرفته و انواع مدارهای جمع کننده و تفریق کننده در حالات دودویی و دهدهی ارائه خواهند شد. همچنین عبارات ضرب دودویی نیز به همراه مدارهای مقایسه ای مورد بررسی قرار خواهند گرفت.

هدف ساختاری

در این فصل عناوین زیر مورد بحث و بررسی قرار می گیرند:

- مفهوم مدارهای ترکیبی
- روش های تحلیل و طراحی مدارهای ترکیبی
- انواع جمع کننده ها و تفریق گر ها
- مدارهای ضرب دودویی
- مدارهای مقایسه گر ها

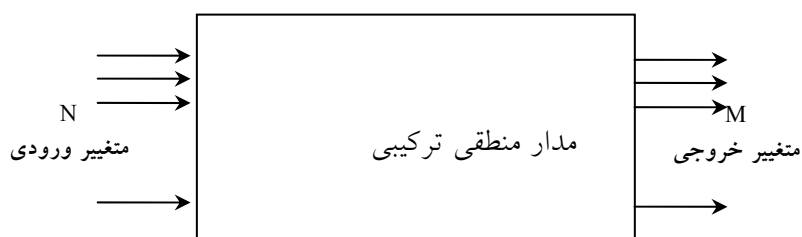
۶-۱ مدارهای ترکیبی

یکی از نکات مهم در طراحی مدارهای منطقی مورد استفاده در سیستم های دیجیتال، بحث ترکیبی بودن و یا ترتیبی بودن مدار می باشد و این مهم به عنوان یکی از

پارامترهای مهم در دسته‌بندی مدارهای دیجیتال مطرح می‌گردد. در این فصل مدارهای ترکیبی مورد بحث و بررسی قرار خواهند گرفت و مدارهای ترتیبی در فصل هشتم توضیح داده خواهند شد.

یک مدار ترکیبی متشکل از تعدادی گیت منطقی است که خروجی آنها در هر لحظه از زمان مستقیماً به وسیله ورودی‌های همان لحظه معین می‌شوند و به ورودی‌های قبلی بستگی ندارد. این نوع مدار، پردازشی را انجام می‌دهد که با مجموعه‌ای از توابع بولی مشخص می‌گردد. مدارهای ترتیبی علاوه بر گیت‌های منطقی از عناصر حافظه نیز استفاده می‌کنند. خروجی‌های آنها تابعی از ورودی‌ها و حالت عناصر حافظه است. در نتیجه خروجی یک مدار ترتیبی نه تنها به مقادیر فعلی ورودی‌ها بلکه به ورودی‌های قبلی وابسته بوده و عملکرد مدار باید به وسیله حالات داخلی و ترتیب زمانی ورودی‌ها مشخص گردد.

یک مدار ترکیبی از متغیرهای ورودی، گیت‌های منطقی، و متغیرهای خروجی تشکیل شده است. گیت‌های منطقی سیگنال‌هایی را از ورودی‌ها دریافت کرده و سیگنال‌هایی را برای خروجی‌ها تولید می‌نمایند. این فرآیند اطلاعات دودویی مفروض در ورودی را به اطلاعات مورد نیاز در خروجی تبدیل می‌کند. نمودار کلی یک مدار ترکیبی در شکل ۶-۱ دیده می‌شود.



شکل ۶-۱: نمودار بلوکی یک مدار ترکیبی

n متغیر دودویی ورودی از منبع بیرونی دریافت و m متغیر خروجی به مقصد بیرونی ارسال می‌شوند. هر متغیر ورودی و یا خروجی به طور فیزیکی به صورت یک سیگنال نشان داده می‌شوند و این سیگنال‌ها نیز 0 و 1 منطقی را نمایش می‌دهند. در بسیاری از کاربردها، منبع و مقصد، ثبات‌های ذخیره‌سازی هستند. اگر ثبات‌ها به همراه گیت‌های منطقی به کار روند، کل مدار با نام مدار ترتیبی شناخته خواهد شد.

برای n متغیر ورودی، 2^n ترکیب ممکن دودویی از ورودی‌ها وجود دارد. برای هر ترکیب ممکن از ورودی‌ها فقط یک مقدار برای خروجی موجود است. بنابراین، یک مدار ترکیبی با یک جدول درستی، که مقادیر خروجی‌ها را در برابر هر ترکیب از متغیرهای ورودی لیست می‌نماید، نشان داده می‌شود. یک مدار ترکیبی با m تابع بولی نیز قابل نمایش است، که هر یک متعلق به یک خروجی است. هر تابع خروجی بر حسب n متغیر ورودی بیان می‌گردد. در این فصل با استفاده از دانش فصل‌های قبل، تحلیل و طراحی مدارهای ترکیبی را فرموله می‌نماییم. با حل مثال‌های نمونه فهرستی از توابع اصلی مهم برای درک سیستم‌های دیجیتال فراهم خواهد شد.

مدارهای ترکیبی متعددی وجود دارند که در طراحی سیستم‌های دیجیتال به کرات به کار می‌روند. این مدارها به صورت مجتمع در دسترس بوده و به عنوان قطعات استاندارد دسته‌بندی شده‌اند. آنها توابع دیجیتال خاصی را که عموماً در طراحی سیستم‌های دیجیتال مورد نیازند، اجرا می‌کنند. در این فصل ما مهمترین مدارهای ترکیبی استاندارد مانند جمع‌کننده‌ها، تفریق‌گرها، مقایسه‌گرها، دیکدرها، انکدرها و مولتی‌پلکسرها را معرفی می‌کنیم. این قطعات به صورت مدارهای مجتمع MIS (مجتمع با فشردگی متوسط) در دسترسند. به آنها در مدارهای پیچیده VLSI، مانند مدارات مجتمع خاص (ASIC)، سلول‌های استاندارد هم می‌گویند. توابع سلول‌های استاندارد در داخل مدارهای VLSI به همان شکل به هم متصل می‌شوند که در طراحی MIS متشکل از چند IC، وصل شدند.

۲-۶ روش تحلیل

در تحلیل یک مدار ترکیبی، ما تابعی را که مدار پیاده‌سازی می‌کند، معین نماییم. این کار با یک نمودار منطقی مفروض آغاز شده و با مجموعه‌ای از توابع بول، یک جدول درستی، یا توضیحاتی از عمل مدار پایان می‌یابد. اگر نمودار منطقی مورد بررسی با نام تابع یا توضیحی از کار آن همراه باشد، آنگاه تحلیل به تصدیق تابع بیان شده کاهش می‌یابد. تحلیل را می‌توان به طور دستی با یافتن توابع بول یا جدول درستی، و یا با استفاده از یک برنامه شبیه‌سازی کامپیوتری اجرا نمود.

اولین قدم در تحلیل این است که مطمئن شویم مدار از نوع ترکیبی است و نه ترتیبی. نمودار یک مدار ترکیبی حاوی گیت‌هایی است که فاقد مسیرهای پس‌خورد یا حافظه است. یک مسیر پس‌خورد، اتصالی است از خروجی یک گیت به ورودی گیت دیگری که خود بخش ورودی آن را (گیت خروجی) تشکیل می‌دهد. مسیرهای پس‌خوردی در یک مدار دیجیتال مدار ترتیبی را تعریف می‌کنند.

به محض این که محقق شد مدار از نوع ترکیبی است، می‌توان برای به‌دست آوردن توابع بول خروجی یا جدول درستی پیش رفت. اگر تابع مدار تحت بررسی است، لازم است عمل مدار را از توابع بول حاصل یا جداول درستی تفسیر کرد. موفقیت در چنین بررسی‌هایی به شرطی میسر است که فرد تجربه قبلی و آشنایی لازم با چنین مدارهایی داشته باشد.

۲-۶-۱ تهیه توابع بول خروجی از یک مدار منطقی

برای به‌دست آوردن توابع بول خروجی از یک مدار منطقی به ترتیب زیر باید عمل کرد:

تمام خروجی‌های گیت‌هایی که تابعی از ورودی هستند باید با سمبل‌های دلخواه نام‌گذاری شوند. برای خروجی هر گیت تابع بول را معین کنید. گیت‌هایی که تابعی از

متغیرهای ورودی و گیت‌های برچسب خورده قبلی‌اند را با سمبل‌های اختیاری دیگری برچسب بزنید. برای این گیت‌ها نیز توابع بول خروجی را به‌دست آورید.

فرآیند مرحله ۲ را تا دستیابی به خروجی‌های مدار ادامه دهید. با جایگزینی توابع به‌دست آمده در قبل، توابع بولی خروجی را بر حسب متغیرهای ورودی اولیه به‌دست آورید. تحلیل مدارهای ترکیبی شکل ۶-۲ روال پیشنهادی را تشریح می‌نماید. توجه دارید که مدار دارای سه ورودی دودویی A، B و C و دو خروجی F_1 و F_2 است. خروجی گیت‌هایی که تابعی از متغیرهای ورودی‌اند عبارتند از T_1 و T_2 . خروجی F_2 به سادگی از متغیرهای ورودی به دست می‌آید. توابع بول برای این سه خروجی عبارتند از:

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

اکنون خروجی گیت‌هایی که تابعی از سمبل‌های قبلی می‌باشند را ملاحظه می‌نماییم.

$$T_3 = F_2' T_1$$

$$F_1 = T_3 + T_2$$

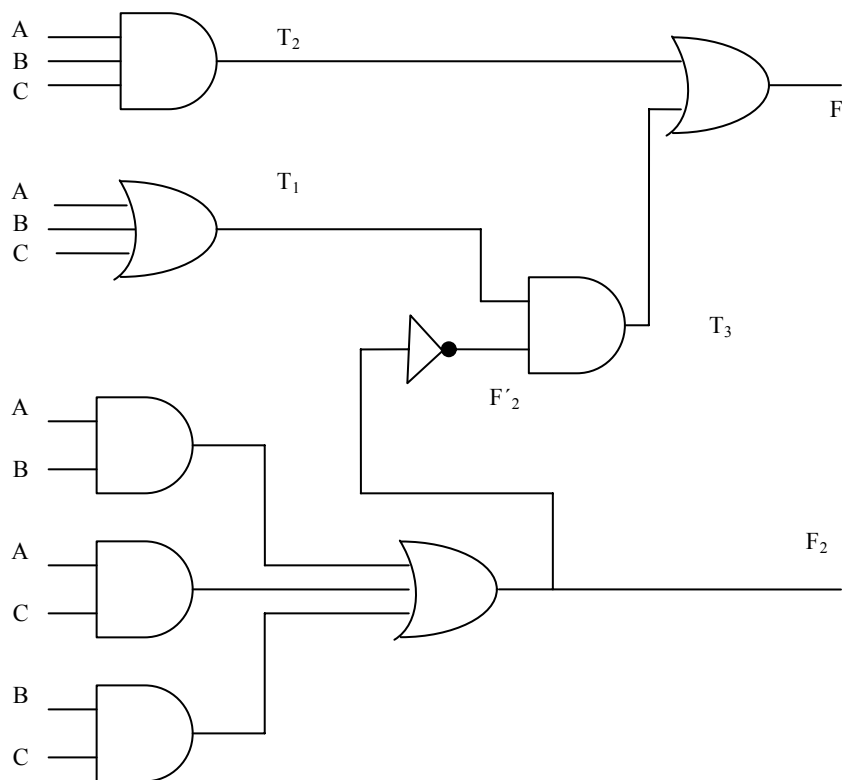
برای به‌دست آوردن F_1 بر حسب A، B و C، یکسری جایگزینی‌ها را به فرم زیر انجام می‌دهیم:

$$\begin{aligned} F_1 = T_3 + T_2 &= F_2' T_1 + ABC \\ &= (AB + AC + BC)'(A + B + C) + ABC \\ &= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC \\ &= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC \\ &= A'BC' + A'B'C + AB'C' + ABC \end{aligned}$$

اگر بخواهیم این بررسی را دنبال کرده و عمل انتقال اطلاعات را با این مدار مشخص نماییم، می‌توانیم مدار را از عبارات بولی فوق رسم کرده و سعی کنیم عملیات آشنا را تشخیص دهیم.

۶-۲-۲ تهیه جدول درستی از نمودار منطقی

به دست آوردن جدول درستی برای مدار، به محض شناختن توابع بولی، خروجی روندی ساده است. برای تهیه مستقیم جدول درستی از نمودار منطقی و بدون نیاز به توابع بول به طریق زیر عمل کنید:



شکل ۶-۲: نمودار منطقی برای مثال تحلیل

۱. تعداد متغیرهای ورودی در مدار را مشخص کنید. برای n ورودی 2^n ترکیب از ورودی‌ها را تشکیل دهید. آنگاه اعداد دودویی را در جدول از ۰ تا $2^n - 1$ لیست نمایید.
۲. خروجی‌های گیت‌های انتخابی را با سمبل‌های دلخواه برچسب بزنید.

۳. برای آن دسته از خروجی گیت‌ها که فقط تابعی از متغیرهای ورودی هستند جدول درستی را به دست آورید.

۴. برای به دست آوردن خروجی گیت‌هایی که تابعی از مقادیر تعریف شده قبلی هستند پیش بروید تا ستون همه خروجی‌ها معین شود.

این فرآیند با استفاده از مدار شکل ۶-۲ تشریح می‌شود. در جدول شکل ۶-۳، هشت ترکیب ممکن را برای سه متغیر ورودی تشکیل می‌دهیم. جدول درستی برای F_2 مستقیماً از مقادیر A ، B و C تشکیل می‌شود که در آن برای هر ترکیبی که دو یا سه ورودی برابر با ۱ دارد، F_2 برابر ۱ است. جدول درستی برای F'_2 متمم F_2 است. جداول درستی برای T_1 و T_2 به ترتیب توابع OR و AND متغیرهای ورودی می‌باشند. مقدار T_3 از T_1 و F'_2 حاصل می‌شود: وقتی که هر دو T_1 و F'_2 برابر با ۱ باشند T_3 نیز برابر ۱ است، در غیر این صورت T_3 برابر ۰ خواهد بود. بالاخره F_1 برای آن دسته از ترکیبات ۱ است که در آنها T_2 یا T_3 یا هر دو برابر ۱ باشند. بررسی ترکیبات جدول درستی برای A ، B و C و F_1 و F_2 نشان می‌دهد که این جدول با جدول جمع‌کننده کامل بخش ۶-۴ با متغیرهای x و y و z و S و C برابر است.

A	B	C	F_2	F'_2	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

شکل ۶-۳: جدول درستی برای نمودار منطقی شکل ۶-۲

۳-۶ روش طراحی

طراحی مدارهای ترکیبی با مشخصات مسئله آغاز و به فرم نمودار مدار منطقی یا مجموعه‌ای از توابع بول که به کمک آنها نمودار منطقی حاصل می‌شود، پایان می‌یابد. روال شامل موارد زیر است:

- با استفاده از مشخصات مدار تعداد ورودی‌ها و خروجی‌ها را معین کرده و به هر کدام سمبلی تخصیص دهید.
- جدول درستی مربوط به ورودی‌ها و خروجی‌های مدار را تشکیل دهید.
- توابع بولی ساده شده را برای خروجی به صورت تابعی از متغیرهای ورودی به دست آورید.
- نمودار منطقی را رسم کرده و صحت طراحی را تحقیق نمایید.

جدول درستی یک مدار ترکیبی، از ستون‌های ورودی و ستون‌های خروجی تشکیل می‌شود. ستون‌های ورودی از 2^n ترکیب مربوط به n متغیر ورودی به دست می‌آید. مقادیر دودویی خروجی‌ها از مشخصات بیان شده در مسئله حاصل می‌گردد. توابع خروجی مشخص شده در جدول درستی تعریف دقیقی از مدار ترکیبی را به دست می‌دهند. تفسیر لفظی صحیح جدول درستی از اهمیت خاصی برخوردار است. اغلب مشخصات لفظی کامل نیستند و تفسیر غلط ممکن است جدول درستی غلطی را تولید کند.

توابع دودویی خروجی لیست شده در جدول با روش‌های موجود مانند دستکاری جبری، جدول کارنو یا برنامه‌های ساده‌سازی مبتنی بر کامپیوتر ساده می‌شوند. غالباً عبارات ساده شده متعددی حاصل می‌شود که باید مناسب ترین راه انتخاب کرد. در یک کاربرد خاص، معیارهای مختلفی در انتخاب یک پیاده‌سازی نقش دارند. یک طرح عملی قیودی چون تعداد گیت‌ها، تعداد ورودی‌ها به یک گیت، زمان انتشار سیگنال‌ها در گیت‌ها، تعداد اتصالات داخلی، محدودیت‌های مربوط به قابلیت راه اندازی هر

گیت، و دیگر معیارهایی که باید در طراحی با مدارهای مجتمع مد نظر باشد، را در نظر می‌گیرد. در بسیاری از حالات ساده‌سازی با تصدیق و تایید یک هدف ساده، مثل تولید توابع بولی به فرم استاندارد آغاز شده و سپس با برآورده کردن دیگر معیارهای رفتاری پیش می‌رود.

۶-۳-۱ مکانیزم‌های تبدیل اعداد در مبناهای متفاوت

وجود کدهای گوناگون و متنوع برای بیان اجزاء اطلاعات گسسته، باعث شده است تا سیستم‌های دیجیتال مختلف از کدهای متفاوتی استفاده کنند. گاهی لازم است خروجی یک سیستم به عنوان ورودی به سیستمی دیگر استفاده شود. اگر این دو سیستم از کدهای متفاوتی برای بیان اطلاعات یکسان استفاده کنند، یک مدار مبدل باید بین آن دو قرار داده شود. بنابراین یک مبدل کد مداری است که دو سیستم را، علیرغم به‌کارگیری کد دودویی متفاوت، با هم سازگار می‌سازد.

برای تبدیل کد دودویی A به کد دودویی B، خطوط ورودی باید ترکیبات بیتی اجزاء مشخص شده با کد A را تهیه نموده و خطوط خروجی نیز باید ترکیبات کد B مربوطه را تولید نمایند. یک مدار ترکیبی به کمک گیت‌ها این تبدیل را انجام می‌دهد. روش طراحی با مثالی که دهدهی کد شده به دودویی (BCD) را به کد افزونی-3 تبدیل می‌نماید، تشریح خواهد شد. چون هر کد، از چهار بیت برای نمایش یک رقم دهدهی استفاده می‌نماید، باید چهار متغیر ورودی و چهار متغیر خروجی داشته باشیم. چهار متغیر دودویی را با A، B، C و D و چهار متغیر خروجی را با x، y و z نام‌گذاری کنید. جدول درستی روابط بین ورودی‌ها و خروجی‌ها در جدول شکل ۶-۴ دیده می‌شود. توجه کنید که چهار متغیر دودویی دارای 16 ترکیب‌اند ولی تنها 10 عدد از آنها در جدول درستی ذکر شده‌اند. 6 ترکیب بیتی ذکر نشده برای متغیرهای ورودی ترکیبات بی‌اهمیت هستند. این مقادیر در BCD مفهوم ندارند و فرض بر این است که

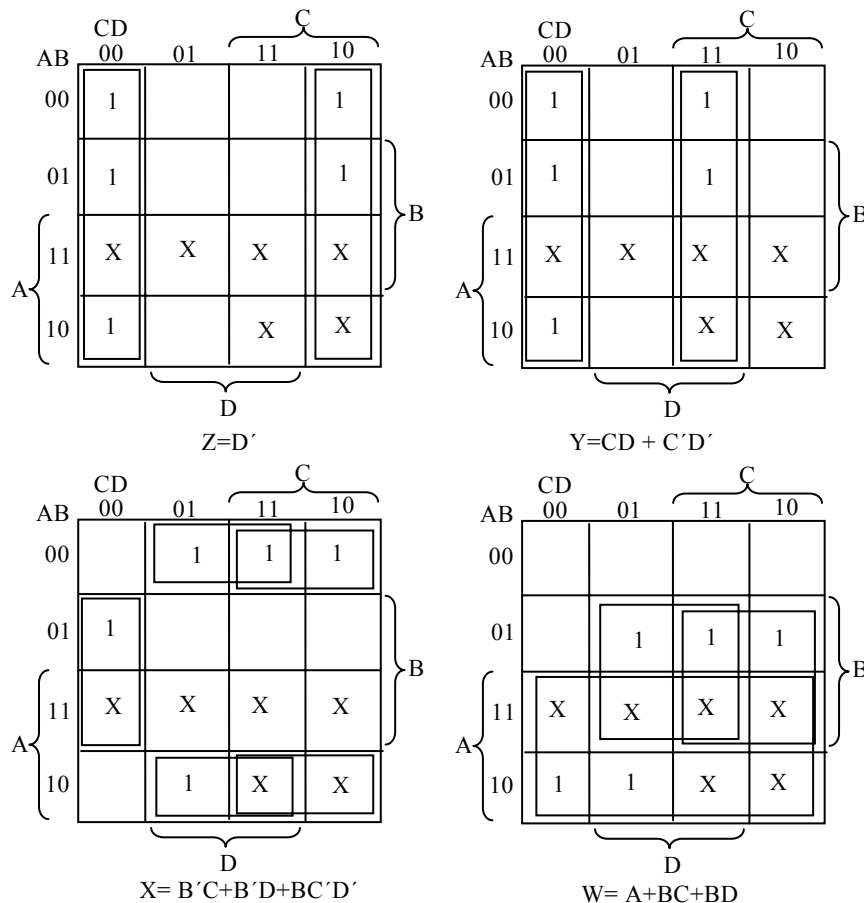
هرگز رخ نمی‌دهند. بنابراین به متغیرهای خروجی مربوط به آنها به دلخواه 0 یا 1 خواهیم داد و این تخصیص به نحوی خواهد بود که از آن مدار ساده‌تری حاصل گردد.

ورودی BCD				خروجی کد افزونی-3			
A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

شکل ۶-۴: جدول درستی برای مثال تبدیل کد

نقشه‌ها در شکل ۶-۵ برای به‌دست آوردن توابع بول خروجی ساده شده رسم شده‌اند. هر یک از چهار نقشه به یکی از خروجی‌های مدار به عنوان تابعی از چهار متغیر ورودی مربوط است. اهایی که در مربع‌ها نوشته شده‌اند از مینترم‌هایی که خروجی را 1 کنند به‌دست می‌آیند. این‌ها با در نظر گرفتن تک تک ستون‌های خروجی در جدول درستی مشخص می‌شوند. مثلاً ستون زیر خروجی Z دارای پنج عدد 1 است؛ بنابراین، نقشه Z دارای پنج 1 می‌باشد که هر یک متعلق به مینترمی است که Z توسط آن برابر 1 می‌شود. شش مینترم بی‌اهمیت از 10 تا 15 با علامت X علامت زده شده‌اند. یکی از نتایج ساده‌سازی توابع در جمع حاصل ضرب‌ها در زیر نقشه هر متغیر خروجی نوشته شده است.

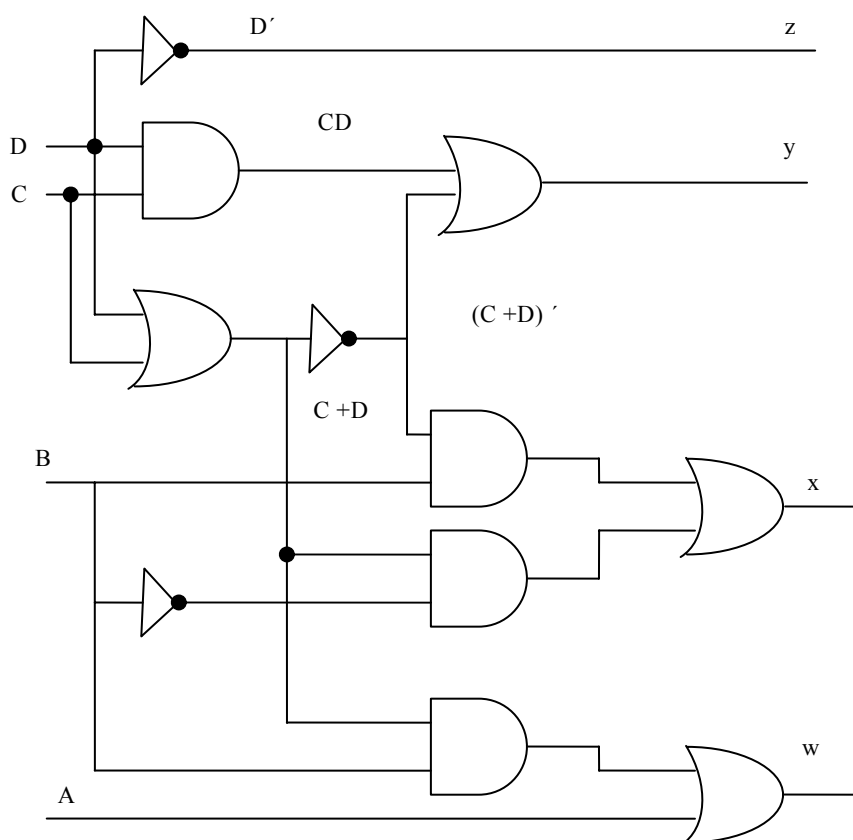
نموار دو سطحی را می‌توان مستقیماً از عبارات بولی حاصل از نقشه‌ها به دست آورد. البته فرم‌های متعدد دیگری نیز برای به‌دست آوردن نمودار منطقی که همین مدار را پیاده‌سازی کند وجود دارند. عبارات حاصل در شکل ۶-۵ را به منظور استفاده از گیت‌های مشترک می‌توان درستکاری جبری نمود. این دستکاری جبری که در زیر آمده



شکل ۵-۶: نقشه‌های تبدیل BCD به افزونی-۳

است انعطاف‌پذیری حاصل با سیستم‌های چند خروجی را وقتی با سه، یا چهار سطر و یا بیشتر پیاده‌سازی می‌شوند، نشان می‌دهد.

نمودار منطقی که این توابع را پیاده‌سازی می‌کند در شکل ۶-۶ دیده می‌شود. مشاهده می‌شود. گیت OR که خروجی‌اش $(C+D)$ است به نحوی در پیاده‌سازی هر سه خروجی به کار رفته است. بدون احتساب گیت‌های وارون‌گر در ورودی، پیاده‌سازی به صورت جمع حاصل‌ضرب‌ها به هفت گیت AND و سه گیت OR نیاز دارد. در



شکل ۶-۶: نمودار منطقی برای تبدیل BCD به افزونی - 3

شکل ۶-۶ همین سیستم به چهار گیت AND، چهار گیت OR و یک وارون‌گر احتیاج دارد. اگر تنها ورودی‌های معمولی یا نرمال در دسترس باشند، پیاده‌سازی اول به وارون‌گرهایی برای متغیرهای B و C و D نیاز خواهد داشت، ولی در پیاده‌سازی دوم

w	$= A + BC + BD$
.	$= A + (C+D)$
.	
x	$= B'C + B'D + BC'D'$
.	$= B'(C+D) + BC'D'$
.	$= B'(C+D) + B(C+D)'$
.	
y	$= CD + C'D'$
.	$= CD + (C+D)'$
.	
z	$= D'$

فقط B و D نیاز به وارون گر دارند.

۶-۴ جمع کننده ها و تفریق گرهای دودویی

اصلی ترین عمل حسابی جمع دو رقم دودویی است. این جمع ساده شامل چهار عمل پایه به شرح ذیل می باشد:

$$\begin{array}{ll} 0+0=0 & 1+0=1 \\ 0+1=1 & 1+1=1 \end{array}$$

سه عمل اول جمعی یک رقمی تولید می کنند، ولی وقتی هر دو بیت مضاف و مضاف الیه برابر 1 باشند، جمع دودویی از دو رقم تشکیل خواهد شد. با ارزش تر این نتیجه را *نقلی* می گویند. وقتی مضاف و مضاف الیه دارای ارقام با ارزش تر بیشتری باشند، نقلی حاصل از جمع دو بیت با جفت بیت با ارزش تر بعدی افزوده می شود. مدار ترکیبی که جمع دو بیت را انجام می دهد، نیم جمع کننده نام دارد. مداری که سه بیت را با هم جمع کند، (دو بیت به علاوه بیت نقلی) جمع کننده کامل یا تمام جمع کننده خوانده می شود. اسم مدارها به این علت انتخاب شده است که از دو نیم جمع کننده می توان در پیاده سازی یک جمع کننده کامل استفاده کرد.

یک جمع-تفریق گر دودویی مداری ترکیبی که عملیات حسابی جمع و تفریق را با اعداد دودویی انجام می دهد. ما این مدار را به صورت سلسله مراتبی طراحی خواهیم کرد. ابتدا طراحی نیم جمع کننده انجام می شود، و با استفاده از آن جمع کننده کامل را طراحی خواهیم کرد. با اتصال سری n جمع کننده کامل جمع دو عدد n بیتی تولید می گردد. مدار تفریق گر با افزودن مدار متمم ساز به آن ساخته می شود.

۶-۴-۱ نیم جمع کننده

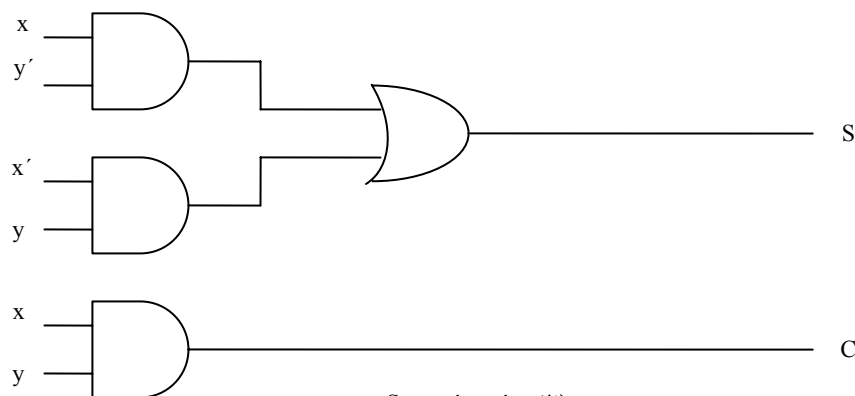
با توجه به توضیحات لفظی یک نیم جمع کننده، در می یابیم که مدار نیاز به دو ورودی دودویی و دو خروجی دودویی دارد. متغیرهای ورودی بیت های مضاف و مضاف الیه را

مشخص می‌کنند. متغیرهای خروجی جمع و نقلی را تولید می‌نمایند. ما سمبل‌های x و y را به دو ورودی و s (برای جمع) و C (نقلی) را به خروجی‌ها تخصیص می‌دهیم. جدول درستی برای نیم جمع‌کننده در جدول شکل ۶-۷ نشان داده شده است.

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

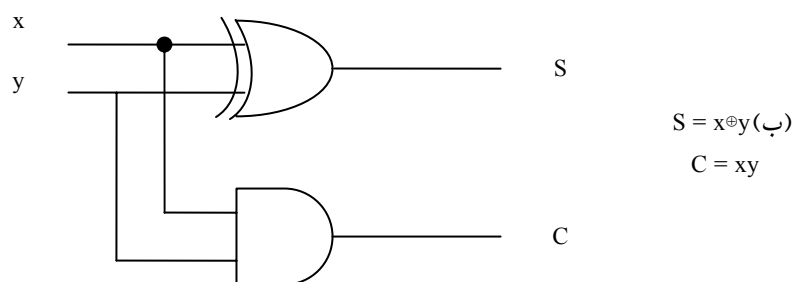
شکل ۶-۷: جدول نیم جمع‌کننده

خروجی C فقط هنگامی 1 است که هر دو ورودی 1 باشند. خروجی S ، بیت کم‌ارزش‌تر حاصل جمع را نشان می‌دهد. توابع بولی ساده شده برای دو خروجی مستقیماً از جدول درستی به دست می‌آیند. عبارات جمع حاصلضرب ساده عبارتند از:



$$S = xy' + x'y \text{ (الف)}$$

$$C = xy$$



$$S = x \oplus y \text{ (ب)}$$

$$C = xy$$

شکل ۶-۸: پیاده‌سازی نیم جمع‌کننده

$$S = x'y + xy'$$

$$C = xy$$

و

نمودار منطقی نیم جمع کننده پیاده شده با جمع حاصلضربها در شکل ۶-۸ (الف) دیده می شود. می توان آن را با گیت های XOR و AND طبق شکل ۶-۸ (ب) هم پیاده کرد. این نوع برای ساخت جمع کننده کامل از دو نیم جمع کننده به کار می رود.

۶-۴-۲ جمع کننده کامل

یک جمع کننده کامل مداری ترکیبی است که جمع حسابی سه بیت را تشکیل می دهد. این مدار دارای سه ورودی و دو خروجی است. دو متغیر ورودی که با x و y نشان داده شده اند. دو بیت با ارزش جمع شونده را نشان می دهند. ورودی سوم، z ، نقلی حاصل از مکان کم ارزش تر قبلی است. دو خروجی لازم است زیرا جمع حسابی سه رقم دودویی بین ۰ تا ۳ می باشد و اعداد ۲ و ۳ به دو رقم دودویی نیاز دارند. دو خروجی با سمبل S برای جمع و C برای نقلی مشخص شده اند. متغیر دودویی S مقدار کم ارزش تر جمع را به دست می دهد. متغیر دودویی C نقلی خروجی را بیان گر است. جدول درستی جمع کننده کامل در جدول شکل ۶-۹ دیده می شود. هشت سطر زیر سه متغیر همه ترکیبات ممکن سه متغیر را نشان می دهند. متغیرهای خروجی از جمع حسابی بیت های ورودی معین می شوند. وقتی همه بیت های ورودی ۰ هستند، خروجی ۰ است. خروجی S هنگامی ۱ می شود که فقط یک ورودی برابر ۱ باشد و یا اگر هر سه ورودی ۱ باشند. خروجی C هم موقعی ۱ است که دو یا سه ورودی برابر ۱ باشند.

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

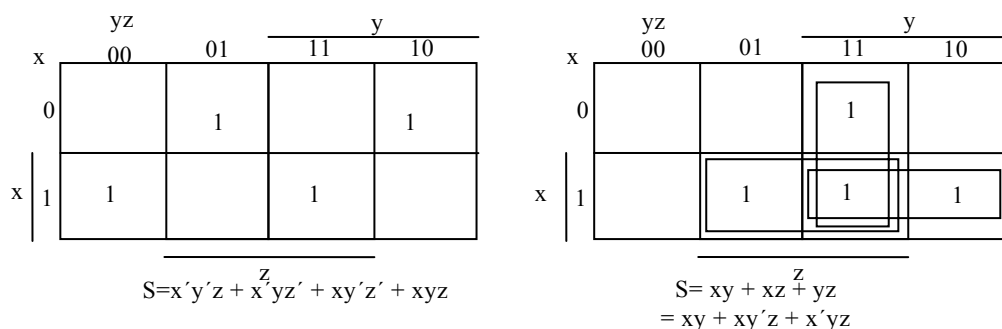
شکل ۶-۹: جدول جمع کننده کامل

تفسیر بیت‌های ورودی و خروجی مدار ترکیبی در مراحل مختلف طراحی متفاوت است. به طور فیزیکی سیگنال‌های دودویی ورودی‌ها ارقامی دودویی تصور می‌شوند که به صورت حسابی باید با هم جمع شده و جمع دو رقمی را در خروجی تولید کنند. از طرف دیگر، در جدول درستی و یا هنگام پیاده‌سازی با گیت‌های منطقی، همان مقادیر به عنوان متغیرهای بول تعبیر می‌شوند. نقشه خروجی‌های جمع‌کننده کامل در شکل ۶-۱۰ ملاحظه می‌شود. عبارات ساده شده عبارتند از:

$$S = x'y'z + x'yz' + xy'z' + xyz$$

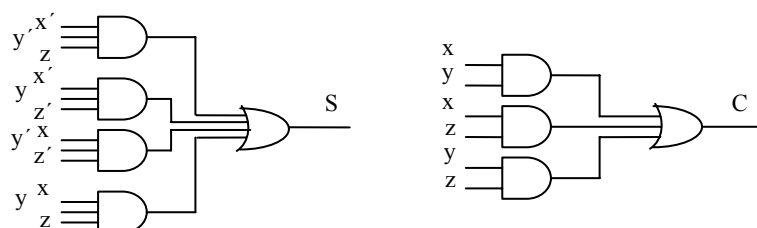
$$C = xy + xz + yz$$

و



شکل ۶-۱۰: نقشه جمع‌کننده کامل به صورت جمع حاصلضرب‌ها

نمودار منطقی پیاده شده به صورت جمع حاصلضرب‌ها در شکل ۶-۱۱ مشاهده می‌شود.



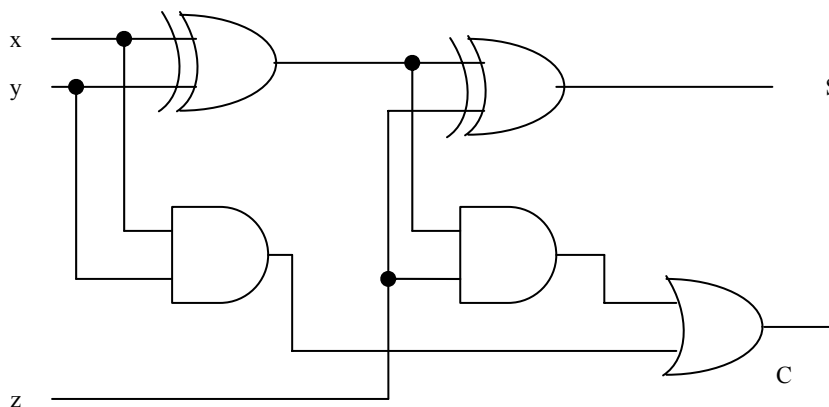
شکل ۶-۱۱: پیاده‌سازی جمع‌کننده کامل با جمع حاصلضرب‌ها

می‌توان با دو نیم جمع‌کننده و یک OR هم طبق شکل ۶-۱۲ آن را پیاده‌سازی کرد. خروجی S از نیم جمع‌کننده دوم XOR متغیر z و خروجی نیم جمع‌کننده اول حاصل می‌شود. زیرا

$$\begin{aligned} S &= z \oplus (x \oplus y) \\ &= z' (xy' + x'y) + z (xy' + x'y)' \\ &= z' (xy' + x'y) + z (xy + x'y') \\ &= xy'z' + x'yz' + xyz + x'y'z \end{aligned}$$

و نقلی خروجی برابر است با

$$\begin{aligned} C &= z (xy' + x'y) + xy \\ &= xy'z + x'yz + xy \end{aligned}$$

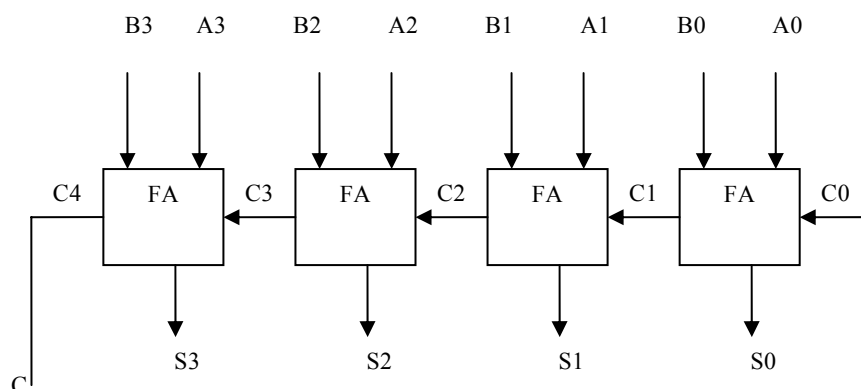


شکل ۶-۱۲: پیاده‌سازی یک جمع‌کننده کامل با دو نیم جمع‌کننده و یک گیت OR

۶-۴-۳ جمع‌کننده دودویی

یک جمع‌کننده دودویی مداری دیجیتال است که جمع حسابی دو عدد دودویی را تولید می‌کند. می‌توان آن را از به هم پیوستن متوالی جمع‌کننده کامل ساخت، و در آن هر

خروجی نقلی از هر جمع‌کننده کامل به ورودی نقلی جمع‌کننده کامل بعدی زنجیر وار بسته می‌شود. شکل ۶-۱۳ اتصالات درونی مدار چهار جمع‌کننده کامل (FA)، برای تهیه جمع‌کننده دودویی ۴ بیت با نقلی موج گونه را نشان می‌دهد. بیت‌های مضاف از A و مضاف‌الیه از B با اعداد اندیس‌دار از راست به چپ و با اندیس ۰ در بیت کم ارزش‌تر مشخص شده است. نقلی‌ها به صورت زنجیر جمع‌کننده‌های کامل را به هم وصل کرده‌اند. نقلی ورودی به جمع‌کننده C₀ وصل بوده و موج‌گونه‌وار تا نقلی خروجی C₄ انتشار می‌یابد. خروجی‌های S بیت‌های حاصل جمع را تولید می‌کنند. یک جمع‌کننده n بیت به n جمع‌کننده کامل نیاز دارد و هر خروجی نقلی به ورودی نقلی جمع‌کننده رتبه بالاتر وصل می‌شود.



شکل ۶-۱۳: جمع‌کننده ۴ بیت

به منظور تشریح بیشتر مثالی را با اعداد دودویی A=1011 و B=0011 در نظر بگیرید. حاصل جمع آنها S=1110 است که از جمع چهار بیت مطابق زیر به دست می‌آید.

اندیس i	:	3	2	1	0	
نقلی ورودی		0	1	1	0	Ci
مضاف		1	0	1	1	Ai
مضاف الیه		0	0	1	1	Bi
حاصل جمع		1	1	1	0	Si
نقلی خروجی		0	0	1	1	Ci+1

شکل ۶-۱۴: جدول وضعیت جمع کننده دودویی

بیت‌ها با کمک جمع کننده کامل و از کم ارزش ترین مکان (اندیس ۰) با هم جمع می شوند تا بیت حاصل جمع و نقلی را تشکیل دهند. نقلی ورودی C_0 در کم ارزش ترین مکان باید ۰ باشد. مقدار C_{i+1} در یک مکان مفروض، نقلی خروجی جمع کننده کامل است. این مقدار به نقلی ورودی تمام جمع کننده ای که بیت های یک مکان بالاتر در سمت چپ را جمع می کند انتقال می یابد. بنابراین بیت های جمع از راست به چپ تولید شده و به محض تولید نقلی قبل از خود در اختیار خواهند بود. برای داشتن خروجی جمع صحیح همه نقلی ها باید تولید شده باشند.

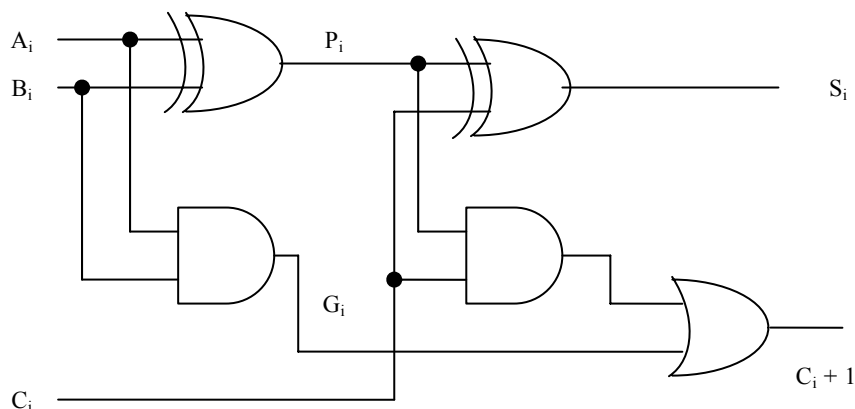
یک جمع کننده ۴ بیت مثالی از یک قطعه استاندارد است. می توان از آن در کاربردهای متعددی مثل عملیات حسابی استفاده کرد. به خاطر بسپارید که طراحی این مدار با روش های کلاسیک به یک جدول درستی با $2^9 = 512$ وارده نیاز دارد زیرا ۹ ورودی به مدار موجود است. با استفاده از روش بستن متوالی یک تابع استاندارد می توان به یک پیاده سازی ساده و مستقیم دست یافت.

۶-۴-۴ انتشار رقم نقلی

جمع دو عدد دودویی به صورت موازی لازم می دارد که مضاف و مضاف الیه به طور همزمان برای محاسبه موجود باشند. همچون دیگر مدارهای ترکیبی، در این مدار هم قبل از داشتن یک جواب صحیح، سیگنال باید از گیت ها عبور کند. زمان کل انتشار برابر است با زمان تاخیر انتشار یک نمونه گیت ضرب در طبقات گیت ها در مدار.

طولانی‌ترین زمان تاخیر انتشار در یک جمع‌کننده زمانی است که نقلی برای انتشار در جمع‌کننده‌ای کامل لازم دارد. چون هر بیت از خروجی جمع به نقلی ورودیش وابسته است مقدار S_i در هر طبقه مفروض در جمع‌کننده تنها موقعی به مقدار پایدار نهایی خود می‌رسند که نقلی ورودی به آن طبقه رسیده باشد. مثلاً خروجی S_3 را در شکل ۶-۱۳ در نظر بگیرید. به محض اعمال ورودی‌ها به جمع‌کننده، ورودی‌های A_3 و B_3 در دسترسند. با این وجود نقلی ورودی C_3 تا تولید C_2 به وسیله طبقه قبل در مقدار نهایی‌اش پایدار نمی‌شود. به‌طور مشابه C_2 منتظر C_1 و C_1 منتظر C_0 خواهد بود. بنابر این پس از انتشار موج گونه نقلی در همه طبقات، خروجی S_3 و نقلی C_4 در مقادیر صحیح نهایی خود پایدار خواهند شد.

تعداد طبقات گیت برای انتشار نقلی را باید از مدار هر جمع‌کننده کامل به‌دست آورد. به منظور سهولت این مدار در شکل ۶-۱۵ دوباره ترسیم شده است. متغیرهای ورودی و خروجی از اندیس i برای مشخص کردن شماره طبقه جمع‌کننده استفاده کرده‌اند سیگنال‌ها در P_i و G_i هنگامی به ثبات می‌رسند که از گیت‌های مربوطه شان انتشار یافته باشند. این دو سیگنال که در همه جمع‌کننده‌های کامل وجود دارند، به بیت‌های مضاف و مضاف‌الیه ورودی وابسته‌اند. سیگنال نقلی ورودی C_i از طریق



شکل ۶-۱۵: جمع‌کننده کامل با P و G

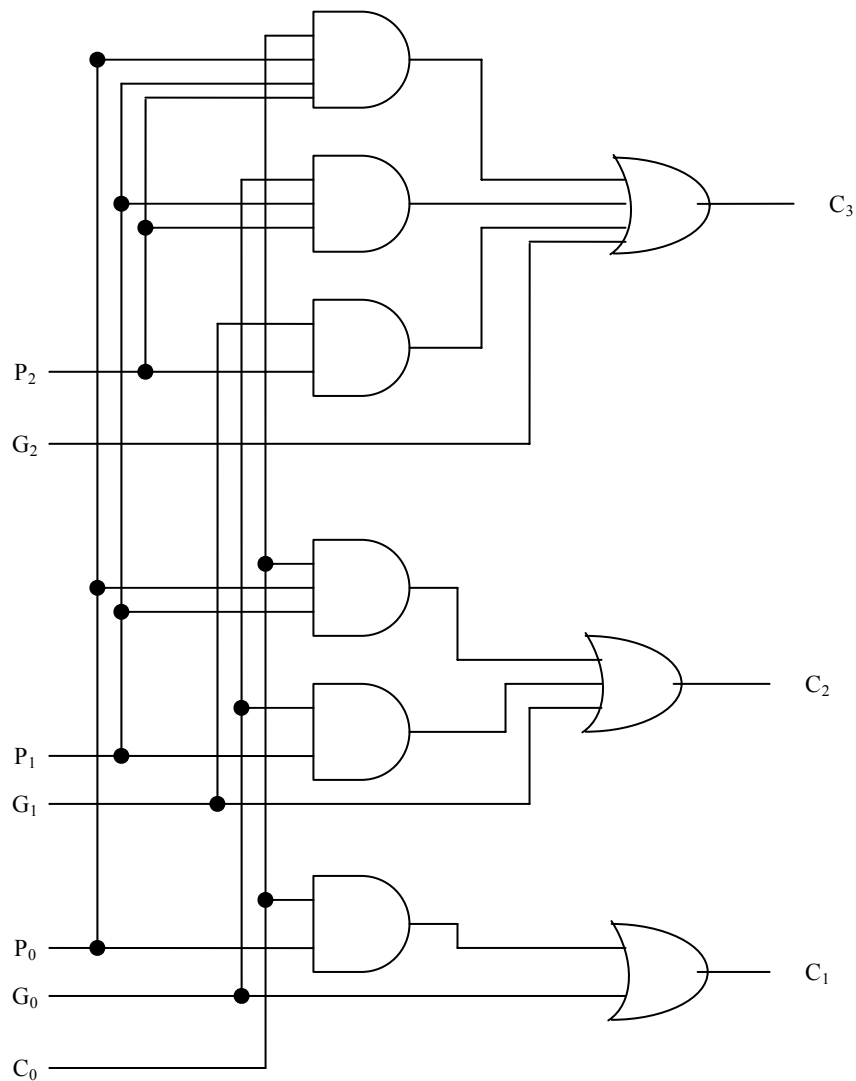
گیت AND یک گیت OR، که دو سطح گیت را تشکیل می‌دهند، به C_{i+1} می‌رسد. اگر در مجموع، چهار جمع‌کننده کامل وجود داشته باشد بین نقلی خروجی C_0 تا C_4 ، $2 \times 4 = 8$ بقیه گیت وجود خواهد داشت. برای یک جمع‌کننده n بیت، $2n$ طبقه گیت وجود دارد تا نقلی ورودی از طریق آنها انتشار یافته و به خروجی برسد.

زمان انتشار نقلی، فاکتور محدود کننده‌ای روی سرعت جمع دو عدد می‌باشد. گرچه جمع‌کننده، یا هر مدار ترکیبی دیگر دارای مقداری در پایانه‌اش است، ولی خروجی‌ها صحیح نخواهند بود مگر اینکه فرصتی کافی برای انتشار سیگنال از گیت‌های متصل بهم از ورودی تا خروجی داده شود. چون همه عملیات حسابی با جمع تکراری صورت می‌گیرد، زمان مصرف شده در طول فرآیند جمع بسیار حساس خواهد بود. راه حل روشنی برای کاهش زمان تاخیر انتشار استفاده از گیت‌های سریعتر است. با این وجود، مدارهای فیزیکی در قابلیت خود محدودیت دارند. راه حل دیگر افزایش پیچیدگی مدار به طریقی است که زمان تاخیر نقلی کاهش یابد. چند تکنیک برای کاهش زمان انتشار نقلی در جمع‌کننده‌های موازی وجود دارد. رایج‌ترین تکنیک استفاده از اصل پیش‌بینی نقلی می‌باشد. مدار جمع‌کننده کامل شکل ۶-۱۵ را ملاحظه نمایید. اگر دو متغیر دودویی جدید زیر را معرفی کنیم:

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

حاصل جمع خروجی و نقلی آن را می‌توان به صورت زیر تعریف کرد.



شکل ۶-۱۶: نمودار منطقی مولد پیش بینی نقلی

$$S_i = P_i \oplus C_i$$

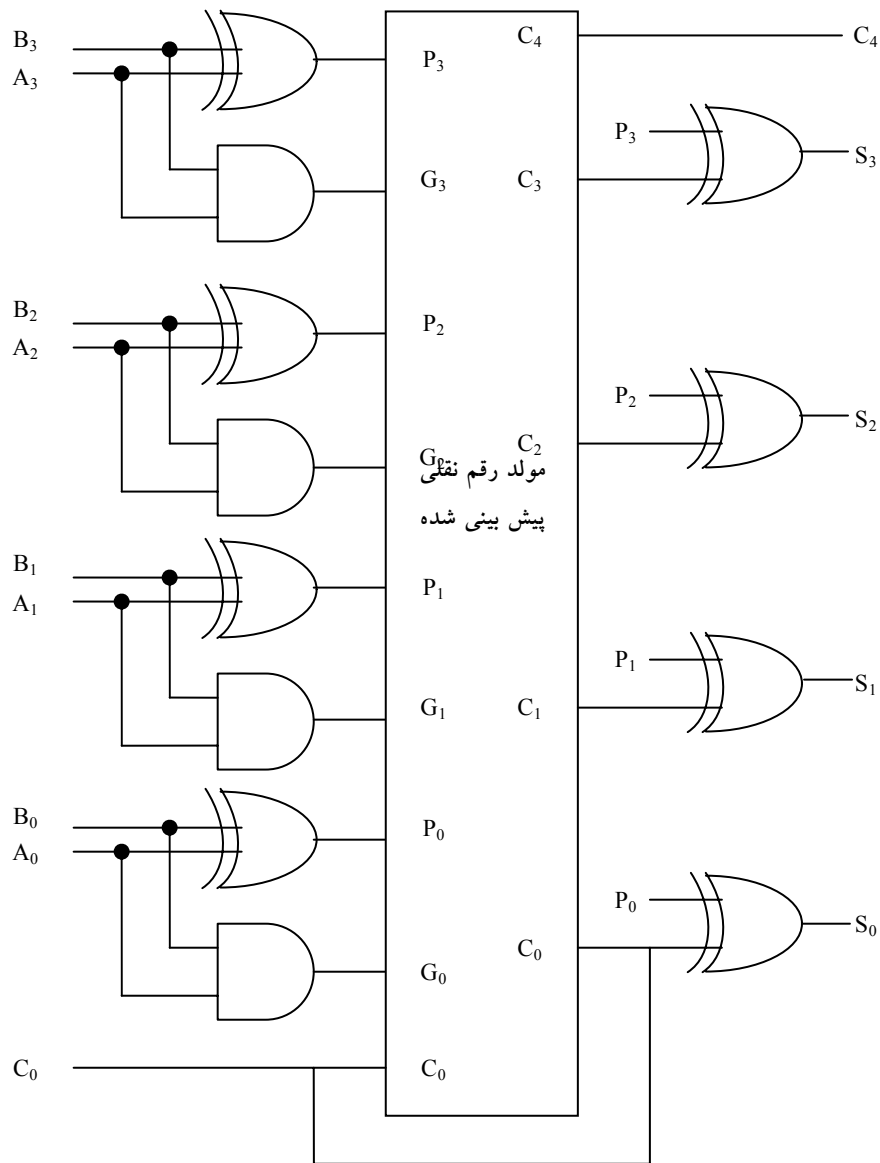
$$C_{i+1} = G_i + P_i C_i$$

مدارهای ترکیبی ۱۸۳

G_i را مولد نقلی نامند که وقتی هر دو A_i و B_i برابر 1 باشند، یک نقلی 1 را تولید می‌نماید، و این تولید مستقل از C_i می‌باشد. P_i را انتشار نقلی گویند زیرا جمله‌ای است که در انتشار نقلی از C_i به C_{i+1} نقش دارد. اکنون توابع بول را برای خروجی‌های نقلی هر طبقه نوشته و هر C_i را با مقدار معادل قبلی جایگزین می‌کنیم:

$$\begin{aligned} C_0 &= \text{نقلی ورودی} \\ C_1 &= G_0 + P_0 C_0 \\ C_2 &= G_1 + P_1 C_1 \\ &= G_1 + P_1 (G_0 + P_0 C_0) \\ &= G_1 + P_1 G_0 + P_1 P_0 C_0 \\ C_3 &= G_2 + P_2 C_2 \\ &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \end{aligned}$$

چون تابع بول برای هر نقلی خروجی برحسب جمع حاصل‌ضربها بیان شده است، هر تابع قابل پیاده‌سازی با یک طبقه گیت AND و بدنبال آن گیت OR (یا با دو طبقه NAND) است. سه تابع بول برای C_1, C_2 و C_3 در مولد نقلی پیش بینی شونده و در شکل ۶-۱۶ دیده می‌شوند. توجه کنید که C_3 نیاز ندارد به انتظار C_2 و C_1 بماند. در واقع C_3 با C_1 و C_2 همزمان منتشر می‌گردد. ساخت یک جمع‌کننده 4 بیت با پیش بینی نقلی در شکل ۶-۱۷ مشاهده می‌شود.



شکل ۶-۱۷: جمع کننده ۴ بیت با پیش بینی نقلی

خروجی اولین گیت XOR متغیر P_i و گیت AND متغیر G_i را تولید می‌نماید. نقلی‌ها از درون مولد پیش بینی نقلی انتشار می‌یابند (مشابه شکل ۶-۱۶) و به عنوان ورودی به

گیت XOR دوم اعمال می گردند. همه نقلی های خروجی پس از یک تاخیر در دو طبقه گیت به طور همزمان تولید می شوند. بنابراین S_1 تا S_3 دارای زمان تاخیر انتشار یکسانی هستند. مدار دو طبقه برای نقلی خروجی C_4 نشان داده نشده است. این مدار هم به سادگی با روش جایگزینی قابل دستیابی است.

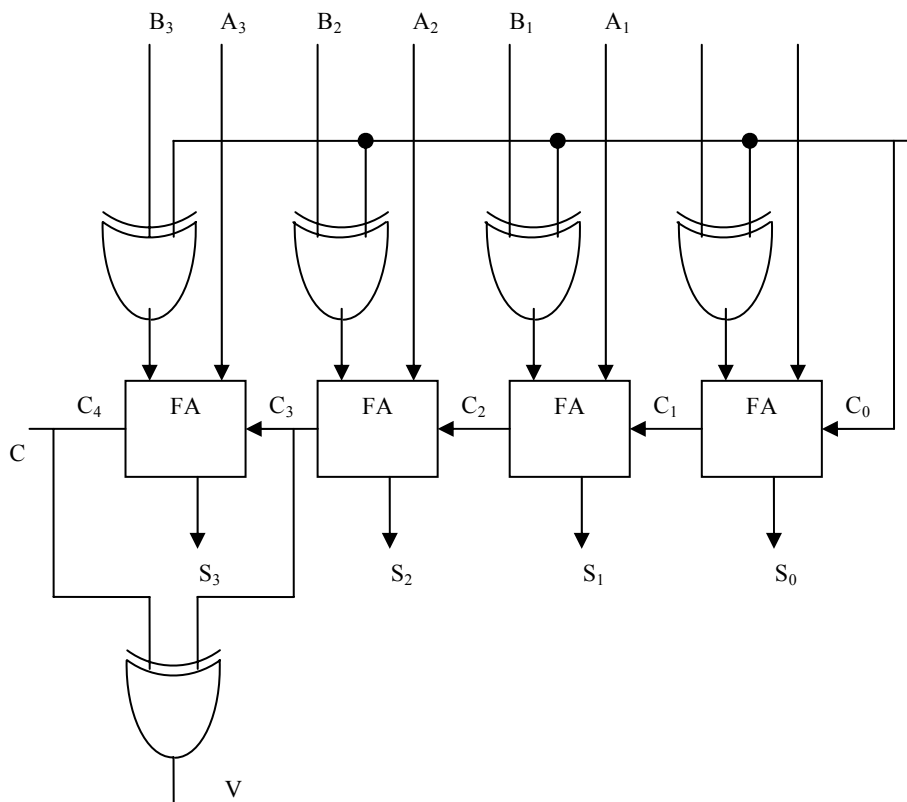
۶-۴-۵ تفریق دودویی

همانطور که در فصل اول در مبحث متممها مطرح شد، تفریق اعداد دودویی بی علامت با استفاده از متمم راحت تر انجام می گیرد. به خاطر دارید که $A-B$ را می توان با محاسبه متمم ۲ عدد B و جمع آن با A معین کرد. متمم ۲ را با به دست آوردن متمم ۱ و جمع آن با ۱ محاسبه می کنند. متمم ۱ را هم با وارون گر به دست آورده و عدد ۱ را هم از طریق ورودی نقلی به آن اضافه می نمایند.

مدار تفریق گر $A-B$ متشکل از یک جمع کننده با وارون گرهای واقع در بین ورودی B و ورودی مربوطه اش در تمام جمع کننده می باشد. نقلی ورودی C_0 به هنگام تفریق باید برابر با ۱ شود. بنابراین عمل به صورت A به علاوه متمم ۱ عدد B به علاوه ۱ اجرا می شود. این عمل برابر با جمع A با متمم ۲ عدد B خواهد بود. برای اعداد بی علامت اگر $A \geq B$ باشد، عمل فوق $A-B$ و اگر $A < B$ باشد $(B-A)$ است. برای اعداد علامت دار، نتیجه $A-B$ است به شرطی که سرریز وجود نداشته باشد.

عملیات جمع و تفریق را می توان با یک مدار در هم ادغام کرده و با یک جمع کننده دودویی مشترک انجام داد. این کار با افزودن یک گیت XOR در هر جمع کننده کامل صورت می گیرد. یک مدار جمع-تفریق گر در شکل ۶-۱۸ دیده می شود. وقتی $M=0$ است، مدار یک جمع کننده و وقتی $M=1$ باشد، مدار یک تفریق گر خواهد بود. هر گیت XOR ورودی M و یکی از ورودی های B را دریافت می کند. وقتی $M=0$ است داریم، $B \oplus 0 = B$. جمع کننده کامل B را دریافت می نماید، نقلی ورودی ۰ است و بنابراین مدار عمل A به علاوه B را اجرا می کند. اگر $M=1$ باشد، $B \oplus 1 = B'$ بوده و $C_0=1$ است.

ورودی‌های B همگی متمم شده و از طریق ورودی نقلی، یک 1 به آن اضافه می‌شوند. در این حالت مدار یک عمل A به علاوه متمم 2 عدد B را انجام می‌دهد. (XOR با خروجی V، یک سرریز را شناسایی می‌نماید).



شکل ۶-۱۸: جمع - تفریق گر

نکته‌ای که در این بین مطرح می‌گردد این است که در سیستم متمم علامت‌دار، اعداد دودویی هم چون اعداد بی علامت، با قوانین جمع و تفریق یکسانی ترکیب می‌شود. بنابراین، کامپیوترها نیاز به یک سخت‌افزار مشترک دارند تا هر دو نوع محاسبه را انجام دهند. کاربر یا برنامه نویس باید نتایج چنین جمع یا تفریقی را متفاوت تفسیر کنند و این به علامت‌دار یا بی علامت بودن اعداد بستگی دارد.

۶-۴-۶ مفهوم سرریز

هر گاه دو عدد n رقمی با هم جمع شوند و حاصل جمع $n+1$ رقم را اشغال کند، گوییم سرریز رخ داده است. این مطلب جدا از علامت‌دار بودن یا نبودن برای اعداد دهدهی یا دودویی صحیح است. وقتی که جمع یا کاغذ و قلم انجام می‌شود، سرریز مسئله‌ای نیست زیرا محدودیتی برای عرض صفحه جهت نوشتن جمع وجود ندارد. ولی سرریز در کامپیوترهای دیجیتال مشکلاتی ایجاد می‌کند، زیرا تعداد بیت‌های نگهداری عدد محدود بوده و نتیجه‌ای را که $n+1$ بیت دارد نمی‌توانند در خود جای دهند. به این دلیل، بسیاری از کامپیوترها وقوع یک سرریز را، اگر رخ دهد، شناسایی می‌کنند و فیلپ فلاپ مربوطه را در 1 می‌نشانند تا بعد به وسیله کاربر چک شود.

تشخیص یک سرریز پس از جمع دو عدد دودویی به این بستگی دارد که آیا اعداد علامت دارند یا بی علامت‌اند. وقتی دو عدد بی علامت با هم جمع شوند، یک سرریز از نقلی با ارزش‌ترین مکان تشخیص داده می‌شود. در حالتی که اعداد علامت‌دار باشند، سمت چپ‌ترین بیت همواره علامت را نشان داده و اعداد منفی هم به صورت متمم 2 هستند. وقتی دو عدد علامت‌دار جمع شوند، با بیت علامت به عنوان بخشی از عدد رفتار می‌شود و رقم نقلی انتهایی هیچ سرریزی را مشخص نمی‌کند.

در جمع وقتی که یکی از اعداد مثبت و دیگری منفی باشد، سرریز رخ نمی‌دهد، زیرا جمع یک عدد مثبت با یک عدد منفی نتیجه‌ای تولید می‌کند که از بزرگترین آن دو کوچکتر است. سرریز هنگامی رخ می‌دهد که هر دو عدد جمع شونده مثبت یا منفی باشند. برای درک بهتر موضوع مثال زیر را ملاحظه کنید. دو عدد علامت‌دار دودویی $+70$ و $+80$ دودویی در دو ثبات 8 بیتی ذخیره شده‌اند محدود اعدادی که هر یک از ثبات‌ها داراست از $+127$ تا -128 دودویی است. چون مجموع دو عدد $+150$ است، حاصل از ظرفیت ثبات 8 بیتی تجاوز خواهد کرد. این مطالب هنگامی که هر دو عدد مثبت یا منفی باشند صحت دارد. دو جمع مذکور همراه با ارقام نقلی در زیر نشان داده شده‌اند:

0	1	نقلی ها :	0	1	نقلی ها :
+70	0	1000110	-70	1	0111010
+80	0	1010000	-80	1	0110000
+150	1	0010110	-150	0	1101010

توجه کنید که حاصل جمع هشت بیتی که باید مثبت باشد یک بیت علامت منفی دارد و نتیجه 8 بیتی که باید منفی باشد دارای بیت علامت مثبت است. با این وجود اگر رقم نقلی خارج شده از بیت علامت به عنوان بیت علامت دار در نظر گرفته شود، آنگاه جواب 9 بیتی حاصل صحیح خواهد بود. چون پاسخ نمی تواند در 8 بیت جای داده شود، گوییم سرریز رخ داده است.

وضعیت سرریز را می توان با وجود رقم نقلی به بیت علامت و نقلی خروجی از بیت علامت مشاهده کرد. اگر این دو نقلی یکی نباشند، یک سرریز رخ داده است. این نکته در مثال های فوق که در آن دو نقلی به طور جداگانه نشان داده شده اند دیده می شود. اگر دو رقم نقلی را به یک گیت XOR اعمال کنیم، وقوع سرریز با 1 شدن خروجی این گیت شناسایی می شود. برای اینکه روش به خوبی کار کند متمم 2 باید از طریق به دست آوردن متمم 1 و جمع آن با 1 انجام گردد. این کار موجب مراقبت از حالتی می شود که در آن عدد منفی ماکزیمم متمم شود.

مدار جمع - تفریق گر با خروجی های C و V در شکل ۶-۱۸ دیده می شود. اگر دو عدد دودویی بی علامت تصور شوند، آنگاه بیت C، نقلی بعد از جمع یا قرض بعد از تفریق است. اگر اعداد علامت دار فرض شوند، آنگاه بیت V یک سرریز را مشخص می کند. اگر $V=0$ بعد از یک جمع یا تفریق باشد، بیانگر نبود سرریز بوده و نتیجه n بیتی حاصل صحیح است. اگر $V=1$ باشد، در این صورت نتیجه عمل حاوی $n+1$ بیت می باشد، ولی بیت $n+1$ ام علامت واقعی است به یک مکان بیرونی منتقل شده است.

۶-۴-۷ جمع کننده دهمی

کامپیوترها یا ماشین‌های حسابی که اعمال محاسباتی را مستقیماً در سیستم اعداد دهمی انجام می‌دهند، اعداد دهمی را به فرم کد دودویی ارائه می‌کنند. یک جمع کننده در این کامپیوترها، از نوعی مدار محاسباتی استفاده می‌کند که اعداد دهمی کد شده را می‌پذیرد و نتایج را در همان کد ارائه می‌نماید. برای جمع دودویی کافی است جفت بیت با ارزش را همراه با رقم نقلی قبلی در نظر بگیرید. یک جمع کننده دهمی به حداقل ده ورودی و پنج خروجی نیاز دارد زیرا برای کد هر رقم دهمی چهار بیت لازم است و مدار باید ورودی و خروجی نقلی هم داشته باشد. برای انجام این گونه جمع، مدارهای جمع کننده دهمی متعددی وجود دارند که انتخاب آنها به کد به کار رفته در نمایش ارقام دهمی بستگی دارد. در اینجا ما جمع کننده دهمی را برای کد BCD بررسی می‌کنیم.

۶-۴-۸ جمع کننده BCD

جمع حسابی دو رقم دهمی در BCD را همراه با یک رقم نقلی از مرحله قبل در نظر بگیرید. چون هر رقم ورودی از ۹ تجاوز نمی‌کند، حاصل جمع خروجی از $19 = 9 + 9 + 1$ بیشتر نخواهد شد. عدد ۱ در جمع فوق، نقلی ورودی است. فرض کنید که دو رقم BCD را به جمع کننده دودویی ۴ بیتی اعمال نماییم. جمع کننده، حاصل جمع را به فرم دودویی اجرا می‌کند و نتیجه تولید شده بین ۰ تا ۱۹ خواهد بود. این اعداد دودویی در جدول (۵-۴) مشاهده می‌شود که با K, z_8, z_4, z_2, z_1 بر چسب خورده‌اند. K یک رقم نقلی است و اندیس زیر حرف z وزن های ۸، ۴، ۲ و ۱ می‌باشند که به چهار بیت کد BCD تخصیص یافته‌اند. ستون زیر حاصل جمع دودویی، مقادیر دودویی ظاهر شده در خروجی‌های جمع کننده چهار بیت را نشان می‌دهد. حاصل جمع خروجی دو رقم دهمی باید به فرم BCD در آید و نیز باید آن طور که در زیرستون جمع BCD ملاحظه می‌شود ظاهر گردد. مسئله این است که برای تبدیل جمع دودویی

به رقم BCD عدد که در ستون جمع BCD مشاهده می‌شود باید قانونی پیدا شود. ضمن بررسی محتوای جدول، ملاحظه می‌شود که وقتی جمع دودویی برابر با یا کمتر از 1001 باشد، با عدد BCD نظیر خود برابر است، و بنابراین تبدیلی لازم نیست. وقتی جمع دودویی بزرگتر از 1001 باشد، نمایش بی اعتباری را برای BCD خواهیم داشت. افزایش دودویی 6 (0110) به جمع دودویی آن را به نمایش BCD صحیح تبدیل می‌کند، ضمن این که یک رقم نقلی نیز در صورت لزوم تولید خواهد کرد.

K	جمع دودویی					جمع BCD				دهمی
	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

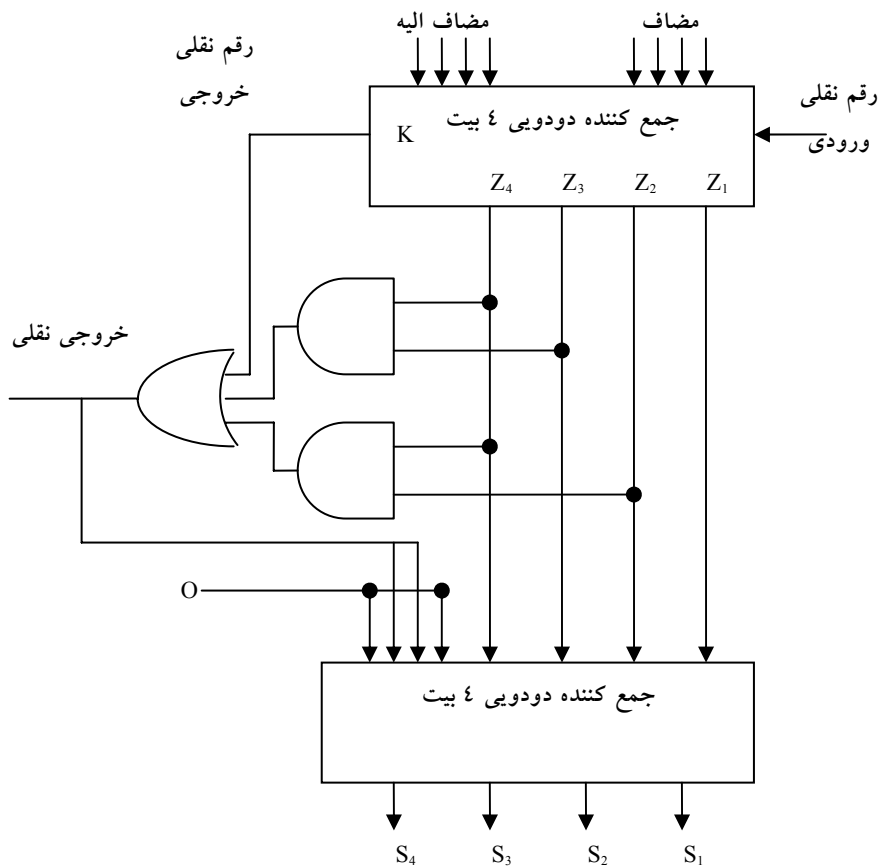
شکل ۶-۱۹: جدول طریقه طراحی جمع کننده BCD

مدار منطقی برای تشخیص این اصلاح، می‌تواند از وارده‌های جدول حاصل گردد. واضح است که وقتی نقلی خروجی $K=1$ باشد نیاز به اصلاح جمع دودویی وجود دارد. دیگر ترکیبات شش گانه از 1010 تا 1111 که به اصلاح نیاز دارند دارای 1 در مکان z_8 می‌باشند. برای تفکیک این شش حالت از 1000 و 1001، که آنها نیز دارای 1 در مکان

Z_8 هستند، به Z_4 و Z_2 مراجعه می‌کنیم که در حال حداقل یکی از آنها 1 است. به این ترتیب شرط اصلاح و داشتن یک نقلی خروجی را می‌توان با تابع بولی زیر بیان کرد:

$$C = K + Z_8 Z_4 + Z_8 Z_2$$

وقتی $C=1$ است، لازم است 0110 به جمع دودویی اضافه شود تا یک نقلی خروجی برای طبقه بعدی فراهم شود.



شکل ۶-۲۰: نمودار بلوکی یک جمع کننده BCD

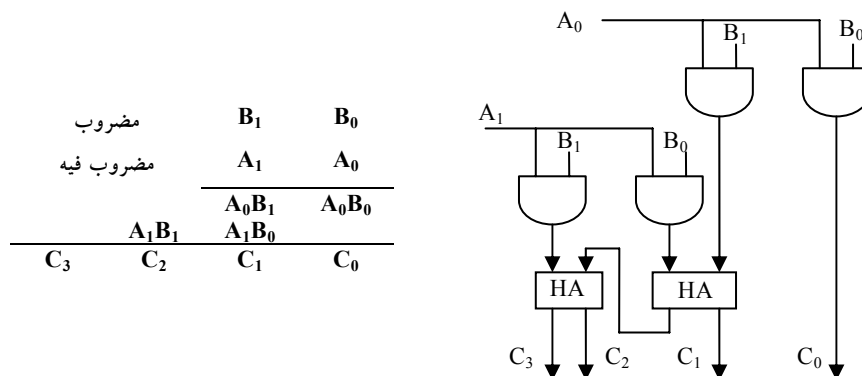
یک جمع کننده BCD که دو رقم BCD را با هم جمع کرده و ارقام جمع را به BCD نشان می‌دهد در شکل ۶-۲۰ ملاحظه می‌گردد. دو رقم دهدهی همراه با نقلی ورودی ابتدا در جمع کننده 4 بیت فوقانی جمع شده و حاصل جمع دودویی تولید می‌کنند.

وقتی نقلی خروجی برابر 0 باشد، چیزی به جمع دودویی اضافه نمی‌شود. وقتی این 6 نقلی برابر 1 باشد، عدد دودویی 0110 از طریق جمع‌کننده 4 بیت پایینی به جمع دودویی اضافه می‌گردد. نقلی خروجی تولید شده در جمع‌کننده پایین می‌تواند صرف نظر شود زیرا اطلاعاتی را حمل می‌کند که قبلاً در پایانه نقلی خروجی وجود داشته است. یک جمع‌کننده دهدهی موازی که n رقم دهدهی را جمع می‌کند به n طبقه جمع‌کننده BCD نیاز دارد. نقلی خروجی هر طبقه باید به ورودی طبقه بالاتر متصل گردد.

۶-۵ ضرب دودویی

یکی از مهمترین مدارات دیجیتال مورد استفاده در سیستم‌ها مدار ضرب اعداد می‌باشد. ضرب اعداد دودویی همچون ضرب اعداد دهدهی انجام می‌شود. هر بیت مضروب، در کم ارزش‌ترین بیت مضروب‌فیه ضرب می‌شود. چنین حاصلضربی، حاصلضرب جزئی خوانده می‌شود. حاصلضرب‌های جزئی هر بار یک مکان به چپ انتقال می‌یابند. حاصلضرب نهایی از جمع حاصلضرب‌های جزئی به‌دست می‌آید.

برای این که ببینیم که یک ضرب‌کننده چگونه با یک مدار ترکیبی پیاده می‌شود،



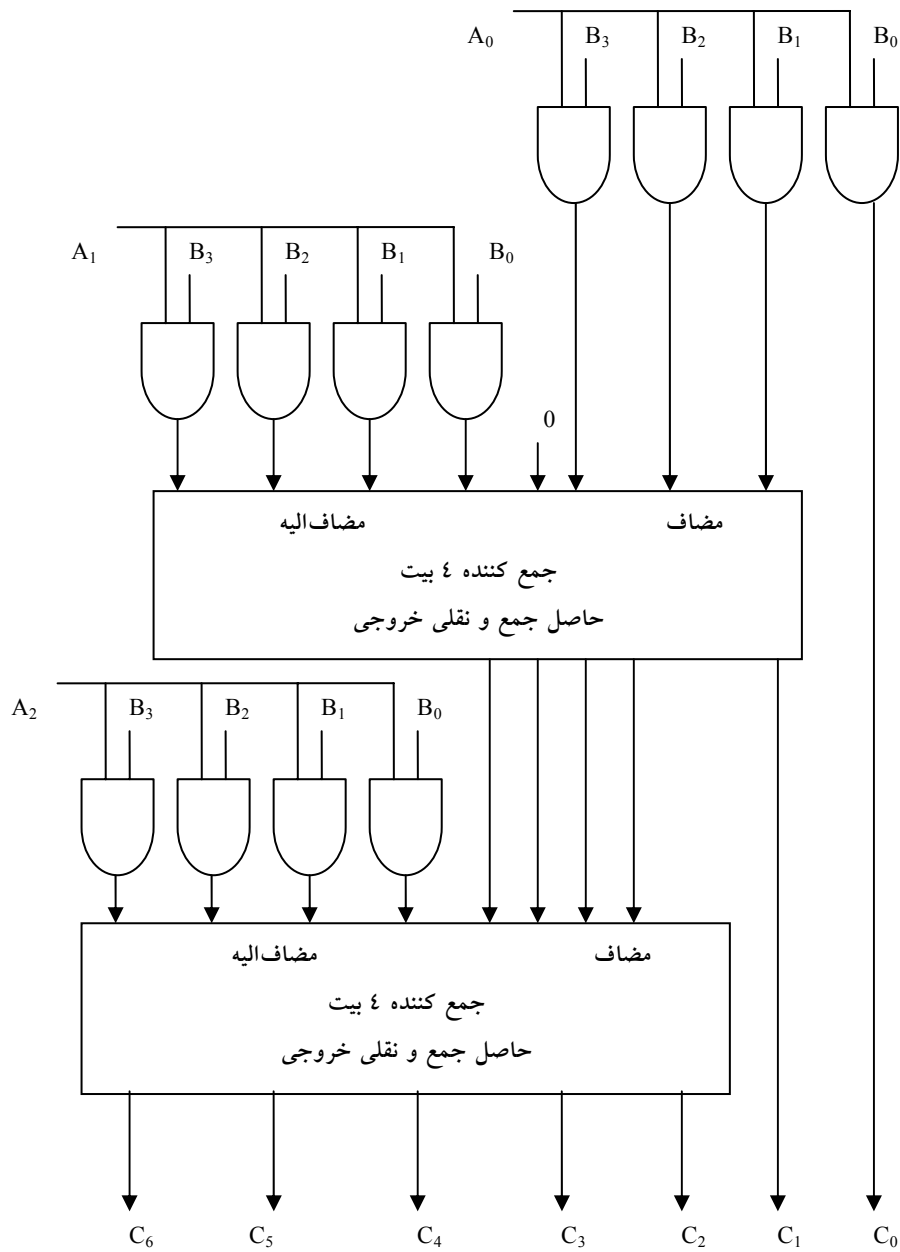
شکل ۶-۲۱: ضرب دودویی 2 بیت در 2 بیت

ضرب اعداد دو بیت را طبق شکل ۶-۲۱ در نظر بگیرید. بیت‌های مضروب، B_1 و B_0 و بیت‌های مضروب فیه A_0 و A_1 و حاصلضرب $C_3C_2C_1C_0$ فرض می‌شوند. اولین حاصلضرب جزئی با ضرب A_0 در B_0 حاصل می‌گردد. ضرب دو بیت مثل A_0 و B_0 هنگامی ۱ تولید می‌کند که هر دوی آنها ۱ باشند، در غیر این صورت ۰ تولید خواهد کرد. این پاسخ مشابه با عمل AND است. بنابراین حاصل ضرب جزئی را می‌توان با گیت‌های AND مطابق شکل پیاده کرد.

دومین حاصلضرب جزئی از ضرب A_1 در B_1 به دست می‌آید که باید یک مکان هم به چپ جابجا شود. دو حاصلضرب جزئی به وسیله مدار دو نیم جمع‌کننده (HA) با هم جمع می‌شوند. معمولاً در حاصلضرب‌های جزئی بیت‌های بیشتری وجود دارند و لازم است از تمام جمع‌کننده برای تولید جمع حاصلضرب‌های جزئی استفاده شود. توجه کنید لزومی ندارد که کم ارزش‌ترین بیت از جمع‌کننده عبور کند زیرا با خروجی اولین گیت AND، تشکیل شده است.

به طریقی مشابه می‌توان یک مدار ترکیبی ضرب دودویی با بیت‌های بیشتر ساخت. هر بیت از مضروب فیه در بیت‌های مضروب، AND می‌گردد. خروجی دودویی در هر سطحی از گیت‌های AND با حاصلضرب جزئی سطح قبلی برای تشکیل حاصلضرب جزئی جدید جمع می‌شود. آخرین سطح حاصلضرب کل را تولید می‌کند. برای J بیت مضروب فیه و K بیت مضروب به $(J \times K)$ گیت AND و $(J - 1)$ عدد جمع‌کننده K بیت نیاز است تا حاصلضرب $J+K$ بیتی تولید شود.

به عنوان دومین مثال مدار ضرب کننده ای را ملاحظه نمایید که یک عدد دودویی ۴ بیتی را در یک عدد ۳ بیتی ضرب می‌کند. فرض کنید مضروب با $B_3B_2B_1B_0$ و مضروب فیه $A_2A_1A_0$ باشد. چون $K=4$ و $J=3$ است ۱۲ گیت AND و دو جمع‌کننده ۴ بیت برای تولید حاصلضرب ۷ بیتی لازم است. نمودار منطقی ضرب کننده در شکل ۶-۲۲ دیده می‌شود.



شکل ۶-۲۲: ضرب کننده دودویی ۴ بیت در ۳ بیت

۶-۶ مقایسه گر مقدار

مقایسه دو عدد عملی است که توسط آن بزرگتر بودن، کوچکتر بودن یا مساوی بودن آنها معین می‌شود. یک مقایسه گر مداری ترکیبی است که دو عدد A و B را مقایسه می‌نماید و اندازه نسبی آنها را تعیین می‌کند. نتیجه این مقایسه با سه متغیر

دودویی که بیانگر $A > B$ یا $A < B$ می‌باشد، مشخص می‌گردد.

مدار مقایسه دو عدد n بیت 2^{2^n} وارده در جدول دارد و حتی با $n=3$ خسته کننده خواهد شد. از طرف دیگر، یک مدار مقایسه ممکن است مقدار قابل توجهی نظم در خود داشته باشد. توابع دیجیتال که ذاتاً دارای نظمی درونی هستند، معمولاً با روال‌های الگوریتمی قابل طراحی‌اند. یک الگوریتم روالی است که مجموعه مراحل معینی را مشخص می‌نماید و اگر دنبال شوند، از آن حلی برای مسئله حاصل می‌گردد. ما در اینجا از این روش برای ارائه یک الگوریتم جهت پیاده‌سازی مقایسه گر مقدار 4 بیتی استفاده خواهیم کرد. دو عدد A و B را که هر کدام چهار رقم دارند در نظر بگیرید. ضرایب اعداد را به ترتیب نزولی زیر هم می‌نویسیم:

$$A = A_3A_2A_1A_0$$

$$B = B_3B_2B_1B_0$$

هر حرف اندیس‌دار یک رقم را در عدد نشان می‌دهد. دو عدد هنگامی مساوی‌اند که همه جفت ارقام متناظر با هم برابر باشند: یعنی

$$A_0 = B_0, A_1 = B_1, A_2 = B_2, A_3 = B_3$$

وقتی که اعداد دودویی باشند، ارقام 1 یا 0 اند و رابطه تساوی هر جفت بیت به طور منطقی با یک تابع XOR نمایش داده می‌شود.

$$X_i = A_iB_i + A'_iB'_i \text{ for } i=0,1,2,3$$

که در آن $x_i=1$ به شرطی صحت دارد که بیت‌های مکان i ام برابر باشند (یعنی اگر هر دو 1 یا هر دو 0).

برابری دو عدد A و B در مدار ترکیبی با یک متغیر خروجی و با علامت $(A=B)$ نشان داده می‌شود. این متغیر دودویی هنگامی 1 است که همه اعداد ورودی A و B مساوی باشند، در غیر این صورت 0 است. برای این که شرایط برابری برقرار باشد همه متغیرهای x_i باید برابر 1 شوند. در این صورت AND همه متغیرها دیکته خواهد شد:

$$(A=B) = x_3x_2x_1x_0$$

عدد دودویی $(A=B)$ هنگامی 1 است که فقط همه جفت ارقام برابر باشند.

برای اینکه معین کنیم آیا A بزرگتر یا کوچکتر از B است، اندازه‌های نسبی دو رقم را با شروع از باارزش‌ترین مکان آغاز می‌نماییم. اگر دو رقم مساوی باشند، دو رقم پایین‌تر را مقایسه می‌کنیم. این مقایسه تا رسیدن به یک جفت غیر مساوی ادامه خواهد داشت و اگر در این هنگام بیت متعلق به A برابر 1 و B برابر 0 باشد، نتیجه می‌گیریم $A > B$ است. اگر بر عکس رقم مربوط به A برابر 0 و B برابر 1 باشد، خواهیم داشت $A < B$. مقایسه فوق را می‌توان با کمک دو تابع بولی به صورت زیر نوشت:

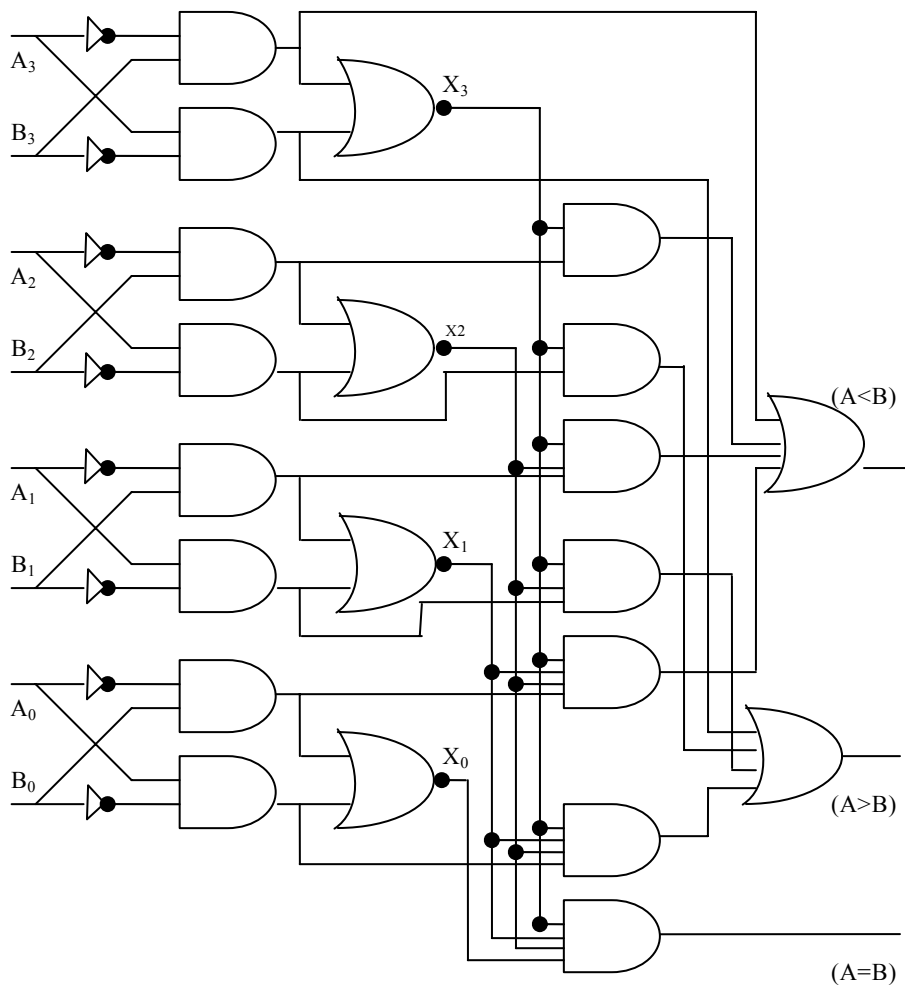
$$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$$

$$(A < B) = A'_3B_3 + x_3A'_2B_2 + x_3x_2A'_1B_1 + x_3x_2x_1A'_0B_0$$

سمبل‌های $(A > B)$ و $(A < B)$ متغیرهای خروجی دودویی هستند که بترتیب هنگام $A > B$ یا $A < B$ برابر 1 می‌شوند.

پیاده‌سازی گیتی سه متغیر خروجی ساده‌تر از آنچه به نظر می‌رسد انجام می‌شود زیرا شامل مقدار قابل توجهی اعمال تکراری است. خروجی‌های نامساوی می‌توانند از گیت‌هایی که برای تولید خروجی مساوی لازم بود استفاده کنند. نمودار منطقی یک مقایسه گر مقدار 4 بیتی در شکل ۶-۲۳ ملاحظه می‌شود.

چهار خروجی x با مدارهای XNOR تولید شده و به گیت AND اعمال شده‌اند تا متغیر دودویی خروجی $(A=B)$ تولید گردد. دو خروجی دیگر از متغیر x برای تولید توابع بولی لیست شده قبلی استفاده می‌کنند. این یک پیاده‌سازی چند طبقه است که الگوی منظمی دارد. روال برای به‌دست آوردن مدارهای مقایسه‌گر اندازه برای اعداد دودویی با بیش از چهار بیت از این مثال کاملاً آشکار است.



شکل ۶-۲۳: مقایسه‌گر مقدار چهار بیتی

سؤالات

- ۱- یک مدار ترکیبی با سه ورودی و یک خروجی طراحی نمایید.
- ۲- اگر تولید و انتشار نقلی را به صورت $P_i = A_i + B_i$ و $G_i = A_i B_i$ داشته باشیم، نشان دهید که نقلی خروجی و جمع خروجی یک جمع کننده کامل به صورت زیر خواهد بود.
$$C_{i+1} = (C'_i G'_i + P'_i)'$$
$$S_i = (P_i G'_i) \oplus C_i$$
- ۳- یک مدار ترکیبی طراحی کنید تا متمم ۹ یک رقم BCD را تولید نماید.
- ۴- یک ضرب کننده دودویی برای ضرب دو عدد چهار بیتی طراحی نمایید.
- ۵- یک مدار جمع کننده کامل BCD بسازید.
- ۶- یک مدار ترکیبی افزایش گر ۴ بیتی را با چهار نیم جمع کننده طراحی نمایید.

فصل ۷

مدارهای رمزگذار و رمزگشا

هدف کلی

در این فصل مباحث اصلی مربوط به مدارهای رمزگذار و رمزگشا مورد بحث و بررسی قرار گرفته و روش پیاده‌سازی این نوع مدارها شرح داده خواهد شد. همچنین مولتی‌پلکسرها نیز در این فصل بررسی خواهند شد.

هدف ساختاری

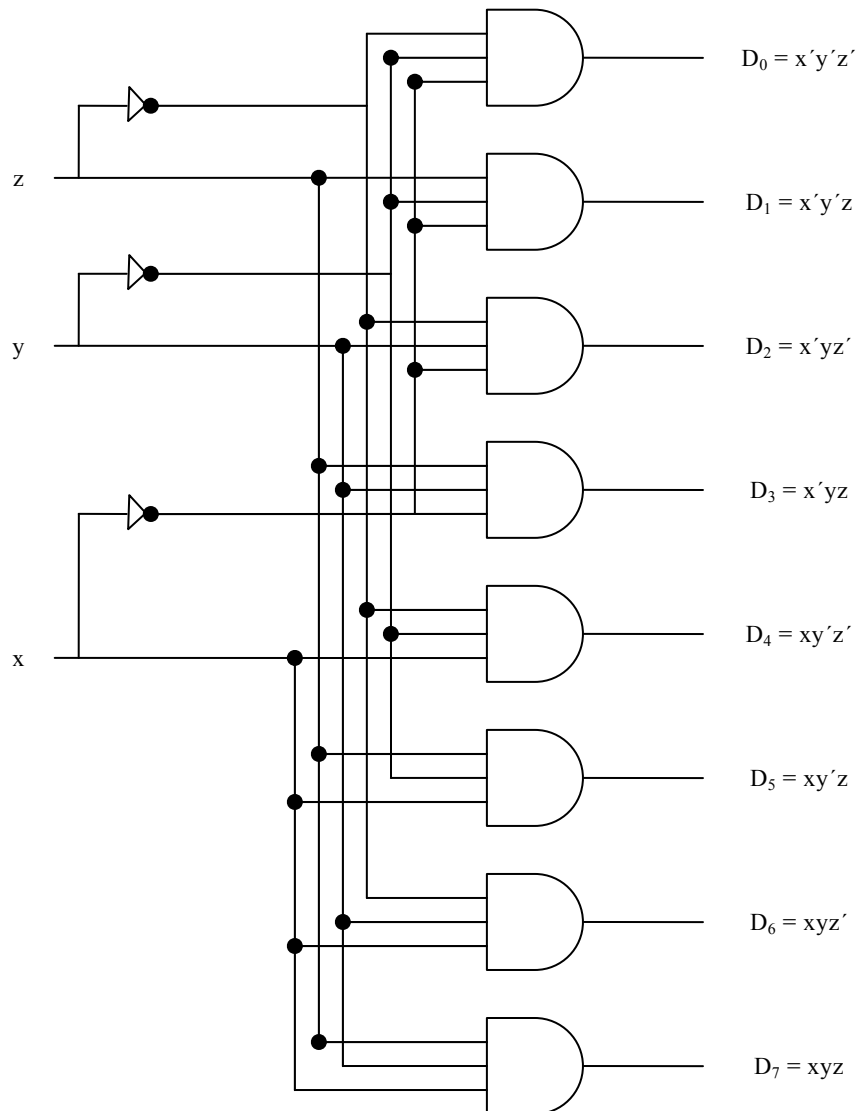
در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند:

- مدارهای رمزگشا (دیکدر)
- مدارهای رمزگذار (انکدر)
- پیاده‌سازی مدارهای رمزگذار و رمزگشا
- مولتی‌پلکسرها

یکی از مباحث مطرح در طراحی مدارات دیجیتال، ایجاد مدارات رمزگذار و رمزگشا می‌باشند. در این فصل این دو نوع مدار به تفصیل مورد بحث و بررسی قرار خواهند گرفت.

۱-۷ مدارات رمزگشا (دیکدر)

کمیت‌های گسسته اطلاعاتی در سیستم‌های دیجیتال با کدهای دودویی نشان داده می‌شوند. یک کد دودویی n بیتی قادر است تا 2^n عنصر گسسته اطلاعات کد شده را



شکل ۱-۷: دیکدر ۳ به ۸

مدارهای رمزگذار و رمزگشا ۲۰۱

نشان دهد. یک دیکدر مداری ترکیبی است که اطلاعات دودویی را از n خط ورودی به حداکثر 2^n خط خروجی منحصر به فرد تبدیل می‌کند اگر کد n بیتی دارای ترکیبات بی استفاده باشد، دیکدر ممکن است خروجی کمتر از 2^n داشته باشد.

دیکدرهایی که در اینجا ارائه شده‌اند دیکدرهای n به m خوانده می‌شوند که $m \leq 2^n$ است. هدف از آنها تولید 2^n مینترم (یا کمتر) از n متغیر ورودی است. نام دیکدر همراه با دیگر مبدل‌های کد مانند دیکدر BCD به هفت قسمتی هم به کار می‌رود. به عنوان مثال دیکدر 3 به 8 قسمتی شکل ۷-۱ را ملاحظه نمایید.

سه ورودی به هشت خروجی دیکدر شده است که هر یک نمایشگر یکی از مینترم‌های متعلق به سه متغیر ورودی است. سه وارون‌گر، متمم ورودی‌ها را تهیه کرده و هشت گیت AND هر کدام یک مینترم تولید می‌کنند. کاربر رایج این نوع دیکدر، تبدیل دودویی به هشت هشتی است. متغیرهای ورودی یک عدد دودویی را نشان می‌دهند، و خروجی بیانگر هشت رقم در سیستم اعداد مبنای هشت است. با این وجود دیکدر 3 به 8 خط را می‌توان برای دیکدر کردن هر کد 3 بیت در تولید هشت خروجی، یکی برای هر عنصر از کد، به کار برد.

ورودی‌ها			خروجی‌ها							
X	Y	Z	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

شکل ۷-۲: جدول درستی دیکدر 3 به 8 خط

طرز کار یک دیکدر می‌تواند با لیستی در جدول درستی شکل ۷-۲ آشکار شود. برای هر ترکیب ورودی ممکن، هفت خروجی وجود دارد که برابر 0 هستند و فقط

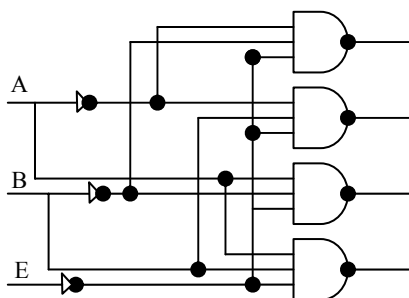
یکی از آنها برابر 1 است. خروجی حاوی 1 بیانگر مینترم عدد دودویی حاضر در خطوط ورودی است.

۷-۱- پیاده‌سازی دیکدر با گیت NAND

از آنجائیکه پیاده‌سازی گیت NAND ساده‌تر است، لذا بعضی از دیکدرها با گیت‌های NAND ساخته می‌شوند. چون گیت NAND عمل AND را با یک خروجی معکوس تولید می‌کند، تولید مینترم‌های دیکدر در شکل متمم اقتصادی‌تر است. به علاوه، دیکدر معمولاً دارای یک یا دو ورودی تواناساز یا فعال‌ساز برای کنترل کار مدار می‌باشند. یک دیکدر 2 به 4 با یک ورودی فعال‌ساز که با گیت‌های NAND ساخته شده در شکل ۷-۳ دیده می‌شود.

E	A	B	D ₀	D ₁	D ₂	D ₃
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

(ب) جدول درستی



(الف) دیاگرام منطقی

شکل ۷-۳: دیکدر 2 به 4 خط با ورودی فعال‌ساز

مدار با خروجی‌های متمم شده و یک ورودی فعال‌ساز متمم شده کار می‌کند. دیکدر هنگامی که E برابر 0 باشد فعال می‌گردد. همانطور که توسط جدول درستی مشاهده می‌شود، هر بار تنها یک خروجی برابر 0 بوده و دیگر خروجی‌ها در وضعیت 1 قرار دارند. وقتی E=1 باشد مدار غیر فعال است و به دو ورودی دیگر بستگی ندارد. هنگام غیر فعال شدن مدار، هیچ یک از خروجی‌ها در 0 نبوده و هیچ یک از مینترم‌ها انتخاب نمی‌شوند. به طور کلی، یک دیکدر ممکن است خروجی‌های متمم شده یا متمم نشده

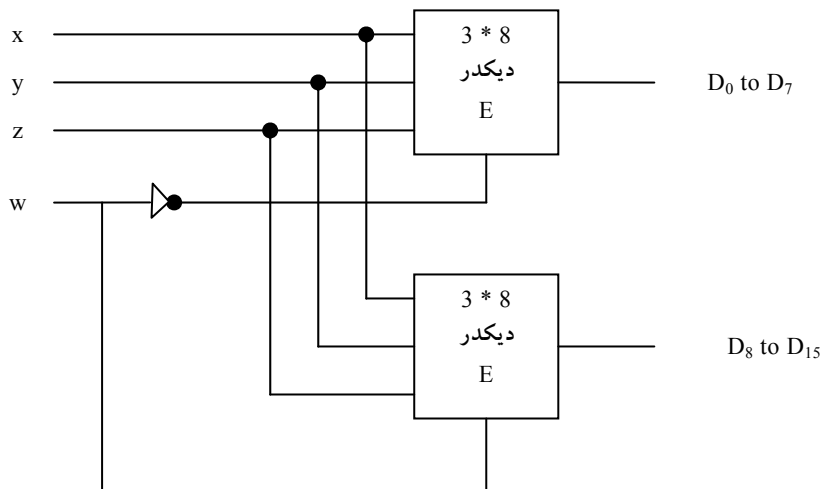
مدارهای رمزگذار و رمزگشا ۲۰۳

داشته باشد. ورودی فعال‌ساز ممکن است با سیگنال 0 یا 1 فعال گردد. بعضی از دیکدرها دارای دو یا چند ورودی فعال‌ساز می‌باشند که باید یک شرط منطقی مفروضی را بر آورده سازند تا مدار فعال شود.

یک دیکدر با ورودی فعال‌ساز می‌تواند به عنوان یک دی مولتی‌پلکسر عمل کند (مباحث مربوط به مولتی‌پلکسر و دی مولتی‌پلکسر در همین فصل توضیح داده خواهد شد). دی مولتی‌پلکسر مداری است که اطلاعات را از یک خط دریافت کرده و آن را به یکی از 2^n خط خروجی ممکن هدایت می‌نماید. انتخاب یک خروجی خاص با ترکیب بیتی n خط انتخاب صورت می‌گیرد. دیکدر شکل ۷-۳ را می‌توان به عنوان یک دی مولتی‌پلکسر 1 به 4 به کار برد. در این مدار E به عنوان ورودی داده و A و B ورودی‌های انتخاب هستند. تنها متغیر ورودی E مسیری به تمام چهار خروجی دارد، ولی اطلاعات ورودی تنها به یکی از خروجی‌ها هدایت می‌شود. این خروجی با ترکیب دودویی دو خط انتخاب A و B انتخاب می‌گردد. می‌توان انتخاب مسیر را از جدول درستی تحقیق کرد. مثلاً اگر خطوط انتخابی $AB=10$ باشند، خروجی D_2 مثل ورودی E خواهد بود در حالی که دیگر خروجی‌ها در 1 نگهداشته خواهند شد. چون عمل دیکدر و در مولتی‌پلکسر با استفاده از یک مدار حاصل می‌شود، یک دیکدر با ورودی فعال‌ساز را دیکدر/ دی مولتی‌پلکسر هم می‌خوانند.

برای تهیه مدار دیکدر بزرگتر می‌توان دیکدرهای فعال‌ساز را به هم متصل کرد. شکل ۷-۴ دو دیکدر 3 به 8 را با ورودی‌های فعال‌ساز به هم پیوسته برای تشکیل یک دیکدر 4 به 16 خط نشان می‌دهد. وقتی $W=0$ است، دیکدر فوقانی فعال می‌شود و دیگری غیرفعال است.

خروجی‌های دیکدر پایینی همگی در 0 خواهند بود و هشت خروجی بالایی مینترم‌های 0000 تا 0111 را تولید می‌کنند. وقتی $W=1$ باشد، وضعیت فعال شدن معکوس می‌گردد. خروجی‌های دیکدر پایینی مینترم‌های 1000 تا 1111 را تولید



شکل ۷-۴: دیکدر 4×16 ساخته شده با دو دیکدر 3×8

می‌نمایند، در حالی که خروجی‌های فوقانی همه 0 هستند. این مثال حسن ورودی‌های فعال‌ساز را در دیکدرها و دیگر قطعات منطقی ترکیبی نشان می‌دهد. به طور کلی ورودی‌های فعال‌ساز ابزارهای مناسبی برای اتصالات درونی دو یا چند قطعه استاندارد برای گسترش آنها با عملکردی مشابه و ورودی‌ها و خروجی‌های بیشتر است.

۲-۱-۷ پیاده‌سازی مدار منطقی ترکیبی با دیکدر

یک دیکدر، 2^n مینترم را برای n متغیر ورودی تهیه می‌کند. چون هر تابع بولی می‌تواند برحسب جمع مینترم‌ها بیان شود، می‌توان از دیکدر برای تولید مینترم استفاده کرده و با گیت OR بیرونی جمع منطقی آنها را تشکیل داد. به این ترتیب هر مدار ترکیبی n ورودی و m خروجی با یک دیکدر n به 2^n و m گیت OR قابل پیاده‌سازی است.

روال پیاده‌سازی یک مدار ترکیبی با دیکدر و گیت‌های OR، لازم می‌دارد که تابع بول مدار برحسب جمع مینترم‌ها بیان شود. سپس یک دیکدر برای تولید همه

مدارهای رمزگذار و رمزگشا ۲۰۵

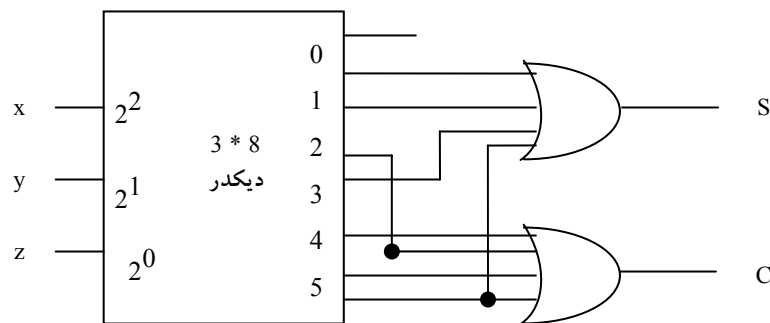
مینترم‌های حاصل از متغیرهای ورودی انتخاب می‌گردد. ورودی‌های هر گیت OR از خروجی‌های دیکدر برحسب لیست مینترم هر تابع انتخاب می‌گردند. این روال با مثالی که مدار جمع‌کننده کامل را به کار می‌برد تشریح می‌گردد.

با توجه به جدول درستی جمع‌کننده کامل که در شکل ۵-۷ ارائه شده است، توابع مدار ترکیبی را به صورت مجموع مینترم‌ها به دست می‌آوریم:

$$S(x,y,z) = \Sigma(1, 2, 4, 7)$$

$$C(x,y,z) = \Sigma(3, 5, 6, 7)$$

چون سه ورودی و جمعاً هشت مینترم وجود دارد، به یک دیکدر ۳ به ۸ خط احتیاج است. پیاده‌سازی در شکل ۵-۷ ملاحظه می‌گردد. دیکدر هشت مینترم را برای y, x و z تولید می‌کند. گیت OR برای خروجی S ، جمع منطقی مینترم‌های ۱، ۲، ۴ و ۷ را تشکیل می‌دهد. گیت OR جمع منطقی مینترم‌های ۳، ۵، ۶ و ۷ را برای تولید خروجی C به کار می‌برد.



شکل ۵-۷: پیاده‌سازی یک جمع‌کننده کامل با دیکدر

یک تابع با لیست طولیلی از مینترم‌ها نیاز به یک گیت OR با ورودی‌های متعدد دارد. تابعی که K مینترم دارد می‌تواند به فرم متمم خود، F' ، با $K - 2^n$ مینترم نشان

داده شود. اگر تعداد میترم‌ها موجود در تابع بزرگتر از $2^n / 2$ باشد، آنگاه می‌توان F' را با تعداد میترم کمتری بیان کرد. در چنین وضعیتی، استفاده از گیت NOR برای تشکیل جمع میترم‌های F' مزیت دارد. خروجی گیت NOR این جمع را متمم کرده و تولید خروجی نرمال F را خواهد کرد. اگر از گیت‌های NAND، مثل شکل ۷-۳ استفاده شود، آنگاه گیت‌های خروجی در عوض OR باید از نوع NAND باشند. دلیل این است که یک مدار گیتی دو طبقه NAND تابع جمع میترم‌ها را پیاده‌سازی می‌کند و معادل با مدار دو طبقه AND-OR است.

۷-۲ مدارات رمز گذار (انکدر)

یک انکدر مداری است که عمل عکس یک دیکدر را انجام می‌دهد. یک انکدر دارای 2^n (یا کمتر) خط ورودی و n خط خروجی است. خطوط خروجی کد دودویی مربوط به مقدار دودویی ورودی را تولید می‌نمایند. مثالی از یک انکدر، انکدر هشت هشتی به دودویی است که جدول درستی آن در شکل ۷-۶ داده شده است. این مدار دارای هشت ورودی (یک ورودی برای هر رقم هشت هشتی) و سه خروجی است که عدد دودویی مربوطه را تولید می‌نماید. فرض بر این است که در هر لحظه‌ای از زمان تنها یک ورودی مقدار ۱ را داشته باشد.

انکدر را می‌توان با گیت‌های OR که ورودی‌هایشان مستقیماً از جدول درستی تهیه می‌شود، پیاده‌سازی کرد. خروجی z هنگامی ۱ است که رقم هشت هشتی ورودی در ۱، ۳، ۵ یا ۷ برابر ۱ باشد. خروجی y به ازاء ارقام هشت هشتی ورودی ۲، ۳، ۶ و ۷، برابر ۱ می‌شود این شرایط را می‌توان با معادلات بولی خروجی بیان کرد.

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

مدارهای رمزگذار و رمزگشا ۲۰۷

این انکدر با سه گیت OR قابل پیاده‌سازی است. انکدري که در جدول شکل ۶-۷ تعريف شد دارای این محدودیت است که در آن در هر لحظه از زمان فقط یک ورودی فعال می‌باشد. اگر دو ورودی به طور هم زمان فعال شوند، خروجی ترکیبات نامفهومی را تولید خواهد کرد. مثلاً اگر D_3 و D_6 همزمان برابر 1 شوند، خروجی انکدر 111 خواهد شد زیرا در این حالت هر سه خروجی برابر 1 است. این خروجی نه 3 و نه 6 را نمایش می‌دهد. برای حل این مشکل، مدارهای انکدر باید اولییتی در ورودی ایجاد کنند تا مطمئن شویم که فقط یک ورودی انکدر شده است. اگر اولویت بالاتر را با اندیس‌های بالاتر ایجاد نماییم، و اگر در یک زمان D_6 و D_3 برابر 1 شوند، خروجی 110 می‌شود زیرا، D_6 اولویت بالاتری نسبت به D_3 دارد.

ورودی‌ها								خروجی‌ها		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	X	Y	Z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

شکل ۶-۷: جدول درستی یک انکدر هشت هشتی به دودویی

مشکل دیگری که در انکدر هشت هشتی به دودویی وجود دارد این است که در آن یک خروجی تمام 0 به ازاء حالتی که همه ورودی‌ها 0 هستند تولید می‌شود. این خروجی برابر با حالتی است که در آن $D_0 = 1$ است. ایراد را می‌توان با تهیه یک خروجی بیشتر حل کرد تا به این ترتیب نشان دهد که حداقل یک ورودی 1 است.

۷-۲-۱ انکدر اولویت

انکدر اولویت مداری است که تابع اولویت دهی را در خود دارد. عملکرد این نوع انکدر چنان است که اگر ده یا چند ورودی به طور همزمان برابر 1 شوند، ورودی با اولویت بالاتر پیش خواهد افتاد. جدول درستی یک انکدر با تقدم چهار ورودی در جدول ۷-۶ ملاحظه می‌گردد. علاوه بر دو خروجی x و y ، مدار دارای سومین خروجی با علامت V است؛ که به معنی بیت معتبر می‌باشد و هرگاه یک یا چند ورودی برابر 1 شوند این خروجی 1 می‌گردد. اگر همه ورودی‌ها 0 باشند بیت معتبر وجود نخواهد داشت و $V=0$ است. وقتی $V=0$ باشد در خروجی دیگر واریسی نمی‌شوند و حالت بی‌اهمیت را خواهند داشت. توجه کنید که گرچه حالات بی‌اهمیت در ستونهای خروجی اهمیت ندارند ولی X ها در ستون ورودی‌ها در کاهش جدول درستی نقش عمده ای دارند. به جای ذکر هر 16 مینترم برای چهار متغیر، جدول درستی از X که می‌تواند 0 یا 1 باشد استفاده می‌کند. مثلاً $X100$ دو مینترم 1100، 0100 را می‌پوشاند.

ورودی‌ها				خروجی‌ها		
D_0	D_1	D_2	D_3	X	Y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

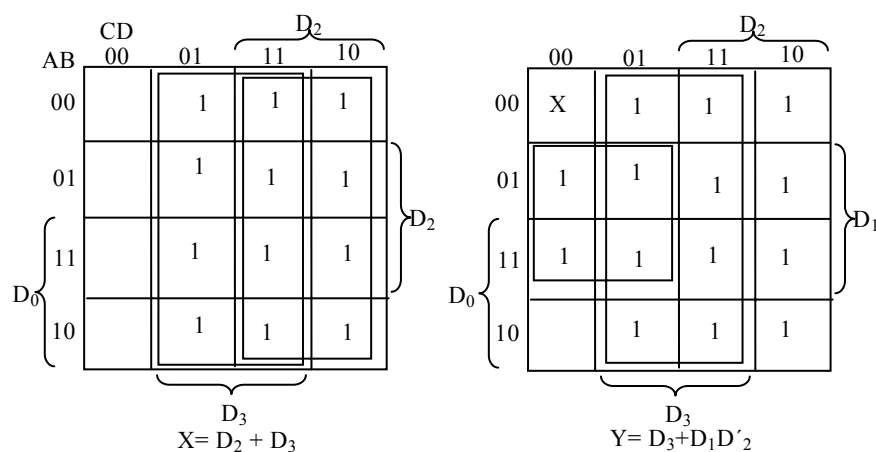
شکل ۷-۷: جدول درستی انکدر اولویت

طبق جدول ارائه شده در شکل ۷-۷، هر عدد با مقدار بالاتر اولویت بالاتری را داراست. در این جدول D_3 اولویت بالاتری دارد، بنابراین بدون توجه به مقادیر دیگر ورودی‌ها، وقتی این ورودی 1 شود، خروجی xy برابر 11 می‌گردد (دودویی 3). D_2 دارای اولویت بعدی است. اگر $D_2=1$ شود خروجی 10 می‌گردد به شرطی که $D_3=0$ باشد، ولی این خروجی مستقل از دو ورودی با اولویت پایین تر است. اگر دیگر

مدارهای رمزگذار و رمزگشا ۲۰۹

ورودی‌های با اولویت تر 0 باشند، خروجی مربوط به D_1 تولید می‌گردد و به همین ترتیب.

نقشه ساده‌سازی خروجی‌های x و y در شکل ۷-۸ نشان داده شده است. مینترم‌های دو تابع از جدول شکل ۷-۷ استنتاج شده‌اند.



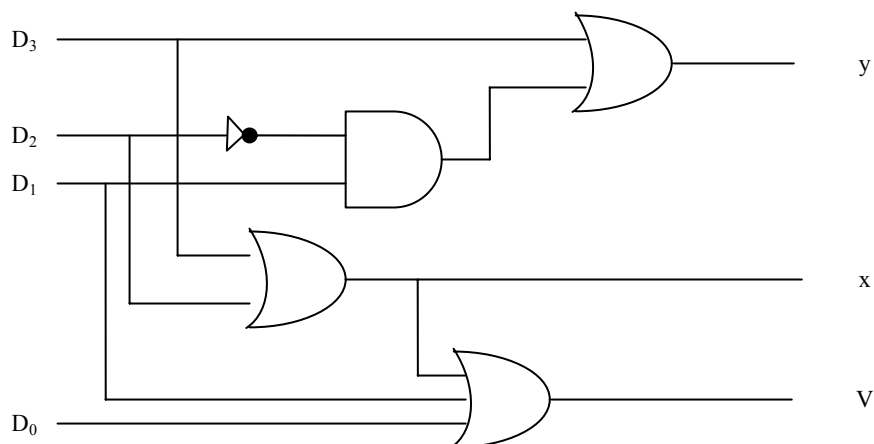
شکل ۷-۸: نقشه یک انکدر اولویت دار

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

$$v = D_0 + D_1 + D_2 + D_3$$

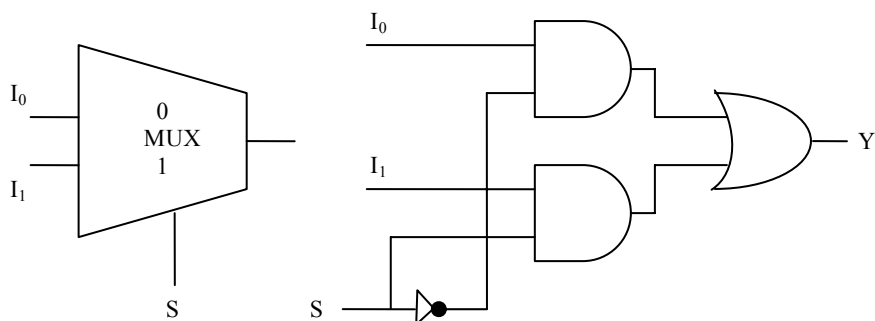
گرچه جدول دارای تنها پنج سطر است، وقتی هر X در هر سطر ابتدا با صفر و سپس با 1 جایگزین شود، آنگاه تمام 16 ترکیب ورودی را خواهیم داشت. مثلاً سطر چهارم در جدول با $XX10$ چهار مینترم 0010، 0110، 1010 و 1110 را نمایش می‌دهد. عبارات بول ساده شده برای انکدر اولویت از نقشه‌ها حاصل می‌شود شرط خروجی V یک تابع OR از همه متغیرهای ورودی است. انکدر اولویت برطبق توابع بولی زیر در شکل ۷-۹ پیاده‌سازی شده است.



شکل ۷-۹: پیاده‌سازی تابع بول با یک مولتی پلکسر

۷-۳ مولتی پلکسر

یک مولتی پلکسر مداری ترکیبی است که اطلاعات دودویی را از تعدادی خط ورودی دریافت کرده و آنها را به یک خط خروجی هدایت می‌نماید. انتخاب یک ورودی خاص به وسیله مجموعه‌ای از خطوط انتخاب انجام می‌شود. معمولاً 2^n خط ورودی و n خط انتخاب وجود دارد و ترکیب بیتی تعیین کننده ورودی انتخاب شده است.



(ب) نمودار بلوکی

(الف) نمودار منطقی

شکل ۷-۱۰: مولتی پلکسر ۲ به ۱

مدارهای رمزگذار و رمزگشا ۲۱۱

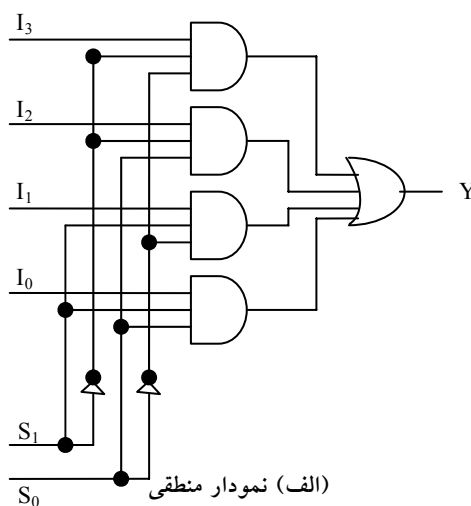
یک مولتی پلکسر ۲ به ۱ یکی از دو منبع ۱ بیت را طبق شکل ۷-۱۰ به یک مقصد مشترک متصل می‌کند. مدار دارای دو خط ورودی داده، یک خروجی و یک خط انتخاب S است. وقتی $S=0$ باشد، گیت AND فوقانی فعال شده و I_0 به خروجی راه می‌یابد. وقتی $S = 1$ باشد، گیت AND تحتانی فعال شده و I_1 به خروجی متصل می‌شود.

مولتی پلکسر مثل یک کلید الکترونیک عمل کرده و یکی از دو منبع را انتخاب می‌نماید. نمودار بلوکی یک مولتی پلکسر گاهی به شکل دوزنقه شکل ۷-۱۰ (ب) نشان داده می‌شود. این مدار چگونگی انتخاب و هدایت منابع متعدد داده را به یک مقصد نشان می‌دهد. مولتی پلکسر اغلب با نمودارهای بلوکی و کلمه MUX نشان داده می‌شود.

یک مولتی پلکسر ۴ به ۱ در شکل ۷-۱۱ دیده می‌شود. هر یک از چهار ورودی I_0 تا I_3 به یک ورودی گیت AND اعمال می‌شود. خطوط انتخاب S_1 و S_0 برای انتخاب گیت AND خاص دیکدر می‌شوند. خروجی گیت‌های AND به یک گیت OR اعمال می‌شوند تا خروجی ۱ خط را ایجاد کنند.

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(ب) جدول تابع



(الف) نمودار منطقی

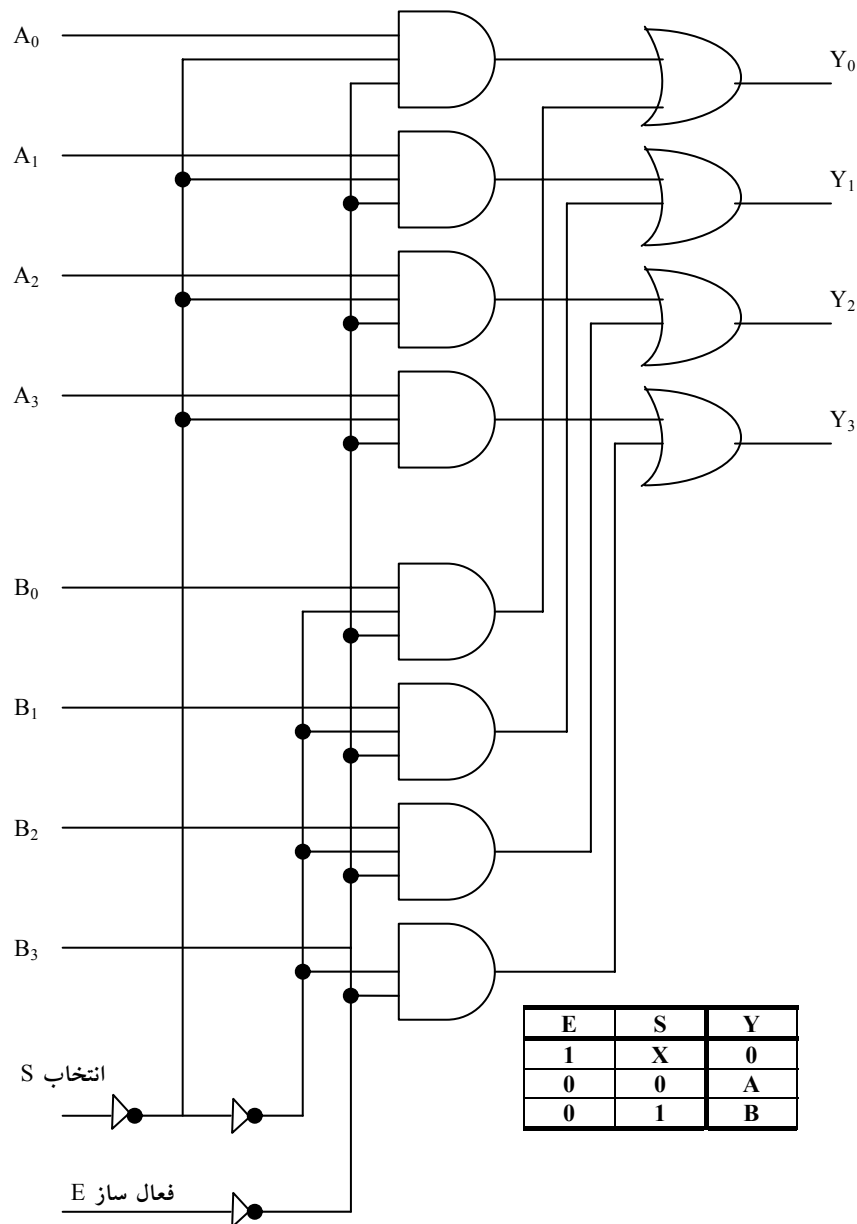
شکل ۷-۱۱: مولتی پلکسر ۴ به ۱

جدول تابع، ورودی را که از مولتی پلکسر عبور کرده نشان می دهد. برای نمایش عمل مدار، حالتی را که $S_1S_0 = 10$ است ملاحظه کنید. گیت مربوط به ورودی I_2 دارای دو ورودی 1 و یک ورودی متصل به I_2 است. سه گیت دیگر هر یک حداقل یک 0 در ورودی خود دارند و بنابراین خروجی شان 0 می شود. خروجی گیت OR، اکنون برابر مقدار I_2 است و به این ترتیب مسیری از ورودی انتخابی به خروجی ایجاد شده است. یک مولتی پلکسر را انتخابگر داده هم می خوانند، زیرا یکی از چند ورودی را انتخاب کرده و اطلاعات دودویی را به خط خروجی هدایت می کند.

وجود گیت های AND و وارون گر ها در مولتی پلکسر، مدار دیکدر را به خاطر می آورد، و به علاوه آنها خطوط ورودی انتخاب را دیکدر می کنند. به طور کلی یک مولتی پلکسر 2^n به 1 از یک دیکدر n به 2^n ، ساخته شده که در آن 2^n خط به 2^n گیت AND، یعنی هر خط به یک گیت وصل شده است. خروجی گیت های AND به تنها گیت OR اعمال می گردند. سائز مولتی پلکسر با 2^n خط ورودی داده و تنها خط خروجی اش مشخص می شود. همچون دیکدر، مولتی پلکسر ها هم ممکن است خط فعال سازی داشته باشند تا عملکرد کل قطعه را کنترل کنند. وقتی که ورودی فعال ساز در وضعیت غیر فعال قرار دارد، خروجی ها غیر فعالند، و وقتی در حالت فعال خود قرار گیرد، مدار به عنوان یک مولتی پلکسر معمولی عمل می کند.

مدار های مولتی پلکسر را می توان برای تهیه مولتی پلکسر چند بیتی با هم ترکیب کرد. به منظور تشریحی بر این مطلب، یک مولتی پلکسر 2 به 1 چهار تایی در شکل ۷-۱۲ نشان داده شده است. مدار دارای چهار مولتی پلکسر است که هر یک قادر است یکی از دو خط را انتخاب نماید. خروجی Y_0 می تواند به یکی از ورودی های A_0 یا B_0 وصل شود و به طور مشابه Y_1 هم می تواند مقدار A_1 یا B_1 را داشته باشند، و به همین ترتیب. خط انتخاب ورودی یکی از خطوط ورودی را در هر یک از چهار مولتی پلکسر انتخاب می کند. خط فعال ساز E با به هنگام کار معمولی فعال شود.

مدارهای رمزگذار و رمزگشا ۲۱۳



شکل ۷-۱۲: مولتی پلکسر ۲ به ۱ چهار تایی

گرچه مدار حاوی چهار مولتی پلکسر 2 به 1 است، ولی ما بیشتر علاقه مندیم به آن به عنوان مداری که یکی از دو مجموعه چهار بیتی خطوط داده را انتخاب می کند، بنگریم. همانطور که در جدول تابع دیده می شود، مدار وقتی که $E = 0$ است فعال می شود. آنگاه اگر $S = 0$ باشد چهار ورودی A مسیری به چهار خروجی دارند. از طرف دیگر اگر $S = 1$ ، چهار ورودی B به خروجی ها اعمال می شوند، وقتی $E = 1$ باشد، مستقل از وضعیت S، همه خروجی ها 0 خواهند بود.

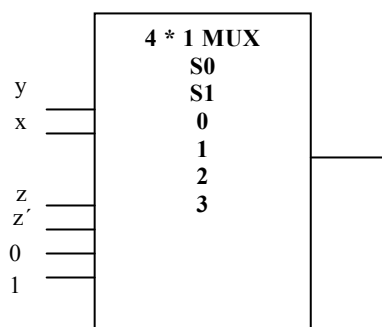
۷-۳-۱ پیاده سازی تابع بول

همانطور که می دانید با افزودن یک گیت OR به خروجی های یک دیکدر می توان از آن برای پیاده سازی توابع بول استفاده کرد. با بررسی نمودار منطقی یک مولتی پلکسر ملاحظه می شود که این مدار در واقع همان دیکدر است که یک گیت OR به آن اضافه شده است. مینترم های یک تابع با خطوط انتخاب مولتی پلکسر تولید می شوند. هر مینترم به وسیله ورودی های داده انتخاب می شود. این مطلب روشی را برای پیاده سازی هر تابع n متغیر به وسیله یک مولتی پلکسر که دارای 2^n ورودی داده و n خط ورودی انتخاب است، فراهم می سازد.

اکنون روش کارتری را برای پیاده سازی یک تابع بول n متغیر با مولتی پلکسری که $n-1$ ورودی انتخاب دارد، معرفی می نماییم. ابتدا $n-1$ متغیر به ورودی های انتخاب مولتی پلکسر وصل می شود. تنها متغیر باقی مانده تابع برای ورودی های داده مورد استفاده قرار می گیرد. اگر متغیر باقی مانده را z بنامیم هر ورودی داده مولتی پلکسر برابر z ، z' و 1 یا 0 خواهد بود. برای نمایش این رویه، تابع سه متغیره زیر را ملاحظه کنید:

$$F(x,y,z) = \Sigma(1,2,6,7)$$

تابع را می توان مطابق شکل ۷-۱۳ با یک مولتی پلکسر 4 به 1 پیاده سازی کرد. دو متغیر x و y به خطوط انتخاب وصل می شود، به این ترتیب که x به ورودی s_1 و y به ورودی s_0 متصل می گردد.



(ب) پیاده سازی مولتی پلکسر

x	y	z	F	
0	0	0	0	F=z
0	0	1	1	
0	1	0	1	F=z'
0	1	1	0	
1	0	0	0	F=0
1	0	1	0	
1	1		1	F=1
1	1	1	1	

(الف) جدول درستی

شکل ۷-۱۳: پیاده سازی یک تابع بول با یک مولتی پلکسر

مقادیر خطوط ورودی از جدول درستی تابع معین می شود. وقتی $xy=00$ باشد خروجی F برابر z است زیرا وقتی $z=0$ است F هم برابر 0 می باشد و وقتی $z=1$ شود F نیز برابر 1 می گردد. این وضع لازم می دارد تا متغیر z به ورودی داده 0 متصل شود. عملکرد مولتی پلکسر به نحوی است که وقتی $xy=00$ گردد، خط داده شماره 0 به خروجی وصل شده F را برابر z می نماید. به طریقی مشابه می توان نشان داد که خطوط ورودی 1، 2 و 3 وقتی که $xy=01$ ، 10 و 11 است بترتیب به F وصل می شوند. این مثال خاص هر چهار حالت ممکن را برای ورودی های داده تهیه می کند.

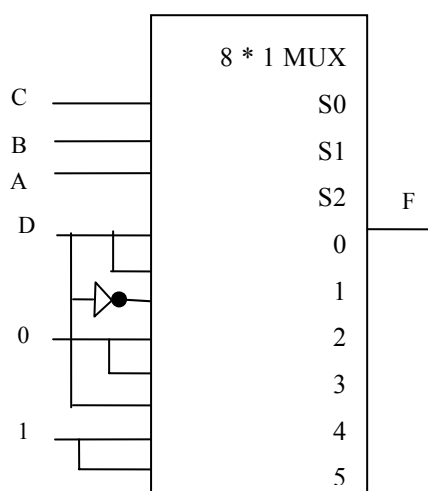
روال کلی برای پیاده سازی هر تابع بول n متغیره با یک مولتی پلکسر که $n-1$ خط ورودی انتخاب و 2^{n-1} ورودی داده دارد از مثال قبل نتیجه می گردد. ابتدا تابع بول را در جدول درستی لیست می کنیم. اولین $n-1$ متغیر در جدول ورودی های انتخاب مولتی پلکسر وصل می شوند. برای هر ترکیبی از متغیرهای انتخاب، خروجی را به عنوان تابعی از آخرین متغیر ارزیابی می کنیم. این تابع می تواند 0، 1، متغیر و یا متمم متغیر باشد. آنگاه این مقادیر به ورودی های داده به نحوی صحیح اعمال می شوند. به عنوان دومین مثال، تابع بول زیر را ملاحظه نمایید:

$$F(A,B,C,D) = \Sigma(1,3,4,11,12,13,14,15)$$

این تابع با مولتی پلکسری که سه ورودی انتخاب دارد، مطابق شکل ۷-۱۴ پیاده سازی می شود. توجه کنید که اولین متغیر A باید به ورودی انتخاب S_2 و دو خط B و C به ترتیب به S_1 و S_0 وصل شوند. مقادیر ورودی داده از جدول درستی لیست شده در شکل معین می گردند. شماره خط داده مربوطه از ترکیب دودویی ABC حاصل می شود. مثلاً وقتی $ABC = 101$ باشد، جدول نشان می دهد که $F=D$ است، بنابراین متغیر D به ورودی داده 5 وصل می شود. ثابت های دودویی 0 و 1 مربوط به دو مقدار سیگنال ثابت است. وقتی از مدارهای مجتمع استفاده کنیم، منطق 0 مربوط به سیگنال زمین و منطق 1 معادل با سیگنال تغذیه است که معمولاً 5 ولت می باشد.

A	B	C	D	F	
0	0	0	0	0	$F=D$
0	0	0	1	1	
0	0	1	0	0	$F=D$
0	0	1	1	1	
0	1	0	0	1	$F=D'$
0	1	0	1	0	
0	1	1	0	0	$F=0$
0	1	1	1	0	
1	0	0	0	0	$F=0$
1	0	0	1	0	
1	0	1	0	0	$F=D$
1	0	1	1	1	
1	1	0	0	1	$F=1$
1	1	0	1	1	
1	1	1	0	1	$F=1$
1	1	1	1	1	

(الف) جدول درستی



(ب) پیاده سازی مولتی پلکسر

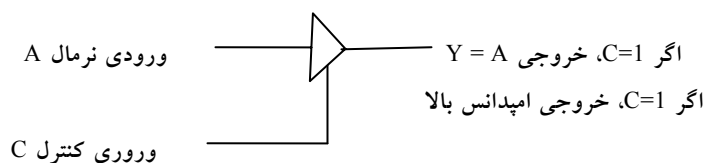
شکل ۷-۱۴: پیاده سازی یک تابع 4 ورودی با یک مولتی پلکسر

۷-۳-۲ گیت های سه حالت

یک مولتی پلکسر را می توان با گیت های سه حالت ساخت. یک گیت سه حالت مداری دیجیتالی است که سه حالت را از خود به نمایش می گذارد. دو حالت، همچون گیت های معمولی همان منطق 1 و 0 است. حالت سوم، حالت امپدانس بالا است.

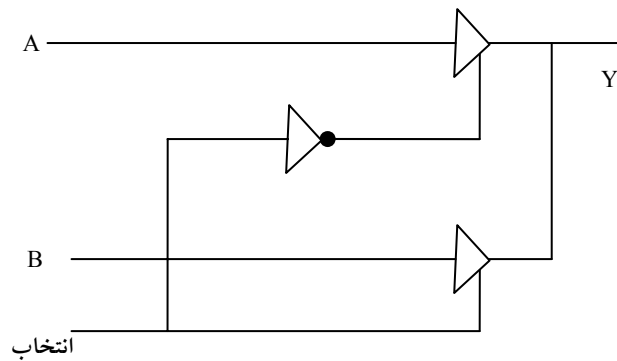
حالت امپدانس بالا مثل مدار باز عمل می‌کند و به این معنی است که خروجی از درون قطع بوده و مدار دارای مفهوم منطقی با ارزشی نیست. گیت‌های سه حالته ممکن است به عنوان گیت‌های AND و NAND نیز عمل کنند. با این وجود اغلب به عنوان بافر مورد استفاده قرار می‌گیرند.

نمودار گرافیکی یک گیت بافر سه حالته در شکل ۷-۱۵ دیده می‌شود این قطعه با ورودی کنترلی که وارد ضلع پایینی آن می‌شود از نوع معمولی‌اش تفکیک می‌شود. یک بافر معمولی دارای یک ورودی، یک خروجی و یک خط کنترل می‌باشد که وضع خروجی را مشخص می‌نماید. وقتی که ورودی کنترل برابر 1 است، خروجی فعال شده و گیت مانند یک بافر معمولی عمل می‌کند و در این حالت خروجی برابر ورودی اصلی است. وقتی که ورودی کنترل 0 شود، خروجی غیر فعال شده و گیت بدون توجه به مقدار اصلی به حالت امپدانس بالا می‌رود. حالت امپدانس بالای یک گیت سه حالته ویژگی خاصی را فراهم می‌کند که در دیگر گیت‌ها وجود ندارد. به علت این ویژگی، تعداد زیادی از خروجی‌های سه حالته می‌توانند به هم وصل و بدون تاثیر بر روی بار شدن، یک خط مشترکی را تشکیل دهند.



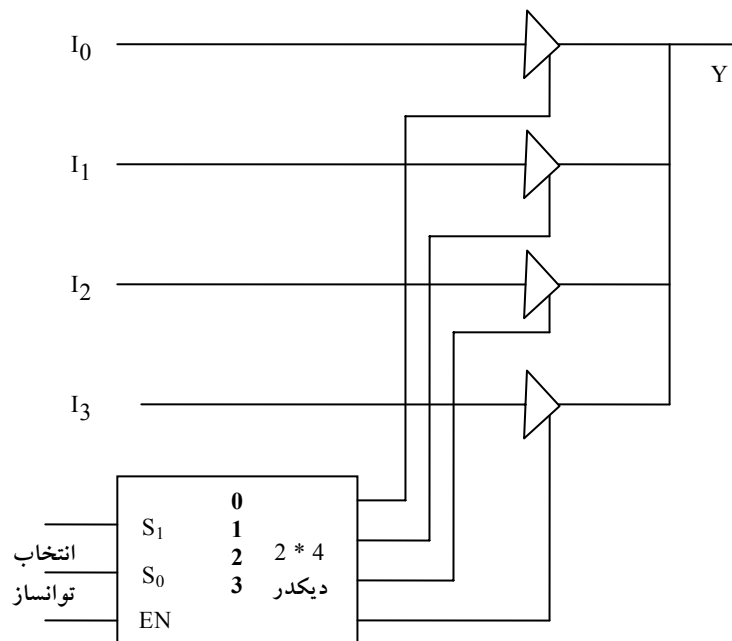
شکل ۷-۱۵: سمبل گرافیکی برای یک بافر سه حالته

ساخت مولتی‌پلکسرها با بافرهای سه حالته در شکل ۷-۱۶ دیده می‌شود. این شکل یک مولتی‌پلکسر 2 به 1 را با دو بافر سه حالته و یک وارون‌گر نشان می‌دهد. دو خروجی به هم وصل شده‌اند تا یک خروجی مشترک را به وجود آورند.



شکل ۷-۱۶: مدار یک مولتی پلکسر ۲ به ۱ با بافر سه حالت

کنترل به بافر مشخص می کنند که کدام یک از چهار ورودی نرمال I_0 تا I_3 به خط باید اشاره کرد که این گونه اتصالات را با گیت هایی که خروجی سه حالت ندارند نمی توان اجرا کرد. وقتی ورودی انتخاب ۰ است، بافر فوقانی به وسیله ورودی کنترلش



شکل ۷-۱۷: مولتی پلکسرهای ۴ به ۱ با گیت های سه حالت

فعال می‌گردد و در این حال بافر پایینی غیر فعال است.

ساختار یک مولتی‌پلکسر 4 به 1 در شکل ۷-۱۷ ملاحظه می‌شود. خروجی‌های چهار بافر سه حالت به هم متصل شده‌اند تا یک خروجی مشترک را بسازند. ورودی‌های خروجی متصل خواهند شد. در هر لحظه از زمان تنها یکی از بافرها در حالت فعال قرار خواهد داشت. بافرهای متصل باید طوری وصل شوند که تنها یکی از بافرهای سه حالت با خروجی ارتباط داشته باشد، ضمن اینکه همه دیگر بافرها در حالت امپدانس بالا قرار خواهند گرفت. برای اطمینان از اینکه تنها یک ورودی در هر لحظه فعال است، از دیکدری طبق نمودار استفاده می‌کنیم. وقتی که ورودی فعال‌ساز دیکدر 0 است، هر چهار خروجی آن 0 خواهد بود و خط گذرگاه در حالت امپدانس بالاست زیرا هر چهار بافر غیر فعال‌اند. وقتی که ورودی فعال‌ساز فعال گردد، یکی از بافرها بسته به مقدار دودویی در ورودی‌های انتخاب دیکدر، فعال خواهد شد. با بررسی دقیق در می‌یابیم که این مدار راهی دیگر در ساخت یک مولتی‌پلکسر 4 به 1 است.

۷-۴ زبان HDL برای مدارهای ترکیبی

در این بخش، روش دیگری را برای توصیف مدارهای ترکیبی HDL نشان خواهیم داد. همانطور که قبلاً ذکر شد، ماژول، یک بلوک ساختاری پایه در Verilog HDL است. ماژول می‌تواند در هر یک از تکنیک‌های مدل‌سازی زیر توصیف گردد.

مدل‌سازی سطح گیت با ذکر گیت‌های اصلی (Primitive) و ماژول‌های تعریف شده به وسیله کاربر.

مدل‌سازی روند داده با به‌کارگیری عبارات تخصیص مداوم (پیوسته) که با کلمه کلیدی assign انجام می‌شوند.

مدل‌سازی رفتاری با استفاده از عبارات تخصیص اجرایی (رویه‌ای) که با کلمه کلیدی always صورت می‌گیرد.

مدل سازی سطح گیت، مدار را با تعیین گیت ها و اینکه چگونه به هم وصل شده اند توصیف می نماید. مدل سازی روند داده اغلب برای توصیف مدارهای ترکیبی به کار می رود. مدل سازی رفتاری برای سیستم های دیجیتال در سطح بالاتر مورد استفاده است. مدل دیگری بجز روش های فوق وجود دارد که به آن مدل سازی سطح سوئیچ گویند. این نوع مدل سازی قابلیت طراحی را در سطح ترانزیستور MOS فراهم می سازد.

۷-۴-۱ مدل سازی سطح گیت

در این نوع نمایش، یک مدار با گیت های منطقی و اتصالات بین آنها نشان داده می شود. این مدل توصیف متنی برای نمودار مداری را فراهم می کند. Verilog قادر است تا ۱۲ گیت را به عنوان گیت های اصلی پیش تعریف شده تشخیص دهد. چهار گیت از آنها از نوع سه حالته است. هشت نوع دیگر آنهایی هستند که در ذیل آمده اند. این گیت ها با کلمات کلید حروف کوچک زیر معرفی می شوند که عبارتند از: and, .buf, .not, .xnor, .xor, .or, .nand

and	۰	۱	x	z
0	۰	0	0	0
1	۰	1	x	x
x	۰	x	x	x
z	۰	x	x	x

or	0	1	x	z
0	0	1	x	x
1	1	1	1	1
x	x	1	x	x
z	x	1	x	x

xor	0	1	x	z
0	0	1	X	X
1	1	0	X	X
X	X	X	X	X
Z	X	X	X	X

not	خروجی ورودی
0	1
1	0
X	X
Z	X

شکل ۷-۱۸: جدول درستی برای گیت های اصلی پیش تعریف شده

وقتی که گیت‌ها شبیه‌سازی شوند، سیستم به هر گیت یک مجموعه چهار مقداری را تخصیص می‌دهد. علاوه بر مقدار منطقی 0 و 1، دو مقدار نامشخص و امیدانس بالا هم لحاظ شده‌اند. مقدار نامشخص با x و امیدانس بالا با z مشخص شده است. مقدار نامشخص در حین شبیه‌سازی برای حالتی است که یک ورودی یا خروجی نامعلوم باشد و به عنوان مثال به آن مقدار 1 یا 0 تخصیص نیافته باشد. حالت امیدانس بالا در خروجی گیت‌های سه حالتی هنگامی رخ می‌دهد که یک سیستم ناخودآگاه باز رها گردد. جدول درستی برای and ، xor ، or و not در جدول شکل ۷-۱۸ دیده می‌شود.

جدول درستی چهار گیت دیگر مشابه است با این تفاوت که خروجی‌ها متمم شده‌اند. توجه کنید که برای گیت and ، خروجی فقط هنگامی 1 است که هر دو ورودی 1 باشند و هنگامی 0 است که هر یک از دو ورودی 0 باشد. در غیر این صورت اگر یک ورودی x یا z باشد خروجی x است. وقتی هر دو ورودی گیت or برابر 0 باشد خروجی آن 0 است و اگر هر یک از ورودی‌ها 1 باشد خروجی هم 1 است، در غیر این صورت x است.

وقتی که یک گیت اصلی در یک ماژول لحاظ شود گوییم در ماژول ذکر شده است. به طور کلی ذکر قطعات عباراتی هستند که به قطعات سطح پایین‌تری در طراحی ارجاع می‌دهند، و کپی‌هایی اساسی (یا نمونه) از آن قطعات در ماژول سطح بالاتر ایجاد می‌نمایند. بنابراین ماژولی که یک گیت را در توصیف خود به کار می‌برد گیت را ذکر کرده است.

اکنون دو مثال از مدل‌سازی سطح گیت را نمایش می‌دهیم. هر دو مثال از عرض چند بیتی که بردار نامیده می‌شوند، استفاده می‌کنند. یک بردار در داخل یک جفت کروشه مشخص می‌شود که از دو عدد و دو نقطه در بین آنها تشکیل شده است. کد زیر دو بردار را نشان می‌دهد:

```
output 3:0 [ ; ] D
wire [ 0:7 ; SUM ]
```

عبارت اول یک بردار خروجی D را با چهار بیت 0 تا 3 نشان می‌دهد. دومین عبارت یک بردار SUM سیمی (wire) با هشت بیت که از 0 تا 7 شماره گذاری شده‌اند را نشان می‌دهد. اولین عدد لیست شده با ارزش‌ترین بیت بردار است. بیت‌های خاص در داخل گروه ذکر می‌گردند، بنابراین [2] D، بیت 2 از D را نشان می‌دهد. همچنین ممکن است بخش‌هایی از یک بردار را ذکر کند. مثلاً [2 : 0] SUM سه بیت کم ارزش‌تر بردار SUM را بیان می‌کند.

۷-۴-۲ گیت‌های سه حالت

هر گیت سه حالت دارای یک ورودی کنترل است که می‌تواند آن را به حالت امپدانس بالا ببرد. این حالت در HDL با سمبل z مشخص می‌شود. چهار نوع گیت سه حالت طبق شکل ۷-۱۹ وجود دارد. گیت bufif1 مانند یک بافر معمولی عمل می‌کند به شرطی که $control=1$ باشد. وقتی $control=0$ است، خروجی به حالت امپدانس بالای z می‌رود. گیت bufif0 رفتاری مشابه دارد با این تفاوت که امپدانس بالا در $control=1$ رخ می‌دهد. دو گیت not به همین ترتیب کار می‌کنند با این تفاوت که وقتی در امپدانس بالا نیستند، خروجی آنها متمم ورودی است. گیت‌ها با عبارت زیر ذکر می‌شوند:

gate name (output , input , control) ;

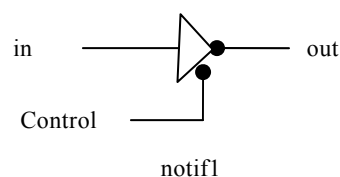
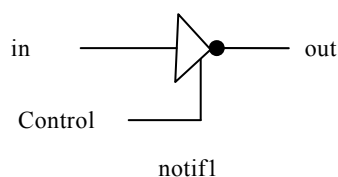
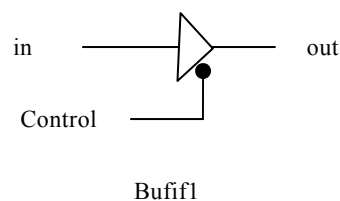
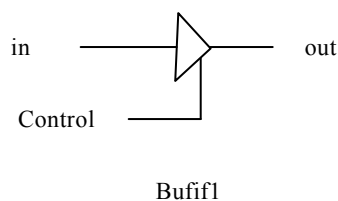
gate name می‌تواند یکی از چهار گیت سه حالت انتخاب گردد. خروجی می‌تواند 0، 1 یا z باشد دو مثال از ذکر گیت در زیر آمده است:

OUT, A , control (bufif1);

notifo (Y,B,enable) ;

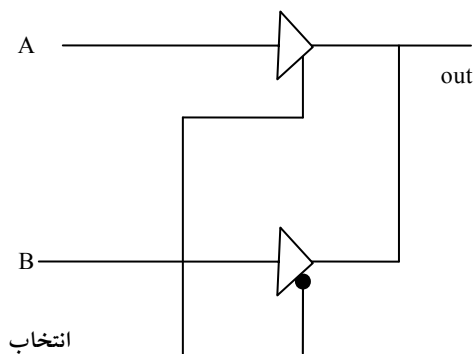
در مثال اولی، ورودی A هنگامی که $control=1$ است به خروجی منتقل می‌گردد وقتی $control=0$ باشد خروجی out به z می‌رود. در مثال دوم وقتی $enable=1$ است $Y=z$ خواهد شد و خروجی $Y=B'$ در $enable=0$ رخ می‌دهد.

۲۲۳ مدارهای رمزگذار و رمزگشا



شکل ۷-۱۹: گیت سه حالت

خروجی گیت‌های سه حالت می‌توانند برای تشکیل یک خروجی مشترک به هم وصل شوند. برای شناخت چنین اتصالی HDL از کلمه کلید tri (برای state tri) استفاده می‌کند تا بیان نماید که خروجی دارای قابلیت راه‌اندازی چندگانه است.



شکل ۷-۲۰: مولتی‌پلکسر ۲ به ۱ با بافرهای سه حالت

مثلاً مولتی‌پلکسر ۲ به ۱ را در شکل ۷-۲۰ با گیت‌های سه حالت در نظر بگیرید. توصیف HDL برای خروجی از داده نوع tri استفاده کند.

```
module muxtri (A , B , select , OUT);
input A , B , select;
```

```

output OUT ;
tri OUT;
bufif1 (OUT , A , select);
bufif0 (OUT , B , select);
endmodule

```

دو بافر سه حالتی دارای خروجی مشترک‌اند. برای اینکه نشان دهیم آنها اتصال مشترک دارند، باید خروجی OUT را با کلمه کلیدی tri همراه کنیم. کلمات کلیدی wire و tri مثال‌هایی از داده نوع net می‌باشند. Net ها اتصال بین دوآلمان یا عنصر را نشان می‌دهند. خروجی آنها با خروجی وسیله ای که نشان می‌دهند مرتباً راه‌اندازی می‌شود. کلمه net (شبکه) یک کلمه کلیدی نیست و کلاسی از انواع داده مانند wire، wor، wand، tri، supply1 و supply0 را نشان می‌دهد. اعلان wire مکرر به کار برده می‌شود. مدل‌های wor پیاده‌سازی سخت‌افزاری آرایش wired-OR را نشان می‌دهند. مدل wand هم برای آرایش wired-AND در نظر گرفته شده‌اند. شبکه‌های supply1 و supply0 منبع تغذیه و زمین هستند. از آنها در توصیف سطح-سوئیچ استفاده می‌شود.

۷-۴-۳ مدل‌سازی روند داده

مدل‌سازی روند داده از تعدادی عملگر روی عملوندها استفاده می‌کند تا خروجی مورد نظر را تولید کند. Verilog HDL حدود 30 عملگر در اختیار می‌گذارد. جدول شکل ۷-۲۱ بخشی از این عملگرها، سمبل‌های آنها و عملی که اجرا می‌کنند را نشان می‌دهند.

لازم است تا بین اعمال حسابی و منطقی تفکیک به عمل آید، بنابراین برای هر یک، از سمبل جداگانه‌ای استفاده شده است. سمبل (+) برای جمع حسابی به کار رفته و AND منطقی از سمبل & استفاده می‌کند. برای OR، NOT و XOR سمبل‌های خاصی وجود دارد. سمبل برابری از دو علامت مساوی و بدون فضا در بین آنها استفاده می‌کند تا با علامت برابری به کار رفته با عبارت تخصیص اشتباه نشود. عملگر ادغام

۲۲۵ مدارهای رمزگذار و رمزگشا

مکانیزمی است که برای ضمیمه کردن چند عملوند استفاده می‌شود. مثلاً دو عملوند دو بیتی را می‌توان برای ایجاد یک عملوند چهار بیتی در هم ادغام کرد.

symbol	operation
+	Binary addition
-	Binary subtraction
&	Bit – wise AND
	Bit – wise OR
^	Bit – wise XOR
~	Bit – wise NOT
=	equality
>	Greater than
<	Less than
{ }	concatenation
? :	conditional

شکل ۷-۲۱: عملگرهای Verilog HDL

```
// dataflow description of a 2- to 4 – line decoder
```

```
// see fig. 4 – 19
```

```
Module decoder – df (A,B,E,D);
```

```
Input A,B,E;
```

```
Output [0:3] D;
```

```
Assign D [0] = ~(~A & ~B & ~E),
```

```
D [1] = ~(~A & B & ~E),
```

```
D [2] = ~(A & ~B & ~E),
```

```
D [3] = ~(A & B & ~E);
```

```
End module
```

مثال ۱: مدل‌سازی روند داده از تخصیص‌های مداوم و کلمه کلیدی assign استفاده

می‌کند. یک تخصیص مداوم عبارتی است که یک مقدار را به یک net تخصیص می‌دهد. net نوع داده در HDL برای نمایش اتصالات بین عناصر مدار به کار می‌رود. یک net خروجی یک گیت را که با عبارت output یا wire بیان شده، تعریف می‌کند. مقدار تخصیص داده شده به net با یک عبارت که عملگر و عملوند را استفاده می‌کند

بیان می‌گردد. به عنوان مثال با فرض اعلان متغیرها، یک مولتی‌پلکسر 2 به 1 با ورودی‌های داده A و B، ورودی انتخاب S و خروجی Y با عبارت مداوم زیر تعریف می‌گردد:

$$\text{assign } Y = (A \& S) | (B \& \sim S);$$

عبارت با کلمه کلیدی assign و به دنبال آن خروجی مورد نظر Y و یک علامت مساوی شروع می‌شود. به دنبال علامت مساوی، یک عبارت بول آورده شده است. در سخت‌افزار، این عبارت معادل با اتصال خروجی گیت OR (|) به سیم Y است. مثال‌های بعدی مدل‌های روند داده دو مثال سطح گیت قبلی را نشان می‌دهند. توصیف روند داده یک دیکدر 2 به 4 در مثال ۱ HDL نشان داده شده است. مدار با توجه به عبارات بولی با چهار عبارت تخصیص مداوم تعریف شده است که هر یک متعلق به یک خروجی است. توصیف روند داده یک جمع‌کننده 4 بیت در مثال زیر HDL آورده شده است.

مثال ۲: در این مثال توصیف روند داده یک جمع‌کننده 4 بیت ارائه شده است. منطق جمع با یک عبارت و استفاده از عملگرهای جمع و ادغام بیان شده است. سمبل جمع (+) بیانگر جمع دودویی 4 بیت A با چهار بیت B و یک رقم نقلی Cin است. خروجی مورد نظر ادغام نقلی خروجی Cout و چهار بیت SUM است.

```
// dataflow description of 4-bit adder
Module binary_adder (A,B,cin,SUM,COUT);
Input [0:3] A,B;
Input cin;
Output [0:3] SUM;
Output COUT;
Assign { COUT,SUM } = A+B+cin;
End module
```

ادغام عملوند ها در پرانتز بیان شده و یک ویرگول آنها را از هم جدا می‌سازد. بنابراین پنج بیت { SUM و Cout }، نتیجه عمل جمع را به نمایش می‌گذارد.

مثال ۳: مدل‌سازی روند داده امکان توصیف مدارهای ترکیبی را با تابع به جای ساختار گیتی اش فراهم می‌سازد برای ملاحظه چگونگی انجام طراحی دیجیتال با روند داده، مقایسه‌گر مقدار چهار بیتی توصیف شده در این مثال را در نظر بگیرید. مدل دو گروه ورودی چهار بیت A و B و سه خروجی را مشخص می‌کند. اگر A کوچکتر از B باشد خروجی (ALTB) در منطق 1 و اگر A بزرگتر از B باشد خروجی (AGTB) در منطق 1 قرار می‌گیرد. ضمن اینکه اگر A برابر با B است خروجی (AEQB) وجود داشته باشد. توجه کنید که تساوی با دو علامت مساوی تعریف می‌شود.

```
// dataflow description of 4-bit comparator
Module magcomp (A,B,ALSB,AGTB,AEQB);
Input [0:3] A,B;
Output ALSB,AGTB,AEQB ;
Assign    ALSB = (A<B) ,
AGTB = (A>B),
AEQB = (A==B);
End module
```

مثال بعدی از عملگر شرطی (?:) استفاده می‌نماید. این عملگر سه عملوند را اختیار می‌کند.

مثال ۴: مولتی‌پلکسر 2 به 1 خط را با عملگر شرطی توصیف می‌نماید.

Condition ? true – expression: false – expression

شرط همواره ارزیابی می‌شود. اگر نتیجه منطق 1 بود true – expression ارزیابی می‌گردد. اگر نتیجه منطق 0 بود، false – expression ارزیابی خواهد شد. این معادل با یک شرط if- else است. تخصیص مداوم

assign OUT = select ? A: B ;

شرط زیر را بیان می‌کند.

OUT = A if select = 1 , else OUT = B if select = 0

```
// dataflow description of 2-to-1-line multiplexer
Module mu2x1-df (A,B,select,OUT);
Input A,B,select;
Output OUT ;
Assign OUT = select? A : B ;
End module
```

۷-۴-۴ مدل سازی رفتاری

مدل سازی رفتاری مدارهای دیجیتال را در سطح الگوریتمی و عملیاتی نمایش می دهد. این مدل اغلب در توصیف مدارهای ترتیبی به کار برده می شود، ولی قابل استفاده در توصیف مدارهای ترکیبی هم می باشد. در اینجا دو مثال از مدارهای ترکیبی ساده برای معرفی موضوع ارائه می گردد.

توصیف های رفتاری از کلمه کلیدی `always` و بدنبال آن لیستی از عبارات تخصیص اجرایی (رویه ای) استفاده می کنند. خروجی مورد نظر این عبارات باید نوع داده `reg` باشد. بر خلاف داده نوع `wire` که خروجی مورد نظر یک تخصیص ممکن است مرتباً به روز شود، داده نوع `reg` مقدارش را تا تخصیص مقدار جدید حفظ می کند.

مثال ۵: مثال ۵ HDL توصیف رفتاری مولتی پلکسر ۲ به ۱ را نشان می دهد (آن را با مثال ۴ HDL مقایسه کنید). چون `OUT` یک خروجی مورد نظر یا مقصد است، باید علاوه بر اعلان `output`، به صورت داده `reg` هم اعلام گردد. عبارات تخصیص اجرایی در داخل بلوک `always` هر زمان که تغییری در هر متغیر بعد از علامت `@` رخ دهد دوباره اجرا می گردد. توجه کنید که در انتهای عبارت `always` علامت `(;)` وجود ندارد. در این حال، آنها عبارتند از `A`، `B` و `select`. توجه کنید که کلمه کلیدی `or` در بین متغیرها به جای عملگر منطقی `OR`، `"|"`، استفاده شده است. عبارت شرطی `if-else` تصمیمی را که مبتنی بر مقدار ورودی `select` است فراهم می کند. عبارت `if` را می توان بدون ذکر سمبل کمیت نیز نوشت:

```
if (select) OUT = A ;
```

و به این معنی است که select برای منطق 1 چک می‌شود.

```
// Behavioral description of 2-to-1-line multiplexer
Module mu2x1-bh (A,B,select,OUT);

Input A,B,select;
Output OUT;
Reg OUT;

Always @(select or A or B)
If (select == 1) OUT = A;
Else OUT = B;

End module
```

مثال ۶: یک مولتی‌پلکسر 4 به 1 را توصیف نمایید.

ورودی select به صورت یک بردار 2 بیت توصیف شده و خروجی y هم با داده reg اعلان شده است. عبارت always دارای یک بلوک ترتیبی در بین کلمات کلیدی case و endcase است. این بلوک هر وقت که هر ورودی بعد از علامت @ تغییر کند، اجرا خواهد شد. عبارت case یک انشعاب شرطی چند مسیری است. عبارت case (select) با مقایر لیست عباراتی که به دنبالش می‌آیند ارزیابی و مقایسه می‌شود. اولین مقداری که با شرط صحیح تطبیق کند اجرا می‌گردد. چون select یک عدد دو بیتی است، می‌تواند 00، 01، 10 و یا 11 باشد. اعداد دودویی با حرف b و قبل از آن یک علامت پریم مشخص می‌گردند. سائز عدد ابتدا نوشته شده و سپس مقدار آن ذکر می‌شود. بنابراین 2'b01 به معنی عدد دودویی دو رقمی است که مقدارش 01 است. اعداد را می‌توان به دهدهی، هشت هشتی یا شانزده شانزدهی و به ترتیب با d'، o' و h' مشخص کرد. اگر مبنای عدد مشخص نباشد، پیش فرض دهدهی خواهد بود. اگر سائز عدد نامشخص باشد، سیستم سائز را 32 بیت فرض خواهد کرد.

```
// Behavioral description of 4-to-1-line multiplexer
// Describes the function table of fig. 4-25 (b)
Module mu4x1-bh (i0,i1,i2,i3,select,y);

Input i0,i1,i2,i3;
```

```

Input [1:0] select;
Output y ;
Reg y ;
Always @ (i0or i1or i2or i3or select )
Case ( select)
2'b 00 : y = i0 ;
2'b 01 : y = i1 ;
2'b 10 : y = i2 ;
2'b 11 : y = i3 ;
End case
End module

```

در اینجا مثال‌های ساده‌ای را از توصیف رفتاری مدارهای ترکیبی نشان دادیم. مدل‌سازی رفتاری و عبارات تخصیص اجرایی دانش مدارهای ترتیبی را لازم دارد که در فصل بعد مورد بحث و بررسی قرار خواهد گرفت.

۷-۴-۵ نوشتن یک برنامه تست ساده

یک برنامه تست (T.B) برنامه‌ای HDL است که برای اعمال محرک به طرح HDL برای تست و مشاهده پاسخ شبیه ساز آن به کار می‌رود. T.B می‌تواند بسیار پیچیده و طولانی باشند تا حدی که ساخت آن از طرح مورد تست بیشتر طول بکشد. با این وجود، برنامه‌ای که در اینجا بررسی می‌شود نسبتاً ساده است زیرا ما فقط مایل به تست مدارهای ترکیبی هستیم. مثال‌ها برای نمایش توصیف‌های نمونه ماژول‌های محرک HDL ارائه شده‌اند.

علاوه بر عبارات always، برنامه تست از عبارت initial برای تهیه محرک به مدار تحت تست استفاده می‌کند. عبارت always حلقه‌ای را به صورت تکراری اجرا می‌کند. عبارت initial فقط یک بار با شروع از $t=0$ شبیه‌سازی را انجام داده و ممکن است هر عملی را با تاخیری که مضربی از واحدهای زمانی است و با سمبل # مشخص شده ادامه دهد. مثلاً بلوک initial زیر را ملاحظه نمایید.

۲۳۱ مدارهای رمزگذار و رمزگشا

```
Initial
begin
A = 0 ; B = 0 ;
#10 A = 1 ;
#20 A = 0 ; B = 1 ;
end
```

بلوک، بین begin و end محصور شده است. در $t=0$ ، A و B در 0 قرار گرفته اند. 10 واحد زمان بعد، A به 1 تغییر یافته است، پس از 20 واحد زمانی، (در $t = 30$) به 0 و B به 1 تغییر پیدا می کند. ورودی ها به جدول درستی 3 بیتی می توانند با بلوک initial تولید شوند.

```
Initial
begin
D = 3 ' b 0 0 0 ;
repeat (7)
#10 D = D + 3 ' b 0 0 0 ;
end
```

برای 3 بیت D در $t = 0$ با مقدار دهی اولیه می شود. کلمه کلیدی repeat یک عبارت حلقه ای را تداعی می کند: یعنی عدد 1 هفت بار به D اضافه شده و این کار هر 10 واحد زمانی یک بار تکرار شده است. نتیجه یک رشته اعدادی از 000 تا 111 خواهد بود.

یک ماژول محرک در یک برنامه HDL ساختار زیر را دارد.

- نام module تست
- اعلان شناسه های reg و wire محلی
- ذکر ماژول طراحی مورد تست
- ایجاد عبارات محرک initial و always
- نمایش پاسخ خروجی
- endmodule

یک ماژول تست معمولاً ورودی یا خروجی ندارد. سیگنال‌هایی که به عنوان ورودی به ماژول طراحی برای شبیه‌سازی اعمال می‌شوند معمولاً در ماژول محرک به عنوان نوع داده یا reg محلی اعلان می‌گردند. خروجی‌های ماژول طراحی که برای تست نمایش داده می‌شوند در ماژول محرک به عنوان داده نوع wire اعلان می‌شوند. آنگاه ماژول تحت تست با به کار گیری شناسنامه‌های محلی ذکر می‌گردد. شکل ۷-۲۲ این ارتباط را نشان می‌دهد.

ماژول طراحی		ماژول محرک
Module circuit(A,B,C);		Module testcircuit
Input A,B;	→	Reg TA , TB;
Output C;	←	Wire TC;
		Circuit cr(TA,TB,TC);

شکل ۷-۲۲: ماژول‌های محرک و طراحی محاوره‌ای

ماژول محرک ورودی‌ها را برای ماژول طراحی با اعلان شناسه‌های TA و TB به عنوان نوعی reg تولید می‌کند و خروجی طرح را با شناسه نوع wire، یعنی TC چک می‌نماید. سپس شناسه‌های محلی برای ذکر ماژول زیر تست به کار می‌روند. پاسخ به محرک تولید شده با بلوک‌های initial و always در خروجی محرک به صورت نمودار زمان بندی ظاهر می‌شود. و نیز می‌توان با استفاده از Verilog system tasks خروجی عددی نیز تولید کرد. این کار در سیستم با شناسایی کلمات کلیدی که با سمبل \$ آغاز می‌شوند ساخته می‌شود. بعضی از این وظایف مفید در نمایش در زیر آمده است:

display\$	یکبار نمایش مقدار متغیر یا رشته‌هایی با بازگشت از انتهای خط
\$write	مثل \$ display ولی بدون رفتن به خط بعد
monitor\$	هر وقت در حین اجرای شبیه‌سازی تغییری تغییر کند، آن را نمایش می‌دهد
time\$	زمان شبیه‌سازی را نشان می‌دهد
finish\$	شبیه‌سازی را پایان می‌دهد

قاعده نوشتن **\$ display**، **\$ write** و **\$monitor** به شکل زیر است.

Task – name (format specification , argument list) ;

و یا

; (لیست آرگومان و مشخصات قالب) نام تکلیف

مشخصات قالب شامل مبنای اعدادی است که با استفاده از سمبل (%) نمایش داده می‌شوند و ممکن است دارای رشته‌ای در داخل (") باشد. مبنا می‌تواند دودویی، دهدهی، هشت هشتی و یا شانزده شانزدهی فرض شود که به ترتیبی با سمبل‌های %b، %d، %o، %h و %H نشان داده می‌شوند. مثلاً عبارت:

$$\text{\$ Display (\% d \% b \% b , C , A , B) ;}$$

به این معنی که C به دهدهی و A و B به دودویی نمایش داده شوند. توجه کنید که در مشخصات قالب، علامت ویرگول وجود ندارد ولی برای جداسازی مشخصات قالب و لیست آرگومان و نیز بین متغیرهای لیست آرگومان، ویرگول وجود دارد. مثالی که یک رشته را داخل علامت کوتیشن یا نقل قول محصور کند مشابه زیر است:

$$\text{\$ Display ("time = \% 0d A = \%b B = \%b" , \$time , A , B);}$$

و نمایش زیر را تولید خواهد کرد:

$$\text{time = 3 A=10 B=1}$$

که (time=)، (A=) و (B=) بخشی از رشته مورد نمایش‌اند. قالب %0d، %b و %B به ترتیب مبنای \$time، A و B را مشخص می‌کنند. هنگام نمایش مقادیر تابع، بهتر است قالب %0d را به جای %d به کار ببریم. این کار رقم‌های با ارزش‌تر را بدون فضای خالی تولید می‌نماید (%d حدود 10 فضای خالی را تولید می‌کند زیرا زمان با عدد 32 بیت تولید می‌گردد). مثالی از مازول محرک در مثال ۷ HDL نشان داده شده است.

مثال ۷: این مثال حالت مازول محرک را نشان می‌دهد. مدار مورد تست یک مولتی‌پلکسر 2×1 است که در مثال ۶ توصیف گردید مازول testmux پورت ندارد.

```
// stimulus for mu×2×1- df .
Module testmux ;
Reg TA, TB, TS ; //inputs for mux
Wire y ; //output from mux
mu×2×1- df mx (TA, TB, TS,y) ; //Instantiate mux
Initial
Begin
TS=1 ; TA=0 ; TB=1 ;
# 10; TA=1 ; TB=0 ;
# 10; TS=0 ;
# 10; TA=0 ; TB=1 ;
End
Initial
$ monitor("select = %bA = %bB=%b OUT= %b Time = % 0d",
TS ,TA, TB, Y ,$ time );
End module
// dataflow description of 2- to - 1 - line multiplexer
// from example 4- 6
Module mu×2×1- df (A,B,select,OUT);
Input A, B, select ;
Output OUT ;
Assign OUT = select? A : B ;
End module
Simulation log :
```

```
Select =1 A=0 B=1 OUT=0 time=0
Select =1 A=1 B=0 OUT=1 time=10
Select =0 A=1 B=0 OUT=0 time=20
Select =0 A=0 B=1 OUT=1 time=30
```

ورودی‌های mux با کلمه کلیدی reg و خروجی‌ها با wire مشخص می‌شوند. mux با متغیرهای محلی ذکر شده است. بلوک initial رشته‌ای از اعداد دودویی را که در حین شبیه‌سازی اعمال می‌گردند، مشخص می‌کند. پاسخ خروجی با تکلیف \$ monitor

۲۳۵ مدارهای رمزگذار و رمزگشا

چک می‌شود. هر بار یک متغیر تغییر مقدار دهد، شبیه‌ساز ورودی‌ها، خروجی و زمان را نمایش می‌دهد. نتیجه شبیه‌سازی در مثال زیر تتر simulation ذکر شده است. می‌بینیم که وقتی $S = 1$ باشد $OUT = A$ و وقتی $S = 0$ باشد $OUT = B$ است، که بدین ترتیب عملکرد مولتی‌پلکسر را تایید می‌کند.

```
// gate- level description  of circuit of Fig. 4-2
Module analysis (A,B,C,F1,F2);
Input A,B,C;
Output F1 , F2 ;
Wire T1 , T2 , T3 , F2not , E1 , E2 , E3 ;
Or g1 (T1 , A, B, C);
And g2 (T2 , A, B, C);
And g3 (E1 , A, B);
And g4 (E2 , A, C);
And g5 (E3 , B, C);
Or g6 (F2 , E1, E2, E3);
Not g7 (F2not , F2 );
And g8 (T3 ,T1, F2 not);
Or g9 (F1 , T2, T3);
End module

// stimulus to analyze the circuit
Module test- circuit ;
Reg [2:0]D;
Wire F1 , F2 ;
Analysis fig 42 ( D[2] , D[1] , D[0] , F1 , F2 );
Initial
Begin
D= 3 'b000;
Repeat (7)
#10 D= D+1 'b1;
End
Initial
```

```
$monitor ("ABC=%b F1=%b F2=%b" , D,F1,F2);
```

```
End module
```

```
Simulation log :
```

```
ABC=000 F1=0 F2=0
```

```
ABC=001 F1=1 F2=0
```

```
ABC=010 F1=1 F2=0
```

```
ABC=011 F1=0 F2=1
```

```
ABC=100 F1=1 F2=0
```

```
ABC=101 F1=0 F2=1
```

```
ABC=110 F1=0 F2=1
```

```
ABC=111 F1=1 F2=1
```

شبیه‌سازی منطقی، روشی سریع و دقیق در تحلیل مدارهای ترکیبی جهت اطمینان از عملکرد صحیح آنهاست. دو نوع تصدیق وجود دارد: عملیاتی و زمانی. در تصدیق عملیاتی، ما عملکرد مدار را جدا از ملاحظات زمانی مورد بررسی قرار می‌دهیم. این کار با تهیه جدول درستی مدار ترکیبی انجام می‌شود. در تصدیق زمانی عملکرد مدار را با احتساب آثار تاخیر در گیت‌ها مطالعه می‌کنیم. این کار با مشاهده امواج در خروجی گیت‌ها وقتی به یک ورودی مفروض پاسخ می‌دهند، صورت می‌گیرد.

سؤالات

۱- نمودار منطقی یک دیکدر ۲ به ۴ را فقط با گیت‌های NOR طراحی نمایید.

۲- یک مدار ترکیبی با سه تابع بولی زیر تعریف شده است. مدار را با دیکدر و گیت‌های بیرونی بسازید.

$$F_1 = x'y' + xyz'$$

$$F_2 = x'yz' + x'y$$

$$F_3 = x'y'z + xz$$

۳- تابع بولی زیر را با استفاده از یک مولتی پلکسر پیاده‌سازی کنید.

$$F(x,y,z,w) = \sum(0,2,5,7,12,14)$$

۴- یک مولتی پلکسر $16 * 1$ را با دو مولتی پلکسر $8 * 1$ پیاده‌سازی نمایید.

۵- جدول درستی یک انکدر اولویت هشت هشتی به دودویی را معین کنید.

۶- یک جمع‌کننده کامل را با دو مولتی پلکسر $4 * 1$ پیاده‌سازی نمایید.

۷- یک دیکدر افزونی 3 به دودویی را با استفاده از ترکیبات به‌کار نرفته بی‌اهمیت طراحی نمایید.

فصل ۸

مدارهای ترتیبی همزمان

هدف کلی

در این فصل مباحث اصلی مربوط به مدارهای ترتیبی بالاخص مدارهای ترتیبی همزمان مورد بحث و بررسی قرار خواهند گرفت. مفاهیم فلیپ‌فلاپ‌ها به همراه مدارها و نحوه کار هر یک از آنها توضیح داده خواهد شد. همچنین در بحث مدارهای ترتیبی ساعت‌دار، روش‌های تحلیل معادلات ورودی با فلیپ‌فلاپ‌ها مورد بحث و بررسی قرار خواهند گرفت.

هدف ساختاری

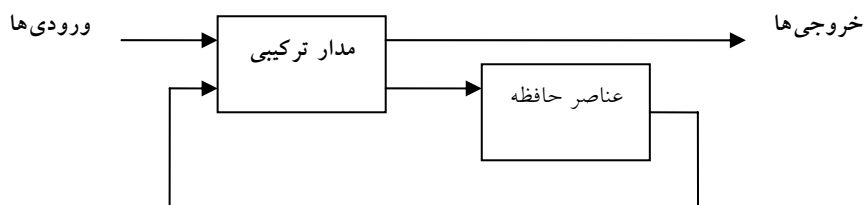
در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند:

- مفهوم مدارهای ترتیبی
- مفاهیم فلیپ‌فلاپ‌ها و لچ‌ها
- فلیپ‌فلاپ T
- فلیپ‌فلاپ D
- فلیپ‌فلاپ K
- مدارهای ترتیبی ساعت‌دار
- تحلیل معادلات با کمک فلیپ‌فلاپ‌ها

۸-۱ مدارهای ترتیبی

کلیه مدارهای دیجیتالی که در فصول ششم و هفتم مورد بررسی قرار گرفته بودند از نوع مدارهای ترکیبی بودند. در این مدارها خروجی‌ها همه به ورودی‌های دیجیتال وابسته‌اند. گرچه به نظر می‌رسد که هر سیستم دیجیتال دارای مدارهای ترکیبی است، بسیاری از سیستم‌هایی که در عمل با آن مواجه هستیم حاوی عناصر حافظه هم می‌باشند و بنابراین لازم است تا این سیستم‌ها بر حسب منطق ترتیبی مورد بررسی قرار گیرند. همچنین لازم است در مواردی در فرایند طراحی گیت عمداً تاخیراتی اعمال گردد. در این فصل مدارهای ترتیبی که قادر به پیاده‌سازی این نوع نیازها هستند، مورد بحث و بررسی قرار می‌گیرند.

نمودار بلوکی یک مدار ترتیبی در شکل ۸-۱ نشان داده شده است. این مدار متشکل از مداری ترکیبی است که عناصر حافظه برای ایجاد یک مسیر پس‌خورد به آن وصل شده‌اند. عناصر حافظه قطعاتی هستند که می‌توانند اطلاعات دودویی را ذخیره کنند. اطلاعات دودویی ذخیره شده در این عناصر در هر لحظه از زمان حالت مدار ترتیبی در آن زمان است. مدار ترتیبی اطلاعات دودویی را از ورودی‌های بیرونی دریافت می‌کند.



شکل ۸-۱: نمودار بلوکی یک مدار ترتیبی

این ورودی‌ها همراه با حالت فعلی عناصر حافظه، مقدار دودویی خروجی‌ها را معین می‌نماید. آنها شرط تغییر حالت در عناصر حافظه را نیز معین می‌سازند. نمودار بلوکی نشان می‌دهد که خروجی‌های یک مدار ترتیبی نه فقط تابعی از ورودی‌ها هستند، بلکه به حالت فعلی عناصر حافظه نیز وابسته می‌باشند. حالت بعدی عناصر به

حافظه نیز تابعی از ورودی‌های بیرونی و حالت فعلی است. بنابراین یک مدار ترتیبی با ترتیب زمانی ورودی‌ها، خروجی‌ها و حالات داخلی مشخص می‌گردد.

۸-۱-۱ انواع مدارهای ترتیبی

دو نوع مدار ترتیبی وجود دارد که دسته‌بندی آنها به زمان‌بندی سیگنال آنها وابسته است. این نوع مدارها عبارتند از:

- مدارهای ترتیبی همزمان
- مدارهای ترتیبی غیر همزمان

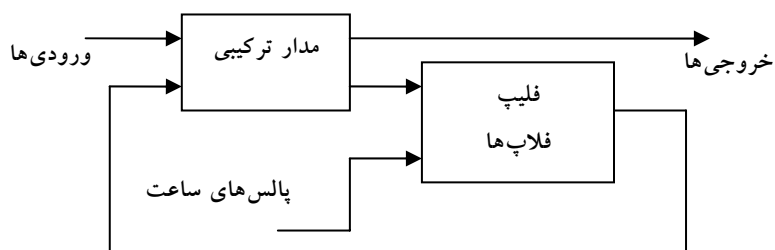
مدار ترتیبی همزمان یا همگام سیستمی است که رفتار آن با توجه به دانش و آگاهی از سیگنال‌هایش در هر لحظه گسسته‌ای از زمان قابل تعریف می‌باشد. رفتار یک مدار ترتیبی غیر همزمان به ترتیب تغییر سیگنال‌های ورودی آن که می‌توانند در هر لحظه از زمان روی مدار تاثیر کنند وابسته می‌باشد. عناصر حافظه‌ای که به طور معمول در مدارهای ترتیبی غیر همزمان به کار می‌روند، نوعی وسایل تاخیر زمانی هستند. قابلیت نگهداری یک وسیله تاخیر زمانی به زمان انتشار سیگنال در وسیله بستگی دارد. در عمل تاخیر انتشار در گیت‌های منطقی درونی برای ایجاد تاخیر کفایت می‌کند بنابراین واحد تاخیر واقعی می‌تواند مورد نیاز نباشد. در سیستم‌های غیر همزمان نوع گیتی، عناصر حافظه متشکل از گیت‌های منطقی است که در واقع تاخیر انتشار آنها عمل ذخیره‌سازی را تداعی می‌نماید. بنابراین در چنین مواقعی یک مدار ترتیبی غیر همزمان را می‌توان مداری با پس‌خورد دانست. به دلیل وجود پس‌خورد در بین گیت‌های منطقی، هر مدار ترتیبی غیر همزمان هر لحظه ممکن است ناپایدار شود. مسئله بی‌ثباتی حاکم مشکلات عدیده‌ای را برای طراح تحمیل خواهد کرد.

با توجه به تعریف، یک مدار ترتیبی همزمان سیگنال‌هایی را مورد استفاده قرار می‌دهد که فقط در لحظات گسسته‌ای از زمان روی عناصر حافظه‌اش اثر می‌گذارد. در این مدارها همزمانی با وسیله‌ای به نام مولد ساعت تحقق می‌یابد و طی آن رشته

متناوبی از پالس ساعت به وسیله این دستگاه تولید می‌گردد. پالس‌های ساعت در سرتاسر سیستم توزیع می‌گردند به نحوی که عناصر حافظه تنها هنگام رسیدن هر پالس تحت تاثیر ورودی خود قرار می‌گیرند. در عمل پالس‌های ساعت به همراه دیگر پالس‌ها که تغییرات لازم را در حافظه ایجاد می‌کند همراه هستند. مدارهای ترتیبی همزمانی که پالس‌های ساعت را در ورودی عناصر ذخیره ساز خود به کار می‌برند، مدارهای ترتیبی ساعت‌دار خوانده می‌شوند. ما غالباً در عمل با مدارهای ترتیبی ساعت‌دار مواجه هستیم. آنها مشکل ناپایداری را ندارند و موضوع زمان‌بندی در آنها به راحتی به مراحل گسسته و مستقل شکسته می‌شود. هر یک از این مراحل یا برش‌های زمانی مستقلاً قابل بررسی می‌باشند.

۸-۲ فلیپ‌فلاپ‌ها و لچ‌ها

یکی از نکات مهم در مدارهای ترتیبی بحث ذخیره‌سازی اطلاعات در هنگام اجرای ترتیبی گیت‌های مدار است. عناصر ذخیره‌سازی در مدارهای ترتیبی ساعت‌دار را



(الف) نمودار بلکی



شکل ۸-۲: مدار ترتیبی ساعت‌دار همزمان

فلیپ فلاپ می گویند. فلیپ فلاپ یک وسیله ذخیره سازی دودویی بوده و قادر است یک بیت از اطلاعات را در خود ذخیره نماید. یک مدار ترتیبی ممکن است در صورت لزوم تعداد قابل توجهی از این فلیپ فلاپ ها را به کار ببرد. نمودار بلوکی یک مدار ترتیبی ساعت دار همزمان در شکل ۸-۲ دیده می شود. خروجی ها می توانند از یک مدار ترکیبی، یا از فلیپ فلاپ ها و یا هر دو حاصل شوند. فلیپ فلاپ ها ورودی های خود را از مدار ترکیبی و نیز از سیگنال ساعت که با فواصل زمانی رخ می دهند، طبق نمودار زمانی دریافت می کنند.

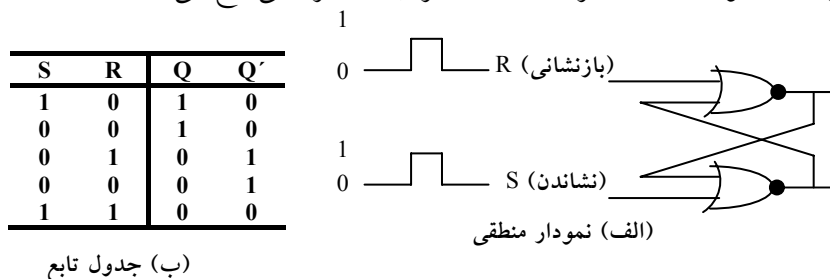
حالت فلیپ فلاپ ها تنها هنگام تغییر وضعیت یک پالس ساعت عوض می شود. وقتی یک پالس ساعت فعال نیست، حلقه پسخورد قطع می شود زیرا حتی اگر خروجی های مدار ترکیبی که ورودی آنها را تغذیه می کند عوض شود خروجی های فلیپ فلاپ تغییر نمی نمایند. بنابراین تغییر وضعیت از یک حالت به بعدی فقط در فواصل زمانی دیکته شده به وسیله پالس های ساعت امکان پذیر است.

۸-۲-۱ لچ ها

یک فلیپ فلاپ می تواند یک حالت دودویی را مادامی که تغذیه به مدارش اعمال شود، تا مدتی نامحدود نگهدارد. تفاوت عمده بین انواع فلیپ فلاپ ها، در تعداد ورودی ها و نحوه تاثیر آنها در تغییر حالت دودویی است. ساده ترین انواع فلیپ فلاپ ها که با سطوح سیگنال عمل می کنند، لچ نامیده می شوند. لچ ها (یا نگهدارها) مدارهای مبنایی هستند که فلیپ فلاپ ها با آنها ساخته می شوند. گرچه لچ ها برای ذخیره اطلاعات دودویی و طراحی مدارهای ترتیبی غیر همزمان مفیدند، ولی عملاً در مدارهای ترتیبی همزمان به کار نمی روند. انواع فلیپ فلاپ هایی که در مدارهای ترتیبی مورد استفاده قرار می گیرند در بخش بعد معرفی شده اند.

۸-۲-۱-۱-۱ SR لچ

لچ SR مداری با دو گیت NAND یا NOR است که به طور متقاطع به هم وصل شده‌اند. این مدار دو ورودی دارد که با S به معنی نشانیدن (set) و R برای بازنشانی (Reset) نام‌گذاری شده‌اند. لچ SR ساخته شده از دو گیت NOR در شکل ۸-۳ دیده می‌شود. لچ دارای دو حالت مفید است. وقتی خروجی $Q = 1$ و $Q' = 0$ باشند گوییم که لچ در حالت نشانده (منطق 1) است. اگر $Q = 0$ و $Q' = 1$ باشد گوییم در حالت بازنشانی (منطق 0). خروجی‌های Q و Q' متمم یکدیگرند. با این وجود، وقتی هر دو ورودی به طور همزمان 1 شوند، حالت تعریف نشده 0 برای دو خروجی رخ می‌دهد.



شکل ۸-۳: لچ SR با گیت NOR

تحت شرایط معمولی، هر دو ورودی در 0 نگهداری می‌شوند مگر اینکه بخواهیم حالت لچ را عوض کنیم. اعمال یک لحظه 1 به ورودی S موجب می‌شود که لچ به حالت 1 برود. قبل از اینکه حالت تعریف نشده ای رخ دهد، ورودی S باید به 0 باز گردد. طبق جدول تابع در شکل ۸-۳(ب)، دو حالت از ورودی موجب می‌شود تا مدار در حالت 1 قرار گیرد. اولین حالت ($S=1, R=0$) نقشی است که طی آن ورودی S، مدار را به حالت 1 می‌برد، که حذف ورودی فعال از S، مدار را در همان حالت باقی می‌گذارد. پس از بازگشت هر دو ورودی به 0، امکان رفتن به حالت 0 میسر خواهد شد، به این ترتیب که برای یک لحظه یک 1 به R اعمال می‌گردد. سپس می‌توان 1 را از R حذف کرد و در این حال مدار در حالت 0 باقی خواهد ماند. لذا وقتی هر دو ورودی S و R برابر 0 اند، بسته به اینکه کدام ورودی اخیراً 1 شده است، لچ می‌تواند در حالت

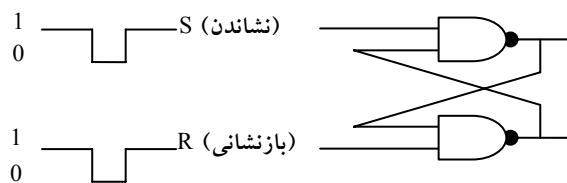
مدارهای ترتیبی همزمان ۲۴۵

1 یا 0 قرار گیرد. اگر به طور همزمان به هر دو ورودی R و S، 1 دودویی را اعمال کنیم، هر دو خروجی به 0 می‌روند. این ورودی‌ها حالت تعریف نشده‌ای را در خروجی ایجاد می‌کنند، زیرا حالت بعدی پیش بینی نشده‌ای را به هنگام بازگشت دو ورودی به 0 نتیجه می‌دهد. در حالت کار معمولی لچ، با اطمینان از اعمال نشدن همزمان 1 به ورودی‌ها، این وضعیت پرهیز می‌گردد.

لچ SR با دو گیت NAND متقاطع در شکل ۸-۴ مشاهده می‌شود. این مدار به طور معمول با 1 در هر دو ورودی‌اش کار می‌کند مگر اینکه بخواهیم حالت لچ را تغییر دهیم. اعمال 0 به S موجب می‌شود Q به 1 برود، و لچ را به حالت نشانده وادارد. وقتی که ورودی S به 1 باز گردد، مدار در همان حالت 1 باقی می‌ماند. پس از بازگشت هر دو ورودی به 1، ما مجاز به تغییر حالت لچ با استقرار 0 در R هستیم. این موجب می‌شود تا مدار به حالت باز نشانی برود و حتی پس از بازگشت هر دو ورودی به 1، لچ در همان حال بماند. حالتی که برای لچ NAND غیر مجاز است، هنگامی است که هر دو ورودی به طور همزمان در 0 باشند. بنابراین از وقوع این حالت باید ممانعت کرد.

S	R	Q	Q'
1	0	0	1
1	1	0	1
0	1	1	0
1	1	1	0
0	0	1	1

(ب) جدول تابع



(الف) نمودار منطقی

شکل ۸-۴: لچ SR با گیت‌های NAND

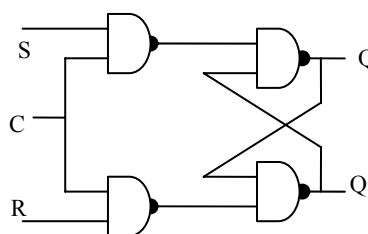
با مقایسه لچ NOR با NAND مشاهده می‌شود که سیگنال‌های ورودی برای NAND متمم ورودی‌های لچ NOR است. چون لچ NAND برای تغییر حالت خود به سیگنال 0 نیاز دارد، گاهی آن را لچ $S'-R'$ می‌خوانند. علامت پریم یا خط بار بر روی حروف،

بیانگر این حقیقت است که ورودی‌ها باید در حالت متمم خود باشند تا مدار را فعال کنند.

عملکرد لچ SR با افزودن یک ورودی کنترل برای تعیین زمان تغییر حالت لچ اصلاح می‌گردد. یک لچ کنترل دار در شکل ۵-۸ مشاهده می‌شود. این مدار شامل یک لچ SR پایه و دو گیت NAND اضافی است. ورودی کنترل C به عنوان یک سیگنال فعال‌ساز برای دو ورودی عمل می‌کند.

C	S	R	حالت بعدی Q
0	X	X	بلا تغییر
1	0	0	بلا تغییر
1	0	1	حالت بازنشانی
1	1	0	حالت نشاندن $Q=1$
1	1	1	نامعین

(ب) جدول درستی



(الف) نمودار منطقی

شکل ۵-۸: لچ SR با ورودی کنترل

مادامی‌که ورودی کنترل در 0 باقی بماند، خروجی گیت‌های NAND در سطح منطقی 1 باقی می‌مانند. این وضعیت حالت سکون برای لچ SR است. حالت نشاندن با $S=1$, $R=0$, $C=1$ حاصل می‌گردد. برای تغییر وضعیت باید $S=0$, $R=1$ و $C=1$ باشد. در هر حال، وقتی که C به 0 بازگردد، مدار در حالت جاری باقی می‌ماند. در ورودی کنترل با اعمال 0 به C، مدار غیر فعال می‌شود، به نحوی که عدم تغییر حالت مستقل از مقادیر S و R، نیز می‌باشد. به علاوه وقتی $C=1$ باشد، و هر دو ورودی S و R برابر 0 باشند، باز هم حالت مدار تغییر نخواهد کرد. این حالات در جدول تابع در کنار نمودار، ملاحظه می‌شوند.

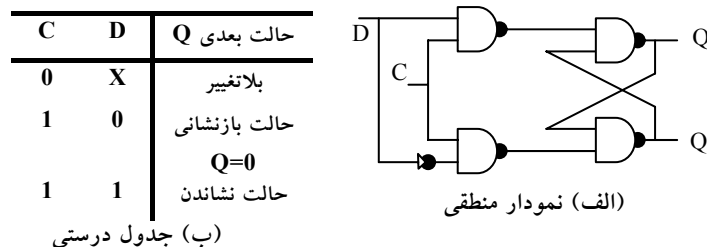
در بعضی شرایط حالت لچ را نامعین می‌نامند. حالت نامعین هنگامی رخ می‌دهد که هر سه ورودی برابر 1 باشند. این وضعیت، مقدار 0 را روی هر دو ورودی لچ SR پایه

مدارهای ترتیبی همزمان ۲۴۷

قرار می‌دهد، که این ورودی‌ها حالت نامعین را برقرار می‌نمایند. وقتی که ورودی کنترل به 0 باز می‌گردد، نمی‌توان حالت بعدی را معین کرد زیرا بستگی دارد که کدام یک از دو ورودی S و R زودتر به 0 بروند. این حالت نامعین موجب می‌گردد تا اداره مدار مشکل باشد و بنابراین به ندرت به کار گرفته می‌شود. با این وجود، مدار از اهمیت لازم برخوردار است زیرا دیگر لچ‌ها و فلیپ‌فلاپ‌ها با آن ساخته می‌شوند.

۸-۲-۱-۲ لچ D

یکی از راه‌های حذف حالت نامطلوب یعنی حالت نامعین یا غیر مجاز در لچ SR این است که مطمئن شویم S و R هرگز به طور همزمان به 1 نمی‌روند. این کار با لچ D شکل ۸-۶ میسر است. این لچ تنها دو ورودی دارد: D (داده) و C (کنترل). ورودی D مستقیماً به ورودی S و متمم آن به ورودی R وصل می‌شود. مادامی‌که ورودی کنترل در 0 قرار دارد، لچ SR متقاطع دارای 1 در هر دو ورودی بوده و مدار نمی‌تواند تغییر حالت دهد. در واقع مقدار D هم نقشی ندارد. وقتی $C=1$ باشد ورودی D نمونه برداری می‌شود. اگر $D=1$ باشد خروجی Q به 1 می‌رود، به این ترتیب مدار در حالت نشانده است. اگر $D=0$ ، خروجی Q به 0 رفته و مدار را به حالت بازنشانی می‌برد.

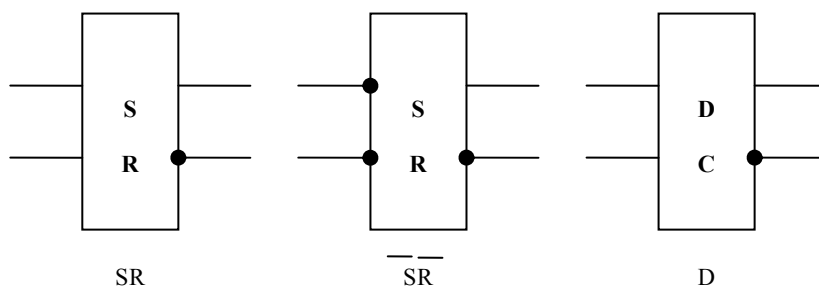


شکل ۸-۶: لچ D

لچ D نامش را از قابلیت نگهداری داده در درون دریافت کرده است. این لچ برای ذخیره موقت اطلاعات دودویی بین یک محیط و یک واحد مناسب است. اطلاعات دودویی حاضر در ورودی داده لچ D هنگامی که ورودی کنترل فعال شود، به خروجی

Q منتقل می‌گردد. مادامی‌که ورودی کنترل فعال است، خروجی تغییرات ورودی را دنبال می‌کند. این وضعیت مسیری از D به خروجی ایجاد می‌کند، و به این دلیل مدار را لچ شفاف هم می‌خوانند. وقتی ورودی کنترل غیر فعال شود، اطلاعات دودویی حاضر قبلی در ورودی، در خروجی Q باقی می‌ماند تا دوباره ورودی کنترل فعال گردد.

نماد گرافیک برای انواع لچ در شکل ۷-۸ آمده است. لچ با یک بلوک مستطیلی مشخص می‌شود، که در آن ورودی‌ها در سمت چپ و خروجی‌ها در سمت راست نشان داده می‌شوند. یکی از خروجی‌ها، خروجی معمولی و دیگری متمم خروجی معمولی را نشان می‌دهد. نمودار گرافیک لچ SR دارای ورودی‌های S و R می‌باشد که در داخل بلوک ذکر شده‌اند. در لچ گیت NAND به ورودی‌ها حباب‌هایی اضافه می‌شود که بیانگر نشانده شدن و بازنشانی با سیگنال منطقی 0 است. نمودار گرافیکی برای ورودی‌های D دارای ورودی‌های D و C است که در داخل بلوک مشخص شده‌اند.



شکل ۷-۸: سمبل‌های گرافیکی لچ

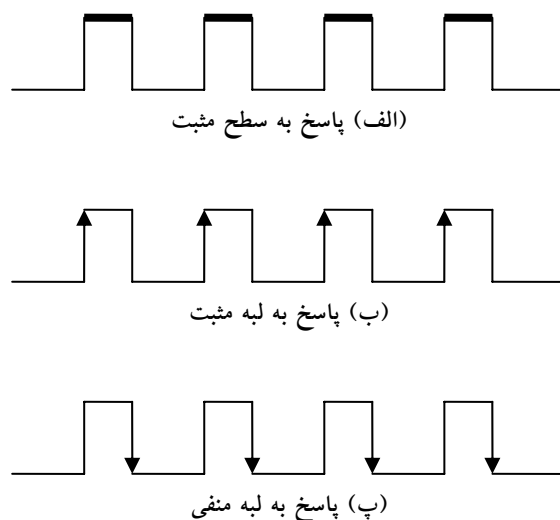
۸-۳ مکانیزم تغییر حالت لچ‌ها

حالت یک لچ یا یک فلیپ‌فلاپ با تغییر در ورودی کنترل عوض می‌شود. این تغییر لحظه‌ای را تریگر گویند و انتقال مربوط به آن را تریگر کردن فلیپ‌فلاپ خوانند. لچ D با پالس‌ها در ورودی کنترلش اساساً یک فلیپ‌فلاپ است که در هر زمان پالس به سطح منطقی 1 برود تریگر می‌شود. مادامی‌که پالس ورودی کنترل در این سطح بماند هر تغییری در ورودی داده، خروجی و حالت لچ را عوض خواهد کرد.

همانطور که از نمودار بلوکی شکل ۸-۲ ملاحظه می‌شود، یک مدار ترتیبی از خروجی‌های فلیپ‌فلاپ به ورودی‌های مدار ترکیبی دارای مسیر پس‌خورد است. در نتیجه ورودی‌های فلیپ‌فلاپ ممکن است از خروجی همان یا دیگر فلیپ‌فلاپ‌ها راه‌اندازی شوند. وقتی که لچ‌ها به عنوان عناصر مورد استفاده قرار گیرند، مشکلی اساسی به وجود می‌آید. به محض تغییر پالس ساعت به منطق 1، انتقال حالت لچ‌ها آغاز می‌شود. در حالی که پالس ساعت هنوز فعال است، حالت جدید لچ در خروجی ظاهر می‌گردد. این خروجی به ورودی لچ‌ها از طریق مدار ترکیبی وصل می‌شود. اگر پالس ساعت در منطق 1، باشد و ورودی اعمال شده به لچ‌ها تغییر کند، لچ به مقادیر جدید واکنش نشان داده و خروجی جدیدی رخ خواهد داد. نتیجه این واکنش وضعیت پیش‌بینی نشده‌ای است زیرا حالت لچ‌ها ممکن است با قرار داشتن پالس ساعت در سطح فعال همچنان به تغییر خود ادامه دهد. به دلیل این عملکرد غیر مطلوب، خروجی یک لچ وقتی همه لچ‌ها به منبع ساعت مشترکی وصلند نمی‌تواند مستقیماً و یا از طریق یک مدار منطقی به همان لچ یا دیگر لچ‌ها وصل شود.

فلیپ‌فلاپ‌ها طوری ساخته می‌شوند که وقتی بخشی از نوع مدار ترتیبی اند و از ساعت مشترکی استفاده می‌کنند، عملکردشان صحیح باشد. مشکل لچ این است که به سطح پالس ساعت پاسخ می‌دهد. همانطور که در شکل ۸-۸ (الف) مشاهده می‌شود،

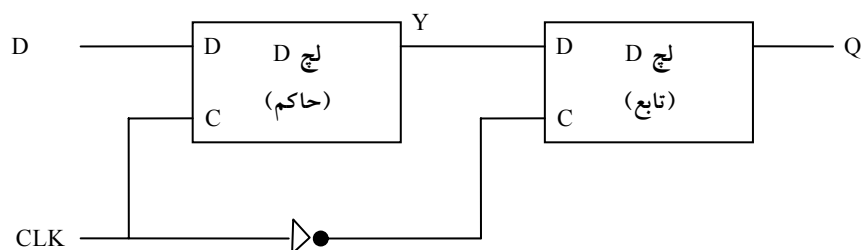
وقتی که پالس ساعت در منطق 1 قرار دارد، هر تغییر مثبت در ورودی کنترل موجب می‌شود تا به ازاء هر تغییر در ورودی D، تغییری در خروجی به وجود آید. نکته کلیدی در یک عملکرد صحیح فلیپ‌فلاپ‌ها تریگر شدن آنها در زمان گذر سیگنال است. پالس ساعت از دو انتقال 1 به 0 و 0 به 1 گذر می‌کند. طبق شکل ۸-۸ گذر مثبت به عنوان لبه مثبت و گذر منفی به نام لبه منفی شناخته می‌شود. برای اصلاح یک لچ به یک فلیپ‌فلاپ، دو راه وجود دارد. یکی از این روش‌ها استفاده از دو لچ با آرایشی خاص است که خروجی فلیپ‌فلاپ را در حین تغییر ورودی، از آن جدا می‌سازد. راه دیگر تهیه فلیپ‌فلاپی است که فقط در حین گذر سیگنال تریگر می‌شود (از 0 به 1 یا از 1 به 0) و در بقیه لحظات پالس ساعت غیر فعال است. اکنون هر دو روش را مطالعه می‌کنیم.



شکل ۸-۸: پاسخ ساعت در لچ و فلیپ‌فلاپ

۸-۳-۱ فلیپ‌فلاپ D حساس به لبه

ساخت یک فلیپ‌فلاپ D با دو لچ D و یک وارون‌گر در شکل ۸-۹ ملاحظه می‌گردد. اولین لچ را حاکم^۱ و دومی را تابع^۲ می‌گویند. مدار، ورودی D را نمونه برداری کرده و خروجی Q را فقط در لبه منفی پالس کنترل ساعت (CLK) تغییر می‌دهد.



شکل ۸-۹: فلیپ‌فلاپ D حاکم - تابع

وقتی که پالس ساعت در ۰ است، خروجی وارون‌گر ۱ می‌باشد. لچ تابع فعال شده و خروجی آن، Q، برابر با خروجی حاکم یعنی Y خواهد شد. لچ حاکم غیر فعال است زیرا $CLK = 0$ می‌باشد. وقتی که پالس ساعت ورودی به سطح ۱ تغییر وضعیت می‌دهد، داده از ورودی بیرونی D به حاکم منتقل می‌گردد در این حال، مادامی‌که ساعت در سطح ۱ بماند، تابع غیر فعال خواهد بود زیرا ورودی C آن برابر ۰ است. هر تغییر در ورودی، خروجی Y را عوض می‌کند، ولی نمی‌تواند خروجی تابع را عوض کند. وقتی که پالس ساعت به ۰ بازگردد، حاکم غیر فعال شده و از ورودی D جدا می‌شود. در همان زمان تابع فعال شده و مقدار Y به خروجی فلیپ‌فلاپ در Q انتقال

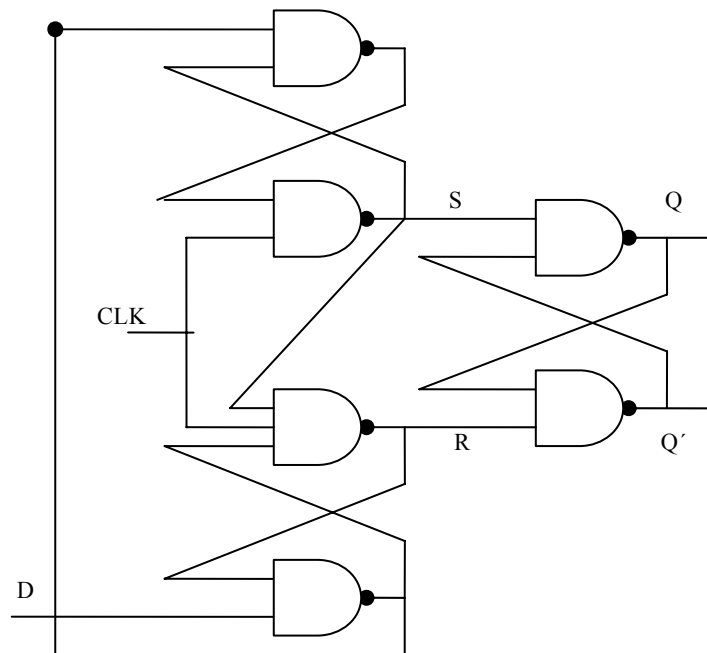
^۱ _ Master

^۲ _ Slave

می‌یابد. بنابراین خروجی فلیپ‌فلاپ فقط در حین گذر پالس ساعت از 1 به 0 تغییر می‌کند.

رفتار فلیپ‌فلاپ حاکم - تابع که در بالا توصیف شد نشان می‌دهد که خروجی فقط در لبه منفی پالس ساعت تغییر می‌نماید. این تغییر را می‌توان در لبه مثبت پالس ساعت هم انجام داد. این کار به این ترتیب صورت می‌گیرد که یک وارون‌گر اضافی بین پایانه CLK و اتصال بین وارون‌گر دیگر و ورودی C لچ حاکم قرار گیرد. چنین فلیپ‌فلاپی با لبه منفی پالس عمل کرده و به این ترتیب لبه منفی حاکم و لبه مثبت نیز تابع و پایانه خروجی را عوض می‌کند.

نمونه دیگری از فلیپ‌فلاپ D حساس به لبه از سه لچ SR، مطابق شکل ۸-۱۰ استفاده می‌کند. دو لچ موجود در این شکل به ورودی‌های بیرونی D (داده) و CLK (ساعت) پاسخ می‌دهد. لچ سوم خروجی را برای فلیپ‌فلاپ تهیه می‌کند. ورودی‌های S و R لچ خروجی در $CLK = 0$ در سطح منطق 1 نگهداری می‌شوند. این موجب می‌شود تا خروجی در حالت فعلی خود باقی بماند. ورودی D ممکن است برابر 0 یا 1 باشد. اگر هنگام 1 شدن CLK، $D = 0$ برقرار باشد، R به 0 تغییر می‌کند. یعنی فلیپ‌فلاپ به حالت بازنشان رفته و در آن $Q = 0$ می‌گردد. اگر در زمان $CLK = 1$ تغییری در ورودی رخ دهد پایانه R در 0 می‌ماند. بنابراین فلیپ‌فلاپ علیرغم تغییر در ورودی خود به حالت قفل باقی خواهد ماند. وقتی که ساعت به 0 باز گردد، R به 1 می‌رود و لچ خروجی در وضعیت ساکن و بدون تغییر در خروجی باقی می‌ماند. به طور مشابه وقتی CLK از 0 به 1 می‌رود، اگر $D = 1$ باشد، S به 0 تغییر می‌کند. این موجب می‌شود تا مدار به حالت 1 رفته و $Q = 1$ گردد. هر تغییر در D، مادامی‌که $CLK = 1$ است، نمی‌تواند خروجی را تحت تاثیر قرار دهد.

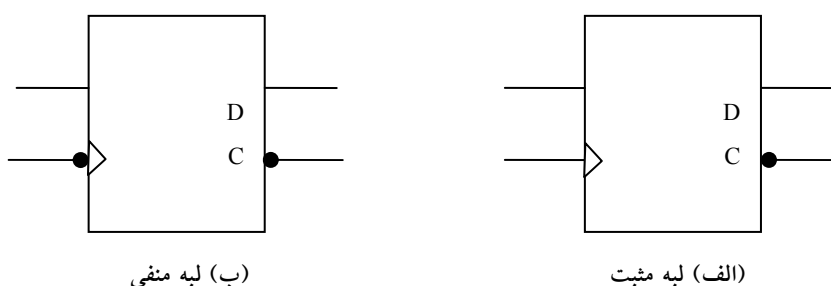


شکل ۸-۱۰: فلیپ‌فلاپ D حساس به لبه مثبت

به طور خلاصه، وقتی ساعت ورودی در فلیپ‌فلاپ حساس به لبه مثبت یک انتقال مثبت انجام دهد، مقدار D به Q منتقل می‌شود. یک لبه منفی از 1 به 0 تاثیری بر روی خروجی ندارد. به همین ترتیب سطح منطق 1، و نیز سطح منطق 0 هم خروجی را عوض نمی‌کنند. از این رو این نوع فلیپ‌فلاپ تنها به لبه 0 به 1 و لاغیر پاسخ می‌دهد.

هنگام استفاده از فلیپ‌فلاپ حساس به لبه باید زمان بندی پاسخ فلیپ‌فلاپ تحت بررسی قرار گیرد. در این زمان بندی، حداقل زمانی به نام زمان برپایی وجود دارد که طی آن قبل از وقوع گذر ساعت، ورودی باید در مقدار ثابت خود نگهداری شود. به همین ترتیب حداقل زمانی بنام زمان نگهداری وجود دارد که طی آن ورودی D نباید پس از اعمال لبه مثبت ساعت، تغییر کند. تاخیر انتشار به صورت فاصله زمانی بین لبه تریگر شدن و تثبیت خروجی در حالت جدید تعریف می‌گردد. این و دیگر پارامترها در برگه‌های اطلاعاتی سازندگان برای هر خانواده منطقی ارائه می‌شوند.

سمبل گرافیکی فلیپ‌فلاپ D حساس به لبه در شکل ۸-۱۱ مشاهده می‌شود. این سمبل مشابه با سمبل لچ D است به جزء اینکه در جلو حرف C علامت فلشی وجود دارد که دینامیکی بودن ورودی را نشان می‌دهد. نشانگر دینامیک به این معنی است که فلیپ‌فلاپ به گذر لبه ساعت حساس است. وجود یک حباب در ورودی دینامیکی به معنی نیاز به لبه منفی ساعت است. عدم وجود حباب پاسخ به لبه مثبت را نشان می‌دهد.



شکل ۸-۱۱: سمبل گرافیکی فلیپ‌فلاپ D حساس به لبه

۸-۴ فلیپ‌فلاپ‌های T و JK

به غیر از فلیپ‌فلاپ نوع D، فلیپ‌فلاپ‌های دیگری نیز وجود دارند. اقتصادی‌ترین و بهترین فلیپ‌فلاپ قابل ساخت، نوع D حساس به لبه می‌باشد که به تعداد کمتری گیت نیاز دارد. دیگر فلیپ‌فلاپ‌ها را می‌توان با فلیپ‌فلاپ D و مقداری مدار بیرونی به وجود آورد. دو فلیپ‌فلاپ رایج در طراحی سیستم‌های دیجیتال عبارتند از: فلیپ‌فلاپ JK و T.

با یک فلیپ‌فلاپ سه عمل را می‌توان انجام داد:

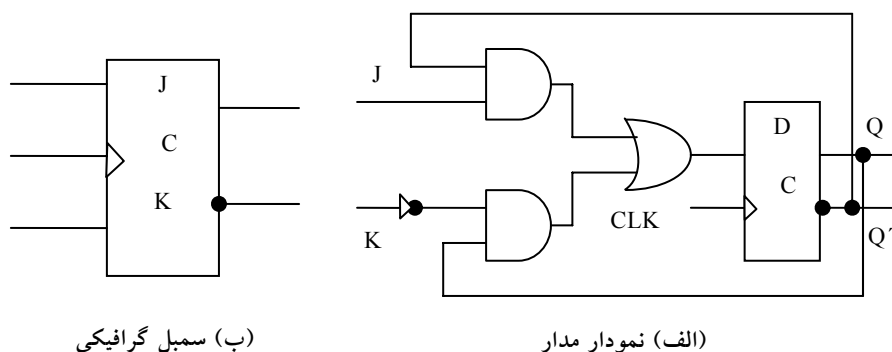
- نشاندن در 1
- بازنشانی در 0
- متمم شدن خروجی

۸-۴-۱ فلیپ‌فلاپ JK

فلیپ‌فلاپ JK هر سه کار را انجام می‌دهد. نمودار مدار ۱ فلیپ‌فلاپ JK که از یک فلیپ‌فلاپ D ساخته شده است، در شکل ۸-۱۲ (الف) دیده می‌شود. ورودی J، فلیپ‌فلاپ را در ۱، و ورودی K، آنرا در ۰ می‌نشانند، و وقتی هر دو ورودی در ۱ قرار گیرند خروجی متمم می‌شود. صحت این مطلب را می‌توان با بررسی مداری که به ورودی D اعمال شده تحقیق کرد:

$$D = JQ' + K'Q$$

وقتی $J = 1$ و $K = 0$ است، $D = Q' + Q = 1$ بوده و بنابراین لبه ساعت بعدی خروجی را در ۱ می‌نشانند. وقتی که $J = 0$ و $K = 1$ باشد، لبه پالس بعدی خروجی را به ۰ باز می‌نشانند. وقتی هر دو ورودی $J = K = 1$ باشد، $D = Q'$ است و بنابراین لبه ساعت



شکل ۸-۱۲: فلیپ‌فلاپ JK

بعدی خروجی را متمم می‌کند. هنگامی که $J = K = 0$ باشد، $D = Q$ است و لبه پالس ساعت بعدی خروجی را بدون تغییر رها خواهد کرد. سمبل گرافیکی برای فلیپ‌فلاپ JK در شکل ۸-۱۲ (ب) ملاحظه می‌گردد. این سمبل مشابه فلیپ‌فلاپ D است به جزء اینکه اکنون ورودی‌ها با J و K نام‌گذاری شده‌اند.

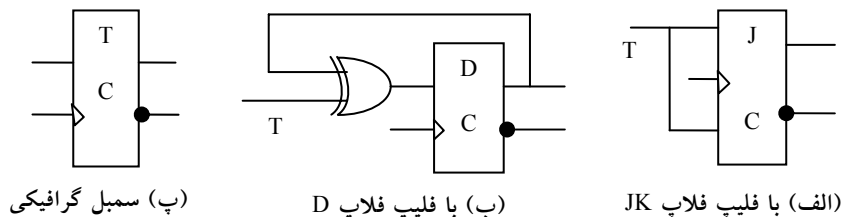
۸-۴-۲ فلیپ فلاپ T

فلیپ فلاپ T (دگر وضع) یک فلیپ فلاپ متمم ساز است و می توان آن را با گره زدن دو ورودی J و K ایجاد کرد. این عمل در شکل ۸-۱۳ (الف) نشان داده شده است. وقتی $T = 0$ باشد، $(J = K = 0)$ ، لبه ساعت، خروجی را عوض نمی کند. وقتی که $T = 1$ است $(J = K = 1)$ ، لبه ساعت، خروجی را متمم مینماید. فلیپ فلاپ متمم ساز در طراحی شمارنده های دودویی بسیار مورد توجه است.

یک فلیپ فلاپ T را می توان با یک فلیپ فلاپ D و یک گیت XOR مطابق شکل ۸-۱۳ (ب) ساخت. عبارت ورودی D در این حالت برابر است با:

$$D = T \oplus Q = TQ' + T'Q$$

وقتی $T = 0$ است، آنگاه $D = Q$ می باشد و بنابراین تغییری در خروجی رخ نمی دهد. وقتی $T = 1$ باشد، آنگاه $D = Q'$ بوده و خروجی متمم می گردد. سمبل گرافیکی برای این نوع فلیپ فلاپ دارای حرف T در ورودی است.



شکل ۸-۱۳: فلیپ فلاپ T

۸-۴-۳ جدول مشخصه فلیپ فلاپ ها

جدول مشخصه خواص منطقی یک فلیپ فلاپ را تعریف می کند و بدین ترتیب عملکرد آن به صورت جدول توصیف می گردد. جداول مشخصه سه نوع فلیپ فلاپ در جدول شکل ۸-۱۴ نشان داده شده است. آنها حالت بعدی را به صورت تابعی از ورودی ها و حالت فعلی تعریف می نمایند. $Q(t)$ به معنی حالت فعلی و یا حالت قبل

از اعمال لبه ساعت است. $Q(t+1)$ ، حالت بعدی پس از اعمال ساعت می‌باشد. توجه کنید ورودی لبه ساعت در جدول مشخصه ذکر نشده است ولی فرض بر این است که بین t و $t+1$ رخ می‌دهد.

فلیپ فلاپ JK			
J	K	$Q(t+1)$	
0	0	$Q(t)$	بلا تغییر
0	1	0	بازنشانی
1	0	1	نشاندن
1	1	$Q'(t)$	متمم

D فلیپ فلاپ		T فلیپ فلاپ	
D	$Q(t+1)$	T	$Q(t+1)$
0	0	0	$Q(t)$
1	1	1	$Q'(t)$

شکل ۸-۱۴: جداول مشخصه فلیپ فلاپ

جدول مشخصه فلیپ‌فلاپ JK نشان می‌دهد که حالت بعدی برابر است با حالت فعلی، به شرطی که $J = K = 0$ باشد. این وضع را می‌توان به صورت $Q(t+1) = Q(t)$ نشان داد و بیان می‌دارد که تغییری در حالت آن ایجاد نمی‌شود. وقتی که $K = 1$ و $J = 0$ باشد، ساعت فلیپ‌فلاپ را به 0 بازنشانی می‌کند و بنابراین $Q(t+1) = 0$ خواهد شد. اگر $J = 1$ و $K = 0$ گردد فلیپ‌فلاپ به $Q(t+1) = 1$ می‌رود. وقتی که هر دو ورودی J و K برابر 1 شوند، حالت بعدی متمم حالت فعلی خواهد بود و می‌توان آن را با $Q(t+1) = Q'(t)$ نشان داد.

حالت بعدی فلیپ‌فلاپ فقط به ورودی D بستگی دارد و مستقل از حالت فعلی است. این حالت را با $Q(t+1) = D$ نشان می‌دهیم. این بدان معنی است که مقدار حالت بعدی برابر با مقدار فعلی (قبل از لبه پالس ساعت) ورودی D است. البته باید توجه کرد

که فلیپ‌فلاپ D حالت بی تغییر را دارا نیست. ولی این کار با غیر فعال کردن ساعت و یا با اتصال خروجی به ورودی D انجام می‌شود. طی آن خروجی یا حالت فلیپ‌فلاپ همواره بی تغییر خواهد ماند.

جدول درستی فلیپ‌فلاپ T فقط دو حالت دارد. وقتی $T = 0$ باشد، لبه ساعت حالت را تغییر نمی‌دهد. وقتی $T = 1$ باشد، لبه ساعت حالت فلیپ‌فلاپ را متمم می‌کند.

۸-۴-۴ معادلات مشخصه

خواص منطقی یک فلیپ‌فلاپ که در جدول مشخصه ملاحظه شد را می‌توان به صورت معادله مشخصه هم بیان کرد. برای فلیپ‌فلاپ D، این معادله به صورت زیر است:

$$Q(t+1) = D$$

این رابطه بیان می‌کند که حالت بعدی خروجی برابر با مقدار ورودی D در حال حاضر است. معادله مشخصه برای فلیپ‌فلاپ JK را از جدول مشخصه و یا از مدار شکل ۸-۱۲ می‌توان به دست آورد. یعنی $Q(t+1) = JK' + K'Q$ که مقدار خروجی فلیپ‌فلاپ قبل از اعمال یک پالس ساعت است. معادله مشخصه برای فلیپ‌فلاپ T از شکل ۸-۱۳ حاصل می‌شود.

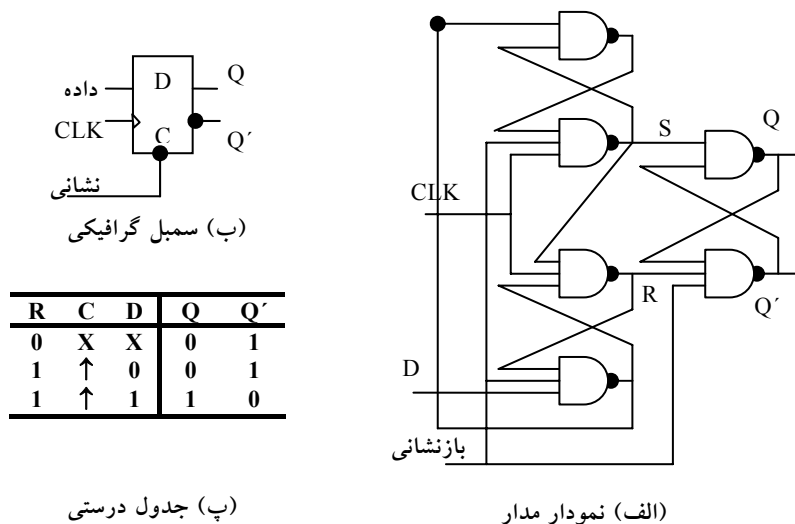
$$Q(t+1) = T \oplus Q = TQ' + T'Q$$

۸-۴-۵ ورودی‌های سیستم

بعضی از فلیپ‌فلاپ‌ها دارای ورودی‌های غیر همزمان برای داشتن آن به یک حالت خاص مستقل از پالس ساعت می‌باشند. ورودی که فلیپ‌فلاپ را در 1 می‌نشاند، پیش تنظیم یا تنظیم مستقیم می‌نامند. ورودی که فلیپ‌فلاپ را به 0 پاک می‌کند، ورودی پاک یا باز نشان مستقیم (غیر همزمان) می‌خوانند. وقتی تغذیه در یک سیستم دیجیتال

روشن شود، حالت فلیپ‌فلاپ نامعلوم است. ورودی‌های مستقیم در استقرار همه فلیپ‌فلاپ‌های سیستم به یک حالت آغازین معلوم، قبل از اعمال پالس ساعت مفید هستند. یک فلیپ‌فلاپ D حساس به لبه مثبت با بازنشانی غیر همزمان R در شکل ۸-۱۵ ملاحظه می‌شود. نمودار مدار مشابه شکل ۸-۱۰ است با این تفاوت که یک ورودی بازنشانی اضافی، به سه گیت NAND متصل شده‌اند. وقتی که این ورودی در 0 است، Q' را همانند در 1 وامی‌دارد، و این به نوبه خود به معنی پاک شدن خروجی Q به 0 است و بنابراین فلیپ‌فلاپ بازنشانی می‌شود. دو اتصال دیگر از ورودی بازنشانی بقاء سومین لچ SR را در منطق 1 تضمین می‌کند. این وضع هنگامی رخ می‌دهد که ورودی بازنشانی، بدون توجه به مقادیر D و CLK، در 0 باشد.

سمبل گرافیکی فلیپ‌فلاپ D با یک ورودی بازنشانی مستقیم دارای یک ورودی اضافی است که با R علامت گذاری شده است. وجود حباب در ورودی به این معنی است که بازنشانی با سطح منطق 0 فعال می‌گردد. فلیپ‌فلاپ‌هایی که از نشان دادن مستقیم استفاده می‌کنند از سمبل S در ورودی نشان دادن غیر همزمان استفاده می‌کنند.



شکل ۸-۱۵: فلیپ‌فلاپ D با بازنشانی غیر همزمان

جدول تابع، عملکرد مدار را مشخص می‌کند. وقتی $R=0$ باشد، خروجی به 0 بازنشانی می‌شود. این حالت مستقل از D و C است. هنگامی مدار می‌تواند به روند عادی خود بازگردد که ورودی بازنشانی به 1 برود. ساعت در C با یک فلش روبه بالا، که به معنی عملکرد فلیپ‌فلاپ در لبه مثبت ساعت می‌باشد، نشان داده شده است. مقدار D با هر لبه مثبت سیگنال ساعت، به شرطی که $R=1$ باشد، به خروجی Q منتقل می‌گردد.

۵-۸ تحلیل مدارهای ترتیبی ساعت دار

رفتار یک مدار ترتیبی ساعت‌دار با ورودی‌ها، خروجی‌ها و حالت فلیپ‌فلاپ‌ها مشخص می‌گردد. خروجی‌ها و حالت بعدی هر دو تابعی از ورودی‌ها و حالت فعلی‌اند. تحلیل یک مدار ترتیبی به معنی تهیه جدول یا نموداری از رشته زمانی ورودی‌ها، خروجی‌ها و حالات درونی است. می‌توان عبارت بول را نوشت و به وسیله آنها رفتار مدار را توصیف کرد. این عبارات باید رشته زمانی لازم را چه مستقیماً و چه غیر مستقیم مشخص کند.

یک نمودار منطقی، وقتی دارای فلیپ‌فلاپ‌ها با ورودی‌های ساعت باشد، مدار ترتیبی ساعت‌دار خوانده می‌شود. فلیپ‌فلاپ‌ها می‌توانند از هر نوع، و نمودار منطقی هم ممکن است شامل گیت‌های ترکیبی باشد یا نباشد. در این بخش، ما یک نمایش جبری را برای تعیین حالت بعدی بر حسب حالت فعلی و ورودی‌ها ارائه می‌کنیم. آنگاه برای توصیف رفتار یک مدار ترتیبی، یک جدول حالت و یک نمودار حالت ارائه می‌شود. یک عبارت جبری دیگر هم برای مشخص کردن نمودار منطقی مدارهای ترتیبی بیان می‌گردد. برای تشریح روال‌های مختلف، مثال‌های خاصی آورده شده است.

۸-۵-۱ معادلات حالت

رفتار مدار ترتیبی ساعت دار را می‌توان با معادلات حالت توصیف کرد. یک معادله حالت (که به آن معادله گذر هم می‌گویند) حالت بعدی را بر حسب تابعی از حالات فعلی و ورودی‌ها بیان می‌نماید. مدار شکل ۸-۱۶ را ملاحظه نمایید. این مدار از دو فلیپ‌فلاپ A و B نوع D و یک ورودی x و یک خروجی y تشکیل شده است. چون ورودی D یک فلیپ‌فلاپ، مقدار حالت بعدی را معین می‌کند، می‌توان مجموعه معادلاتی را به صورت زیر برای مدار نوشت:

$$A(t+1) = A(t)x(t) + B(t)x(t)$$

$$B(t+1) = A'(t)x(t)$$

یک معادله حالت معادله ای است که شرایط گذر حالت را برای یک فلیپ‌فلاپ بیان می‌کند. سمت چپ معادله با (t+1) حالت بعدی فلیپ‌فلاپ را پس از یک لبه ساعت معین می‌نماید. سمت راست معادله عبارتی است بولی که حالت فعلی و وضعیت ورودی‌هایی را مشخص می‌نمایند که در قبال آنها حالت بعدی 1 می‌گردد. چون همه متغیرها در عبارت بول تابعی از حالت فعلی هستند، ما از نوشتن (t) پس از متغیر صرف نظر کرده و معادلات حالت را به صورت فشرده تری مطابق زیر

$$A(t+1) = Ax + Bx \quad \text{می‌نویسیم:}$$

$$B(t+1) = A'x$$

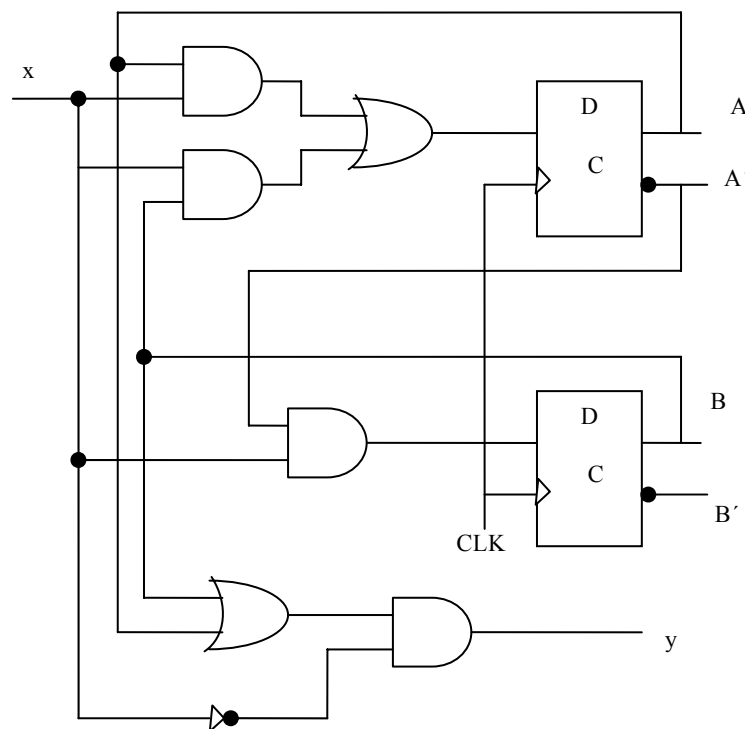
عبارت بولی برای معادلات حالت مستقیماً از گیت‌های تشکیل دهنده بخش ترکیبی در مدار ترتیبی به‌دست می‌آیند، زیرا مقادیر D در مدار ترکیبی حالت بعدی را تعیین می‌کنند.

به طور مشابه مقدار فعلی خروجی نیز قابل ارائه به صورت جبری زیر است:

$$y(t) = [A(t) + B(t)] x'(t)$$

با حذف سمبل (t) از مقدار فعلی، معادله بولی خروجی زیر به‌دست می‌آید:

$$y = (A + B)x'$$



شکل ۸-۱۶: مثال مدار ترتیبی

۸-۵-۲ جدول حالت

رشته‌های زمانی ورودی‌ها، و خروجی‌ها و حالات فلیپ‌فلاپ را می‌توان در یک جدول حالت (به آن جدول گذر هم می‌گویند) جمع‌آوری کرد. جدول حالت برای مدار شکل ۸-۱۶ در جدول شکل ۸-۱۷ دیده می‌شود. جدول متشکل از چهار بخش با نام‌های حالت فعلی، ورودی، حالت بعدی و خروجی است. بخش حالت فعلی، حالت فلیپ‌فلاپ‌های A و B را هر لحظه از زمان t نشان می‌دهد. بخش ورودی مقدار x را برای حالت فعلی ممکن به دست می‌دهد. بخش حالت بعدی، وضعیت فلیپ‌فلاپ‌ها را

مدارهای ترتیبی همزمان ۲۶۳

یک سیکل ساعت بعد، در زمان $t+1$ بیان می‌دارد. بخش خروجی مقدار y را در هر زمان t در قبال هر حالت فعلی با توجه به شرایط ورودی، مشخص می‌کند. تهیه جدول حالت به لیستی از همه ترکیبات دودویی حالت فعلی ورودی‌ها نیاز دارد.

خروجی		حالت بعدی		ورودی	حالت فعلی	
y		A	B	X	A	B
0	0	0	0	0	0	0
0	1	0	1	1	0	0
1	0	0	0	0	1	0
1	1	1	1	1	1	0
0	0	0	0	0	0	1
0	1	1	1	1	0	1
1	0	0	0	0	1	1
1	1	1	1	1	1	1

شکل ۸-۱۷: جدول حالت برای شکل ۸-۱۶

در این حال، ما هشت ترکیب دودویی 000 تا 111 را داریم. سپس مقادیر حالت بعدی از نمودار منطقی یا از معادلات حالت به دست می‌آیند. حالت بعدی فلیپ‌فلاپ A باید در معادله زیر صدق کند.

$$A(t+1) = Ax + Bx$$

بخش حالت بعدی در جدول حالت در زیر ستون A دارای سه عدد 1 است که در قبال آنها حالت فعلی و مقدار ورودی را که حالت فعلی A و ورودی x هر دو برابر 1 هستند و یا حالت فعلی B و ورودی x هر دو برابر 1 می‌باشند بر آورده می‌سازند. به‌طور مشابه حالت بعدی فلیپ‌فلاپ B از معادله حالت زیر حاصل می‌گردد.

$$B(t+1) = A'x$$

و هنگامی برابر 1 است که حالت فعلی $A=0$ و ورودی $x=1$ باشد. ستون خروجی از معادله زیر حاصل می‌شود:

$$y = Ax' + Bx'$$

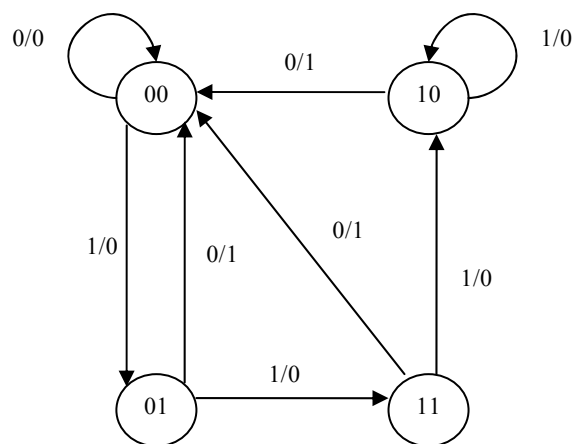
حالت فعلی	حالت بعدی		خروجی	
	X=0	X=1	X=0	X=1
AB	AB	AB	y	y
00	00	01	0	0
01	00	11	1	0
10	00	10	1	0
11	00	10	1	0

جدول حالت یک مدار ترتیبی با فلیپ‌فلاپ‌های نوع D با روال مشابهی به‌دست می‌آید. به طور کلی، یک مدار ترتیبی با m فلیپ‌فلاپ و n ورودی نیاز به 2^{m+n} سطر در جدول حالت دارد. اعداد دودویی از 0 تا $2^{m+n}-1$ در زیر ستون‌های حالت فعلی و ورودی لیست شده‌اند. بخش حالت بعدی دارای m ستون، یعنی یک ستون در ازاء هر فلیپ‌فلاپ، می‌باشد. مقادیر دودویی برای حالت بعدی مستقیماً از معادلات حالت حاصل می‌گردند. بخش خروجی دارای ستون‌هایی به تعداد خروجی‌هاست. مقدار دودویی این بخش مشابه با جدول درستی از مدار یا تابع بولی آن به‌دست می‌آید. گاهی بهتر است تا جدول حالت را با کمی تغییر نشان دهیم. در آرایشی دیگر، جدول حالت تنها سه بخش دارد که عبارتند از: حالت فعلی، حالت بعدی و خروجی. حالت ورودی در زیر ستون حالت بعدی و ستون خروجی ذکر می‌شود. جدول حالت شکل ۸-۱۷ با توجه به این روش، به جدول زیر تبدیل شده است. برای هر حالت فعلی، بسته به مقدار ورودی، دو حالت ممکن برای حالت بعدی و خروجی وجود دارد. بسته به نوع کاربرد هر یک از دو روش فوق بر دیگری ارجحیت دارد. اطلاعات موجود در جدول حالت را می‌توان به صورت گرافیکی با نمودار حالت نشان داد. در این نوع نمودار، یک حالت با یک دایره نشان داده می‌شود و گذر در بین حالات با خطوط جهت‌داری که دو دایره را به هم وصل می‌کنند نمایش داده می‌شود.

نمودار حالت مدار ترتیبی شکل ۸-۱۶ در شکل ۸-۱۸ ملاحظه می‌گردد. نمودار حالت همان اطلاعات جدول حالت را بیان می‌کند که مستقیماً از جدول شکل‌های ۸-۱۷ به‌دست می‌آید. عدد دودویی داخل هر دایره حالت فلیپ‌فلاپ‌ها را بیان می‌نماید.

مدارهای ترتیبی همزمان ۲۶۵

خطوط جهت دار با دو عدد که با یک خط مورب از هم جدا شده‌اند، برچسب خورده‌اند. مقدار ورودی در حالت فعلی در سمت چپ این خط و عدد پس از خط مورب، خروجی را در حالت فعلی در قبال ورودی مربوطه‌اش نشان می‌دهد. باید توجه داشت مقدار خروجی ذکر شده در کنار خطوط جهت دار، در حین حالت فعلی و ورودی مربوطه رخ می‌دهد و هیچ ارتباطی به حالت بعدی ندارد. مثلاً خط واصل جهت‌دار که از حالت 00 به 01 می‌رود با 1/0 برچسب خورده است و به این معنی است که وقتی مدار ترتیبی در حالت فعلی 00 است ورودی 1 و خروجی 0 می‌باشد، در پالس ساعت بعدی، مدار به حالت بعدی 01 می‌رود. اگر ورودی به 0 تغییر یابد، آنگاه خروجی 1 می‌گردد ولی اگر ورودی در 1 باقی بماند، خروجی در 0 خواهد ماند. این اطلاعات از نمودار حالت و خطوط جهت دار که از دایره 01 سرچشمه گرفته، حاصل شده است. یک خط جهت دار که دایره را به خودش وصل کند، به معنی عدم وجود تغییر در حالت است.



شکل ۸-۱۸: نمودار حالت مدار شکل ۸-۱۶

بین جدول حالت و نمودار حالت تفاوتی به جزء نحوه ارائه وجود ندارد. جدول درستی به راحتی از یک معادله حالت و نمودار منطقی حاصل می‌گردد. نمودار حالت مستقیماً از جدول حالت به دست می‌آید. نمودار حالت تصویری از گذر حالات را مجسم می‌کند و برای تفسیر عملکرد مدار مناسب‌تر است. مثلاً، نمودار حالت شکل ۸-۱۸ به وضوح نشان می‌دهد که، با شروع از حالت 00، مادامی‌که ورودی در 1 باشد خروجی برابر 0 است. اولین ورودی 0 بعد از رشته‌ای از 1، خروجی 1 را تولید کرده و مدار را به 00 اولیه باز می‌گرداند.

۸-۶ تحلیل معادلات ورودی با فلیپ‌فلاپ

نمودار منطقی یک مدار ترتیبی متشکل از فلیپ‌فلاپ‌ها و گیت‌هاست. اتصالات میان گیت‌ها مدار ترکیبی را می‌سازند و ممکن است با عبارات بولی نشان داده شوند. آگاهی از نوع فلیپ‌فلاپ‌ها و لیست عبارات بولی مدار ترکیبی، اطلاعات لازم را برای ترسیم نمودار منطقی مدار ترتیبی فراهم می‌سازد. بخشی از مدار ترکیبی که خروجی‌های بیرونی را تولید می‌کند و به صورت توابع بولی توصیف می‌گردند معادلات خروجی نامیده می‌شوند. بخشی از مدار که ورودی‌های فلیپ‌فلاپ‌ها را تولید می‌کنند با توابع بولی به نام معادلات ورودی فلیپ‌فلاپ نام‌گذاری شده‌اند (گاهی به آنها معادلات تحریک هم می‌گویند). ما از سمبل ورودی فلیپ‌فلاپ برای نام‌گذاری متغیر معادله ورودی و نام خروجی فلیپ‌فلاپ‌ها به عنوان اندیس استفاده خواهیم کرد. مثلاً معادله ورودی زیر یک گیت OR، با ورودی‌های x و y که به ورودی D از فلیپ‌فلاپ متصل‌اند و خروجی آن با Q نام‌گذاری شده است را نشان می‌دهد.

$$DQ = x + y$$

مدار ترتیبی شکل ۸-۱۶ متشکل از دو فلیپ‌فلاپ A و B از نوع D ، یک ورودی x و یک خروجی y است. نمودار منطقی مدار می‌تواند به صورت جبری با دو معادله ورودی و یک معادله خروجی بیان شود:

مدارهای ترتیبی همزمان ۲۶۷

$$D_A = Ax + Bx$$

$$D_B = A'x$$

$$y = (A + B)x'$$

سه معادله فوق اطلاعات لازم را برای ترسیم نمودار منطقی مدار ترتیبی فراهم می‌سازند. سمبل D_A یک فلیپ‌فلاپ D با نام A را مشخص می‌نماید. به همین ترتیب D_B فلیپ‌فلاپ B از نوع D است. عبارات بولی مربوط به این دو متغیر و عبارات خروجی y ، بخش ترکیبی مدار ترتیبی را معین می‌کنند.

معادلات ورودی فلیپ‌فلاپ‌ها فرم جبری مناسبی را برای نمودار منطقی یک مدار ترتیبی تشکیل می‌دهند. آنها نوع فلیپ‌فلاپ را با توجه به سمبل فلیپ‌فلاپ مشخص می‌نمایند و مدار ترکیبی که فلیپ‌فلاپ‌ها را راه می‌اندازند هم با آنها مشخص می‌شود. توجه کنید که عبارت معادله ورودی با عبارت مربوط به معادله حالت یکی است. دلیل این است که معادله مشخصه با مقدار ورودی به D برابر است. یعنی:

$$Q(t+1) = DQ$$

۸-۶-۱ تحلیل معادلات با کمک فلیپ‌فلاپ‌های D

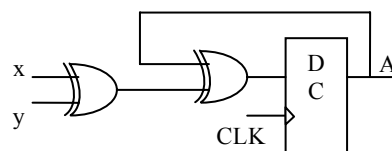
در اینجا روال تحلیل یک مدار ترتیبی متشکل از فلیپ‌فلاپ‌های D را با یک مثال ساده خلاصه می‌کنیم. مداری که برای این هدف در نظر گرفته شده با معادله ورودی زیر توصیف گردیده است.

$$DA = A \oplus x \oplus B$$

سمبل DA یک فلیپ‌فلاپ نوع D با خروجی A را بیان می‌کند. متغیرهای x و y ، ورودی‌ها به مدار هستند. هیچ معادله خروجی مشخص نشده، بنابراین خروجی مدار از خروجی فلیپ‌فلاپ اخذ شده است. نمودار منطقی از معادله ورودی حاصل و در شکل ۸-۱۹(الف) رسم شده است.

حالت بعدی	ورودی‌ها		حالت فعلی
A	x	y	A
0	0	0	0
1	0	1	1
1	1	0	1
0	1	1	0
1	0	0	1
0	0	1	0
0	1	1	0
1	1	1	1

(ب) جدول حالت



(الف) نمودار مدار

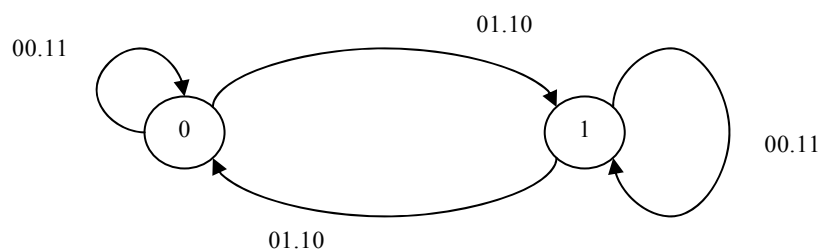
شکل ۸-۱۹: مدار ترتیبی با فلیپ‌فلاپ D

جدول حالت برای حالت فعلی یک ستون داشته و متعلق به فلیپ‌فلاپ A است، دو ستون هم برای ورودی‌ها و یک ستون برای حالت بعدی A لازم است. اعداد دودویی زیر ستون Axy از 000 تا 111 مطابق شکل ۸-۱۹ (ب) لیست شده‌اند. مقادیر حالت بعد، از معادله حالت زیر حاصل می‌شوند:

$$A(t+1) = A \oplus x \oplus y$$

این عبارت یک تابع فرد را بیان می‌دارد و هنگامی برابر 1 است که فقط یک یا سه متغیر برابر 1 باشد. این نکته در ستون حالت بعدی A قابل ملاحظه است.

مدار دارای یک فلیپ‌فلاپ و دو حالت است. نمودار حالت از دو دایره که هر یک مطابق شکل ۸-۲۰ متعلق به یک حالت می‌باشد تشکیل گردیده است. حالت فعلی و خروجی، همانطور که با اعداد داخل دایره نشان داده شده، می‌تواند 0 یا 1 باشد. روی خطوط جهت دار به خطوط مورب نیازی نیست زیرا برای مدار ترکیبی هیچ خروجی در نظر گرفته نشده است. دو ورودی، چهار ترکیب ممکن را برای هر حالت ممکن می‌سازند. دو ترکیب ورودی برای هر گذر حالت با یک ویرگول از هم جدا شده‌اند تا شکل مفهوم‌تر باشد.



شکل ۸-۲۰: نمودار حالت مدار ترتیبی با فلیپ فلاپ D

۸-۶-۲ تحلیل معادلات با کمک فلیپ فلاپ های JK

یک جدول حالت متشکل از چهار بخش، حالت فعلی، ورودی ها، حالت بعدی و خروجی هاست. دو مورد اول با لیست حاصل از همه ترکیبات به دست می آیند. بخش خروجی از معادلات خروجی حاصل می شوند. مقادیر حالت بعدی از معادلات حالت ارزیابی می گردند. در فلیپ فلاپ نوع D معادله حالت با معادله ورودی یکی است. هنگامی که فلیپ فلاپ هایی به جز D مثل JK یا T به کار روند، لازم است به جدول مشخصه یا معادله مشخصه آنها مراجعه شود تا مقادیر حالت بعدی به دست آیند. ما رویه را ابتدا با به کارگیری جدول مشخصه و سپس با معادله مشخصه تشریح خواهیم کرد. مقادیر حالت بعدی یک مدار ترتیبی که از فلیپ فلاپ هایی چون نوع JK و T استفاده می کنند از رویه زیر به دست می آید.

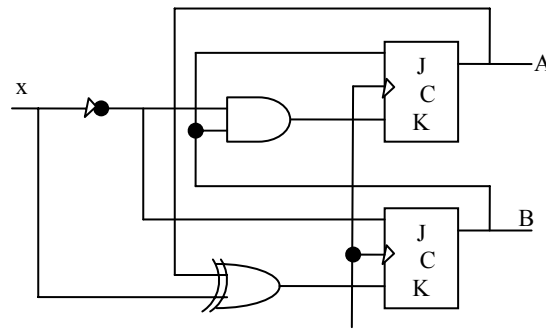
- تعیین معادلات ورودی بر حسب حالت فعلی و متغیرهای ورودی
- لیست مقادیر دودویی هر معادله ورودی
- استفاده از جدول مشخصه فلیپ فلاپ برای تعیین مقادیر حالت در جدول حالت.

به عنوان یک مثال، مدار ترتیبی متشکل از دو فلیپ فلاپ A و B از نوع JK و یک ورودی x را طبق شکل ۸-۲۱ ملاحظه نمایید. مدار دارای خروجی خاص نیست و

بنابراین نیازی به ستون خروجی در جدول حالت وجود ندارد. می‌توان مدار را با

$$JA = B \quad KA = Bx' \quad JB = x' \quad KB = A'x + Ax' = A \oplus x$$

$$JB = x' \quad KB = A'x + Ax' = A \oplus x$$



شکل ۸-۲۱: مدار ترتیبی با فلیپ‌فلاپ JK

جدول حالت مدار ترتیبی در جدول شکل ۸-۲۲ نشان داده شده است. ستون‌های حالت فعلی و ورودی، هشت حالت ممکن را لیست کرده. مقادیر دودویی زیر ستون‌های "ورودی فلیپ‌فلاپ‌ها" بخشی از جدول حالت نیستند، ولی برای ارزیابی حالت بعدی که در مرحله ۲ از رویه ذکر شده لازم‌اند. این مقادیر دودویی مستقیماً از چهار معادله ورودی مشابه با آنچه برای جدول درستی یک عبارت بول حاصل می‌شود، به‌دست آمده‌اند.

حالت فعلی		ورودی	حالت بعدی		ورودی‌های فلیپ‌فلاپ			
A	B		A	B	JA	KA	JB	KB
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0

شکل ۸-۲۲: جدول حالت برای مدار ترتیبی با فلیپ‌فلاپ JK

حالت بعدی هر فلیپ‌فلاپ از ورودی‌های J و K و جدول مشخصه فلیپ‌فلاپ JK در جدول شکل ۸-۱۴ حاصل می‌گردند. چهار حالت برای بررسی وجود دارد. وقتی $J=1$ و $K=0$ باشد، حالت بعدی 1 است. وقتی $J=0$ و $K=1$ است، حالت بعدی 0 می‌باشد. با $J=K=0$ ، تغییری در حالت وجود ندارد و حالت بعدی با حالت فعلی یکی است. وقتی $J=K=1$ باشد، بیت حالت بعدی متمم بیت حالت فعلی است. مثال‌های دو حالت فوق‌الذکر در جدول هنگام $AB=10$ و $x=0$ رخ می‌دهند. بنابراین $JA = KA=0$ بوده و حالت فعلی $A=1$ است. به این ترتیب حالت بعدی A با حالت فعلی تفاوتی نداشته و مقدار آن 1 است. در همان سطر از جدول $JB = KB=1$ است. چون حالت فعلی $B=0$ می‌باشد، حالت بعدی B متمم شده و به 1 تغییر می‌یابد. می‌توان مقادیر حالت بعدی را ارزیابی معادلات حالت در معادله مشخصه هم به‌دست آورد. این کار با دنبال کردن روال زیر میسر است:

معادلات ورودی فلیپ‌فلاپ را بر حسب حالت فعلی و متغیرهای ورودی به‌دست آورید.

معادلات ورودی را در معادلات مشخصه فلیپ‌فلاپ جایگزین نمایید تا معادلات حالت حاصل شود.

از معادلات حالت برای تعیین مقادیر حالت بعدی در جدول حالت استفاده نمایید. معادلات ورودی فلیپ‌فلاپ JK شکل ۸-۲۱ در فوق ملاحظه گردید. معادلات مشخصه برای فلیپ‌فلاپ‌ها از جایگزینی A و B به جای اسم Q به‌دست می‌آیند:

$$A(t+1) = JA' + K'A$$

$$B(t+1) = JB' + K'B$$

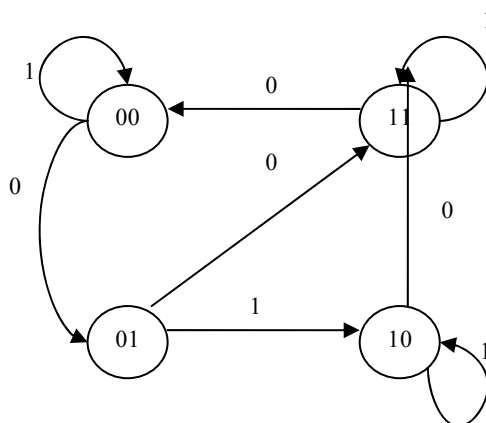
از جایگزینی JA و KA از معادلات ورودی، معادله حالت برای A به‌دست می‌آید:

$$\begin{aligned} A(t+1) &= BA' + (Bx')'A \\ &= A'B + AB' + Ax \end{aligned}$$

معادله حالت مقادیر بیتی ستون زیر "حالت بعدی" A را در جدول حالت فراهم می‌سازد. و به‌طور مشابه، معادله حالت برای فلیپ‌فلاپ B با جایگزینی مقادیر JB و KB به‌دست می‌آید.

$$\begin{aligned} B(t+1) &= x'B' + (A \oplus x)'B \\ &= B'x' + ABx + A'Bx' \end{aligned}$$

معادله حالت مقادیر بیتی را برای ستون زیر "حالت بعدی" B را در جدول حالت فراهم می‌نماید. توجه کنید که معادله حالت به کار رود ستون‌های زیر ورودی‌های "فلیپ‌فلاپ" در جدول شکل ۸-۲۲ لازم نیستند. نمودار حالت مدار ترتیبی در شکل ۸-۲۳ نشان داده شده است. چون مدار دارای خروجی نیست اعداد روی خطوط جهت‌دار خارج شده از دوایر، تنها مقدار ورودی x را بیانگر هستند.



شکل ۸-۲۳: نمودار حالت شکل ۸-۲۱

۸-۶-۳ تحلیل معادلات با کمک فلیپ‌فلاپ‌های T

تحلیل یک مدار ترتیبی با فلیپ‌فلاپ‌های T روال یکسانی با نوع JK دارد. مقادیر حالت بعدی در جدول حالت با جدول مشخصه شکل ۸-۲۴ یا با معادله مشخصه زیر به‌دست می‌آیند.

برابر 1 است، مدار به عنوان یک شمارنده دودویی با رشته 00، 01، 10 و 11 عمل می‌کند و در نهایت به 00 باز می‌گردد. وقتی $x=0$ است، مدار در همان حال باقی می‌ماند. در حالت 11، خروجی $y=1$ است. در اینجا خروجی فقط به حالت فعلی وابسته بوده و مستقل از ورودی است. دو مقدار داخل هر دایره با یک خط مورب از هم جدا شده‌اند تا حالت فعلی و خروجی از هم تفکیک شوند.

حالت فعلی		ورودی	حالت بعدی		خروجی
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

شکل ۸-۲۵: جدول حالت برای مدار ترتیبی با فلیپ‌فلاپ‌های T

سؤالات

۱- با استفاده از فلیپ فلاپ D و یک مولتی پلکسر و یک وارونگر، یک فلیپ فلاپ JK بسازید.

۲- یک مدار ترتیبی با دو فلیپ فلاپ D و معادلات زیر مشخص شده است که در آن A و B فلیپ فلاپ ها، x و y ورودی ها و z خروجی می باشد.

$$A(t+1) = x'y + xA$$

$$B(t+1) = x'B + xA$$

$$Z = B$$

نمودار منطقی و نمودار حالت مرتبط را رسم کرده و جدول حالت را برای مدار ترتیبی لیست کنید.

۳- یک مدار ترتیبی با دو فلیپ فلاپ از نوع D و یک ورودی x طراحی کنید . شرایط زیر باید در طراحی در نظر گرفته شود

- وقتی $x = 0$ است، حالت مدار بدون تغییر باقی می ماند.
- وقتی $x = 1$ است، مدار وارد حالات 00 و 01 و 11 و 10 و بازگشت به 00 شده و کار تکرار شود.

۴- مدار نمودار ترتیبی حالت شکل ۸-۱۸ را با استفاده از فلیپ فلاپ T طراحی نمایید.

۵- یک مدار ترتیبی با دو فلیپ فلاپ JK و دو ورودی E و x طراحی نمایید که شرایط ذیل را داشته باشد:

- اگر $E = 0$ باشد، مدار بدون توجه به x در حالت فعلی خود می ماند
- اگر $E = 1$ و $x = 1$ باشد مدار وارد حالات 00 و 01 و 10 و 11 و بازگشت به 00 شده و کار را تکرار کند.

۲۷۶ مدار منطقی

- اگر $E = 1$ و $x = 0$ باشد، مدار وارد حالات 00 و 11 و 10 و 01 شده و به حالت 00 رفته و عمل را تکرار نماید.

فصل ۹

ثبات‌ها و شمارنده‌ها

هدف کلی

در این فصل مباحث اصلی مربوط ثبات‌ها مورد بحث و بررسی قرار خواهند گرفت و انواع عملیات شامل انتقال و شیفت و ... در ثبات‌ها بررسی خواهند شد. همچنین شمارنده‌ها به عنوان مدارهای پایه مورد بررسی قرار گرفته و انواع شمارنده‌ها با جزئیات بررسی خواهند شد.

هدف ساختاری

در این فصل عناوین زیر مورد بحث و بررسی قرار می‌گیرند:

- ثبات‌ها
- روش‌های بار شدن ثبات‌ها
- انتقال و شیفت اطلاعات بین ثبات‌ها
- مفاهیم شمارنده‌ها
- شمارنده‌های موج گونه
- شمارنده‌های دودویی
- شمارنده‌های همزمان
- انواع دیگر شمارنده‌ها

۹-۱ ذخیره‌سازی دودویی و ثبات‌ها

اطلاعات دودویی در یک کامپیوتر دیجیتال، نوعی موجودیت فیزیکی در یک محیط ذخیره‌سازی اطلاعات برای ذخیره تک تک بیت‌ها باید داشته باشد. یک سلول دودویی وسیله‌ای است که از خود دو حالت با ثبات را به نمایش می‌گذارد و قابل استقرار در یکی از دو حالت است. ورودی سلول سیگنال‌های تحریک کننده‌ای را برای استقرار در یکی از دو حالت دریافت می‌کند. خروجی سلول کمیتی فیزیکی است که دو حالت را از هم تفکیک می‌نماید. وقتی خروجی در سلول در یکی از دو حالت باشد اطلاعات ذخیره شده 1 و یا 0 است.

یک مدار ترتیبی ساعت دار متشکل از گروهی از فلیپ‌فلاپ و گیت‌های ترکیبی است که به منظور تشکیل یک مسیر پس‌خورد به هم متصل شده‌اند. فلیپ‌فلاپ‌ها عناصر ضروری مدار هستند زیرا در غیاب آنها، مدار به یک مدار ترکیبی محض تقلیل می‌یابد. (به شرطی که بین گیت‌ها هم مسیر پس‌خورد وجود نداشته باشد.) اما مدارهای با فلیپ‌فلاپ حتی در نبود گیت‌های ترکیبی باز هم یک مدار ترتیبی است. مدارهای حاوی فلیپ‌فلاپ‌ها معمولاً بر حسب کارشان و نه با نام مدار ترتیبی دسته‌بندی می‌شوند. دو نوع از این مدارها ثبات‌ها و شمارنده‌ها هستند.

۹-۱-۱ ثبات‌ها

یک ثبات در مفهومی ساده و ابتدایی، گروهی از سلول‌های دودویی است. یک ثبات با n سلول، هر کمیت گسسته اطلاعاتی را که حاوی n بیت باشد، می‌تواند ذخیره کند. حالت یک ثبات عددی n تایی از ۱ها و 0ها است که هر بیت حالت یک سلول را در ثبات بیان می‌کند. محتوای یک ثبات تابعی از تفسیر اطلاعات ذخیره شده در آن است. مثلاً یک ثبات 16 بیتی را با محتوای زیر در نظر بگیرید:

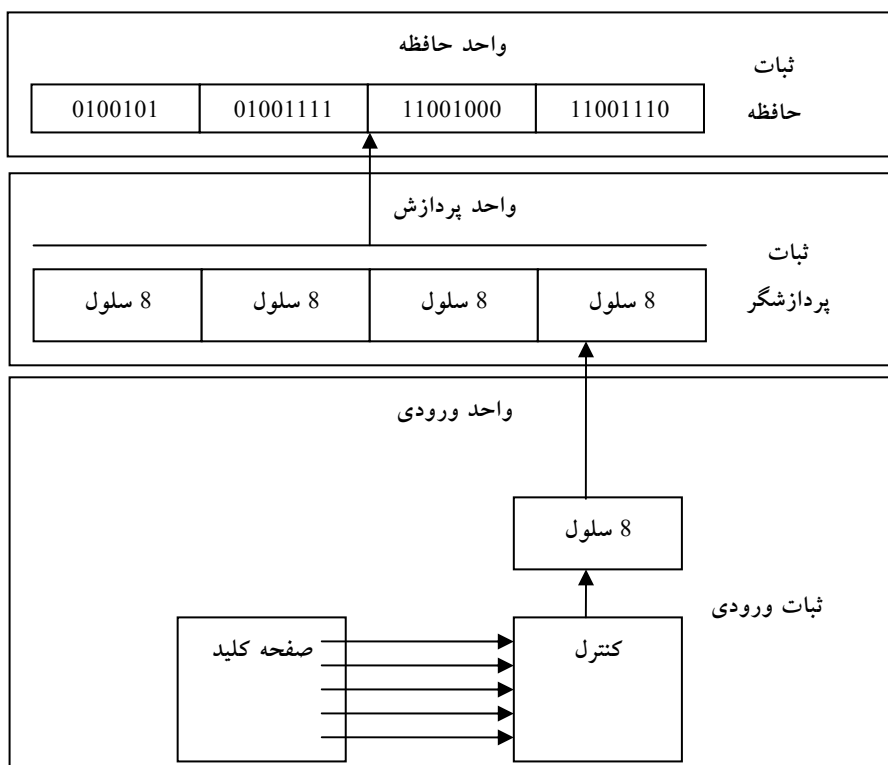
1100001111001001

یک ثبات با 16 سلول می‌تواند در یکی از 2^{16} حالت ممکن باشد. اگر فرض کنیم که محتوای یک ثبات عدد صحیح دودویی را نشان می‌دهد، آنگاه ثبات می‌تواند هر عدد دودویی از 0 تا $2^{16}-1$ را ذخیره کند. برای مثال خاص فوق، محتوای ثبات عدد دودویی معادل با 50121 دهدهی است. اگر فرض کنیم که ثبات کاراکترهای کد هشت بیتی الفبا عددی را ذخیره کرده است، محتوای ثبات می‌تواند هر دو کاراکتر با معنی باشد. برای کد اسکی با توازن زوج واقع در هشتمین بیت با ارزش، ثبات حاوی دو کاراکتر C (هشت بیت سمت چپ) و I (هشت بیت سمت راست) می‌باشد. از طرف دیگر اگر محتوای ثبات به صورت چهار رقم دهدهی تفسیر شود، محتوای ثبات یک عدد دهدهی چهاررقمی خواهد بود. در کد افزونی -3 مثال بالا عدد دهدهی 9096 است. در کد BCD این محتوا بی‌معنی است زیرا ترکیب بیتی 1100 به هیچ رقم دهدهی تخصیص نیافته است. با توجه به مثال، واضح است که یک ثبات قادر است اجزاء گسسته‌ای از اطلاعات را در خود ذخیره نماید و نیز آرایش بیتی یکسانی ممکن است برای انواع دیگر داده، تفسیر متفاوتی داشته باشد.

۹-۱-۲ انتقال بین ثباتی

یک سیستم دیجیتال با ثبات‌هایش و قطعاتی که پردازش داده را اجرا می‌کنند مشخص می‌شود. عمل انتقال بین ثباتی یک عمل بنیادی در سیستم‌های دیجیتال است. این عمل متشکل از انتقال اطلاعات دودویی از یک مجموعه ثبات به مجموعه دیگر است. انتقال ممکن است مستقیماً از یک ثبات به دیگری باشد و یا از طریق مدارهای پردازش داده برای انجام یک عمل صورت گیرد. شکل ۹-۱ انتقال اطلاعات را در میان ثبات‌ها نشان می‌دهد و نیز انتقال اطلاعات دودویی از یک صفحه کلید به یک حافظه به تصویر کشیده شده است. فرض می‌شود واحد ورودی دارای یک صفحه کلید، مدار کنترل و یک ثبات ورودی است. هر بار کلیدی فشرده شود، کنترل یک کد کاراکتر الفبا عددی هشت بیتی معادل را وارد ثبات ورودی می‌نماید.

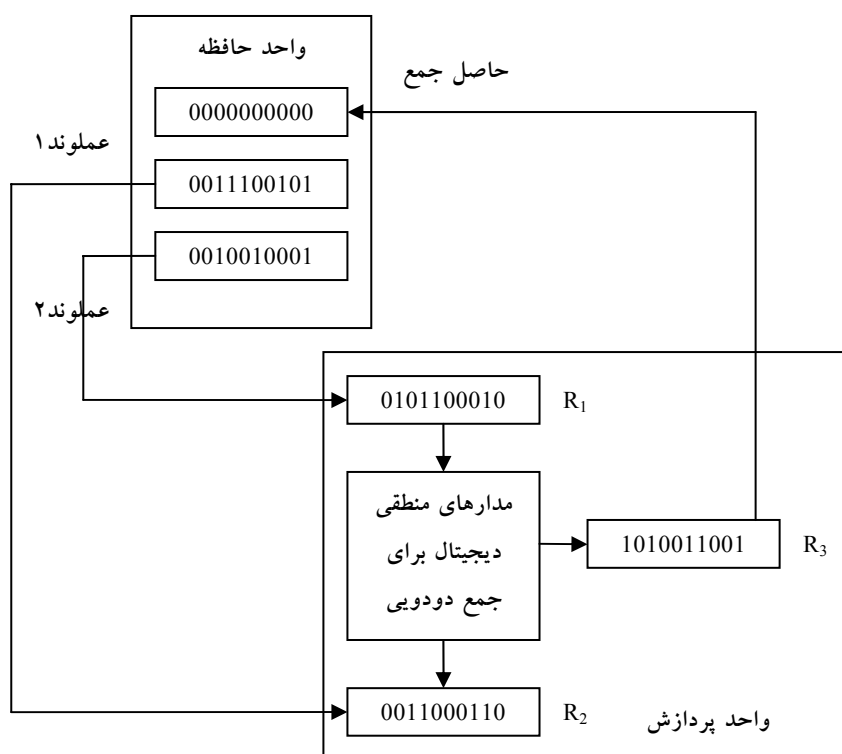
فرض می‌کنیم کد وارده از نوع اسکی و دارای توازن فرد باشد. اطلاعات از ثبات ورودی وارد هشت سلول کم ارزش‌تر یک ثبات پردازنده می‌گردد. پس از هر انتقال، ثبات ورودی پاک می‌شود تا کنترل بتواند پس از زدن کلید، کد هشت بیت جدید را وارد کند. قبل از ورود یا انتقال هر هشت بیت کاراکتر به ثبات پردازنده، کاراکتر قبلی به هشت سلول بعدی در سمت چپ خود منتقل می‌شود. وقتی کار انتقال چهار کاراکتر کامل شد، ثبات پردازنده پر شده و محتوای آن به یک ثبات حافظه منتقل می‌گردد. محتوای ذخیره شده در ثبات حافظه که در شکل ۹-۱ ملاحظه می‌گردد از انتقال کاراکترهای "J"، "O"، "H"، "N" پس از زدن چهار کلید مناسب حاصل شده است.



شکل ۹-۱: انتقال اطلاعات به وسیله ثباتها

ثباتها و شمارنده‌ها ۲۸۱

برای پردازش کمیت‌های گسسته‌ای از اطلاعات به فرم دودویی، کامپیوتر باید مجهز به وسایلی باشد تا داده مورد پردازش را حفظ و نیز بیت‌های اطلاعات را دستکاری کند. وسیله‌ای که برای نگهداری داده به کار می‌رود ثبات است. دستکاری متغیرهای دودویی به کمک مدارهای منطقی دیجیتال انجام می‌شود. شکل ۹-۲ فرایند جمع دو عدد دودویی ۱۰ بیتی را نشان می‌دهد. واحد حافظه که معمولاً متشکل از میلیون‌ها ثبات است، در نمودار تنها با سه ثباتش نشان داده شده است.



شکل ۹-۲: مثالی برای پردازش اطلاعات دودویی

بخشی از واحد پردازشگر که در شکل آمده شامل سه ثبات R_1 ، R_2 ، R_3 به همراه مدارهای منطقی دیجیتال است که بیت‌های ثبات R_1 و ثبات R_2 را دستکاری کرده و نتیجه را که یک حاصل جمع حسابی است به R_3 منتقل می‌سازد. ثبات‌های تشکیل

دهنده حافظه اطلاعات را ذخیره می‌کنند و قادر نیستند دو عملوند را پردازش نمایند. با این وجود، اطلاعات ذخیره شده در حافظه قابل انتقال به ثبات‌های پردازش‌گر است. نتایج حاصل در ثبات‌های پردازنده می‌تواند مجدداً به ثبات‌های حافظه برای ذخیره کاربر بعدی باز فرستاده شود. نمودار، محتوای ارسال شده دو عملوند در ثبات‌های حافظه را به R_1 ، R_2 نشان می‌دهد. مدارهای منطقی حاصل جمع را تولید می‌کنند، که بعد به ثبات R_3 منتقل می‌گردد. اکنون محتوای R_3 قابل بازگشت به یکی از ثبات‌های حافظه است.

۹-۱-۳ شمارنده‌ها

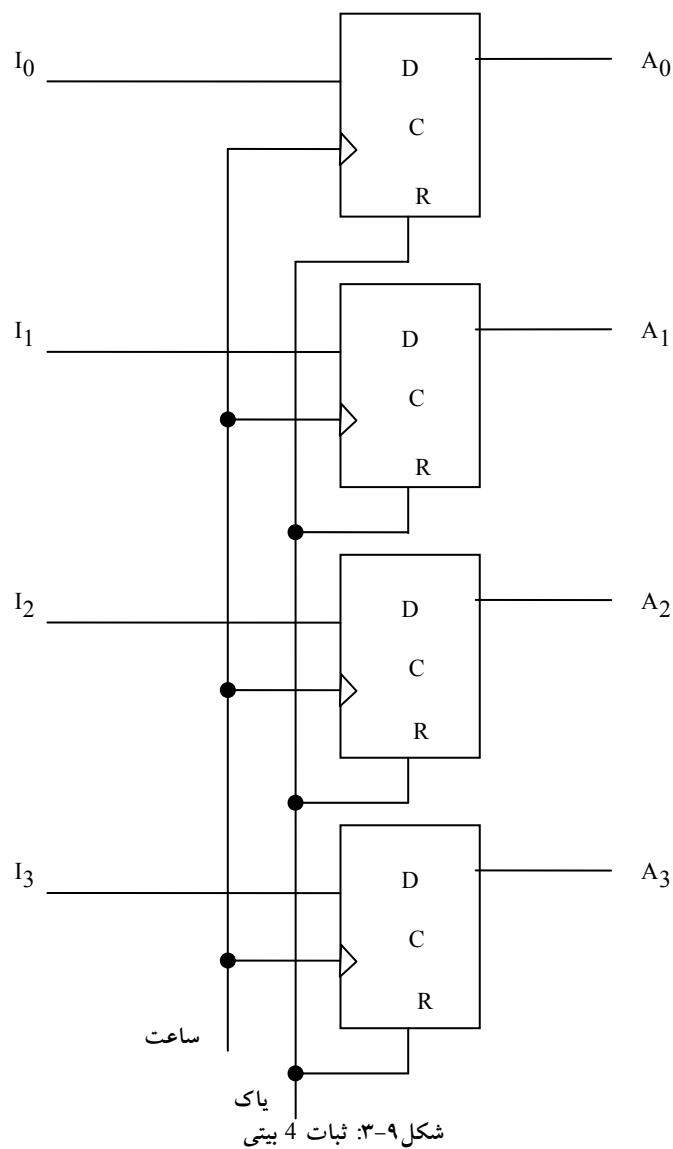
یک شمارنده در مفهومی ساده، اساساً یک ثبات است که وارد یک رشته از حالات از پیش تعیین شده می‌شود. گیت‌ها در شمارنده‌ها چنان به هم متصل شده‌اند تا رشته از پیش تعیین شده‌ای از حالات را تولید نمایند. هرچند که شمارنده‌ها نوع خاصی از ثبات می‌باشند، معمولاً آنها را با نام‌های متفاوت از ثبات‌ها جدا می‌کنند.

۹-۲ کاربرد فلیپ‌فلاپ در ثبات‌ها

از آنجائیکه فلیپ‌فلاپ‌ها به عنوان اصلی‌ترین عضو مدارهای ترتیبی هستند و همچنین در طراحی ثبات بحث استفاده مدارهای ترتیبی مطرح است، لذا می‌توان گفت یک ثبات گروهی از فلیپ‌فلاپ‌ها ست. هر فلیپ‌فلاپ قادر است یک بیت از اطلاعات را در خود ذخیره نماید. یک ثبات n بیت، مجموعه‌ای از n فلیپ‌فلاپ می‌باشد که قادر است n بیت از اطلاعات دودویی را در خود ذخیره نماید. علاوه بر فلیپ‌فلاپ، یک ثبات ممکن است گیت‌های ترکیبی را نیز برای اجرای کارهای پردازشی مختلف داشته باشد. در تعریف جامع‌تر، یک ثبات متشکل از یک گروه فلیپ‌فلاپ و گیت‌هاست که در عمل انتقال با یکدیگر تشریک مساعی دارند. فلیپ‌فلاپ‌ها اطلاعات دودویی را نگه می‌دارند و گیت‌ها چگونگی انتقال اطلاعات را به ثبات معین می‌کنند.

ثباتها و شمارنده‌ها ۲۸۳

انواع متنوعی از ثبات‌ها در بازار وجود دارند. ساده‌ترین ثبات، فقط از فلیپ‌فلاپ‌ها و بدون هر گونه گیتی تشکیل شده است. شکل ۹-۳ چنین ثباتی را که از چهار فلیپ‌فلاپ D ساخته شده نشان می‌دهد.



ساعت ورودی مشترک همه فلیپ‌فلاپ‌ها را با لبه مثبت هر پالس تریگر می‌کنند و به این ترتیب اطلاعات دودویی در چهار ورودی به داخل ثبات 4 بیت منتقل می‌گردند. می‌توان هر لحظه چهار خروجی را نمونه برداری کرد و اطلاعات دودویی ذخیره شده در ثبات را به دست آورد. ورودی پاک به ورودی بازنشان (R) همه فلیپ‌فلاپ‌ها می‌رود. وقتی این ورودی به 0 رود، همه فلیپ‌فلاپ‌ها به طور غیر همزمان بازنشانی (0) می‌شوند. ورودی پاک کردن برای 0 کردن ثبات قبل از عمل ساعت‌زنی مفید است. در حین عمل معمول ساعت زنی، ورودی‌های R باید در منطق 1 قرار گیرند. توجه کنید که برای 0 کردن همه حالات در یک ثبات، می‌توان از پاک کردن، یا بازنشانی استفاده کرد.

۹-۲-۱ ثبات با بارشدن موازی

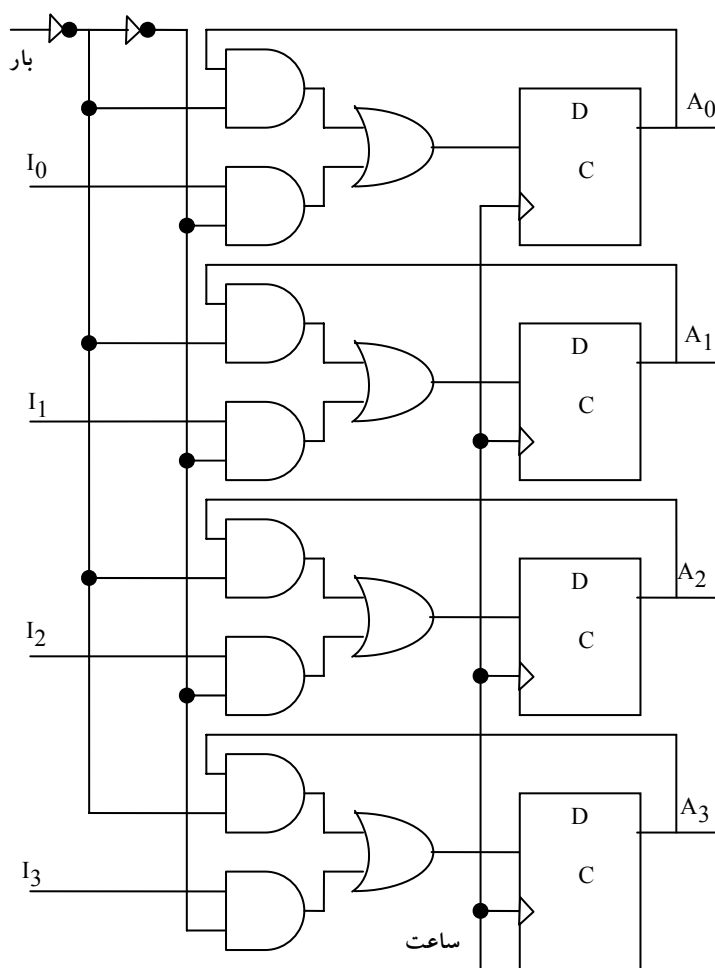
سیستم‌های دیجیتال هم زمان دارای یک مولد ساعت اصلی اند که رشته ای از پالسهای ساعت را به طور پیوسته فراهم می‌سازند. پالسهای ساعت به همه فلیپ‌فلاپ‌ها و ثبات‌ها در سیستم اعمال می‌گردند. ساعت اصلی مانند پمپی است که ضربان ثابتی را برای همه بخش های سیستم فراهم می‌نماید. برای تاثیر یک پالس ساعت خاص بر روی یک ثبات خاص، باید یک کنترل جداگانه به کار برده شود.

انتقال اطلاعات جدید به یک ثبات را بارشدن ثبات نامند. اگر همه بیت‌های ثبات به طور همزمان با یک پالس بار شوند گوییم بارشدن موازی است. لبه ساعت اعمال شده به ورودی‌های C ثبات شکل ۹-۳ موجب می‌شود تا هر چهار ورودی به طور موازی بار گردند. در این آرایش اگر بخواهیم ثبات بدون تغییر رها شود، باید ساعت از مدار قطع گردد. این کار با کنترل سیگنال ورودی ساعت به وسیله گیت فعال‌ساز انجام می‌شود. با این وجود قرار دادن گیت‌ها در مسیر ساعت به این معنی است که یک کار منطقی صورت گرفته است. استقرار گیت‌ها موجب تولید تاخیرهای نابرابر در فلیپ‌فلاپ‌ها می‌گردد. برای همزمانی کامل سیستم، باید مطمئن بود که همه پالس‌های

ثباتها و شمارنده‌ها ۲۸۵

ساعت به طور همزمان به هر نقطه از سیستم می‌رسد و بنابراین همه فلیپ‌فلاپ‌ها به طور همزمان تریگر می‌شوند. اعمال پالس ساعت از طریق گیت، تاخیرهای متغیری را موجب می‌شود و ممکن است سیستم را از همزمانی خارج کند. به این دلیل پیشنهاد می‌شود که کنترل عمل یک ثبات با ورودی‌های D به جای کنترل ساعت در ورودی‌های C فلیپ‌فلاپ‌ها انجام گیرد.

یک ثبات 4 بیتی با ورودی کنترل بارشدن که از طریق گیت‌ها به ورودی‌های D



شکل ۹-۴: ثبات 4 بیتی با بارشدن موازی

فلیپ‌فلاپ هدایت شده در شکل ۹-۴ ملاحظه می‌گردد. ورودی بارشدن به ثبات عملی را که در هر پالس ساعت اتفاق می‌افتد مشخص می‌کند. وقتی که ورودی بار برابر با 1 است، داده در چهار ورودی در لبه مثبت پالس ساعت بعدی به داخل پبات منتقل

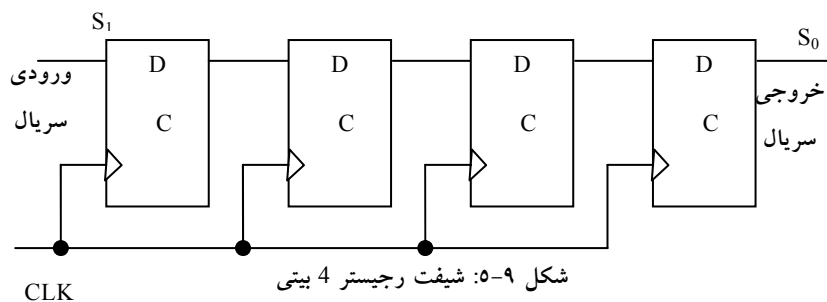
می‌شود. وقتی ورودی بار شدن 0 است، خروجی‌های فلیپ‌فلاپ‌ها به ورودی‌های خودشان وصلند. اتصال پس‌خوردی از خروجی به ورودی لازم است زیرا فلیپ‌فلاپ D دارای حالت "بی تغییر" نیست. در لبه پالس ساعت، ورودی D حالت بعدی فلیپ‌فلاپ را مشخص می‌نماید. برای بدون تغییر نگهداشتن فلیپ‌فلاپ، لازم است D را با حالت فعلی فلیپ‌فلاپ یکسان نماییم.

پالس‌های ساعت مرتباً به ورودی‌های C اعمال می‌گردند. ورودی بارشدن، پذیرش اطلاعات جدید و یا حفظ اطلاعات فعلی را در ثبات معین می‌کند. انتقال اطلاعات از ورودی‌ها یا خروجی‌های ثبات در پاسخ به یک لبه ساعت به طور همزمان در هر چهار بیت انجام می‌گیرد.

۹-۲-۲ شیفت رجیسترها

ثباتی که بتواند اطلاعات دودویی‌اش را به سمت راست یا چپ شیفت یا جابه‌جا کند، شیفت رجیستر یا ثبات جابه‌جایی نامیده می‌شوند. ساختار منطقی یک شیفت رجیستر، از زنجیره‌ای از فلیپ‌فلاپ‌ها تشکیل شده که در آن خروجی یک فلیپ‌فلاپ به ورودی فلیپ‌فلاپ دیگر متصل است. همه فلیپ‌فلاپ‌ها پالس ساعت مشترکی دریافت می‌کنند. پالس‌های ساعت اطلاعات را از یک طبقه به طبقه دیگر جابه‌جا می‌کنند.

ساده‌ترین شیفت رجیستر طبق شکل ۹-۵ فقط از فلیپ‌فلاپ‌ها استفاده می‌کند. خروجی یک فلیپ‌فلاپ مفروض به ورودی D فلیپ‌فلاپ سمت راست خود متصل است. هر پالس ساعت محتوای ثبات را یک بیت به راست جابه‌جا می‌کند.



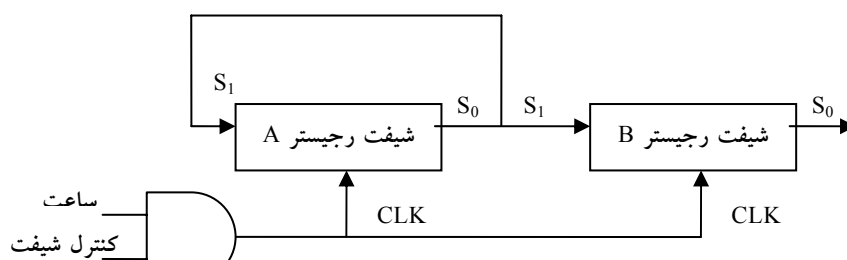
ورودی سریال، تعیین کننده اطلاعاتی است که از منتهی الیه سمت چپ در حین جابه‌جایی وارد می‌شود. خروجی سریال از خروجی سمت راست ترین فلیپ‌فلاپ اخذ می‌گردد. گاهی لازم است تا جابه‌جایی را طوری کنترل کنیم که فقط با پالس‌های معینی رخ دهد. این کار با ممانعت از پالس ساعت در رسیدن به ثبات امکان‌پذیر است. بعد نشان خواهیم داد که عمل جابه‌جایی می‌تواند از ورودی‌های D به جای ورودی ساعت ثبات کنترل گردد. در هر صورت اگر شیفت رجیستر شکل ۹-۵ به کار رود، می‌توان عمل جابه‌جایی را به وسیله یک گیت AND و ورودی که جابه‌جایی را کنترل می‌کند تحت کنترل در آورد.

۹-۲-۳ انتقال سریال

اگر یک سیستم دیجیتال هر بار یک بیت را انتقال دهد و یا دستکاری نماید، آنگاه سیستم را فعال در مد سریال می‌نامیم. با جابه‌جایی یک بیت به خارج ثبات مبدا و ورود به ثبات مقصد، اطلاعات هر بار یک بیت انتقال می‌یابد. این بر خلاف انتقال موازی است که در آن همه بیت‌های ثبات به طور همزمان انتقال می‌یابند.

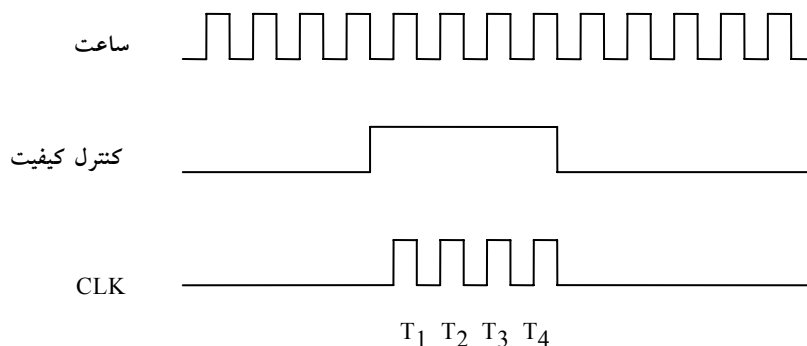
انتقال سریال اطلاعات از ثبات A به ثبات B طبق نمودار بلوکی شکل ۹-۶ با شیفت رجیستر انجام می‌شود. خروجی سریال (SO) از ثبات A به ورودی سریال (SI) در ثبات B وصل است. برای پیش‌گیری در از دست دادن اطلاعات ذخیره شده در ثبات مبدا، اطلاعات ثبات A از خروجی سریال به ورودی سریال چرخانده می‌شود. در حین عمل

جابه‌جایی مقدار اولیه ثابت B به بیرون منتقل شده و از بین می‌رود، مگر اینکه به ثابت سومی انتقال یابد. ورودی کنترل جابه‌جایی زمان و تعداد دفعاتی که ثابت‌ها جابه‌جا می‌شوند را معین می‌سازد. این کار با یک گیت AND انجام می‌گردد و طی آن پالس‌های ساعت اجازه عبور به پایانه‌های CLK را به هنگام فعال بودن کنترل جابه‌جا خواهد داشت.



شکل ۹-۶: نمودار بلوکی انتقال سریال از یک ثابت به دیگری

فرض کنید که شیف رجیسترها هر کدام دارای چهار بیت باشند. واحد کنترلی که انتقال را مدیریت می‌کند باید طوری طراحی شود که شیف رجیسترها را در طول سیگنال کنترل جابه‌جایی برای مدت چهار پالس ساعت فعال سازد.



شکل ۹-۷: نمودار زمانی انتقال سریال از یک ثابت به دیگری

این مطلب در نمودار زمان بندی شکل ۹-۷ ملاحظه می‌شود. سیگنال کنترل جابه‌جایی با ساعت هنگام است و مقدارش درست پس از لبه منفی پالس ساعت تغییر می‌یابد. در چهار پالس ساعت بعدی سیگنال کنترل جابه‌جایی فعال است و خروجی گیت AND متصل به ورودی‌های CLK چهار پالس T_1 ، T_2 ، T_3 و T_4 را تولید می‌نماید. هر لبه بالارونده پالس یک جابه‌جایی را در هر ثبات انجام می‌دهد. چهارمین پالس کنترل جابه‌جایی را 0 نموده و موجب می‌شود تا شیفت رجیسترها غیر فعال شوند.

فرض کنید که محتوای دودویی A قبل از جابه‌جایی 1011 و B برابر 0010 باشد. انتقال سریال از A به B در چهار مرحله رخ می‌دهد، جدول شکل ۹-۸. با اولین پالس، T_1 ، سمت راست‌ترین بیت A به سمت چپ‌ترین بیت B منتقل می‌گردد و نیز به سمت چپ‌ترین بیت A می‌چرخد. در همان زمان تمام بیت‌های A و B یک مکان به راست جابه‌جا می‌شوند. خروجی سریال قبلی از B در سمت راست‌ترین مکان از بین رفته و مقدار آن از 0 به 1 تبدیل می‌گردد. سه پالس بعدی اعمال مشابهی را انجام می‌دهند و بیت‌های A و B را هر بار یک بیت به راست جابه‌جا می‌کنند. پس از چهارمین جابه‌جایی، کنترل جابه‌جایی به 0 رفته و هر دو ثبات A و B دارای مقدار 1011 خواهند بود. بنابراین محتوای A به B منتقل شده است، ضمن اینکه A همچنان بدون تغییر باقی می‌ماند.

پالس زمانی	شیفت رجیستر A				شیفت رجیستر B			
	1	0	1	1	0	0	1	0
مقدار اولیه	1	1	0	1	1	0	0	1
پس از T_1	1	1	1	0	1	1	0	0
پس از T_2	0	1	1	1	0	1	1	0
پس از T_3	1	0	1	1	1	0	1	1
پس از T_4								

شکل ۹-۸: مثالی از انتقال سریال بین ثبات‌ها

با توجه به این مثال تفاوت بین مد های سریال و موازی کاملاً آشکار است. در مد موازی، اطلاعات همه بیت های ثابت در دسترس است و همگی می توانند با یک پالس ساعت به طور همزمان انتقال یابند. در مد سریال ثابت ها دارای یک ورودی سریال و یک خروجی سریال هستند، اطلاعات هر بار یک بیت انتقال می یابد و ثابت ها در یک جهت جابه جا می شوند.

۹-۲-۴ جمع کننده سریال

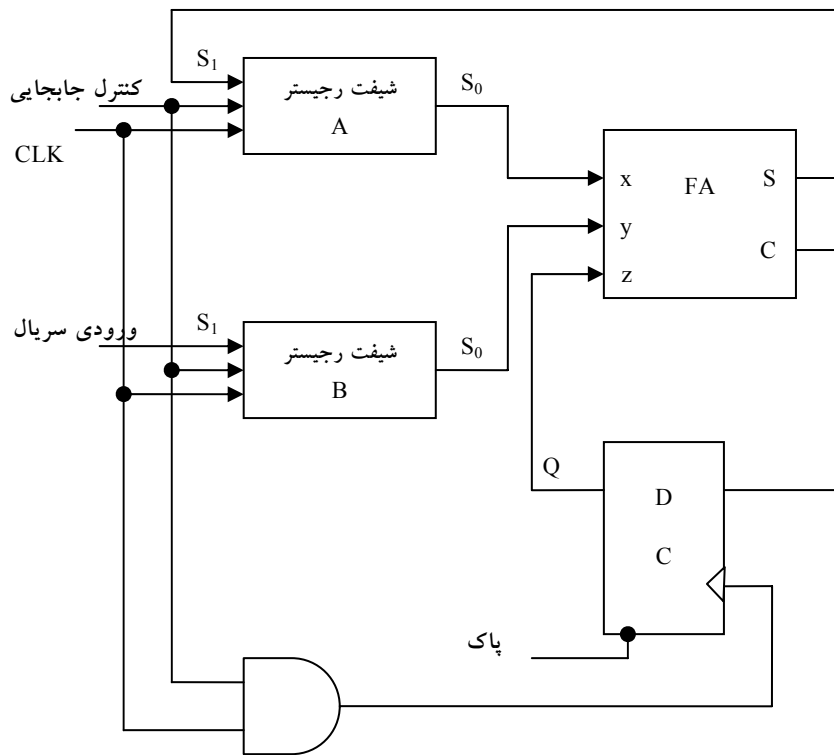
همانگونه که در فصول قبل مشاهده گردید، عملیات در کامپیوترهای دیجیتال معمولاً به صورت موازی صورت می گیرد، زیرا این روش سریع ترین نوع است. در مقابل عملیات سریال کندتر است، ولی به قطعات کمتری نیاز دارد. برای ارائه مد سریال، در اینجا یک جمع کننده سریال را نشان می دهیم.

دو عددی که قرار است به طور سریال با هم جمع شوند در دو شیفت رجیستر ذخیره می شوند. بیت ها، هر جفت یک بار به وسیله یک جمع کننده کامل سریال (FA)، با هم جمع می شوند، شکل ۹-۹. نقلی خروجی جمع کننده کامل به فلیپ فلاپ D انتقال می یابد. آنگاه خروجی این فلیپ فلاپ به عنوان نقلی ورودی به جفت بیت با ارزش تر بعدی اضافه می شود. با انتقال حاصل جمع به A، که با جا به جایی A انجام می گردد، می توان از یک ثابت برای هر دو مقدار مضاف الیه و حاصل جمع استفاده کرد. ورودی سریال ثابت B می تواند برای انتقال یک مقدار دودویی جدید به کار رود و در همان زمان بیت های مضاف در حین جمع، به خارج جابه جا می شوند.

طرز کار یک جمع کننده سریال به شرح زیر است. در ابتدا ثابت A مقدار مضاف الیه و ثابت B مقدار مضاف را نگه می دارند، ضمن اینکه فلیپ فلاپ نقلی به 0 پاک شده است. خروجی Q از فلیپ فلاپ نقلی ورودی در z را تهیه می کند. کنترل جابجایی هر دو ثابت و فلیپ فلاپ نقلی را فعال می سازد، به نحوی که در پالس ساعت بعدی، هر دو ثابت یک بار به راست جابجا می شوند، بیت حاصل جمع از S به سمت چپ

ثباتها و شمارنده ها ۲۹۱

فلیپ فلاپ A می رود و رقم نقلی به فلیپ فلاپ Q منتقل خواهد شد. کنترل جابجایی ثباتها را به تعداد پالس هایی برابر با تعداد بیت های ثباتها فعال می کند. در قبال هر یک پالس ساعت جدید، یک بیت حاصل جمع جدید به A می رود. یک نقلی جدید به Q رفته و هر دو ثبات یک بار به سمت راست جابجا می گردند. این روند تا از کار افتادن کنترل جابجایی ادامه می یابد. بنابراین، جمع با عبور هر جفت بیت به همراه نقلی قبلی از یک جمع کننده کامل و انتقال حاصل جمع به ثبات A، در هر بار یک بیت، انجام می گردد.



شکل ۹-۹: جمع کننده سریال

در آغاز، ثبات A در فلیپ فلاپ نقلی به 0 پاک می شود و سپس اولین عدد از B به آن اضافه می گردد. ضمن جابجایی B به جمع کننده، دومین عدد از طریق ورودی

سریال وارد می‌شود. این عدد به محتوای ثبات A اضافه می‌شود، و در همان هنگام سومین عدد وارد ثبات می‌گردد. این کار می‌تواند برای تشکیل جمع دو، سه یا چند عدد و ذخیره حاصل جمع در ثبات A تکرار شود.

از مقایسه جمع‌کننده سریال با جمع‌کننده موازی (که در فصول قبل ارائه شد)، چندین تفاوت ملاحظه می‌گردد. جمع‌کننده موازی از ثبات‌های با امکان بار شدن موازی استفاده می‌کند، در حالی که جمع‌کننده سری شیفتر رجیسترها را به کار می‌برد. تعداد جمع‌کننده‌های مدار موازی برابر تعداد بیت‌های اعداد دودویی است، در صورتی که جمع‌کننده سریال از یک جمع‌کننده کامل و یک فلیپ‌فلاپ برای ذخیره نقلی و خروجی استفاده می‌نماید. دلیل ذخیره نقلی این است که در اعمال سریال نتیجه یک جمع بیتی در هر زمان نه تنها به ورودی‌های فعلی بلکه به ورودی‌های قبلی که باید در فلیپ‌فلاپ‌ها ذخیره شوند نیز بستگی دارد.

برای نمایش روش طراحی اعمال سریال با مدارهای ترتیبی دوباره جمع‌کننده سریال را با استفاده از جدول حالت طراحی می‌کنیم. ابتدا فرض می‌نماییم که دو شیفتر رجیستر برای ذخیره اعدادی که قرار است با هم به طور سریال جمع شود موجود باشد. خروجی‌های سریال از ثبات‌ها را x و y می‌نامیم. مدار ترتیبی مورد نظر در حال حاضر فاقد شیفتر رجیستر است ولی هنگام تکمیل آن به مدار اضافه خواهد شد. مدار ترتیبی دارای دو ورودی x و y است که دو بیت با ارزش‌تر دو عدد را برای مدار تهیه می‌کنند، یک خروجی S که بیت حاصل جمع را تولید می‌نماید و یک فلیپ‌فلاپ Q برای ذخیره رقم نقلی است. جدول حالتی که مدار ترتیبی را تعریف می‌کند در جدول شکل ۹-۱۰ نشان داده شده است. حالت فعلی Q مقدار فعلی نقلی است. نقلی فعلی در Q با ورودی‌های x و y جمع می‌شود تا بیت جمع را در خروجی S تولید نماید. حالت بعدی Q برابر نقلی فعلی است. توجه کنید که وارده‌های جدول حالت، با وارده‌های جدول جمع‌کننده کامل یکی است جز اینکه نقلی ورودی اکنون حالت فعلی Q ، و نقلی خروجی اکنون حالت بعدی آن است.

حالت فعلی	ورودی‌ها		حالت بعدی	خروجی	ورودی‌های فلیپ فلاپ‌ها	
	x	y			JQ	KQ
0	0	0	0	0	0	X
0	0	1	0	1	0	X
0	1	0	0	1	0	X
0	1	1	1	0	1	X
1	0	0	0	1	X	1
1	0	1	1	0	X	0
1	1	0	1	0	X	0

شکل ۹-۱۰: جدول حالت یک جمع‌کننده سریال

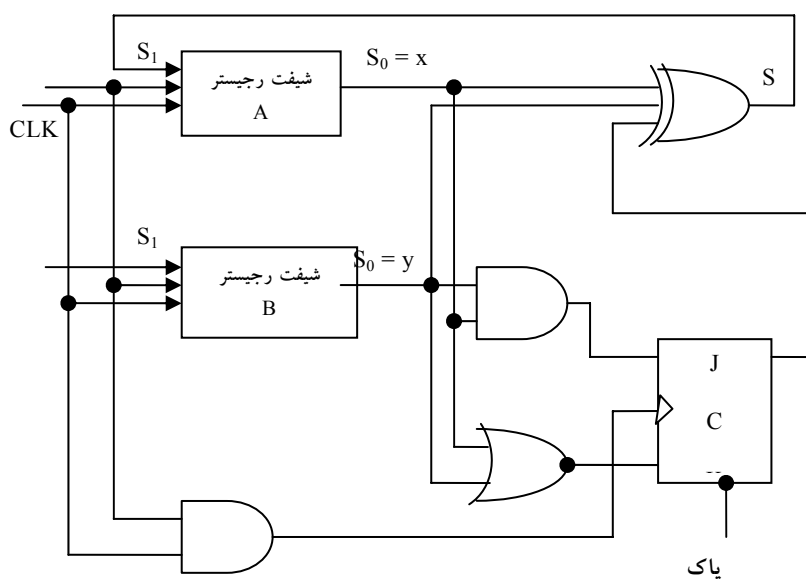
اگر از فلیپ‌فلاپ D برای Q استفاده شود مدار به شکل ۹-۹ کاهش می‌یابد. اگر برای Q از JK استفاده نماییم، لازم است مقادیر ورودی‌های J و K را با ارجاع به جدول تحریک فصل قبل مربوط به فلیپ‌فلاپ‌های JK معین کنیم. این کار در دو ستون آخر جدول ارائه شده در شکل ۹-۱۰ انجام شده است. دو معادله ورودی فلیپ‌فلاپ و معادله خروجی با نقشه به صورت زیر ساده می‌شوند.

$$JQ = xy$$

$$KQ = x'y' = (x+y)'$$

$$S = x \oplus y \oplus Q$$

نمودار مدار در شکل ۹-۱۱ نشان داده شده است. مدار متشکل از سه گیت و یک فلیپ‌فلاپ JK می‌باشد. برای تکمیل جمع‌کننده سریال دو شیفت رجیستر هم به آن اضافه شده است. توجه کنید که خروجی S نه فقط تابعی از x و y است، بلکه تابعی از Q هم می‌باشد. حالت بعدی Q تابعی از حالت فعلی Q و مقادیر x و y رسیده از خروجی‌های شیفت رجیسترهاست.



شکل ۹-۱۱: فرم دوم یک جمع کننده سریال

۹-۲-۵ شیفت رجیستر

- کنترل پاک برای پاک کردن ثبات به 0.
- ورودی ساعت برای همزمانی اعمال.
- کنترل جابجایی به راست برای فعال کردن عمل جابجایی به راست و خطوط ورودی و خروجی سرپال مربوط به جابجایی به راست.

ثباتها و شمارنده‌ها ۲۹۵

- کنترل جابجایی به چپ برای فعال کردن عمل جابجایی به چپ و خطوط ورودی و خروجی سریال مربوط به جابجایی به چپ.
- یک کنترل بارکردن موازی برای فعال کردن انتقال موازی و n خط ورودی مربوط به انتقال موازی.
- n خط خروجی موازی.
- حالت کنترلی که علیرغم وجود پالس ساعت اطلاعات را در ثبات بدون تغییر نگه می‌دارد.

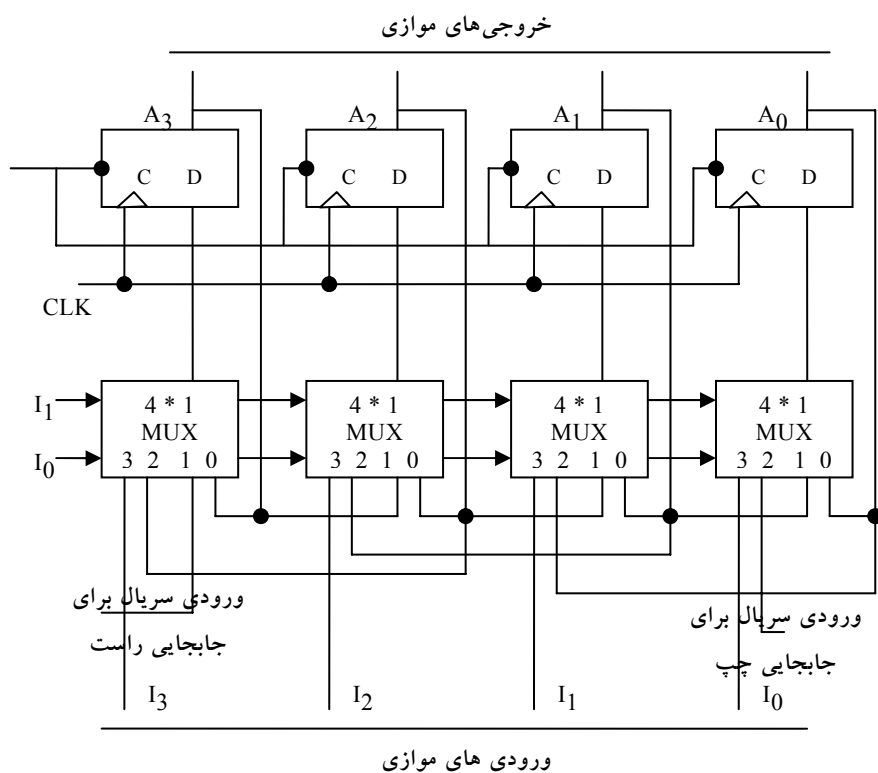
۹-۲-۶ انواع شیفت رجیسترها

دیگر شیفت رجیسترها ممکن است بعضی از امکانات فوق را با حداقل یک عمل جابجایی یا شیفت داشته باشد. انواع شیفت رجیسترها عبارتند از:

- یک جهته
- دو جهته
- یونیورسال

ثباتی که فقط قادر به جابجایی داده در یک جهت باشد را شیفت رجیستر یک جهته گویند. اگر در دو جهت جابجا نماید آنرا شیفت رجیستر دو جهته می‌نامند. اگر ثبات قادر به جابجایی دو جهته و بار شدن موازی باشد به آن شیفت رجیستر یونیورسال گویند.

نمودار یک شیفت رجیستر ۴ بیت یونیورسال با همه فلیپ‌فلاپ‌های فوق‌الذکر در شکل ۹-۱۲ نشان داده شده است. این مدار از چهار فلیپ‌فلاپ D و چهار مولتی‌پلکسر ساخته شده است. چهار مولتی‌پلکسر دو ورودی انتخاب مشترک S_1 و S_0 دارند. ورودی ۰ در هر مولتی‌پلکسر وقتی انتخاب می‌شود که $S_1S_0 = 00$ باشد، ورودی ۱ با $S_1S_0 = 01$ انتخاب می‌گردد، و به طور مشابه دو ورودی باقی مانده انتخاب می‌گردند.



شکل ۹-۱۲: شیفت رجیستر یونیورسال ۴ بیتی

ورودی‌های انتخاب مد عملیات ثبات را طبق وارده‌های جدول شکل ۹-۱۳ کنترل می‌کنند. وقتی $S_1S_0 = 00$ است، مقدار فعلی ثبات به ورودی‌های D از فلیپ‌فلاپ‌ها اعمال می‌گردد. این وضعیت مسیری را از خروجی هر فلیپ‌فلاپ به ورودی‌اش ایجاد می‌نماید. لبه ساعت بعدی، مقداری را که از قبل در آن ذخیره شده وارد فلیپ‌فلاپ می‌کند و بنابراین هیچ تغییری در حالت رخ نمی‌دهد. وقتی $S_1S_0 = 01$ است، پایه ورودی 1 از مولتی‌پلکسر دارای مسیری به ورودی‌های D فلیپ‌فلاپ‌هاست. این موجب عمل جابجایی به راست می‌گردد، که در آن ورودی سریال به فلیپ‌فلاپ A_3 وارد می‌شود. وقتی $S_1S_0 = 10$ است، یک عمل جابجایی به چپ صورت می‌گیرد و طی آن دیگر ورودی به فلیپ‌فلاپ A_0 خواهد رفت. بالاخره وقتی $S_1S_0 = 11$ است،

ثباتها و شمارنده‌ها ۲۹۷

اطلاعات دودویی روی خطوط ورودی موازی به طور همزمان در لبه پالس بعدی وارد ثبات می‌گردند.

وضعیت کنترل		عملکرد ثبات
S_1	S_0	
0	0	بلا تغییر
0	1	شیفت - راست
1	0	شیفت - چپ
1	1	بار کردن موازی

شکل ۹-۱۳ جدول عملکرد ثبات شکل ۹-۱۲

شیفت رجیسترها اغلب برای اتصال و ارتباط سیستم‌های دیجیتالی که با فواصل دوری از یکدیگر قرار دارند به کار می‌روند. مثلاً فرض کنید که بخواهیم کمیتی n بیتی را بین دو نقطه جابجا کنیم. اگر فاصله زیاد باشد، استفاده از n خط موازی گران تمام می‌شود. استفاده از یک خط و انتقال سریال اطلاعات به صورت یک بیت در هر بار اقتصادی تر است. فرستنده داده n بیتی را به صورت موازی وارد شیفت رجیستر کرده و داده را به صورت سریال در طول خط ارسال می‌دارد. گیرنده داده را به طور سریال وارد یک شیفت رجیستر می‌نماید. وقتی هر n بیت دریافت شد، می‌توان از خروجی‌های ثبات آنها را به صورت موازی دریافت کرد. بنابراین فرستنده یک عمل تبدیل موازی به سریال داده و گیرنده یک عمل تبدیل سریال به موازی را انجام می‌دهد.

۹-۳ شمارنده‌های موج‌گونه

ثباتی که بر اساس اعمال پالس‌های ورودی وارد رشته حالات از پیش تعیین شده‌ای می‌گردد، شمارنده نام دارد. پالس‌های ورودی ممکن است پالس‌های ساعت و یا از یک

منبع بیرونی با توالی ثابت و یا متغیر باشند. رشته حالات ممکن است رشته اعداد دودویی و یا رشته حالات دیگری باشد. شمارنده ای که رشته اعداد دودویی را دنبال می‌کند، شمارنده دودویی نامیده می‌شود. یک شمارنده n بیتی متشکل از n فلیپ‌فلاپ بوده و می‌تواند از ۰ تا $2^n - 1$ را بشمارد. شمارنده‌ها به دو صورت وجود دارند:

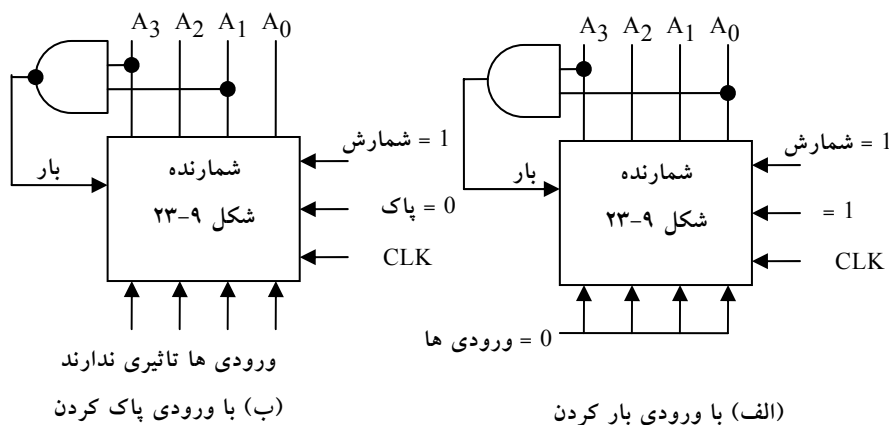
- شمارنده‌های موج گونه
- شمارنده‌های همزمان

در یک شمارنده موج گونه، تغییر وضعیت خروجی فلیپ‌فلاپ به عنوان منبع تریگر کردن دیگر فلیپ‌فلاپ‌ها عمل می‌کند. به بیان دیگر، ورودی C بعضی از و یا همه فلیپ‌فلاپ‌ها با پالس‌های ساعت مشترکی تریگر یا راه اندازی نمی‌شوند. بر عکس در شمارنده همزمان ورودی‌های C همه فلیپ‌فلاپ‌ها ساعت مشترکی، را دریافت می‌نمایند. در اینجا شمارنده‌های موج گونه BCD و دودویی را ارائه کرده و نحوه کار آنها را توضیح می‌دهیم.

۹-۳-۱ شمارنده موج گونه دودویی

یک شمارنده موج گونه دودویی از یک سری اتصال بین فلیپ‌فلاپ‌های متمم ساز تشکیل شده است. که خروجی هر فلیپ‌فلاپ به ورودی C فلیپ‌فلاپ مرتبه بالاتر وصل است. فلیپ‌فلاپی که کم ارزش‌ترین بیت را نگه می‌دارد، پالس‌های مورد شمارش را دریافت می‌کند. فلیپ‌فلاپ متمم ساز را می‌توان با یک فلیپ‌فلاپ JK که در آن J و K به هم وصل اند و یا از یک فلیپ‌فلاپ T ساخت. سومین امکان استفاده از فلیپ‌فلاپ D است که در آن خروجی متمم به ورودی D وصل است. به این ترتیب، ورودی D همواره متمم حالت فعلی بوده و پالس ساعت بعدی موجب متمم شدن خروجی اصلی آن خواهد شد. نمودار منطقی دو شمارنده دودویی ۴ بیت در شکل ۹-۱۴ نشان داده شده است. شمارنده با فلیپ‌فلاپ‌های متمم ساز نوع T در بخش (الف) و نوع D در بخش (ب) ساخته شده است. خروجی هر فلیپ‌فلاپ به

ورودی فلیپ‌فلاپ بعدی در رشته متصل است. همانطور که گفته شد فلیپ‌فلاپی که کم ارزش‌ترین بیت را نگه می‌دارد پالس‌های شمارش را دریافت می‌کند. ورودی‌های T همه فلیپ‌فلاپ‌ها در (الف) به طور دایم به منطق 1 متصل‌اند. این شرایط موجب می‌شود تا با گذر منفی در ورودی C فلیپ‌فلاپ متمم شود. حباب جلوی نشانه گر دینامیک (*) در کنار C به این معنی است که فلیپ‌فلاپ‌ها به لبه منفی ورودی واکنش نشان می‌دهند. گذر منفی هنگامی رخ می‌دهد که خروجی فلیپ‌فلاپ قبل که به C وصل است از 1 به 0 برود.



شکل ۹-۱۴: شمارنده موج گونه دودویی 4 بیتی

برای درک عملکرد شمارنده دودویی 4 بیت، به 9 عدد دودویی اول در جدول شکل ۹-۱۵ مراجعه کنید. شمارش از 0 دودویی شروع و با هر پالس در ورودی افزایش می‌یابد. پس از شماره 15 شمارنده برای تکرار به 0 باز می‌گردد. بیت کم ارزش‌تر A_0 با هر پالس شمارش ورودی متمم می‌شود. هر بار که A_0 از 1 به 0 برود، A_1 را متمم می‌سازد. هر بار که A_1 از 1 به 0 برود، A_2 را متمم می‌نماید. هر بار که A_2 از 1 به 0 برود، A_3 را متمم می‌کند. و به همین ترتیب بیت‌های بالاتر در شمارنده موج گونه تغییر می‌کنند. به عنوان مثال، گذر از 0011 به 0100 را در نظر بگیرید. A_0 با پالس ساعت

متمم می‌شود. چون A_0 از 1 به 0 می‌رود، A_1 تریگر شده و متمم می‌گردد. در نتیجه A_1 از 1 به 0 می‌رود که به نوبه خود موجب متمم شدن A_2 گشته و آن را از 0 به 1 خواهد برد. A_2 نمی‌تواند A_3 را تریگر کند زیرا A_2 یک گذر مثبت را تولید می‌کند و فلیپ‌فلاپ هم تنها به گذر منفی واکنش نشان می‌دهد.

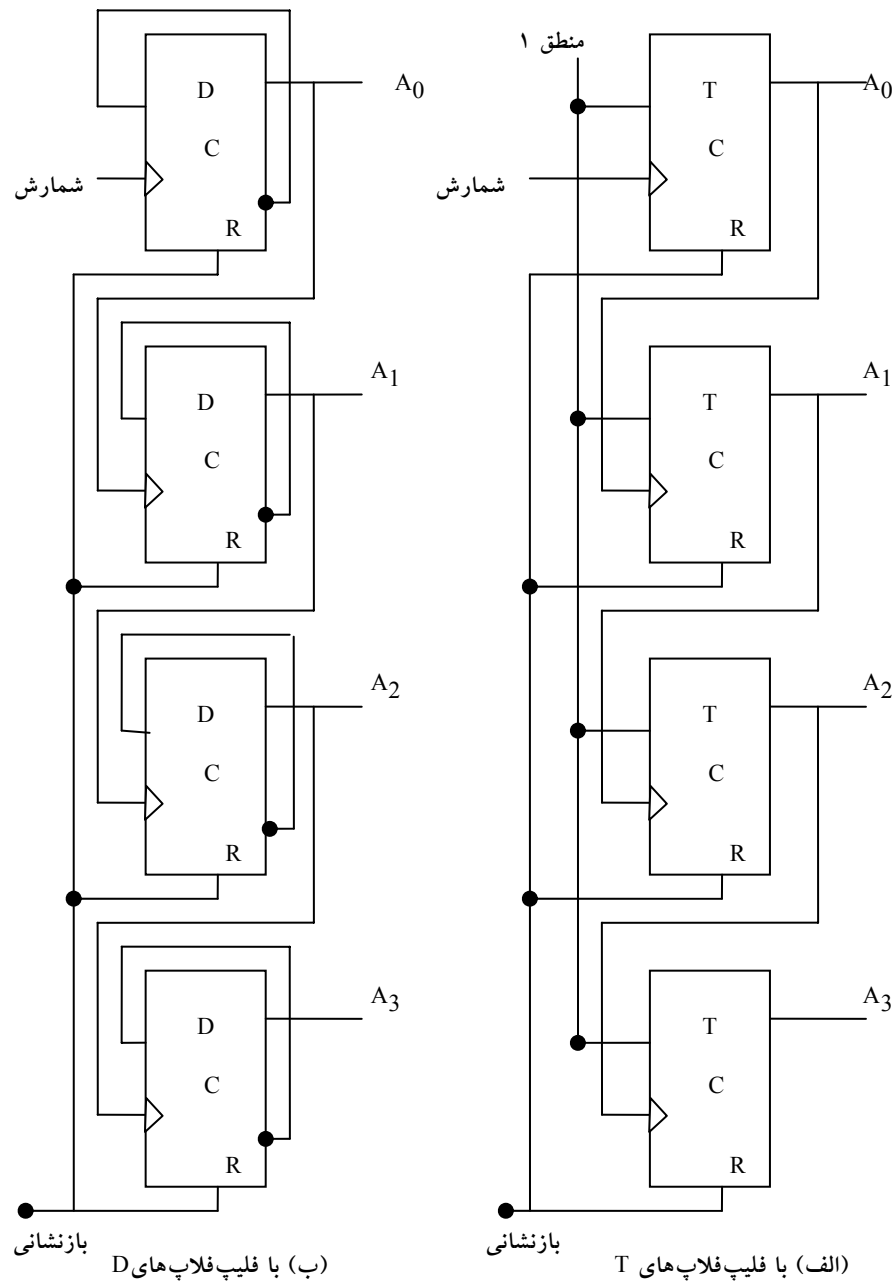
A_3	A_2	A_1	A_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0

شکل ۹-۱۵: جدول نمایانگر رشته شمارش دودویی

بنابراین شمارش از 0011 به 0100 با تغییر نوبتی بیت‌ها رخ می‌دهد، به طوری که شمارنده از 0011 به 0010، سپس به 0000 و بالاخره به 0100 خواهد رفت. هر بار یکی از فلیپ‌فلاپ‌ها تغییر کرده و تغییر به پیش می‌رود و انتشار سیگنال در شمارنده از یک طبقه به طبقه دیگر مثل حرکت موج می‌ماند. برای درک بهتر موضوع به شمارنده ارائه شده در شکل ۹-۱۶ توجه نمایید.

یک شمارنده دودویی با شمارش معکوس را پایین شمار گویند. در پایین شمار، شمارش با هر ورودی پالس شمارش، یک واحد کم می‌شد. شمارش یک پایین شمار 4 بیت از 15 شروع و به صورت 14، 13، 12،، 0 پایان یافته و سپس به 15 باز می‌گردد. لیستی از شمارش یک شمارنده پایین شمار نشان می‌دهد که کم ارزش‌ترین بیت با هر پالس شمارش متمم شده است. هر بیت دیگر در رشته، اگر بیت کم ارزش‌تر قبل از آن از 0 به 1 برود، متمم می‌گردد. بنابراین نمودار یک پایین شمار مشابه شکل ۹-۱۶ خواهند بود، به شرطی که همه فلیپ‌فلاپ‌ها با لبه مثبت ساعت تریگر شوند. (حباب در

ثباتها و شمارنده‌ها ۳۰۱

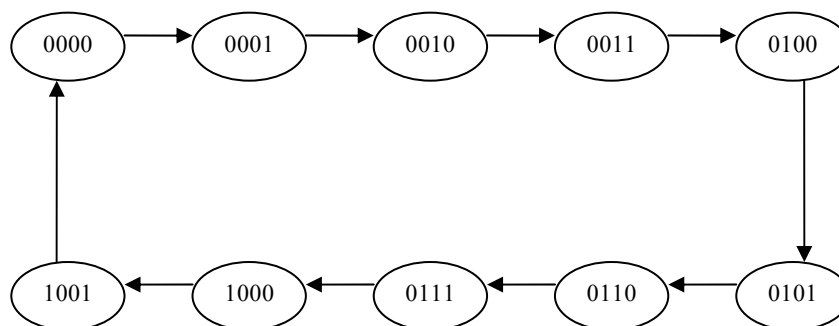


شکل ۹-۱۶: شمارنده موج گونه دودویی ۴ بیتی با فلیپ‌فلاپ‌های D و T

ورودی C باید حذف شود). اگر از فلیپ‌فلاپ‌های حساس به لبه منفی استفاده شود، آنگاه ورودی C هر فلیپ‌فلاپ باید به خروجی متمم فلیپ‌فلاپ قبلی وصل گردد. آنگاه وقتی که خروجی غیر متمم از 0 به 1 برود، متمم از 1 به 0 رفته و فلیپ‌فلاپ بعدی را آنطور که باید متمم خواهد کرد.

۹-۳-۲ شمارنده BCD موج گونه

یک شمارنده دهدهی رشته‌ای از ده حالت را دنبال کرده و پس از 9 به 0 باز می‌گردد.



شکل ۹-۱۷: نمودار حالت یک شمارنده دهدهی BCD

چنین شمارنده‌ای باید حداقل چهار فلیپ‌فلاپ برای نمایش هر رقم دهدهی داشته باشد، زیرا یک رقم دهدهی با کد چهار بیتی نشان داده می‌شود. رشته حالات در شمارنده دهدهی به وسیله کد دودویی مربوطه برای نمایش هر رقم دیجیتال معین می‌گردد اگر BCD به کار رود، رشته حالات مطابق نمودار حالت شکل ۹-۱۷ خواهد بود. این جدول مشابه با جدول دودویی است. به جز اینکه پس از 1001 برای عدد دهدهی 9، 0000 را برای رقم دهدهی 0 خواهیم داشت.

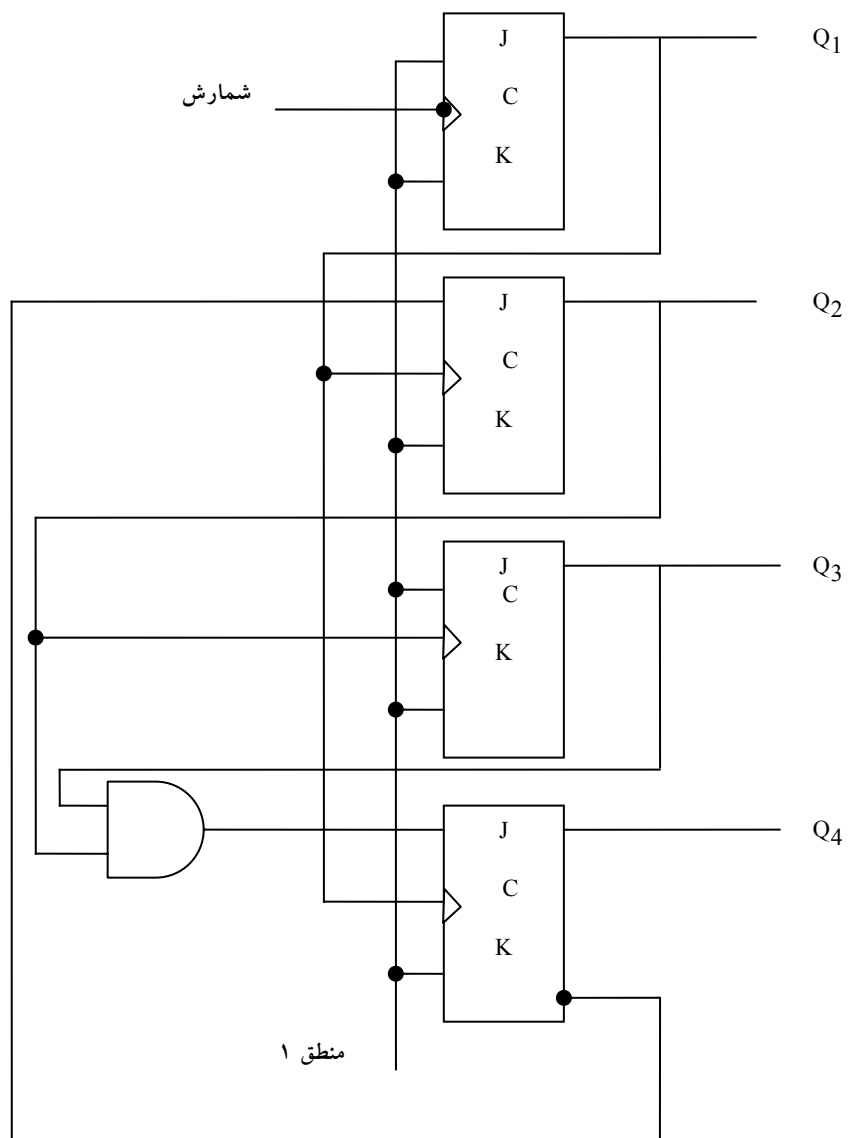
نمودار منطقی یک شمارنده BCD موج گونه با استفاده از فلیپ‌فلاپ JK در شکل ۹-۱۸ دیده می‌شود. چهار خروجی با حروف Q و اندیسی در زیر آن برای مشخص کردن وزن آن در BCD علامت گذاری شده است. توجه کنید که خروجی Q₁ به ورودیهای C در هر دو ورودی Q₂ و Q₃ اعمال شده است و خروجی Q₂ هم به

ثباتها و شمارنده‌ها ۳۰۳

ورودی C از Q_4 وصل است. ورودی‌های J و K یا دائما به 1 وصلند و یا به خروجی فلیپ‌فلاپ‌های دیگر وصل شده‌اند. شمارنده موج گونه یک مدار غیر همزمان است. سیگنال‌ها بسته به ترتیبی که در آن از 1 به 0 می‌روند، روی فلیپ‌فلاپ‌ها اثر می‌گذارند. عمل یک شمارنده با لیستی از حالات گذر هر فلیپ‌فلاپ قابل تفسیر است. این حالات از نمودار منطقی و دانستن چگونگی عملکرد یک فلیپ‌فلاپ JK حاصل می‌شود. به خاطر بسپارید وقتی که ورودی C از 1 به 0 می‌رود، اگر $J = 1$ باشد، فلیپ‌فلاپ 1 می‌شود و اگر $K = 1$ باشد به 0 پاک می‌گردد و نیز اگر $J = K = 1$ باشد متمم شده و بلاخره با $J = K = 0$ حالت فلیپ‌فلاپ بی تغییر خواهد بود.

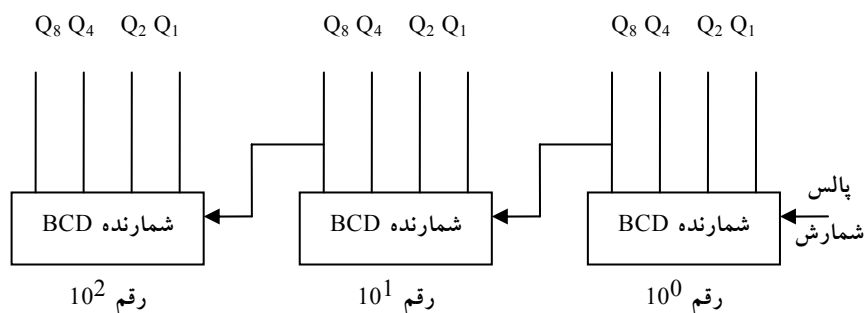
برای تحقیق و اطمینان از اینکه این حالات به ترتیب در شمارنده BCD رخ می‌دهند باید مطمئن شویم که گذر حالات فلیپ‌فلاپ‌ها رشته ای را که به وسیله نمودار حالت شکل ۹-۱۷ مشخص شده دنبال می‌کند. حالت Q_1 پس از هر پالس ساعت عوض می‌شود. هر بار Q_1 از 1 به 0 برود و $Q_3 = 0$ باشد Q_2 متمم می‌شود. وقتی $Q_3 = 1$ شود، Q_2 در 0 می‌ماند. هر بار Q_2 از 1 به 0 برود Q_4 متمم می‌گردد. مادامی که Q_2 و Q_4 در 0 باشند، Q_3 در 0 خواهد ماند. وقتی هر دو Q_2 و Q_4 برابر 1 شوند، با تغییر 1 به 0 خروجی Q_1 ، خروجی Q_2 متمم می‌شود. با گذر بعدی Q_1 ، Q_3 پاک می‌شود.

شمارنده BCD شکل ۹-۱۸ یک شمارنده دهدهی است، زیرا از 0 تا 9 می‌شمارد. برای شمارش از 0 تا 99 دو شمارنده دهدهی لازم داریم. شمارش 0 تا 999 سه شمارنده دهدهی لازم دارد. شمارنده‌های دهدهی چند رقمی با اتصال سری شمارنده‌های BCD ساخته می‌شوند که هر کدام برای یک دهه است. یک شمارنده دهدهی سه رقمی در شکل ۹-۱۹ دیده می‌شود. ورودی‌ها به دومین و سومین دهه از Q_3 دهه قبل وارد می‌شوند. وقتی Q_3 در یک دهه از 1 به 0 می‌رود، شمارنده دهه بالاتر را تریگر می‌کند، ضمن اینکه خودش از 9 به 0 باز می‌گردد.



شکل ۹-۱۸: شمارنده موج گونه BCD

ثباتها و شمارنده‌ها ۳۰۵



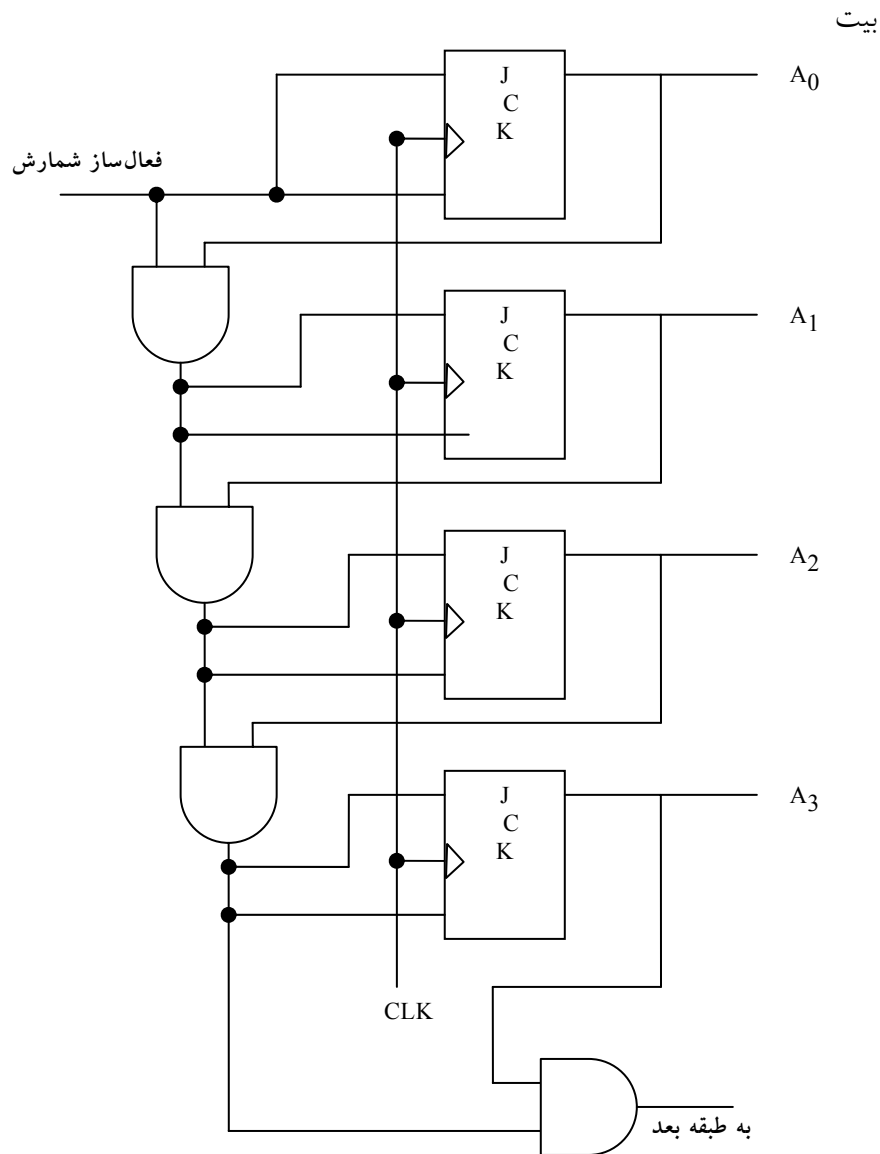
شکل ۹-۱۹: نمودار بلوکی یک شمارنده دهدهی BCD با سه دهه

۹-۴ شمارنده‌های همزمان

تاکنون مباحث مختلفی در خصوص شمارنده‌های موج گونه مطرح شد. دسته دوم شمارنده‌ها، عبارتند از شمارنده‌های همزمان. شمارنده‌های همزمان در اعمال پالس ساعت به ورودی فلیپ‌فلاپ‌ها با شمارنده‌های موج گونه تفاوت دارند. یک ساعت مشترک همه فلیپ‌فلاپ‌ها را به طور همزمان تریگر می‌کند در صورتی که در نوع شمارنده‌های موج گونه هر بار فقط یکی از فلیپ‌فلاپ‌ها تریگر می‌شود. تصمیم بر متهم شدن یک فلیپ‌فلاپ از مقادیر داده‌های ورودی مانند T و J و K در لبه ساعت معین می‌شود. اگر $T=0$ یا $J=K=0$ باشد، حالت فلیپ‌فلاپ تغییر نمی‌نماید. اگر $T=1$ یا $J=K=1$ باشد، فلیپ‌فلاپ متهم می‌گردد.

۹-۴-۱ شمارنده دودویی

در شمارنده دودویی همزمان، فلیپ‌فلاپ واقع در کم ارزش‌ترین مکان با هر پالس یکبار متهم می‌شود. فلیپ‌فلاپ‌های واقع در هر مکان هنگامی متهم می‌شود که همه فلیپ‌فلاپ‌های پایین‌تر 1 باشند. مثلاً اگر حالت فعلی یک شمارنده 4



شکل ۹-۲۰: شمارنده دودویی همزمان ۴ بیتی

$A_3A_2A_1A_0 = 0011$ باشد، شماره بعدی 0100 خواهد بود. لازم به یاد آوری است که در مثال فوق A_0 مرتباً متمم می‌شود. A_1 هنگامی متمم می‌گردد که A_0 برابر 1 باشد. A_2 هنگامی 1 می‌شود که $A_1A_0 = 11$ باشد. با این وجود A_3 متمم نمی‌شود زیرا حالت فعلی

ثباتها و شمارنده‌ها ۳۰۷

$A_2A_1A_0 = 011$ است، چون حالت تمام 1 وجود ندارد. شمارنده‌های دودویی همزمان الگوی منظمی دارند و می‌توان آنها را با متمم کردن فلیپ‌فلاپ‌ها و گیت‌ها ساخت. نظم الگو را می‌توان با توجه به شکل ۹-۲۰ ملاحظه کرد. ورودی‌های C همه فلیپ‌فلاپ‌ها به ساعت مشترکی وصل‌اند. شمارنده با ورودی فعال‌ساز شمارش، فعال می‌گردد. اگر ورودی فعال‌ساز 0 باشد، ورودی همه J ها و K ها برابر 0 خواهند بود و بنابراین ساعت قادر نخواهد بود حالت شمارنده را عوض کند. در اولین طبقه، A_0 ، اگر شمارنده فعال شود $J=K=1$ خواهد بود. در دیگر طبقات، J ها و K ها به شرطی 1 هستند که همه طبقات کم ارزش‌تر آنها برابر 1 و ورودی شمارش هم فعال شده باشد. در هر طبقه، زنجیره گیت‌های AND منطق لازم را برای ورودی‌های J و K فراهم می‌کنند. شمارنده را می‌توان به هر تعداد از طبقات گسترش داد که در آن هر طبقه یک گیت AND و یک فلیپ‌فلاپ اضافی خواهد داشت و هرگاه همه فلیپ‌فلاپ‌های طبقات قبل 1 شوند خروجی AND برابر با 1 خواهد بود.

توجه داشته باشید که فلیپ‌فلاپ‌ها در لبه مثبت ساعت تریگر می‌شوند. قطبیت ساعت، آنطور که در شمارنده‌های موج گونه مهم بود، در اینجا اهمیت ندارد. شمارنده همزمان با هر یک از دو لبه مثبت یا منفی پالس ساعت تریگر می‌گردد. فلیپ‌فلاپ‌های متمم ساز در یک شمارنده دودویی می‌توانند از نوع JK، T یا D با گیت‌های XOR باشند.

۹-۴-۲ شمارنده BCD

یک شمارنده BCD دهدهی کد شده به دودویی از 0000 تا 1001 و بعد 0000 می‌شمارد. به دلیل بازگشت از 9 به 0، یک شمارنده BCD دارای الگوی منظمی همچون شمارنده دودویی نیست. برای راه اندازی مدار یک شمارنده همزمان BCD، لازم است از روال طراحی یک مدار ترتیبی استفاده شود.

جدول حالت یک شمارنده BCD در جدول ارائه شده در شکل ۹-۲۱ لیست شده است. وضعیت ورودی فلیپ‌فلاپ‌های T از حالت فعلی و بعدی به دست می‌آیند. یک خروجی y هم در جدول دیده می‌شود. وقتی حالت فعلی 1001 باشد این خروجی برابر 1 است. به این ترتیب y می‌تواند شمارش دهه با ارزش‌تر بعدی را فعال کرده و به طور همزمان از 1001 به 0000 برود.

حالت فعلی				حالت بعدی				خروجی	ورودی‌های فلیپ‌فلاپ			
Q ₈	Q ₄	Q ₂	Q ₁	Q ₈	Q ₄	Q ₂	Q ₁	y	TQ ₈	TQ ₄	TQ ₂	TQ ₁
0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	0	1
0	0	1	1	0	1	0	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	0	1
0	1	0	1	0	1	1	0	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	0	1
0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	0	1
1	0	0	1	0	0	0	0	1	1	0	0	1

شکل ۹-۲۱: جدول حالت یک شمارنده BCD

معادلات ورودی فلیپ‌فلاپ‌ها را می‌توان به کمک نقشه ساده کرد. حالات بی استفاده برای مینترم‌های 10 الی 15 جملات بی‌اهمیت تلقی می‌شوند. توابع ساده شده عبارتند از:

$$TQ_1 = 1$$

$$TQ_2 = Q_8'Q_1$$

$$TQ_4 = Q_2Q_1$$

$$TQ_8 = Q_8Q_1 + Q_4Q_2Q_1$$

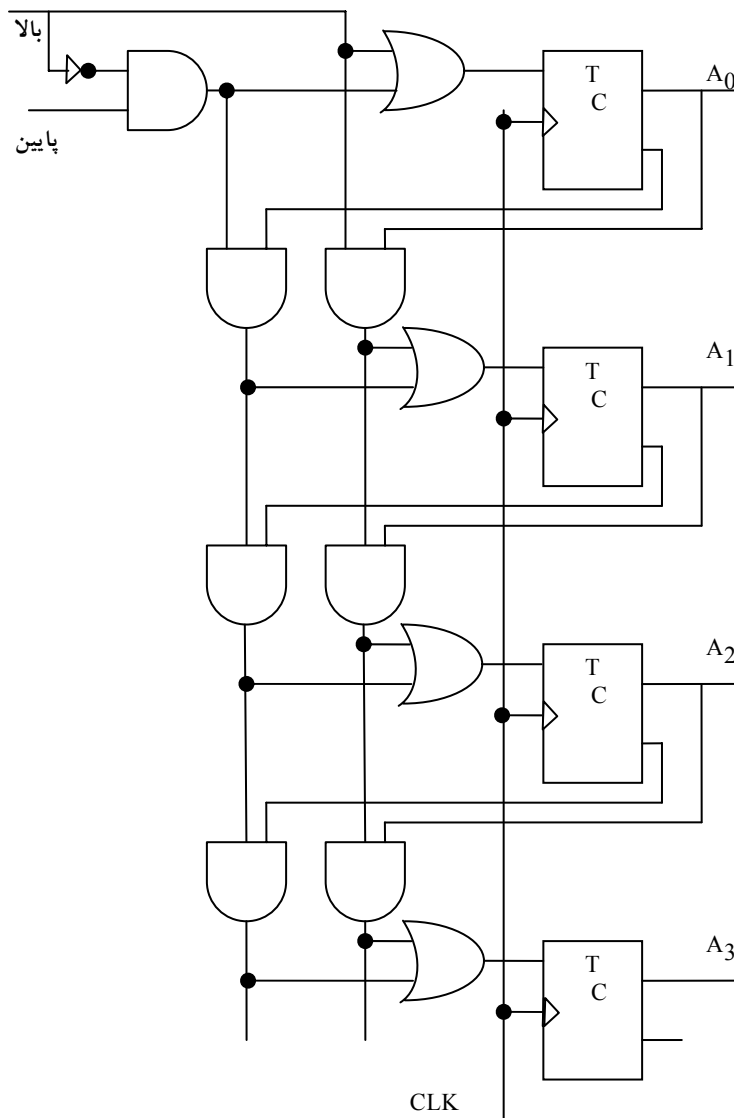
$$y = Q_8Q_1$$

ثباتها و شمارنده‌ها ۳۰۹

می‌توان به سادگی با چهار فلیپ‌فلاپ T و پنج گیت AND و یک OR طراحی کرد. شمارنده‌های BCD همزمان را می‌توان با شمارش اعداد دهدهی با هر طول به صورت متوالی به یکدیگر متصل کرد. این نوع سری‌سازی در شکل ۹-۱۹ دیده شد، با این تفاوت که خروجی y باید به ورودی شمارش دهه با ارزش‌تر بعدی وصل گردد.

۹-۴-۳- پایین شمار دودویی

یک شمارنده پایین شمار دودویی همزمان وارد حالات معکوسی از 1111 به سمت 0000 و سپس به 1111 می‌شود. تا شمارش را تکرار کند. می‌توان شمارنده پایین شماری به روش معمول ساخت، ولی نتایج از واریسی شمارش دودویی پایین شمار قابل پیش بینی هست. به این ترتیب که بیت مکان کم ارزش‌تر با هر پالس متمم می‌شود. هر بیت در هر مکان دیگر اگر همه بیت‌های کم ارزش‌تر 0 باشند، متمم می‌گردد. مثلاً پس از حالت فعلی 0100، حالت بعدی 0011 قرار دارد. کم ارزش‌ترین بیت همواره متمم می‌گردد. بیت با ارزش‌تر دوم چون بیت اول 0 است، متمم می‌شود. سومین بیت متمم می‌شود زیرا دو بیت اول برابر 0 اند. ولی چهارمین بیت تغییر نمی‌کند چون همه بیت‌های پایین رتبه 0 نیستند. یک شمارنده پایین شمار می‌تواند مشابه شکل ۹-۲۰ ساخته شود، با این تفاوت که ورودی گیت‌ها از خروجی‌های متمم فلیپ‌فلاپ‌های قبلی می‌آیند. می‌توان دو عمل بالا و پایین شمار را با هم ترکیب کرد و به این ترتیب بالا-پایین شمار ساخت. مدار چنین شمارنده‌ای که از فلیپ‌فلاپ‌های T استفاده می‌کند در شکل ۹-۲۲ آمده است.



شکل ۹-۲۲: شمارنده دودویی چهار بیتی، بالا-پایین شمار

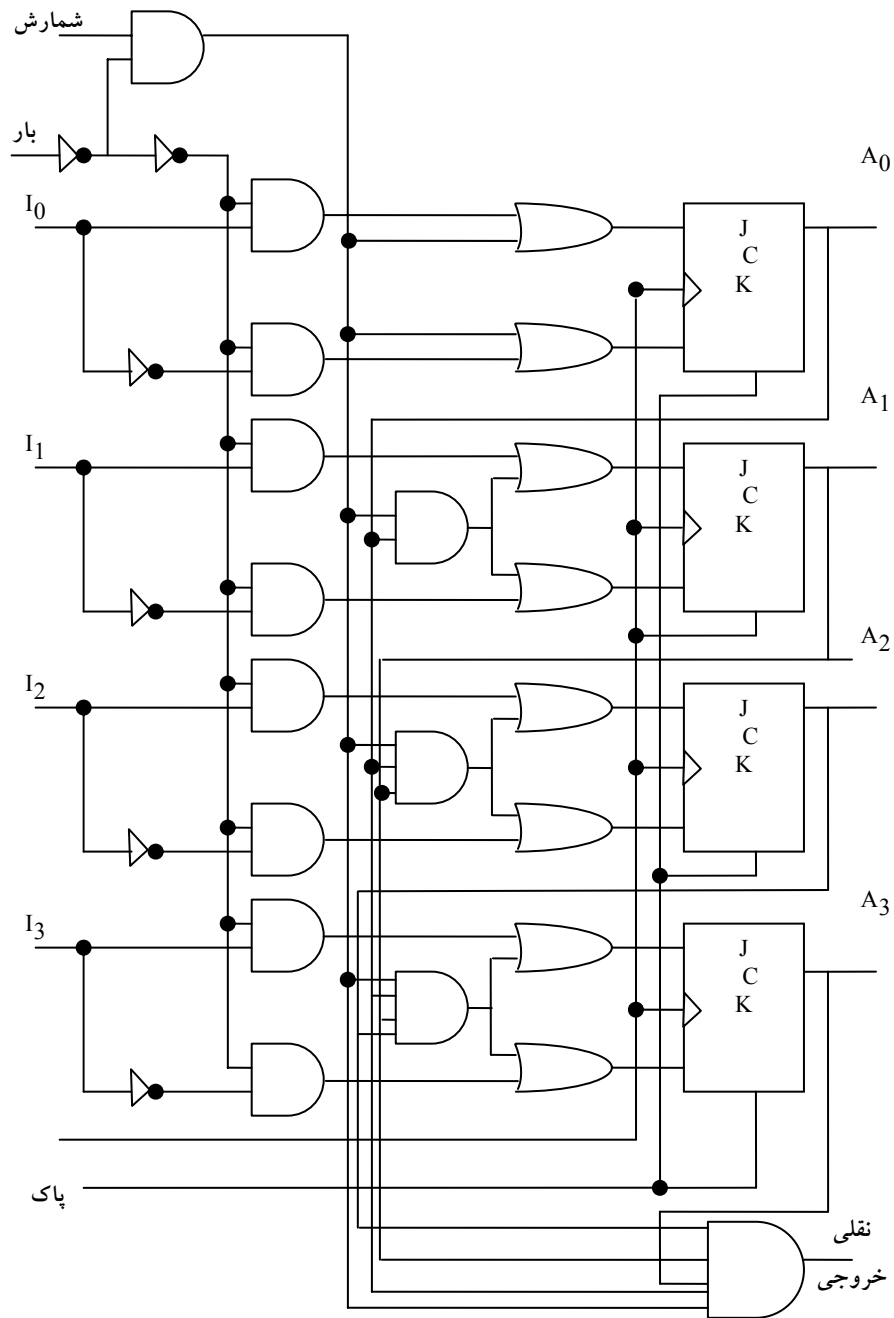
این مدار دارای یک ورودی کنترل بالا و یک ورودی کنترل پایین است. وقتی ورودی بالا برابر 1 است، مدار رو به بالا می‌شمارد، زیرا ورودی‌های T سیگنال‌های خود را از مقادیر خروجی نرمال فلیپ‌فلاپ‌ها دریافت می‌کنند. وقتی ورودی پایین

برابر 1 است ورودی بالا برابر 0 است، مدار رو به پایین می‌شمارد زیرا خروجی‌های متمم شده فلیپ‌فلاپ‌های قبلی به ورودی‌های T اعمال شده‌اند. وقتی هر دو مقدار بالا و پایین 0 باشند، مدار تغییر نکرده و شماره ثابت می‌ماند. وقتی هر دو ورودی بالا و پایین 1 باشند، مدار رو به بالا می‌شمارد. این موجب می‌شود تا همیشه تنها یک عمل اجرا گردد.

۹-۴-۴ شمارنده دودویی با بار شدن موازی

اغلب شمارنده‌های که در سیستم‌های دیجیتال به کار می‌روند نیاز به قابلیت انتقال موازی یک عدد دودویی اولیه به داخل شمارنده، قبل از شروع دارند. شکل ۹-۲۳ نمودار منطقی یک ثبات 4 بیت را نشان می‌دهد، که قابلیت بار شدن موازی دارد و می‌تواند به عنوان یک شمارنده به کار رود. اگر ورودی کنترل بار شدن در وضعیت 1 باشد، عمل شمارش را غیر فعال می‌کند و موجب انتقال داده از چهار ورودی داده به چهار فلیپ‌فلاپ می‌گردد.

اگر هر دو ورودی کنترل 0 باشد، پالس‌های ساعت حالت ثبات را عوض نمی‌کند. ضمن فعال بودن ورودی شمارش، اگر همه فلیپ‌فلاپ‌ها در 1 باشند، خروجی نقلی 1 می‌شود. این حالتی است که طی آن فلیپ‌فلاپ بیت با ارزش‌تر بعدی متمم می‌گردد. خروجی نقلی برای گسترش شمارنده به بیش از چهار بیت نیز مفید است. هنگام تولید مستقیم نقلی خروجی به دلیل کاهش تاخیر آن سرعت شمارنده افزایش می‌یابد. در رفتن از 1111 به 0000، تنها یک گیت تاخیر وجود دارد، در صورتی که در زنجیره گیت AND شکل ۹-۲۳، چهار گیت تاخیر موجود است.



شکل ۹-۲۳: شمارنده دودویی ۴ بیتی با امکان بار شدن موازی

ثباتها و شمارنده‌ها ۳۱۳

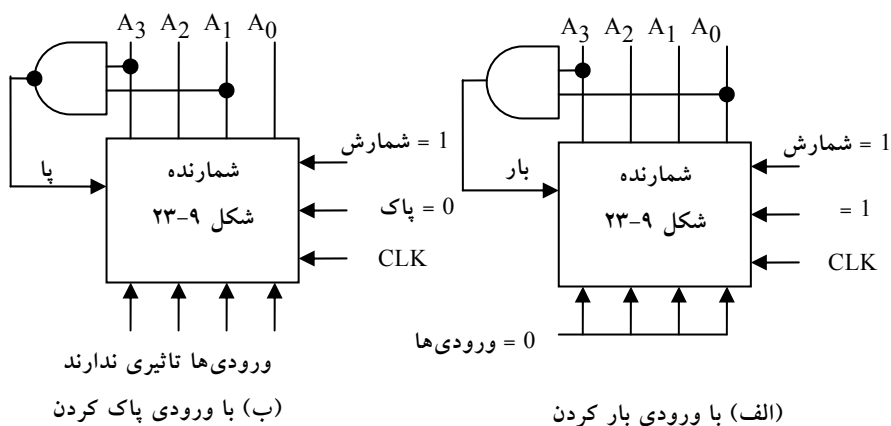
به طور مشابه هر فلیپ‌فلاپ به گیت AND مرتبط است که مستقیماً خروجی همه فلیپ‌فلاپ‌ها را دریافت می‌کند.

عملکرد این شمارنده در جدول شکل ۹-۲۴ خلاصه شده است. چهار ورودی کنترل، یعنی پاک، CLK، بار و شمارش حالت بعدی را معین می‌کنند. ورودی پاک غیر همزمان است و هر وقت 0 شود بدون توجه به وجود پالس ساعت یا دیگر ورودی‌ها، شمارنده را پاک می‌کند. این مطلب با وارده X در جدول ذکر شده که به معنی حالات بی‌اهمیت برای ورودی‌هاست. در دیگر حالات ورودی پاک در 1 قرار دارد. با قرار داشتن ورودی‌های بار و شمارش در 0، خروجی‌ها عوض نمی‌شوند. با ورودی بار در 1، انتقال از I_0-I_3 به ثبات در لبه مثبت پالس ساعت انجام می‌شود. مستقل از مقدار ورودی شمارش، داده ورودی در ثبات بار می‌شود، زیرا ورودی شمارش با فعال شدن ورودی بار، غیر فعال می‌شود. اگر ورودی شمارش کنترل شده باشد، ورودی بار در 0 خواهد بود.

تابع	شمارش	بار	CLK	پاک کننده
به 0 پاک می‌شود	X	X	X	0
ورودی بار شونده	X	1	↑	1
شمارش حالت دودویی بعدی	1	0	↑	1
بلا تغییر	0	0	↑	1

شکل ۹-۲۴: جدول عملکرد شمارنده شکل ۹-۲۳

یک شمارنده با بار شدن موازی را می‌توان برای تولید هر رشته شمارش مورد نظر به کار برد. شکل ۹-۲۵ دو راه تولید شمارش BCD با بار شدن موازی را نشان می‌دهد. در هر حال، برای فعال کردن شمردن از طریق ورودی CLK، کنترل شمارش در 1 قرار داده می‌شود. همچنین به خاطر بسپارید که کنترل بار از شمردن جلوگیری می‌کند و عمل پاک مستقل از دیگر ورودی‌های کنترل است.



شکل ۹-۲۵: دو روش دسترسی به شمارنده BCD با امکان بار شدن موازی

گیت AND در شکل ۹-۲۵ (الف) وقوع حالت 1001 را شناسایی می‌کند. شمارنده در آغاز به 0 پاک می‌شود و سپس ورودی‌های پاک و بار به 1 برده می‌شوند بنابراین شمارنده همواره فعال خواهد بود. مادامی که خروجی گیت AND برابر 0 است، هر لبه ساعت مثبت، شمارنده را یکبار افزایش می‌دهد. وقتی که خروجی به 1001 برسد، A0 و A3 برابر 1 خواهند شد و بنابراین خروجی گیت AND برابر 1 می‌گردد. این حالت ورودی بار را فعال می‌کند و بنابراین در لبه ساعت بعدی ثبات نمی‌شمارد، ولی از طریق چهار ورودی بار خواهد شد. چون همه ورودی‌ها به منطق 0 مرتبط هستند و همه مقادیر 0 به دنبال شماره 1001، وارد ثبات می‌گردند، بنابراین طبق آنچه در BCD لازم است از 0000 تا 1001 شمرد و سپس به 0000 باز می‌گردد. در شکل ۹-۲۵ (ب)، گیت AND شماره 1010 را تشخیص می‌دهد، و به محض وقوع آن، ثبات پاک می‌شود. شماره 1010 برای ماندن فرصت چندانی نمی‌یابد زیرا ثبات بلافاصله به 0 خواهد رفت. هنگام رفتن از 1010 به 1011 در خروجی A0 یک جرقه کوچک رخ داده و بلافاصله به 0000 خواهد رفت. این جرقه لحظه‌ای ممکن است مطلوب نباشد و به این دلیل، این آرایش پیشنهاد نمی‌گردد. اگر شمارنده دارای ورودی پاک کردن باشد، می‌توان پس از 1001 آن را پاک کرد.

۹-۵ انواع دیگر شمارنده‌ها

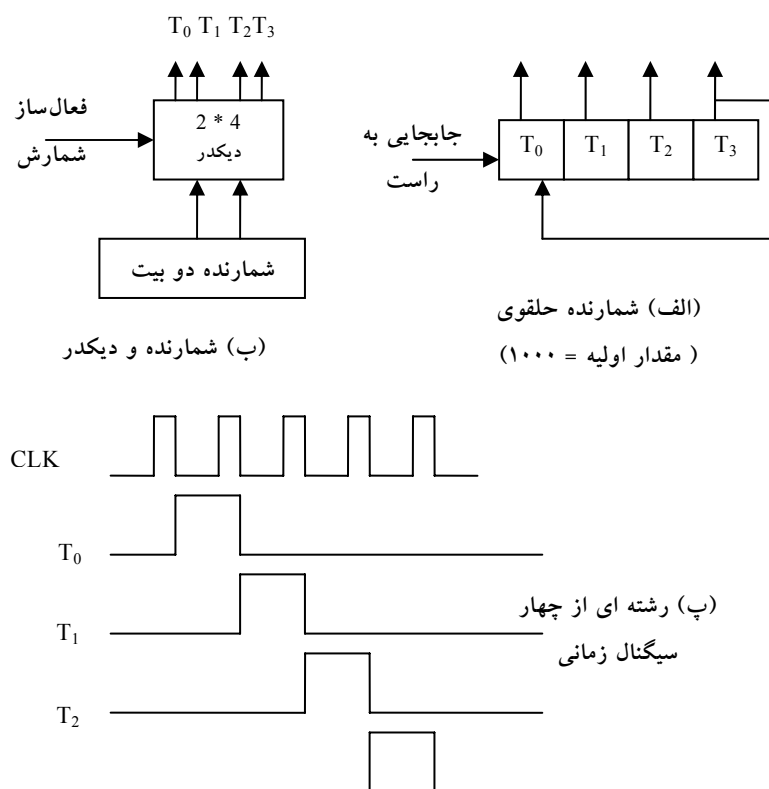
بغیر از دو گروه اصلی شمارنده‌ها که به آنها شمارنده‌های موج گونه و همزمان می‌گفتند، انواع دیگری از شمارنده‌ها نیز وجود دارند. می‌توان شمارنده‌ها را برای تولید هر رشته از حالات طراحی کرد. یک شمارنده تقسیم بر N که به آن N شمار هم می‌گویند، شمارنده ای است که یک رشته N حالتی را تکرار می‌کند. رشته ممکن است شمارش دودویی یا دیگر رشته‌های اختیاری را دنبال کند. از شمارنده‌ها برای تولید سیگنال‌های زمانبندی جهت کنترل یک رشته از اعمال در یک سیستم دیجیتال استفاده می‌شود. شمارنده را می‌توان با شیفت رجیسترها هم ساخت. در ادامه چند نوع از شمارنده‌های غیر دودویی را ارائه خواهیم نمود:

۹-۵-۱ شمارنده حلقوی

سیگنال‌های زمانبندی که رشته عملیاتی را در یک سیستم دیجیتال کنترل می‌کنند با یک شیفت رجیستر یا یک شمارنده و یک دیکدر قابل تولیدند. یک شمارنده حلقوی یک شیفت رجیستر چرخشی است که در آن هر بار تنها یک فلیپ‌فلاپ در حالت 1 است و همه دیگر فلیپ‌فلاپ‌ها صفراند. تنها بیت نشانده (1) به فلیپ‌فلاپ بعدی جابجا می‌شود تا رشته‌ای از سیگنال‌های زمانبندی تولید گردد.

شکل ۹-۲۶(الف) یک شیفت رجیستر 4 بیت متصل به یک شمارنده حلقوی را نشان می‌دهد. مقدار اولیه ثبات 1000 است. تنها بیت فوق‌الذکر با هر پالس ساعت به سمت راست جابجا می‌شود و چرخش هم از T_3 به T_0 اتفاق می‌افتد. هر چهار پالس ساعت یکبار یکی از فلیپ‌فلاپ‌ها در حالت 1 قرار می‌گیرد. هر خروجی پس از یک گذر لبه منفی پالس ساعت، 1 می‌گردد و در طول سیکل ساعت بعدی در 1 باقی می‌ماند.

سیگنال‌های زمانبندی با یک شمارنده ۲ بیت که به چهار حالت جدا از هم می‌رود نیز تولید می‌گردد. دیکدر شکل ۹-۲۶ (ب) چهار حالت شمارنده را دیکدر می‌کند و رشته سیگنال‌های زمانبندی لازم را ایجاد می‌نماید.



شکل ۹-۲۶: تولید سیگنال‌های زمانی با شمارنده حلقوی

برای تولید 2^n سیگنال، به یک شیفت رجیستر با 2^n فلیپ‌فلاپ و یا به یک شمارنده دودویی n بیتی همراه با یک دیکدر n به 2^n نیاز است. مثلاً ۱۶ سیگنال زمانبندی می‌تواند با یک شیفت رجیستر ۱۶ بیتی به عنوان یک شمارنده حلقوی یا یک شمارنده ۴ بیت و یک دیکدر ۴ به ۱۶ تولید گردد. در حالت اول، به ۱۶ فلیپ‌فلاپ نیاز است. در دومی، ۴ فلیپ‌فلاپ و ۱۶ گیت AND چهار ورودی برای دیکدر لازم

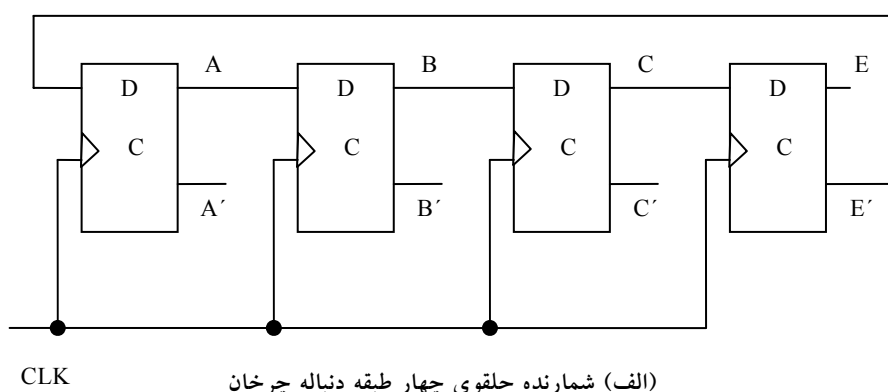
می‌باشد. در این حالت تعداد فلیپ‌فلاپ‌ها کمتر از شمارنده حلقوی است و دیکدر تنها گیت دو ورودی نیاز دارد. این ترکیب را شمارنده جانسون می‌نامند.

۹-۵-۲ شمارنده جانسون

یک شمارنده حلقوی K بیتی، یک بیت را بین K فلیپ‌فلاپ گردش می‌دهد تا بدین وسیله K حالت قابل تفکیک تولید شود. اگر شیفت رجیستر به صورت یک شمارنده حلقوی دنباله چرخان در آید، تعداد این حالات دو برابر می‌شود. شمارنده حلقوی دنباله چرخان یک شیفت رجیستر دوار است که خروجی متمم آخرین فلیپ‌فلاپ به ورودی اولین فلیپ‌فلاپ متصل شده است. شکل ۹-۲۷ (الف) چنین شیفت رجیستری را نشان می‌دهد. اتصال پس‌خوردی از خروجی متمم سمت راست ترین فلیپ‌فلاپ به ورودی سمت چپ ترین فلیپ‌فلاپ ایجاد می‌گردد. شیفت رجیستر مذکور محتوای خود را ب هر پالس ساعت یکبار به سمت راست جابجا می‌کند، و در همان زمان مقدار متمم فلیپ‌فلاپ E به فلیپ‌فلاپ A منتقل می‌گردد.

با شروع از حالت پاک شده، شمارنده حلقوی وارد رشته حالات هشتگانه ای می‌شود، شکل ۹-۲۷ (ب). به طور کلی، یک شمارنده حلقوی دنباله چرخان K بیتی یک رشته ۲K حالت را دنبال می‌نماید. این شمارنده از حالتی که همه بیت‌های آن 0 است شروع می‌نماید و هر عمل شیفت یک 1 را از سمت چپ وارد می‌کند تا همه ثبات‌ها با 1 پر شود. در دنباله عملیات 0 ها از سمت چپ وارد می‌شوند تا وقتی که همه بیت‌های ثبات مجدداً با 0 پر شوند. شمارنده جانسون، شمارنده حلقوی دنباله چرخان K بیتی به همراه 2K گیت برای دیکدر کردن و تهیه 2K سیگنال زمانبندی خروجی است. گیت‌های دیکدر در شکل ۹-۲۷ نشان داده نشده‌اند، اما در آخرین ستون جدول مشخص گردیده‌اند. پس از وصل هشت گیت لیست شده در جدول، ساختار شمارنده جانسون کامل خواهد بود. نظر به اینکه هر گیت در حین یک حالت ویژه توانا می‌شود، خروجی گیت‌ها، به ترتیب هشت سیگنال زمانبندی را تولید خواهند کرد.

رمز گشایی یک شمارنده K بیتی حلقوی دنباله چرخان در به دست آوردن $2K$ دنباله زمانی، از یک الگوی منظم پیروی می‌کند. حالتی که همه بیتها 0 هستند توسط خروجی‌های متمم دو فلیپ‌فلاپ واقع در منتهی‌الیه دیکد می‌شود. کلیه حالات دیگر با الگوی 0 و 1 یا 1 و 0 مجاور دیکد می‌شوند. به عنوان مثال، دنباله 7 دارای یک الگوی 0 و 1 در فلیپ‌فلاپ‌های B و C است. بنابراین خروجی دیکد شده با گرفتن متمم B و خروجی طبیعی C یعنی $B'C$ به دست می‌آید.



رشته اعداد	خروجی های فلیپ فلاپ				گیت AND لازم برای خروجی
	A	B	C	E	
1	0	0	0	0	$A'E'$
2	1	0	0	0	AB'
3	1	1	0	0	BC'
4	1	1	1	0	CE'
5	1	1	1	1	AE
6	0	1	1	1	$A'B$
7	0	0	1	1	$B'C$
8	0	0	0	1	$C'E$

(ب) رشته شمارش و دیکد کردن مورد نیاز

شکل ۹-۲۷: جدول حالت شمارنده

ثباتها و شمارنده‌ها ۳۱۹

یکی از معایب مدار شکل ۹-۲۷ (الف) این است که اگر مدار به حالت بی استفاده وارد شود، شروع به دنبال کردن حالات نا معتبر کرده و هرگز راهش را به یک حالت معتبر نخواهد یافت. این شکل را می‌توان با تصحیح مدار به صورتی که از حالت نامعتبر دوری جوید، رفع کرد. یکی از روش‌های اصلاح، قطع خروجی فلیپ‌فلاپ B است که به ورودی D فلیپ‌فلاپ C می‌رود.

$$DC = (A + C) B$$

که DC معادله ورودی فلیپ‌فلاپ برای ورودی D در فلیپ‌فلاپ C است. شمارنده جانسون را می‌توان با هر طول زمانی ساخت. تعداد فلیپ‌فلاپ‌های لازم نصف تعداد سیگنال‌های زمانبندی است. تعداد گیت‌ها برای ساخت آن نصف تعداد سیگنال‌های زمانبندی بوده و فقط گیت دو ورودی نیاز دارد.

۹-۵-۳ شمارنده با حالات بی استفاده

مداری با n فلیپ‌فلاپ دارای 2^n حالت دودویی است. مواردی وجود دارد که در آن یک مدار ترتیبی حالات کمتری از حداکثر فوق را به کار می‌برد. حالاتی که به کار نروند در جدول حالت لیست نمی‌شوند. هنگام ساده کردن معادلات ورودی، حالات به کار نرفته را می‌توان به عنوان حالت بی‌اهمیت یا حالت خاص بعدی تلقی کرد. به محض طراحی و ساخت مدار عوامل خارجی ممکن است آن را وارد یکی از حالات به کار نرفته کنند. در این حال لازم است مطمئن شویم که مدار بالاخره وارد یکی از حالات معتبر خواهد شد و بنابراین عملکرد معمول خود را دنبال خواهد کرد. در غیر این صورت، اگر مدار ترتیبی در میان حالات به کار نرفته به عنوان حالات بی‌اهمیت تصور شوند، آنگاه پس از طراحی مدار، باید در مورد اثر آنها تحقیق به عمل آید. پس از طراحی می‌توان حالت بعدی حاصل از حالت به کار نرفته را با تحلیل مدار به دست آورد.

به منظور تشریح، شمارنده مربوط به جدول شکل ۹-۲۸ را ملاحظه کنید. شمارش، رشته‌ای تکراری از شش حالت است، که فلیپ‌فلاپ‌های B و C شماره‌های دودویی 00، 01 و 10 را تکرار می‌کنند و A نیز هر سه شماره یک بار از 0 به 1 و بالعکس تغییر می‌نماید. رشته شمارش شمارنده دودویی سر راست نیست و دو حالت 111 011 در شمارش لحاظ نشده‌اند. انتخاب فلیپ‌فلاپ‌های JK شرایط ورودی جدول را به دنبال خواهد داشت. ورودی‌های KB و KC تنها 1 و X را در ستون‌های خود دارند، بنابراین این حالات همیشه برابر 1 هستند. معادلات ورودی دیگر را می‌توان با مینترم‌های 3 و 7 به عنوان بی‌اهمیت مشخص کرد. معادلات ساده شده به قرار زیر می‌باشند:

$$KA = B$$

$$JA = B$$

$$KB = 1$$

$$JB = C$$

$$KC = 1$$

$$JC = B'$$

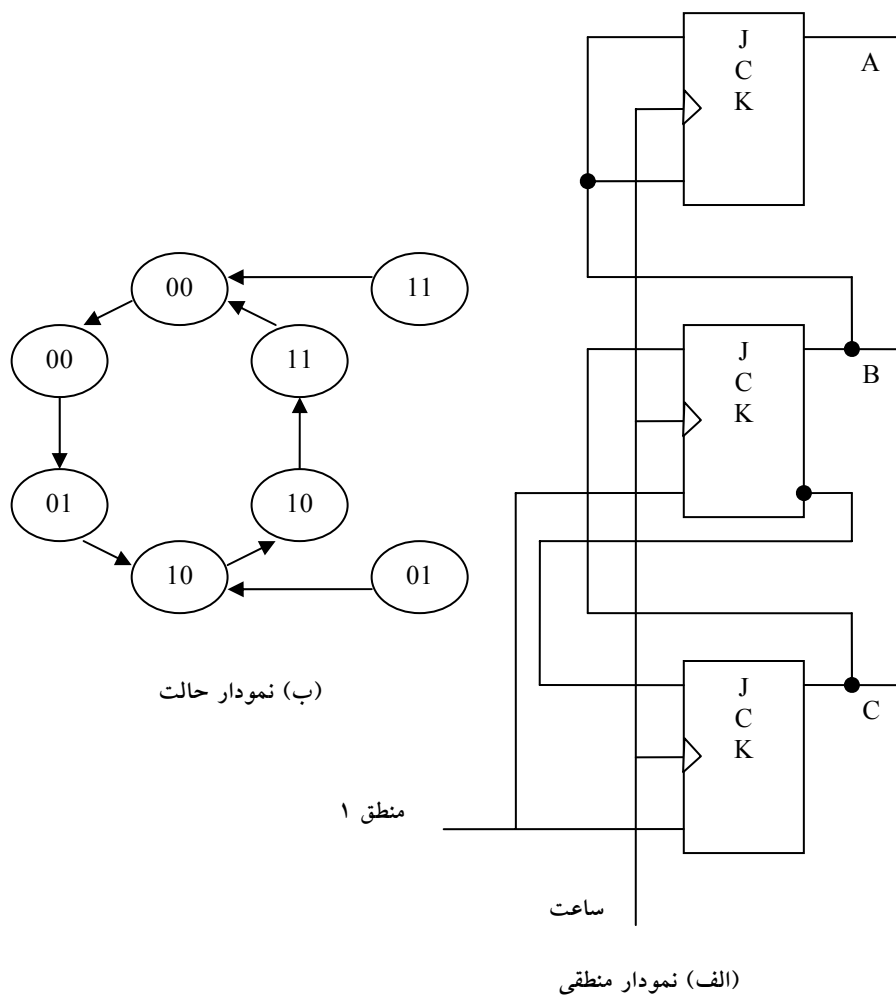
حالت فعلی			حالت بعدی			ورودی‌های فلیپ‌فلاپ‌ها						
A	B	C	A	B	C	JA	KA	JB	KB	JC	KC	
0	0	0	0	0	1	0	X	0	X	1	X	
0	0	1	0	1	0	0	X	1	X	X	1	
0	1	0	1	0	0	1	X	X	1	0	X	
1	0	0	1	0	1	X	0	0	X	1	X	
1	0	1	1	1	0	X	0	1	X	X	1	
1	1	0	0	0	0	X	1	X	1	0	X	

شکل ۹-۲۸: جدول حالت شمارنده

نمودار منطقی شمارنده در شکل ۹-۲۹ (الف) دیده می‌شود. چون دو حالت به کار نرفته وجود دارد، مدار را برای تعیین اثر آنها تحلیل می‌کنیم. اگر مدار به علت وقوع یک سیگنال خطا به حالت 011 برود، پس از اعمال یک پالس ساعت به 100 خواهد رفت. با بررسی نمودار منطقی و توجه به این که با $B = 1$ لبه ساعت بعدی A متمم و C به 0 می‌رود، و وقتی $C = 1$ باشد لبه پالس بعدی B را متمم می‌کند، این نتیجه‌گیری به عمل آمده است. به طریقی مشابه، می‌توان حالت بعدی را از حالت فعلی 111 به 000

ثباتها و شمارنده‌ها ۳۲۱

ارزیابی کرد. نمودار حالت همراه با حالات به کار نرفته در شکل ۹-۲۹ (ب) ملاحظه می‌شود. اگر مدار به علت عوامل خارجی وارد یکی از حالات بی استفاده شود، پالس شمارش بعدی آن را به یکی از حالات معتبر خواهد برد و آنگاه مدار به طور صحیحی به شمارش خود ادامه خواهد داد. بنابراین، مدار خود تصحیح است. یک شمارنده خود تصحیح اگر در یکی از حالات به کار نرفته برود، نهایتاً پس از یک یا چند پالس ساعت به رشته شمارش طبیعی باز خواهد گشت.



شکل ۹-۲۹: شمارنده‌ای با حالات به کار رفته

سؤالات

- ۱- محتوای یک ثبات ۴ بیتی در آغاز 1101 است. ثبات شش بار با ورودی سریال 101101 به راست جابجا می‌شود. محتوای ثبات پس از هر جابجایی چیست؟
- ۲- یک شیفت رجیستر ۴ بیتی با بار شدن موازی را با استفاده از فلیپ‌فلاپ D و دو ورودی shift و load بگونه‌ای طراحی نمایید که شرایط زیر را داشته باشد:
 - وقتی $shift = 1$ است محتوای ثبات یک مکان جابجا می‌شود.
 - وقتی $load = 1$ و $shift = 1$ است داده جدید وارد شیفت رجیستر می‌گردد.
 - اگر هر دو ورودی کنترل برابر 0 باشند، محتوای ثبات تغییر نمی‌کند.
- ۳- نمودار منطقی یک پایین شمار موج گونه دودویی ۴ بیت را بار با فلیپ‌فلاپ‌های زیر رسم کنید:
 - فلیپ‌فلاپ‌هایی که در لبه مثبت ساعت تریگر می‌شوند
 - فلیپ‌فلاپ‌هایی که در لبه منفی ساعت تریگر می‌شوند
- ۴- یک مدار زمانبندی که یک سیگنال خروجی را به مدت هشت پالس ساعت نگهداری می‌کند، طراحی نمایید. سیگنال شروع حالت 1 را خارج می‌کند و پس از هشت سیکل ساعت سیگنال به حالت 0 باز می‌گردد.
- ۵- شمارنده‌ای را با استفاده از فلیپ‌فلاپ JK و با رشته دودویی 0, 1, 2, 4, 6 طراحی نمایید.
- ۶- نشان دهید که یک شمارنده جانشون با n فلیپ‌فلاپ یک رشته $2n$ حالتی تولید می‌کند.

مجموعه سؤالات خودآزمایی

۱- عدد ${}_2(110101/111011)$ را در مبنای ۸ به دست آورید.

الف) ${}_8(72/65)$ ب) ${}_8(73/65)$

ج) ${}_8(37/65)$ د) ${}_8(32/65)$

۲- عدد ${}_4(232/223)$ را در مبنای ۱۶ بنویسید.

الف) ${}_{16}(2C/A3)$ ب) ${}_{16}(2A/B2)$

ج) ${}_{16}(2D/89)$ د) ${}_{16}(2C/53)$

۳- مکمل ۹ عدد ${}_{10}(38940)$ برابر است با:

الف) ${}_{10}(61037)$ ب) ${}_{10}(61059)$

ج) ${}_{10}(69178)$ د) ${}_{10}(61060)$

۴- مکمل ۲ عدد ${}_2(10111011)$ برابر است با:

الف) ${}_2(01000101)$ ب) ${}_2(01000100)$

ج) ${}_2(01001101)$ د) ${}_2(01001000)$

۵- عدد ${}_{10}(35)$ را با کد دهدهی ۲۴۲۱ بنویسید.

الف) (1001101) ب) (0011101)

ج) (10011011) د) (00111011)

۶- مکمل تابع $F = xy + yz(x+y')$ را به دست آورید.

الف) $F' = (x'+y')(y+z'+x')(y+z')$ ب) $F' = (x'+y')(y+z'+x')z'$

ج) $F' = (x' + y') (y' + z' + x')$ (د) $F' = (x' + y') (y' + zx + zy')$

۷- عبارت $F = AB'C + A'BC + A'B'C + ABC + AB'C' + A'BC'$ برابر است با:

الف) $F = C' + BC$ (ب) $F = C + AB$

ج) $F = C + A$ (د) $F = C' + B$

۸- تابع $F(A, B, C, D) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 10, 12, 13, 14)$ برابر است با:

الف) $F = D' + B'$ (ب) $F = C' + A$

ج) $F = C' + D'$ (د) $F = D + BC$

۹- تابع $F(A, B, C) = \Sigma(0, 2, 3, 7)$ را با شرایط بی اثر $d(A, B, C) = \Sigma(4, 5, 6)$ در نظر بگیرید تابع F برابر است با:

الف) $F = B + C'$ (ب) $F = BC + A$

ج) $F = B' + C$ (د) $F = AB + C'$

۱۰- اگر $A + B = 1010$ و $AB = 10101$

الف) $A = 100$ و $B = 110$ (ب) $A = 011$ و $B = 111$

ج) $A = 010$ و $B = 111$ (د) $A = 101$ و $B = 010$

۱۱- با اتصال ورودی J و K به هم در یک JK-FF اگر خط کنترلی UP برابر 1 باشد، کدام FF حاصل می‌گردد؟

الف) JK-FF کاهنده (ب) D-FF افزاینده

ج) RS-FF کاهنده (د) T-FF افزاینده

۱۲- حاصلضرب کلیه ماکسترم‌های یک تابع بول X متغیره برابر است با:

الف) $X/0$ (ب) $0/X$

مجموعه سؤالات خود آزمایی ۳۲۵

(د) X^X

(ج) XX

۱۳- کدام پاسخ صحیح است؟

(ب) $(156)_{10} = (12310)_3$

(الف) $(156)_{10} = (10111001)_2$

(د) $(156)_{10} = (213)_7$

(ج) $(156)_{10} = (2130)_4$

۱۴- متمم دو کدام عدد صحیح نیست؟

(ب) $7 \rightarrow 1000$

(الف) $3 \rightarrow 1101$

(د) $12 \rightarrow 0101$

(ج) $9 \rightarrow 0110$

۱۵- در یک دریچه NOR در چه حالتی خروجی پایین است؟

(ب) هر ورودی بالا باشد

(الف) هر ورودی پایین باشد

(د) تمام ورودی ها بالا باشند

(ج) تمام ورودی ها پایین باشند

۱۶- کدام گروه از FF های زیر دارای حالت مبهم نخواهد بود؟

(ب) RS ، D ، T

(الف) RS ، JK ، T

(د) D ، T ، JK

(ج) JK-MS ، RS ، JK

۱۷- یک حافظه با n خط ورودی داده و k خط آدرس دارای چه ظرفیتی است .

(ب) $2^{k \times n}$

(الف) $2^{n \times k}$

(د) $n k$

(ج) $2nk$

۱۸- کدام پاسخ صحیح نیست؟

(ب) $(0/625)_{10} = (0/101)_2$

(الف) $(0/6875)_{10} = (0/1011)_2$

(د) $(0/65625)_{10} = (0/10101)_2$

(ج) $(0/6875)_{10} = (0/1101)_2$

۱۹- عدد $10(511)$ در مبنای ۱۶ چه مقدار است؟

الف) 1EF ب) 1FE

ج) FF1 د) 1FF

۲۰- متمم ۸ عدد $N = 176$ کدام یک می باشد؟

الف) ۶۰۲ ب) ۶۰۱

ج) ۷۱۱ د) ۷۱۲

۲۱- کدام گزینه برای معادل تمام NOR یک گیت XOR صحیح می باشد؟

الف) سه دریچه NOR در سه سطح ب) پنج دریچه NOR در سه سطح

ج) چهار دریچه NOR در چهار سطح د) شش دریچه NOR در چهار سطح

۲۲- فرض کنید $xy=0$ باشد آنگاه x برابر کدامیک از گزینه های زیر است؟

الف) $x+y$ ب) xy'

ج) $x-y$ د) $x'y$

۲۳- یک PAL دارای چند سطح و کدام دریچه و چه ساختار برنامه ریزی است؟

الف) دارای دو سطح AND ثابت و OR قابل برنامه ریزی

ب) دارای دو سطح AND و OR و هر دو قابل برنامه ریزی

ج) دارای دو سطح OR ثابت و AND قابل برنامه ریزی

د) دارای دو سطح OR و AND هر دو ثابت

مجموعه سؤالات خود آزمایی ۳۲۷

۲۴- هر FF دارای چند حالت است؟

الف) ۱ ب) ۲

ج) ۴ د) n

۲۵- در یک SR-FF، به ازای کدام حالت Q_{n+1} مبهم است؟

الف) $R=1$ ، $S=1$ ب) $R=1$ ، $S=0$

ج) $R=0$ ، $S=1$ د) $R=0$ ، $S=0$

۲۶- دوره تناوب ضربان ساعت یک سیستم که از یک ساعت ۵۰۰ مگا هرتز استفاده می کند چند میکرو ثانیه است؟

الف) ۸ ب) ۴

ج) ۲ د) ۱/۲

۲۷- ساده شده عبارت بولی $ABC + A'B + ABC'$ به صورت:

الف) B' ب) B

ج) $A+B$ د) A

۲۸- عبارت بولی زیر را به منظور کاهش حروف به تعداد تعیین شده ساده کنید؟

$(A' + C) (A' + C') (A + B + C'D)$ به چهار متغیر

الف) $A'B + A'C'D$ ب) $B'A + A'C'D$

ج) $A + A'C'D$ د) $B' + AC'D'$

۲۹- مکمل تابع $F1 = xyz' + x'y'z$

الف) $(x' + y + z) (x + y + z)$ ب) $(x + y' + z) (x + y + z)$

ج) $(x + y + z) (x' + y + z)$ د) $(x + y' + z) (x + y + z')$

۳۲۸ مدار منطقی

۳۰- مجموع حاصلضرب $(AB + C)(B + C'D)$

الف) $A'B + BC$ (ب) $AB' + BC$

ج) $AB + BC$ (د) هیچکدام

۳۱- ساده شده عبارت بولی $xy + x'y'z' + x'yz'$

الف) $x'z' + xy$ (ب) $xz' + xy$

ج) $x'z' + xy'$ (د) هیچکدام

۳۲- معادل مبنای ۱۰ بزرگترین عدد n بیتی برابر است با:

الف) 2^n (ب) 2^{n-1}

ج) 3^{n-1} (د) هیچکدام

۳۳- عدد $(198)_{10}$ در مبنای ۱۰:

الف) $(260)_{10}$ (ب) $(258)_{10}$

ج) $(256)_{10}$ (د) هیچکدام

۳۴- عدد $(1231)_{10}$ به عدد دودویی برابر است با:

الف) $(11011001111)_2$ (ب) $(10111001111)_2$

ج) $(10011001111)_2$ (د) هیچکدام

۳۵- عدد 8620 در مبنای ۱۰ به فرم BCD

الف) $0001 \quad 0010 \quad 0110 \quad 1000$ (ب) $0010 \quad 0010 \quad 0110 \quad 1000$

ج) $0010 \quad 0010 \quad 0110 \quad 1000$ (د) هیچکدام

مجموعه سؤالات خود آزمایی ۳۲۹

۳۶- کدام عبارت درست است؟

الف) یک فلیپ فلاپ RS را می توان با ترکیب دو گیت OR و یک گیت NAND ساخت.

ب) یک فلیپ فلاپ RS را می توان با ترکیب دو گیت OR و دو گیت NAND ساخت.

ج) یک فلیپ فلاپ RS را می توان با ترکیب دو گیت NAND و یا دو گیت NOR که فیدبک دارند ساخت.

د) هیچکدام

۳۷- در یک مدار مولتی پلکسر (ادغام کننده) بین تعداد ورودی ها و خطوط آدرس چه ارتباطی برقرار است. (تعداد خطوط آدرس را برابر با n در نظر می گیریم و تعداد خطوط ورودی را m می نامیم).

$$\text{الف) } m = 2^n \quad \text{ب) } n = 2^m$$

$$\text{ج) } m = 2^{n+1} \quad \text{د) } n = 2^{m+1}$$

۳۸- معادل عدد $(10011/1011)_2$ در مبنای ۱۰ مساوی:

$$\text{الف) } 11/19 \quad \text{ب) } 19/11$$

$$\text{ج) } 11/59375 \quad \text{د) } 19/6875$$

۳۹- کدامیک از پاسخ ها صحیح است ؟

$$\text{الف) } (34)_8 = (29)_{10} = (43)_7 \quad \text{ب) } (14/8)_8 = (13/5)_{10} = (1101/1)_7$$

$$\text{ج) } (23)_4 = (11)_{10} = (1101)_7 \quad \text{د) } (31)_4 = (13)_{10} = (1101)_7$$

۳۳۰ مدار منطقی

۴۰- متمم ۲ عدد $(101101)_2$ برابر است با:

الف) $(010010)_2$ ب) $(010011)_2$

ج) $(010001)_2$ د) $(011110)_2$

۴۱- عدد $(11001/101101)_2$ در مبنای ۸ مساوی است با:

الف) $(31/45)_8$ ب) $(55/31)_8$

ج) $(31/55)_8$ د) $(55/14)_8$

۴۲- مکمل ۹ و ۱۰ عدد $(0/3267)_1$ چیست؟

الف) $0/6732$ ، $0/6732$ ب) $0/6733$ ، $0/6734$

ج) $0/6733$ ، $0/6733$ د) $0/6732$ ، $0/6733$

۴۳- تفاوت فلیپ فلاپ RS با JK در چیست؟

الف) هیچ تفاوتی ندارند.

ب) بر خلاف RS اگر در JK هر دو ورودی فعال باشد خروجی مکمل حالت فعلی خواهد شد.

ج) خروجی JK مکمل خروجی RS است.

د) اصلاً این دو قابل مقایسه نیست.

۴۴- اگر رشته زیر را به فلیپ فلاپ T بدهیم خروجی برابر خواهد بود:

$1000110 =$ ورودی

$(Q=0)$ حالت اولیه خروجی

الف) 1111011 ب) 1000110

ج) 0111001 د) 1010110

مجموعه سؤالات خود آزمایی ۳۳۱

۴۵- تفاوت مدار ترتیبی و ترکیبی چیست؟

الف) مدارات ترتیبی حافظه دارند. ب) هیچ تفاوتی ندارند.

ج) مدارات ترکیبی حافظه دارند. د) مدارات ترتیبی نیاز به ورودی ندارند.

۴۶- عدد دهدهی 250.5 در مبنای 3 کدامست؟

الف) 100021.111 ب) 3322.2

ج) 10021.21 د) 21211.111

۴۷- عدد دهدهی 12.0625 در مبنای 2 کدامست ؟

الف) 1101.01 ب) 1100.10

ج) 1100.0001 د) 1010.1010

۴۸- عدد دودویی 1110101.110 در مبنای ده کدامست ؟

الف) 101.25 ب) 117.75

ج) 117.11 د) 101.11

۴۹- عدد هشت هشتی 623.77 در مبنای 16 کدامست ؟

الف) 193.FC ب) 402.984

ج) 311.88 د) C47.54

۵۰- عدد دهدهی 620 به صورت BCD و کد افزونی 3 به ترتیب کدام گزینه است؟

الف) قابل نمایش به صورت کد افزونی 3 نمی باشد .

ب) 0001 0011 1001 و 0000 0010 0110

ج) 1001 1101 1111 و 0110 0010 0000

د) 0101 1001 0011 و 0110 0010 0000

۵۱- در ارزیابی عبارات جبر بول کدام گزینه صحیح نمی باشد ؟

الف) تقدم اول با پرانتز است. ب) تقدم AND از NOT بیشتر است.

ج) تقدم NOT از OR بیشتر است. د) تقدم AND از OR بیشتر است.

۵۲- عبارت $(A + B)' (A' + B')'$ معادل است با:

الف) صفر ب) یک

ج) $A + B$ د) $A' + B'$

۵۳- متمم تابع بول $(AB' + CD') (BC' + A'D)$ برابر است با :

الف) ABCD ب) $(A + C') (B + D')$

ج) صفر د) یک

۵۴- کدام گزینه بیان دیگری از تابع $F(x, y, z) = (xy + z)(y + xz)$ میباشد:

الف) $\Sigma(3, 5, 6, 7)$ ب) $\Sigma(2, 3, 5, 6)$

ج) $\Pi(0, 1, 2, 4)$ د) الف و ج

۵۵- ساده شده تابع $xyz + x'y + x'yz + x'yz + xyz$ کدام گزینه است ؟

الف) $xy + xz + yz$ ب) $(xy' + yx')z$

ج) xyz د) $x + yz$

۵۶- تابع $F(x, y, z) = \Pi(0, 1, 4, 5)$ معادل است با:

الف) y ب) $x'y$

ج) xy د) صفر

مجموعه سؤالات خود آزمایی ۳۳۳

۵۷- کدام گزینه صحیح نمی باشد ؟

الف) در نیم جمع کننده $C = xy$ است.

ب) در نیم جمع کننده $S = x \oplus y$ است.

ج) در تمام جمع کننده $S = z \oplus (x \oplus y)$ است.

د) در نیم جمع کننده $B = xy$ است.

۵۸- عدد دهدهی (۱۲۰/۵) در مبنای ۸ کدام یک از موارد ذیل است؟

الف) ۱۷۰/۴ (ب) ۲۲۶/۴

ج) ۲۲۶/۵ (د) ۶۴/۵

۵۹- عدد دودویی ۱۱۰۱۱۰۱ / ۱۰۱ در مبنای ۱۰ چه عددی است ؟

الف) ۱۴/۷ (ب) ۱۰۹/۱۲۵

ج) ۱۰۹/۶۲۵ (د) ۲۰۹/۰۵

۶۰- عدد شانزده شانزدهی (در مبنای ۱۶) ۷۶ معادل کدام یک از موارد ذیل است؟

الف) ۱۱۸ در مبنای ۱۰ (ب) ۱۱۱۰۱۱۰ در مبنای ۲

ج) ۲۳۴ در مبنای ۷ (د) ۱۱۶ در مبنای ۸

۶۱- مکمل ۲ عدد ۱۰۰۱۰۰ چیست؟

الف) ۰۱۱۱۰۰ (ب) ۰۱۱۰۱۱

ج) ۰۱۱۱۰۱ (د) ۱۱۱۱۰۰

۶۲- کدام یک از روابط ذیل غلط است؟

الف) $x(x + y) = x$

۳۳۴ مدار منطقی

ب) $x + xy = x$

ج) $\Sigma(1, 4, 5, 6, 7) = \Pi(0, 1, 2, 3)$

د) هیچکدام

۶۳- تابع $F(A, B, C, D) = \Pi(0, 1, 2, 4, 5, 6, 8, 9, 10, 12, 13)$ (با

استفاده از جدول کارنو) معادل کدام یک از توابع ذیل است؟

ب) $F = A'B' + B'D'$

الف) $F = C'D' + AB + A'$

د) $F = C' + A'D' + B'D'$

ج) $F = D' + B'C'$

۶۴- اگر با استفاده از جدول کارنو تابع $F(A, B, C, D) = \Sigma(1, 2, 3, 4, 9)$ با

حالات بی اهمیت $d(A, B, C, D) = \Sigma(10, 11, 12, 13, 14, 15)$ ساده شود ساده

ترین تابع ممکن کدام یک از موارد ذیل است؟

الف) $CB'A' + DB'A + BD'C' + DA$

ب) $B'C + DB'A' + BD'C' + DA$

ج) $B'C + B'D + BC'D'$

د) $A + B'D + BC'D'$

۶۵- با توجه به جدول تحریک فلیپ فلاپ ذیل ورودی y کدام یک از موارد است.

ورودی y	حالت بعدی $Q(t+1)$	حالت فعلی $Q(t)$
0	0	0
1	1	0
1	0	1
0	1	1

ب) ورودی فلیپ فلاپ T

الف) ورودی فلیپ فلاپ D

مجموعه سؤالات خود آزمایی ۳۳۵

ج) ورودی s فلیپ فلاپ r s (د) ورودی r فلیپ فلاپ r s

۶۶- عدد 0.6875 در مبنای 10 معادل چه عددی در مبنای 4 است ؟

الف) 0.23 (ب) 0.32

ج) 3.2 (د) 2.3

۶۷- عدد FD7F در مبنای 16 معادل چه عددی در مبنای 2 است؟

الف) 11111010111111 (ب) 1101011110111101

ج) 1110110111011111 (د) 1101111111100110

۶۸- مکمل مبنای کاهش یافته (مکمل 4) عدد 231 در مبنای 5 کدامیک از موارد ذیل است؟

الف) 213 (ب) 214

ج) 324 (د) 325

۶۹- کدامیک از روابط ذیل در مورد نتیجه $M - N$ در سیستم مکمل 2 صحیح است؟

الف) $M - N = M + (\hat{r}^n - N)$ (ب) $M - N = M + (\hat{r}^n - N - 1)$

ج) $M - N = M + (N - \hat{r}^n)$ (د) $M - N = M + (N - \hat{r}^n + 1)$

۷۰- تابع $f(x, y, z) = x + y + z$ معادل کدامیک از توابع ذیل است؟

الف) $f(x, y, z) = \prod(1)$ (ب) $f(x, y, z) = \prod(0)$

ج) $f(x, y, z) = \prod(7)$ (د) $f(x, y, z) = 1$

۷۱- معادل تابع $f(A, B, C, D) = \sum(0, 2, 4, 6, 8)$ و

$d(A, B, C, D) = \sum(10, 11, 12, 13, 14, 15)$ کدامیک از موارد ذیل است؟

۳۳۶ مدار منطقی

ب) $CD + C'D'$

الف) $A' + D'$

د) AD

ج) D'

۷۲- با استفاده از کدامیک از ترکیبهای ذیل نمی‌توان توابع منطقی را پیاده‌سازی کرد؟

ب) $NAND - NAND$

الف) $NAND - AND$

د) $NOR - NAND$

ج) $AND - NOR$

۷۳- اگر در کد گری، 0011 برای عدد 2 در نظر گرفته شود برای عدد 3 کدامیک از کدهای ذیل را می‌توان استفاده کرد؟

ب) 1010

الف) 0010

د) 0110

ج) 1100

۷۴- کدامیک از توابع ذیل نشان دهنده توابع مدار نیم جمع‌کننده است؟

ب) $c = x \oplus y$ و $s = xy + y$

الف) $c = xy$ و $s = x \oplus y$

د) $c = x'y$ و $s = x' + (x \oplus y)$

ج) $c = xy'$ و $s = x + (x \oplus y)$

۷۵- در استفاده از کد همینگ اگر تعداد بیت‌های تست 4 باشد ($k=4$) حداکثر تعداد بیت‌های داده کدامیک از موارد ذیل است؟

ب) 12

الف) 11

د) 14

ج) 13

۷۶- اگر به ورودی یک T فلیپ فلاپ که در آن $Q = 1$ است رشته صفر و یک $x = 10101$ وارد شود رشته خروجی Q کدام یک از موارد ذیل است؟

ب) $Q = 11001$

الف) $Q = 01100$

مجموعه سؤالات خود آزمایی ۳۳۷

$$Q = 01010 \text{ (د)}$$

$$Q = 10101 \text{ (ج)}$$

۷۷- کدامیک از موارد ذیل غلط است ؟

الف) به وسیله یک دیکدر و 3 گیت OR می توان 3 تابع منطقی را پیاده سازی کرد.

ب) به وسیله مالتی پلکستر فقط می توان یک تابع را پیاده سازی کرد.

ج) یک دیکدر با تواناساز معادل یک دی مالتی پلکستر است.

د) با 8 گیت AND می توان یک انکدر 8×3 ساخت.

۷۸- برای پیاده سازی تابع $f(A, B, C)$ از یک مولتی پلکستر 4×1 با دو خط انتخاب

S_0 و S_1 و چهار خط ورودی I_0 تا I_3 استفاده می شود به طوریکه ورودی B به S_1 و

ورودی C به S_0 وصل است و داده های ورودی I_0 تا I_3 بدین قرارند:

$I_0 = 1$ و $I_1 = 1$ و $I_2 = A$ و $I_3 = A'$. خروجی این مالتی پلکستر کدامیک از توابع

ذیل را پیاده سازی می کند؟

$$f(A, B, C) = \Sigma(1, 3, 5, 6) \text{ (الف)}$$

$$f(A, B, C) = \Sigma(0, 2, 4) \text{ (ب)}$$

$$f(A, B, C) = \Sigma(3, 1, 5) \text{ (ج)}$$

$$f(A, B, C) = \Sigma(0, 1) \text{ (د)}$$

۷۹- بزرگترین عدد دودویی 16 بیتی در مبنای ده کدام است؟

$$2^{16} - 1 \text{ (ب)}$$

$$2^{16} \text{ (الف)}$$

$$2^{15} - 1 \text{ (د)}$$

$$2^{15} \text{ (ج)}$$

۳۳۸ مدار منطقی

۸۰- یک مدار ترکیبی دارای سه ورودی و یک خروجی است. خروجی زمانی یک می شود که دو ورودی از سه ورودی یک باشند. تابع خروجی کدامیک از موارد ذیل است؟

الف) $\Sigma(3, 5, 7)$ ب) $\Sigma(3, 5, 6)$

ج) $\Sigma(5, 7)$ د) $\Sigma(3, 6)$

۸۱- کدامیک از موارد ذیل در مورد تولید سیگنالهای زمانی غلط است؟

الف) در شمارنده حلقوی برای تولید 4 سیگنال زمانی به یک شیفت رجیستر 4 بیتی احتیاج است.

ب) در شمارنده دیکدر برای تولید 4 سیگنال زمانی به یک شمارنده 2 بیتی با یک دیکدر 2×4 احتیاج است.

ج) در شمارنده حلقوی برای تولید 8 سیگنال زمانی به یک شیفت رجیستر با 3 بیت احتیاج است.

د) در شمارنده دیکدر برای تولید 8 سیگنال زمانی احتیاج به شمارنده 3 بیتی با یک دیکدر 3×8 است.

۸۲- عدد 1010.011 در مبنای 2 معادل چه عددی در مبنای 10 است؟

الف) 10.375 ب) 10.5

ج) 10.3 د) 6.8

۸۳- کدامیک از روابط ذیل غلط است؟

الف) $x \oplus y' = (x \oplus y)'$ ب) $x' \oplus y = x \oplus y'$

ج) $x' \oplus y' = (x \oplus y)'$ د) $(x' \oplus y) = (x \oplus y)'$

مجموعه سؤالات خود آزمایی ۳۳۹

۸۴- کدام نامساوی زیر می‌تواند شرط وجود یک کد برای تصحیح کننده یک بیت غلط در هر حرف دانست (حروف m بیتی و k بیت اضافه به حروف می‌شود).

الف) $m \leq 2^k - k - 1$ ب) $m \leq 2^k + k$

ج) $m - 1 \leq 2^k + k$ د) $m - k \leq 1$

۸۵- جدول شکل روبرو

D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

الف) جدول دیکدر است

ب) جدول اینکدر است با اولویت

ج) جدول دیکدر اولویت‌دار

د) جمع کننده دو عدد دو بیتی است

۸۶- جدول شکل مقابل کدام فلیپ فلاپ را نشان می‌دهد.

$Q(t + \Delta t)$	$Q(t)$	ورودی X
0	0	0
0	1	1
1	0	0
1	1	1

ب) JK فلیپ فلاپ

الف) RS فلیپ فلاپ

د) D فلیپ فلاپ

ج) T فلیپ فلاپ

۳۴۰ مدار منطقی

۸۷- از کدام یک از موارد زیر می‌توان برای نمایش میترم‌های یک تابع بولی استفاده کرد؟

الف) مالتی پلکستر ب) دیکدر

ج) اینکدر د) مالتی پلکستر و دیکدر

۸۸- با توجه به نحوه کار JK فلیپ فلاپ در جای خالی چه چیزی باید در جدول قرار داد.

J	K	Q (t+Δt)
0	0	Q(t)
0	1	0
1	0	?
1	1	?

الف) به ترتیب 1 و Q ب) به ترتیب 1 و Q'(t)

ج) به ترتیب 0 و Q'(t) د) به ترتیب 1 و نامشخص

۸۹-تریگر نمودن (Triggering) فلیپ فلاپها یعنی:

الف) تغییرات در ورودی ب) تغییرات در خروجی

ج) سنکرون کردن آنها د) همه موارد

۹۰- با توجه به جدول زیر با استفاده از فلیپ فلاپ نوع T تابع ورودی به فلیپ فلاپ را به دست آورید .

Q (t)	X (ورودی)	Q (t+1)
0	0	1
0	1	0
1	0	1
1	1	0

۳۴۱ مجموعه سؤالات خود آزمایی

ب) $T(Q, X) = 1$

الف) $T(Q, X) = \Sigma(2, 3)$

د) $T(Q, X) = \Sigma(0, 1, 3)$

ج) $T(Q, X) = \Sigma(0, 3)$

پاسخ نامه

ب-۱	۲-الف	۳-ب	۴-الف
د-۵	۶-ج	۷-د	۸-ج
۹-الف	۱۰-ب	۱۱-د	۱۲-ب
۱۳-ج	۱۴-الف	۱۵-ب	۱۶-د
۱۷-ب	۱۸-ج	۱۹-د	۲۰-الف
۲۱-د	۲۲-الف	۲۳-ج	۲۴-ب
۲۵-الف	۲۶-ج	۲۷-ب	۲۸-الف
۲۹-د	۳۰-ج	۳۱-الف	۳۲-ب
۳۳-الف	۳۴-ج	۳۵-د	۳۶-ج
۳۷-الف	۳۸-د	۳۹-د	۴۰-ب
۴۱-ج	۴۲-د	۴۳-ب	۴۴-الف
۴۵-الف	۴۶-الف	۴۷-ج	۴۸-ب
۴۹-الف	۵۰-د	۵۱-ب	۵۲-الف
۵۳-د	۵۴-د	۵۵-الف	۵۶-الف
۵۷-د	۵۸-الف	۵۹-ج	۶۰-ج
۶۱-الف	۶۲-ج	۶۳-د	۶۴-ج
۶۵-ب	۶۶-الف	۶۷-الف	۶۸-الف
۶۹-الف	۷۰-ب	۷۱-ج	۷۲-د
۷۳-الف	۷۴-الف	۷۵-الف	۷۶-الف
۷۷-د	۷۸-الف	۷۹-ب	۸۰-ب
۸۱-ج	۸۲-الف	۸۳-ج	۸۴-الف
۸۵-ج	۸۶-د	۸۷-د	۸۸-د
۸۹-د	۹۰-ج		

سؤالات تشریحی

۱- تابع بولی زیر را با یک متمرکز کننده (multiplexer) 8×1 پیاده کنید.

$$F(A,B,C,D) = \Sigma(0,1,4,5,7,10,11,15)$$

۲- با فلیپ فلاپ های نوع T شمارنده ای طراحی کنید تا اعداد بایری 000 تا 110 را شمارش نموده و دوباره به 000 برگردد.

۳- توابع منطقی f و d به ترتیب بیانگر جمع حاصل ضرب ها و حالات بی اهمیت یک مدار می باشند. مدار را طوری طراحی کنید که:

الف) با حداکثر صرفه جویی قابل پیاده سازی باشد.

ب) فاقد هرگونه خروجی تصادفی باشد.

$$f(A, B, C, D) = \Sigma(1, 2, 5, 6, 7, 13, 15)$$

$$d(A, B, C, D) = \Sigma(0, 10, 12)$$

۴- مدار تابع $F(w, x, y, z) = \Sigma(1, 3, 6, 7, 8, 11, 13, 15)$ را با استفاده از یک تسهیم کننده طراحی کرده و اتصالات آنرا با رسم ساختار کلی مدار نشان دهید.

۵- فرکانس ورودی یک شمارنده همزمان 3 بیتی 120 مگاهرتز است و کلیه FF ها در حالت سکون قرار دارند.

الف) فرکانس خروجی آنرا حساب کنید.

ب) ساختار کلی شمارنده را با استفاده از D-FF رسم کنید.

ج) این شمارنده اعداد فرد صعودی تک رقمی را تولید می کند.

مدار آن را طراحی کنید.

۶- تسهیم کننده تابع زیر را پیاده سازی کرده و بلوک دیاگرام آنرا رسم نمایید.

$$F(w, x, y, z) = \Sigma(0, 3, 4, 8, 9, 12, 15)$$

۷- فرکانس ورودی یک شمارنده همزمان 3 بیتی 180 مگاهرتز است و کلید FF در حالت سکون می‌باشند.

الف) فرکانس خروجی آنرا به دست آورید.

ب) بلوک دیاگرام این شمارنده را به کمک D-FF رسم نمایید.

ج) مدار شمارنده را به گونه‌ای طراحی کنید که اعداد فرد صعودی تک رقمی را تولید کند.

۸- مدار ترکیبی طراحی کنید که یک عدد ۲ بیتی را به توان ۲ برساند.

۹- مدار تولید توازن سه بیتی را طراحی نمایید. (خروجی در صورتی یک شود که تعداد بیت‌های یک زوج باشد).

۱۰- شمارنده‌ای با استفاده از فلیپ فلاپ T طراحی نمایید که سری زیر را بشمارد.

$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 0$

۱۱- تابع زیر را به کمک مالتی پلکسر طراحی کنید.

$$f(a, b, c, d) = \Sigma (0, 1, 3, 4, 8, 9, 15)$$

۱۲- یک مدار ترکیبی طراحی کنید که چهار ورودی و سه خروجی داشته باشد به طوریکه خروجی نشان دهنده مجموع ارقام ورودی باشد.

راهنمایی: به طور مثال اگر ورودی ۱۰۱۱ باشد خروجی باید ۰۱۱ باشد زیرا مجموع

ارقام ورودی ۳ است ($1+0+1+1=3$)

۱۳- بلوک دیاگرامی رسم کنید که نشان دهد چگونه می‌توان یک دیکدر 4×16 را با

استفاده از دو دیکدر 3×8 که دارای ورودی تواناساز هستند؟ (بلوک دیاگرام را با رسم کلیه ورودیها و خروجیها مشخص نمایید).

مجموعه سؤالات خود آزمایی ۳۴۵

۱۴- با استفاده از مولتی پلکسر تابع $F(A, B, C) = \prod(0, 2, 3, 6)$ را پیاده سازی نمایید؟

۱۵- فقط با استفاده از تمام جمع کننده ها یک مدار ترکیبی طراحی کنید که کد BCD را به کد افزونی 3 معادل آن تبدیل کند.

۱۶- یک مدار ترکیبی طراحی کنید که مساوی بودن دو عدد دو بیتی را چک کند، مدار دارای یک خروجی است به طوریکه اگر دو عدد ورودی برابر باشند خروجی یک است و اگر دو عدد ورودی نامساوی باشند خروجی صفر است. (ارائه تابع خروجی پس از ساده سازی و رسم مدار و ارائه جدول درستی مدار الزامی است)

۱۷- یک مدار ترتیبی با استفاده از دو فلیپ فلاپ T به نام A و B و یک ورودی x طراحی کنید، به طوری که وقتی $x = 0$ است حالت مدار ثابت می ماند (یعنی در هر حالتی که قرار دارد، همان حالت تکرار می شود) و اگر $x = 1$ باشد مدار به ترتیب به حالات 00 و 01 و 10 و 11 رفته و این سری تکرار می شود.

۱۸- مدار یک شیفت رجیستر 4 بیتی دو جهته با امکان بار شدن موازی را با استفاده از فلیپ فلاپ D بسازید به طوریکه عملکرد آن به صورت زیر باشد. (S_0S_1) خطوط انتخاب هستند)

$S_0S_1 = 00$ بلا تغییر

$S_0S_1 = 01$ شیفت به راست

$S_0S_1 = 10$ شیفت به چپ

$S_0S_1 = 11$ بار کردن موازی

۱۹- یک دیکدر کد سه افزونی (3- EXCESS) به BCD طراحی کنید.

۲۰- تابع $F(A, B, C) = \Sigma(3, 4, 6, 7)$ را

الف) به وسیله جدول کارنو ساده کنید.

ب) به وسیله دیکدر پیاده سازی نمائید.

ج) به وسیله مالتی پلکستر پیاده سازی نمائید.

۲۱- با استفاده از مدارهای ترتیبی طراحی کنید که مسئله زیر را حل نمائید. یک سیستم ترتیبی دارای دو متغیر X_1, X_2 و یک تابع خروجی Z است. اگر X_1 حداقل یکبار از صفر به یک و از یک به صفر تغییر نماید و X_2 از صفر به یک و از یک به صفر و دوباره از صفر به یک تغییر نماید در این صورت Z از صفر به یک تبدیل می شود اگر بعد از اینکه X_2 از صفر به یک رسید بلافاصله از یک به صفر تغییر نماید آنوقت Z برابر صفر خواهد شد.

۲۲- یک شمارشگر سه بیتی BCD با فلیپ فلاپ k طراحی کنید.

۲۳- با ثبات شیفت دهنده 4 بیتی مداری طراحی نمائید که به ازاء هر پالس ساعت فقط یکی از خروجی های آن یک می شود.

واژه نامه

انگلیسی به فارسی

case sensitive	حساس به حالت	A	
central processing unit	واحد پردازش مرکزی	algorithmic state machine (ASM)	ماشین حالت الگوریتمی
check bit	بیت تست	American standard code for information interchange	کد دودویی استاندارد برای کاراکترهای الفبا
chip	تراشه	application standard IC	مدارات مجتمع خاص
chip select	انتخابگر تراشه	application specific	کاربرد خاص
circuit description module	توصیف مدار	associative	اصل شرکت پذیری
Clear	ورودی پاک	asynchronous	غیرهمزمان
clock generator	مدارهای ترتیبی ساعت دار	B	
closure	بسته بودن	behavioral modeling	مدلسازی رفتاری
communication control characters	کاراکترهای کنترل تبادل اطلاعات	binary coded decimal	کد دهدهی به دودویی
commutative	اصل جابجایی	binary coded decimal	دهدهی کد شده به دودویی
complex programmable logic device	وسیله منطقی برنامه پذیر پیچیده	bitwise	عمل بیتی
composite map	نقشه مرکب	blocking	بلوکی
computer aided design	طراحی با کمک کامپیوتر	bottom- up	پایین به بالا
concatenation	عملگر ادغام	C	
consensus theorem	تنوری وفاق	canonical	متعارف
continuous state assignment	مدل سازی رونده داده	carriage return	بازگشت نورد
count- down counter	پایین شمار	carry	نقلی
critical race	رقابت بحرانی		

۳۴۹ واژه‌نامه انگلیسی به فارسی

excess-3	کدافزونی-۳	cut-and-try	سعی و کاهش
excitation table	جدول تحریک	cycle	چرخه-سیکل
F		D	
fan in	گنجایش ورودی	data path	مسیر داده
fan out	گنجایش خروجی	de multiplexer	دی مولتی پلکسر
field programmable gate array	آرایه گیتی برنامه پذیر موردی	decoder	رمز گشا - دیکدر
file separator	جداساز فایل	delay propagation	تاخیر انتشار
finite state machine	حالت متناهی	digital versatile disk	دیسک چندکاره دیجیتال
flow table	جدول روند	direct set	تنظیم مستقیم
format effector	افکتور های فورمت	direct reset	باز نشان مستقیم
free-running clock	ساعت آزادگرد	distributive low	اصل توزیع پذیری
full adder	جمع کننده کامل	E	
full-custom IC	IC سفارشی		
fundamental mode operation	عملیات اساسی		
G		edge qualifier	نشانه لبه
Gate	گیت	enable	فعال
		encoder	رمز گذار - انکدر
		end around carry	نقلی چرخشی
end carry		نقلی انتهایی	
H		end of text	ختم متن
half adder	نیم جمع کننده	essential prime implicant	موجب های اصلی اساسی
hardware description language	زبان توصیف سخت		

۳۵۰ مدار منطقی

Master	حاکم	افزاری	
medium scale	فشرده‌گی متوسط	hardware design language	زبان طراحی سخت افزار
merging	ادغام	hierarchical description	سلسله مراتبی
mixed notation	علائم مخلوط	high impedance	امپدانس بالا
module	ماژول-قطعه	hold time	زمان نگهداری
	نرم افزاری یا سخت افزاری	horizontal tabulation	جدول بندی افقی
Modulus	مدولوس: عملی ریاضی که نتیجه آن باقیمانده یک تقسیم است	I	
		identity element	عنصر شناسه
multiplexer	مولتی پلکسر	implication table	جدول ایجاب
		incompletely specified function	تابع غیر کامل
	N	information separator	جداسازی اطلاعات
negative acknowledge	تصدیق منفی	instantiation	ذکر
noise margin	حد پارازیت	inverse	معکوس
non critical race	رقابت غیر بحرانی		
non-blocking	غیر بلوکی	L	
not connected	غیر متصل	large scale	فشرده‌گی زیاد
		logic programmable array	منطق آرایه ای
	O		برنامه پذیر
overflow	سرریز	look a head carry	پیش بینی نقلی
	P	M	

واژه‌نامه انگلیسی به فارسی ۳۵۱

row address strobe	آگاه گر آدرس سطر	parity	توازن
		parity check	تست توازن
S		power dissipation	توان مصرفی
Schematic	نمودار تصویری	preset	پیش تنظیم
schematic capture	ضبط تصویری	prime implicant	موجب های اصلی
selector	انتخابگر	primitive gate	گیت اصلی
self complement	کدهای خود متمم	procedural state assignment	تخصیص اجرایی
sequence register	ثبات توالی	processor	پردازشگر
Sequential (or simple) programmable logic device set	وسیله منطقی برنامه پذیر ترتیبی نشاندن	programmable logic array	آرایه منطقی برنامه پذیر
settling time	زمان نشست	programmable logic device	وسیله منطقی برنامه پذیر
setup time	زمان برپایی	R	
signed complement system	متمم علامت دار منفی	race condition	وضعیت رقابتی
signed-magnitude system	سیستم مقدار علامت دار منفی	radix	ممیز
simulation	شبیه سازی	random access memory	حافظه با دستیابی تصادفی
slave	تابع	read only memory	حافظه فقط خواندنی
small scale integration	فشرده‌گی کم	record separator	جداساز رکورد
start of text	شروع متن	register level transfer	سطح انتقال ثباتی
state	حالت	register transfer level	سطح انتقال بین ثباتی
stimulus module	ماژول محرک	Reset	باز نشانی
stray capacitance	ظرفیت پارازیتی		

۳۵۲ مدار منطقی

	کاربر تعریف می شود	structural modeling	مدل سازی ساخت یافته
user-defined	تعریف شده به وسیله کاربر	structural description	توصیف ساختاری
V		switch-level modeling	مدل سازی سطح سوئیچ
Valid	معتبر	switch-tail ring counter	شمارنده حلقوی دنباله چرخان
Vector	بردار	synchronization	همگام سازی
vendor-specific	مختص فروشنده	synchronous	همزمان
very large scale	فشرده گی خیلی زیاد	synthesis	سنتز - ترکیب - ادغام

W

Wire	سیم
wire Less	بی سیم

T

target output	خروجی مقصد یا هدف
task	تکلیف
test bench	برنامه تست
time units	واحدهای زمانی
toggle	دگر وضع
top-down	بالا به پایین
total state	حالت کلی
transition table	جدول گذر
transparent	شفاف
tri-state(three-state)	سه حالت

U

user defined primitives	مواردی که به وسیله
-------------------------	--------------------

واژه نامه

فارسی به انگلیسی

wire Less	بی سیم	الف	
check bit	بیت تست	field programmable gate array	آرایه گیتی برنامه پذیر موردی
	پ	programmable logic array	آرایه منطقی برنامه پذیر
bottom- up count- down counter	پایین به بالا پایین شمار	row address strobe	آگاه گر آدرس سطر
processor	پردازشگر	full-custom IC	آی سی سفارشی
look a head carry	پیش بینی نقلی	merging	ادغام
preset	پیش تنظیم	distributive low	اصل توزیع پذیری
	ت	commutative	اصل جابجایی
consensus theorem	تئوری وفاق	associative	اصل شرکت پذیری
slave	تابع	format effector	افکتور های فورمت
incompletely specified function delay	تابع غیر کامل تاخیر انتشار	high impedance	امپدانس بالا
propagation procedural state assignment	تخصیص اجرایی	selector	انتخابگر
chip	تراشه	chip select	انتخابگر تراشه
parity check negative acknowledge	تست توازن تصدیق منفی تعریف شده به وسیله	direct reset	باز نشان مستقیم
user-defined	کاربر	reset	باز نشانی
task	تکلیف	carriage return	بازگشت نور
direct set	تنظیم مستقیم	top-down	بالا به پایین
parity	توازن	vector	بردار
power dissipation	توان مصرفی	test bench	برنامه تست
		closure	بسته بودن
		blocking	بلوکی

واژه‌نامه فارسی به انگلیسی ۳۵۵

total state	حالت کلی	structural description	توصیف ساختاری
finite state machine	حالت منتهای	circuit description module	توصیف مدار
noise margin	حد پارازیت		ث
case sensitive	حساس به حالت	register	ثبات
	خ	sequence register	ثبات توالی
end of text	ختم متن		ج
target output	خروجی مقصد یا هدف	record separator	جداساز رکورد
	د	file separator	جداساز فایل
toggle	دگر وضع	information separator	جداسازی اطلاعات
binary coded decimal	دهدهی کد شده به دودویی	implication table	جدول ایجاب
de multiplexer	دی مولتی پلکسر	horizontal tabulation	جدول بندی افقی
digital versatile disk	دیسک چندکاره دیجیتال	excitation table	جدول تحریک
	ذ	flow table	جدول روند
	ذکر کردن - توضیح دادن	transition table	جدول گذر
instantiation	- نمایش با نمونه	full adder	جمع کننده کامل
	ر		چ
critical race	رقابت بحرانی	cycle	چرخه-سیکل
non critical race	رقابت غیر بحرانی		ح
encoder	رمز گذار - انکدر	random access memory	حافظه با دستیابی تصادفی
decoder	رمز گشا - دیکدر	read only memory	حافظه فقط خواندنی
	ز	master	حاکم
hardware description language	زبان توصیف سخت افزاری	state	حالت

۳۵۶ مدار منطقی

	ط	hardware design language	زبان طراحی سخت افزار
design	طراحی	setup time	زمان برپایی
computer aided design	طراحی با کمک کامپیوتر	settling time	زمان نشست
	ظ	hold time	زمان نگهداری
stray capacitance	ظرفیت پارازیتی		س
	ع	free-running clock	ساعت آزادگرد
mixed notation	علائم مخلوط	overflow	سرریز
bitwise	عمل بیتی	register transfer level	سطح انتقال بین ثباتی
concatenation	عملگر ادغام	register level transfer	سطح انتقال ثباتی
fundamental mode operation	عملیات اساسی	cut-and-try	سعی و کاهش
identity element	عنصر شناسه	hierarchical description	سلسله مراتبی
	غ	synthesis	سنتز - ترکیب - ادغام
not connected	غیر متصل	tri-state(three-state)	سه حالت
non-blocking	غیر بلوکی	signed-magnitude system	سیستم مقدار علامت دار منفی
asynchronous	غیر همزمان	wire	سیم
	ف		ش
medium scale	فشرده‌گی متوسط	simulation	شبیه سازی
very large scale	فشرده‌گی خیلی زیاد	start of text	شروع متن
large scale	فشرده‌گی زیاد	transparent	شفاف
small scale integration	فشرده‌گی کم	switch-tail ring counter	شمارنده حلقوی دنباله چرخان
enable	فعال		ض
	ک	schematic capture	ضبط تصویری

واژه‌نامه فارسی به انگلیسی ۳۵۷

structural modeling	مدل‌سازی ساخت یافته	communication control characters	کاراکترهای کنترل تبادل اطلاعات
switch-level modeling	مدل‌سازی سطح سوئیچ	application specific	کاربرد خاص
behavioral modeling	مدلسازی رفتاری	american standard code for information interchange	کد دودویی استاندارد برای کاراکترهای الفبا عددی اسکی
modulus	تقسیم است	binary coded decimal	کد دهمی به دودویی
data path	مسیر داده	excess_3	کد افزونی-۳
valid	معتبر	self complement	کدهای خود متمم
inverse	معکوس		گی
radix logic programmable array	ممیز	gate	گیت
user defined primitives	منطق آرایه ای برنامه پذیر مواردی که به وسیله کاربر تعریف می شود	fan in	گنجایش ورودی
prime implicant	موجب های اصلی	fan out	گنجایش خروجی
essential prime implicant	موجب های اصلی اساسی	primitive gate	گیت اصلی
multiplexer	مولتی پلکسر		م
	ن	module	ماژول - قطعه نرم افزاری یا سخت افزاری
set	نشاندن	stimulus module	ماژول محرک
edge qualifier	نشانه لبه	algorithmic state machine (ASM)	ماشین حالت الگوریتمی
composite map	نقشه مرکب	canonical signed complement system	متعارف
carry	نقلی		متمم‌علامت دار منفی
end carry	نقلی انتهایی	vendor-specific application standard IC	مختص فروشنده
end around carry	نقلی چرخشی		مدارات مجتمع خاص
schematic	نمودار تصویری	clock generator	مدارهای ترتیبی ساعت دار
		continuous state assignment	مدل‌سازی رونده داده

half adder	نیم جمع کننده
	و
central processing unit	واحد پردازش مرکزی
time units	واحدهای زمانی
clear	ورودی پاک
programmable logic device	وسیله منطقی برنامه پذیر
complex programmable logic device	وسیله منطقی برنامه پذیر پیچیده
Sequential (or simple) programmable logic device	وسیله منطقی برنامه پذیر ترتیبی
race condition	وضعیت رقابتی
	ه
synchronization	همگام سازی
synchronous	همزمان

لیست منابع و مراجع

۱. Logic and Computer Design Fundamentals _ 2ndEd Prentice Hall
Morris M . Mano 2000
۲. Digital Logic Circuit Analysis and Design Prentice Hall
Nelson _ Nagle _ Irwin _ Carroll 1995
۳. Introduction to Logic Design Addison_wesley
Hayes 1993
۴. Digital Design : Principles and Practices _ 3rd Ed Prentice Hall
Wakerly 2000
۵. Contemporary Logic Design Prentice Hall
Katz 1994
۶. Logic Design of Digital Systems _ 3rd Ed Allyn Bacon

مدار الکتریکی ۳۶۰

Dietmeyer 1988

۷. Principles of Digital Design Prentice Hall

Gajski 1997

۸. Fundamentals of Logic Design _ 4th Ed West

Roth 1992

۹. ویرایش سوم _ طراحی دیجیتال انتشارات خراسان

دکتر قدرت سپید نام ۱۳۸۴